Formal Logic

Language, Semantics, Deduction, and Metatheory

A Graduate-Level Reference

Version 1.0

9 August 2026

This document fixes one technical term for each concept and records the scope of every notation before use. Definitions precede use. Every calculus is identified before the derivability symbol is used, and every semantic claim states the semantic data over which it is made.

Preface

Formal logic studies formal languages, interpretations of those languages, and rule-governed derivations. Correct work in formal logic therefore requires more than symbol manipulation. It requires exact control of the level of language, the grammar of expressions, variable binding, interpretation, assumptions, inference rules, and the metatheoretic hypotheses under which a result is stated.

This reference is organized at graduate level rather than as a catalogue of disconnected systems. The dependency order is deliberate. Syntax is defined before semantics; semantics is defined before soundness; a calculus is defined before derivability; classical propositional logic is established before first-order logic; classical first-order logic is established before model theory, computability, incompleteness, proof theory, and nonclassical logic. The organization follows the core treatments in [19, 18, 47, 45], while the historical and advanced results are tied to their original sources where practical.

The document has four purposes:

1. to teach the formal-logical content of the subject;
2. to teach the construction and checking of derivations, proofs, countervaluations, structures, and countermodels of sentence arguments;
3. to teach the writing and typesetting of formal logic; and
4. to provide a dependency-ordered reference for graduate study, teaching, and preparation for qualifying examinations.

The reference is self-contained at the level of notation and foundational definitions. Advanced model theory and proof theory are presented to the point required for a broad graduate-level reference, but a research specialization still requires the monographs cited in the relevant chapters.

Terminology and notation policy

Literature in formal logic does not impose one universal notation. Pretending otherwise would be inaccurate. This document instead fixes a registered global convention. A subsystem-specific departure is permitted only when its scope and standing are declared before use, and no registered notation is used for a second role within that scope.

Whenever this document introduces a new technical name, five conditions are met at the same location:

1. *Necessity*: an existing term cannot express the required distinction without ambiguity;
2. *Definition*: an exact eliminable definition or an explicit axiom is supplied;
3. *Standing*: the grammatical and formal standing of the name is stated;
4. *Nonredundancy*: the distinction from previously defined terms is explained; and
5. *Use*: later definitions or theorems require the distinction.

Until those conditions are met, the text describes the required object or relation without assigning it a technical name.

Throughout this document, the word *iff* abbreviates the metalinguistic biconditional "if and only if"; it is the registered connective of definitions and metatheoretic equivalences.

The following distinctions are permanent:

- A *formula* is a grammatical object-language expression. A *sentence* is a formula with no free variable occurrence.
- A *structure* supplies a domain and interpretations for the nonlogical symbols of a signature. A *model* is a structure that satisfies a specified set of sentences.
- A *valuation* assigns truth values to propositional variables. A *variable assignment* assigns domain elements to first-order variables.
- *Satisfaction* is the relation between a formula and the complete semantic data permitted by the active semantics. *Truth* is satisfaction relative to specified such data. *Validity* is satisfaction for every permitted tuple of semantic data. *Semantic consequence* is preservation of satisfaction from a set of assumptions to a conclusion. For a first-order sentence, assignment independence later permits the specialized phrase "truth in a structure."
- A *derivation* is a finite rule-governed object in a specified calculus. A *proof* is a derivation with no undischarged assumptions other than the stated premises.
- *Soundness* proceeds from derivability to semantic consequence. *Completeness* proceeds from semantic consequence to derivability.
- *Consistency* excludes derivability of a contradiction. A theory is *deductively complete* iff every sentence or its negation is derivable from the theory. A theory is *semantically complete* iff it semantically entails every sentence or its negation. For the fixed sound and complete first-order calculus, these two properties are equivalent; neither is consistency.

Evidence and verification

The status of a formal result must be reported at the level actually checked. These levels are not interchangeable:

1. a displayed calculation checks only its displayed instances;
2. a truth table checks all valuations of the finitely many variables it contains;
3. a derivation checks derivability in the stated calculus;
4. a proof establishes the stated result under its stated hypotheses;
5. a proof-assistant kernel checks a formal proof relative to the kernel, imported declarations, and admitted axioms;
6. successful LaTeX compilation checks typesetting syntax, not logical correctness; and
7. editorial or formal-logical review is independent evidence and must not be inferred from any preceding level.

When this document states a result status explicitly, it uses exactly one of three terms. *Complete proof* means that all substantive formal-logical steps are supplied in this document. *Proof sketch* identifies the construction and states which substantive verification is omitted. *Cited theorem* states exact hypotheses and a conclusion but refers to a named source for the proof. A dependency used within a complete proof is stated separately as a cited theorem; it is not hidden in a qualified status.

Graduate-Level Scope

Prerequisites

The reader should be able to work with sets, functions, relations, finite strings, countable sets, induction on natural numbers, and ordinary rigorous proofs. Prior exposure to elementary logic is useful but not required. Chapters that use cardinal arithmetic, algebra, or topology state the additional facts they require.

Learning outcomes

A reader completing the reference should be able to:

1. define a formal language by an inductive grammar and prove results by structural induction;
2. define capture-avoiding substitution and prove a substitution lemma;
3. prove soundness for a natural-deduction or sequent calculus;
4. prove completeness for classical propositional logic and explain a Henkin proof of completeness for classical first-order logic;
5. derive compactness and the downward and upward Löwenheim–Skolem theorems with their cardinal hypotheses;
6. use back-and-forth arguments, elementary chains, and ultraproducts;
7. prove undecidability by a many-one reduction;
8. state and apply the diagonal lemma and derive the first incompleteness theorem, the second incompleteness theorem, and undefinability of truth with exact hypotheses;
9. explain cut elimination, normalization, and the subformula property;
10. give relational semantics for intuitionistic and normal modal logic and prove representative soundness results;
11. distinguish full semantics from Henkin semantics for second-order logic; and
12. audit a written or mechanized proof for hidden assumptions, variable capture, rule misuse, and shifts between object language and metalanguage.

Contents

Preface i

Graduate-Level Scope iii

I Foundations of Formal Work 1

1 Sets, Relations, Functions, and Cardinality 2
1.1 Sets and classes . . . 2
1.2 Ordered pairs and products . . . 2
1.3 Relations and functions . . . 3
1.4 Equivalence relations and quotients . . . 4
1.5 Finite, countable, and uncountable sets . . . 4
1.6 Choice principles used later . . . 5

2 Levels, Objects, and Consequence 8
2.1 Formal logic and a logical system . . . 8
2.2 Object language and metalanguage . . . 8
2.3 Syntax, semantics, and proof theory . . . 9
2.4 Truth, satisfaction, validity, and models . . . 10
2.5 Arguments and formalization . . . 11

3 Finite Strings, Induction, and Recursion 12
3.1 Finite sequences and strings . . . 12
3.2 Inductive definitions . . . 13
3.3 Structural recursion . . . 14
3.4 Induction on derivations . . . 15
3.5 Relations and closures . . . 15

4 Writing Formal Definitions and Proofs 17
4.1 The dependency order of a formal development . . . 17
4.2 Definitions . . . 17
4.3 Theorem statements . . . 18
4.4 Proof methods . . . 18
4.4.1 Direct proof . . . 18
4.4.2 Proof by contrapositive . . . 18
4.4.3 Proof by contradiction . . . 18
4.4.4 Induction . . . 18

4.4.5 Construction . . . 19
4.4.6 Counterexample . . . 19
4.5 Displayed formulas and prose . . . 19
4.6 Inference-rule typography . . . 19
4.7 Error taxonomy . . . 19

II Classical Propositional Logic 21

5 Propositional Syntax and Semantics 23
5.1 Alphabet and grammar . . . 23
5.2 Valuations and recursive truth clauses . . . 25
5.3 Translation from controlled prose . . . 26
5.4 Normal forms . . . 26

6 Natural Deduction and Sequents 30
6.1 Classical natural deduction . . . 30
6.2 Classical sequent calculus . . . 32

7 Soundness, Completeness, Compactness, and Resolution 33
7.1 Soundness . . . 33
7.2 Maximal consistent sets . . . 34
7.3 Completeness and compactness . . . 35
7.4 Propositional resolution . . . 36
7.5 Common errors . . . 38
7.6 Exercises . . . 38

III Classical First-Order Logic 39

8 First-Order Syntax 41
8.1 Signatures and terms . . . 41
8.2 Formulas and unique readability . . . 42
8.3 Free variables and sentences . . . 42
8.4 Capture-avoiding substitution . . . 43

9 Structures, Satisfaction, and Translation 46
9.1 Structures and variable assignments . . . 46
9.2 Satisfaction . . . 47
9.3 Translation with quantifiers . . . 48

10 Theories and Axiomatization 50
10.1 Axiomatization errors . . . 52

11 First-Order Deduction 53
11.1 Natural-deduction rules for quantifiers and equality . . . 53
11.2 First-order sequent rules . . . 54
11.3 Soundness . . . 55
11.4 Quantifier-rule errors . . . 56

11.5 Exercises . . . 56

12 Henkin Completeness and Compactness **58**
12.1 Syntactic preparation . . . 58
12.2 The term structure . . . 61
12.3 Completeness . . . 62
12.4 Compactness . . . 64

13 Cardinality, Normal Forms, and Resolution **65**
13.1 Elementary substructures . . . 65
13.2 The Löwenheim–Skolem theorems . . . 66
13.3 Prenex form and Skolemization . . . 66
13.4 First-order resolution . . . 71
13.5 Common errors . . . 75
13.6 Exercises . . . 75

IV Model Theory **77**

14 Maps, Substructures, and Elementarity **78**
14.1 Homomorphisms, embeddings, and isomorphisms . . . 78
14.2 Substructures . . . 79
14.3 Elementary maps and elementary substructures . . . 80
14.4 Diagrams and their embedding tests . . . 81
14.5 Elementary chains . . . 82

15 Types, Saturation, and Ultraproducts **83**
15.1 Types over parameter sets . . . 83
15.2 Homogeneity . . . 85
15.3 Omitting types . . . 86
15.4 Filters and ultrafilters . . . 86
15.5 Ultraproducts . . . 87

16 Quantifier Elimination and Categoricity **90**
16.1 Quantifier elimination . . . 90
16.2 Categoricity . . . 91

V Computability **94**

17 Machines and Effective Procedures **95**
17.1 Deterministic single-tape Turing machines . . . 95
17.2 Primitive recursion . . . 97
17.3 Decidable and computably enumerable sets . . . 98

18 Universal Computation, Reductions, and Fixed Points **100**
18.1 An effective numbering of programs . . . 100
18.2 Many-one reductions . . . 102
18.3 The self-application domain problem . . . 103

18.4 The recursion theorem . . . 103
18.5 Rice's theorem . . . 104

19 Logical Decision Problems and the Arithmetical Hierarchy 105
19.1 The classical decision problem . . . 105
19.2 Quantifier complexity over arithmetic . . . 106
19.3 Relative computation and Post's theorem . . . 112

VI Formal Arithmetic and Incompleteness 115

20 Arithmetic, Representation, and Coding 116
20.1 The language and standard structure of arithmetic . . . 116
20.2 Standard and nonstandard models of arithmetic . . . 117
20.3 Representability . . . 118
20.4 Gödel coding . . . 119

21 Diagonalization and the First Incompleteness Theorem 121
21.1 The diagonal lemma . . . 121
21.2 Consistency and soundness conditions . . . 122
21.3 The Gödel sentence . . . 122
21.4 Rosser's strengthening . . . 123

22 Provability, the Second Incompleteness Theorem, and Truth 125
22.1 Derivability conditions . . . 125
22.2 Löb's theorem . . . 126
22.3 The second incompleteness theorem . . . 126
22.4 Tarski's undefinability theorem . . . 127
22.5 A final audit table . . . 127

VII Axiomatic Set Theory 129

23 The Axioms of ZF and ZFC 130
23.1 The language and the axiom schemas . . . 130
23.2 First constructions and exact uses of the axioms . . . 132

24 Hierarchy, Ordinals, and Cardinals 134
24.1 Transitive sets and ordinals . . . 134
24.2 The cumulative hierarchy and rank . . . 135
24.3 Well-founded relations and the Mostowski collapse . . . 136
24.4 Cardinals and the Axiom of Choice . . . 136

25 Constructibility and Forcing 139
25.1 The constructible hierarchy . . . 139
25.2 Forcing over a countable transitive set: a pedagogical device . . . 140
25.3 Exact relative-consistency status . . . 144

VIII Proof Theory **147**

26 Classical and Intuitionistic Calculi **148**
26.1 Sequents and contexts . . . 148
26.2 The propositional rules of LK . . . 149
26.3 Quantifier rules . . . 149
26.4 The calculus LJ . . . 150
26.5 Primitive, derived, and admissible rules . . . 151

27 Cut Elimination and Analytic Derivations **152**
27.1 Measures on cuts . . . 152
27.2 What cut elimination establishes . . . 154
27.3 Interpolation and definability . . . 154

28 Normalization and Ordinal Analysis **156**
28.1 Natural-deduction normalization . . . 156
28.2 The simply typed lambda calculus . . . 157
28.3 Curry–Howard correspondence . . . 158
28.4 Scope of ordinal analysis . . . 158

IX Second-Order and Higher-Order Logic **160**

29 Second-Order Syntax and Two Semantics **161**
29.1 Second-order formulas . . . 161
29.2 Full semantics . . . 161
29.3 Full second-order categoricity . . . 162
29.4 Failure of compactness and effective completeness . . . 163
29.5 Henkin semantics . . . 164

30 Simple Type Theory **167**
30.1 Types, terms, and formulas . . . 167
30.2 Standard and Henkin semantics . . . 168
30.3 Type discipline and foundational scope . . . 169

X Nonclassical Logic **171**

31 Intuitionistic Logic **172**
31.1 Language, calculus, and Kripke structures . . . 172
31.2 Relations to classical derivability . . . 174

32 Modal, Temporal, and Epistemic Logic **175**
32.1 Normal propositional modal logic . . . 175
32.2 Linear temporal logic . . . 176
32.3 Epistemic logic . . . 178

33 Many-Valued and Paraconsistent Logic **180**
33.1 Logical matrices . . . 180
33.2 A value-labelled tableau calculus . . . 180
33.3 Kleene's three-valued matrix . . . 181
33.4 The Logic of Paradox . . . 182

34 Relevance and Linear Logic **183**
34.1 Structural rules as logical data . . . 183
34.2 Routley–Meyer semantics for relevance . . . 183
34.3 Intuitionistic multiplicative linear logic . . . 185
34.4 Relations among the systems . . . 186

XI Formal Practice, Automation, and Verification **187**

35 Constructing Derivations, Models, and Countermodels **188**
35.1 A derivation-search method . . . 188
35.2 A propositional countervaluation method . . . 188
35.3 A first-order semantic-refutation method . . . 189
35.4 Writing a metatheoretic proof . . . 189
35.5 Failure analysis . . . 190

36 Automated Reasoning **191**
36.1 Propositional satisfiability . . . 191
36.2 Equivalence and equisatisfiability . . . 192
36.3 First-order unification . . . 192
36.4 First-order resolution . . . 193
36.5 Equality and superposition . . . 193
36.6 SAT, SMT, and first-order proving . . . 194
36.7 Complexity and practical status . . . 194

37 Mechanized Proof Checking **196**
37.1 Kernel, elaboration, and automation . . . 196
37.2 Propositions as types . . . 196
37.3 Trusted base . . . 197
37.4 A small propositions-as-types example . . . 197

38 Writing and Typesetting Formal Logic **198**
38.1 Academic prose carries the argument . . . 198
38.2 One symbol, one role within a declared scope . . . 199
38.3 A complete before-and-after formalization . . . 199
38.4 LaTeX structure . . . 200
38.5 Auditing source and rendered output . . . 200
38.6 Rule displays . . . 201
38.7 Worked correction of an invalid rule application . . . 201
38.8 Citation practice . . . 201
38.9 Revision audit . . . 202

39 Research Workflow and Reproducibility **203**
39.1 Table for theorem status . . . 203
39.2 Counterexample-first testing . . . 203
39.3 Reproducible computational supplements . . . 204
39.4 Independent review . . . 204

A Registered Notation **205**
A.1 Structure convention . . . 207
A.2 Natural numbers convention . . . 207

B Binding and Precedence **208**

C Rule Sheets **209**
C.1 Classical natural deduction . . . 209
C.2 Classical sequent calculus . . . 210
C.3 Modal calculus for K . . . 211

D Qualifying-Examination Problems and Selected Solutions **212**
D.1 Problem 1: substitution and satisfaction . . . 212
D.2 Problem 2: compactness . . . 212
D.3 Problem 3: incompleteness hypotheses . . . 213
D.4 Problem 4: modal frame condition . . . 213
D.5 Problem 5: cut and subformula property . . . 213
D.6 Additional problems . . . 213

E Exercise Feedback **215**
E.1 Language levels, formal objects, and proof writing . . . 215
E.2 Classical propositional logic . . . 216
E.3 Classical first-order logic . . . 217
E.4 Model theory . . . 220
E.5 Computability and arithmetical hierarchy . . . 222
E.6 Arithmetic and incompleteness . . . 222
E.7 Set theory and forcing . . . 224
E.8 Sequent calculus, normalization, and typed proof terms . . . 225
E.9 Second-order, typed, intuitionistic, and modal systems . . . 226
E.10 Many-valued, relevance, and linear logic . . . 227
E.11 Formal practice, automation, mechanization, and typesetting . . . 228
E.12 Keys for the qualifying problems . . . 229

F Editorial and Formal-Logical Release Checklist **231**
F.1 Definitions and notation . . . 231
F.2 Theorems and proofs . . . 231
F.3 Sources and computation . . . 231
F.4 Production . . . 232

Bibliography **233**

Index **237**

Part I

Foundations of Formal Work

Chapter 1

Sets, Relations, Functions, and Cardinality

This chapter fixes the metatheoretic set theory used before the dedicated study of axiomatic set theory. The treatment assumes ordinary classical set theory in the metalanguage. It does not treat an informal collection as a set unless the ambient set theory licenses its existence.

1.1 Sets and classes

Definition 1.1 (Subset)**.** For sets A and B, write $A \subseteq B$ iff

$$\forall x\,(x \in A \to x \in B).$$

The sets are equal iff each is a subset of the other.

Definition 1.2 (Power set)**.** For a set A, its *power set* is

$$\mathcal{P}(A) := \{X : X \subseteq A\}.$$

Definition 1.3 (Class)**.** A *class* is a metalinguistically specified collection of sets. A class need not be a set. A *proper class* is a class that is not a set.

An unrestricted collection described by a condition may be a proper class. Quantification over such a collection is metalinguistic quantification in the ambient foundation, not object-language quantification over a set.

Warning 1.4. Set-builder notation does not itself prove that a set exists. The expression $\{x : P(x)\}$ may denote a proper class. When existence matters, identify a previously available bounding set or cite the relevant set-existence axiom.

1.2 Ordered pairs and products

Definition 1.5 (Ordered pair)**.** The *ordered pair* of sets a and b is

$$\langle a, b\rangle := \{\{a\}, \{a, b\}\}.$$

Lemma 1.6 (Ordered-pair identity)**.** *For all sets a, b, c, d,*

$$\langle a, b\rangle = \langle c, d\rangle \quad \Longleftrightarrow \quad a = c \text{ and } b = d.$$

Proof. The reverse implication follows by substitution. For the forward implication, if $a = b$, then $\langle a, b\rangle = \{\{a\}\}$, so equality of the ordered pairs forces $c = d = a$. If $a \neq b$, then the ordered pair has the two distinct members $\{a\}$ and $\{a, b\}$. Equality forces $\{c\} = \{a\}$, hence $c = a$, and then comparison of the two-element members gives $d = b$. The remaining apparent case $c = d$ would make the right ordered pair a singleton and is excluded by $a \neq b$. □

Definition 1.7 (Cartesian product)**.** For sets A and B,

$$A \times B := \{\langle a, b\rangle : a \in A \text{ and } b \in B\}.$$

For every $n \in \mathbb{N}$, define A^n as the set of functions from $n = \{0, \ldots, n-1\}$ to A. When $n = 0$, the domain is empty, so A^0 contains exactly the empty function.

1.3 Relations and functions

Definition 1.8 (Binary relation)**.** A *binary relation from A to B* is a subset of $A \times B$. A *binary relation on A* is a subset of $A \times A$.

Definition 1.9 (Properties of a binary relation)**.** Let R be a binary relation on A. It is *reflexive* iff aRa for every $a \in A$; *irreflexive* iff aRa for no $a \in A$; *symmetric* iff aRb implies bRa for all $a, b \in A$; and *transitive* iff aRb and bRc imply aRc for all $a, b, c \in A$.

The relation R is a *strict linear order* on A iff it is irreflexive and transitive and, for every distinct $a, b \in A$, exactly one of aRb and bRa holds. It *strictly well-orders* A iff it is a strict linear order and every nonempty $B \subseteq A$ has an R-least member: some $b \in B$ such that no $a \in B$ satisfies aRb. These clauses define the relation properties used below; none is a second name for another clause, and strict well-ordering is required in the definition of ordinal.

Definition 1.10 (Function)**.** A *function from A to B*, written $f : A \to B$, is a binary relation $f \subseteq A \times B$ such that for every $a \in A$ there exists exactly one $b \in B$ with $\langle a, b\rangle \in f$. The set A is the domain, and B is the codomain.

The value at a is written $f(a)$. The range is $\mathrm{rng}(f) = \{f(a) : a \in A\}$. The codomain is part of the typed declaration and need not equal the range.

Definition 1.11 (Partial function)**.** A *partial function from A to B*, written $f : A \rightharpoonup B$, is a function $f : D \to B$ for some $D \subseteq A$. Its domain of definition is D.

Definition 1.12 (Injection, surjection, and bijection)**.** Let $f : A \to B$.
1. The function is *injective* iff $f(a) = f(a')$ implies $a = a'$ for all $a, a' \in A$.
2. The function is *surjective* iff for every $b \in B$ there is an $a \in A$ such that $f(a) = b$.
3. The function is *bijective* iff it is injective and surjective.

Definition 1.13 (Image and inverse image)**.** For $X \subseteq A$ and $Y \subseteq B$, define

$$f[X] := \{f(x) : x \in X\}, \qquad f^{-1}[Y] := \{x \in A : f(x) \in Y\}.$$

The notation $f^{-1}[Y]$ does not assert that f has an inverse function.

1.4 Equivalence relations and quotients

Definition 1.14 (Equivalence relation)**.** A binary relation E on A is an *equivalence relation* iff it is reflexive, symmetric, and transitive.

Definition 1.15 (Equivalence class and quotient)**.** If E is an equivalence relation on A, define

$$[a]_E := \{x \in A : xEa\}, \qquad A/E := \{[a]_E : a \in A\}.$$

Method 1.16 (Definition on a quotient)**.** To define $\bar{f} : A/E \to B$ by $\bar{f}([a]_E) = f(a)$, prove

$$aEa' \quad \Longrightarrow \quad f(a) = f(a').$$

This proof establishes that the displayed value is independent of the chosen representative. Without it, the formula does not define a function.

1.5 Finite, countable, and uncountable sets

Definition 1.17 (Finite set)**.** A set A is *finite* iff there exists $n \in \mathbb{N}$ and a bijection from $n = \{0, \dots, n-1\}$ to A.

Definition 1.18 (Countable set)**.** A set A is *countable* iff there exists an injection $A \to \mathbb{N}$. It is *countably infinite* iff there exists a bijection $\mathbb{N} \to A$.

This document includes finite sets among countable sets. The convention is used in the countable-language versions of the completeness and omitting-types theorems.

Definition 1.19 (Cardinal comparison)**.** For sets A and B, write $|A| \leq |B|$ iff there exists an injection $A \to B$, and write $|A| = |B|$ iff there exists a bijection $A \to B$. Write $|A| < |B|$ iff $|A| \leq |B|$ and $|A| \neq |B|$.

Theorem 1.20 (Cantor–Schröder–Bernstein)**.** *If there is an injection $f : A \to B$ and an injection $g : B \to A$, then there is a bijection $h : A \to B$.*

Proof. Let $A_0 = A \setminus g[B]$, and recursively let $A_{n+1} = g[f[A_n]]$. Put $C = \bigcup_{n\in\mathbb{N}} A_n$. Define

$$h(a) = \begin{cases} f(a), & a \in C, \\ g^{-1}(a), & a \notin C. \end{cases}$$

The second case is defined because $A_0 \subseteq C$, so $a \notin C$ implies $a \in g[B]$, and injectivity of g makes the preimage unique. The two ranges are disjoint: if $a \in C$, then $g(f(a)) \in C$; if $a' \notin C$, then $a' = g(b)$ with $g(b) \notin C$, so $b \notin f[C]$. Each branch is injective. For surjectivity, let $b \in B$. If $b \in f[C]$, it is in the first range. Otherwise $g(b) \notin C$: since $g(b) \in g[B]$, the element $g(b)$ is not in $A_0 = A \setminus g[B]$; and if $g(b) \in A_{n+1} = g[f[A_n]]$, injectivity of g would give $b \in f[C]$, contrary to the case assumption. Thus $h(g(b)) = b$. Hence h is bijective. □

Theorem 1.21 (Cantor)**.** *For every set A, there is no surjection $f : A \to \mathcal{P}(A)$. Consequently, $|A| < |\mathcal{P}(A)|$.*

Proof. Suppose $f : A \to \mathcal{P}(A)$. Define

$$D := \{a \in A : a \notin f(a)\}.$$

If f were surjective, there would be $d \in A$ with $f(d) = D$. Then

$$d \in D \iff d \notin f(d) \iff d \notin D,$$

a contradiction. Thus no such surjection exists. The singleton map $a \mapsto \{a\}$ is an injection $A \to \mathcal{P}(A)$, so the cardinal inequality is strict. □

Definition 1.22 (Transitive set)**.** A set A is *transitive* iff

$$\forall x \in A \, \forall y \in x \, (y \in A),$$

equivalently, iff $\bigcup A \subseteq A$.

Definition 1.23 (Ordinal and cardinal)**.** An *ordinal* is a transitive set α such that $\in$ strictly well-orders α. For ordinals α, β, write $\alpha < \beta$ iff $\alpha \in \beta$. A *cardinal* is an ordinal κ for which no smaller ordinal is bijective with κ. Thus a cardinal is already the least ordinal of its bijection type; the phrase "initial ordinal" will not be used as a second term. A cardinal is *infinite* iff it is not bijective with any $n \in \mathbb{N}$.

For an ordinal α, its *successor ordinal* is $\alpha + 1 := \alpha \cup \{\alpha\}$. A *limit ordinal* is a nonzero ordinal that is not a successor ordinal. These terms classify ordinals by their immediate-predecessor behavior; they do not introduce a second order or a second notion of cardinality. Limit ordinals index the elementary chains used later, and successor and limit stages distinguish the two clauses of the Henkin recursion.

Theorem 1.24 (Transfinite recursion)**.** *Let δ be an ordinal. Suppose a functional rule G assigns a set $G(g)$ to every function g whose domain is an ordinal smaller than δ. Then there is a unique function F with domain δ such that*

$$F(\alpha) = G(F \restriction \alpha) \qquad \text{for every } \alpha < \delta.$$

Status: cited theorem.

See [33] for a complete proof. The theorem is the exact metalinguistic recursion principle used below; it is not a new inference rule of any object-language calculus.

1.6 Choice principles used later

Definition 1.25 (Axiom of Choice)**.** The *Axiom of Choice* states that for every set X of nonempty sets there exists a function $c : X \to \bigcup X$ such that $c(A) \in A$ for every $A \in X$.

Theorem 1.26 (Well-ordering theorem)**.** *Assuming the Axiom of Choice, every set can be well-ordered. Status: cited theorem.*

Corollary 1.27 (Cardinal assignment)**.** *Assuming the Axiom of Choice, every set A is bijective with a unique cardinal. Status: cited theorem.*

See [33] for complete proofs. The cardinal bijective with A is denoted by $|A|$, and its order agrees with the injection-based comparison defined above. The cardinal of $\mathbb{N}$ is denoted by $\aleph_0$. For a nonempty finite list of cardinals, $\max(\kappa_1, \ldots, \kappa_n)$ denotes its greatest member in the cardinal order. The well-ordering theorem makes that order total under the stated Axiom of Choice, so this maximum exists. These conventions are the exact cardinal conventions used by the Henkin and Löwenheim–Skolem constructions.

Theorem 1.28 (Finite coding over an infinite cardinal)**.** *Assume the Axiom of Choice, and let κ be an infinite cardinal. Then:*
1. *$|\kappa^n| = \kappa$ for every positive $n \in \mathbb{N}$;*
2. *the set of all finite sequences from κ has cardinality κ;*
3. *if $\alpha < \kappa$ and A_β is finite for every $\beta < \alpha$, then $|\bigcup_{\beta<\alpha} A_\beta| < \kappa$; and*
4. *if $|A_n| \leq \kappa$ for every $n \in \mathbb{N}$, then $|\bigcup_{n\in\mathbb{N}} A_n| \leq \kappa$.*

Status: cited theorem.

Corollary 1.29 (Syntax counting)**.** *Assume the Axiom of Choice and let κ be an infinite cardinal.*
1. *Every set A with $|A| \leq \kappa$ satisfies $|A^{<\omega}| \leq \kappa$, where $A^{<\omega}$ is the set of finite sequences from A.*
2. *If Σ is an alphabet with $|\Sigma| \leq \kappa$, then the finite strings over Σ, the finite tuples of such strings, and the formulas generated by any fixed finitary grammar whose alphabet extends Σ by at most countably many further symbols each form a set of cardinality at most κ.*

Status: complete proof.

Proof. For item 1, fix an injection $f : A \to \kappa$; such an injection is the meaning of $|A| \leq \kappa$. The map $(a_1, \ldots, a_n) \mapsto (f(a_1), \ldots, f(a_n))$ is a length-preserving injection of $A^{<\omega}$ into $\kappa^{<\omega}$, and item 2 of theorem 1.28 gives $|\kappa^{<\omega}| = \kappa$.

For item 2, the finite strings over Σ lie in $\Sigma^{<\omega}$: apply item 1 with $A = \Sigma$. The finite tuples of such strings lie in $(\Sigma^{<\omega})^{<\omega}$: apply item 1 with $A = \Sigma^{<\omega}$, a set of cardinality at most κ by the previous step. Finally, every expression generated by a finitary grammar is a finite string over the grammar's alphabet; the string chapters below record this fact for formulas. Write X for the at most countable set of adjoined symbols hypothesized in the statement—logical symbols, punctuation, and variables. Item 4 of theorem 1.28, applied to the sequence $\Sigma, X, \varnothing, \varnothing, \ldots$, gives $|\Sigma \cup X| \leq \kappa$, so the formula set injects into $(\Sigma \cup X)^{<\omega}$ and item 1 applies. □

See [33, Theorem 3.5 and Chapter 5] for complete proofs of theorem 1.28: Theorem 3.5 gives $\aleph_\alpha \cdot \aleph_\alpha = \aleph_\alpha$, and with the well-ordering theorem cited above every infinite cardinal is an aleph; Chapter 5 gives the cardinal sums and the union bound $|\bigcup_{i\in I} A_i| \leq |I| \cdot \sup_{i\in I} |A_i|$ under the Axiom of Choice. Theorem 1.28 and corollary 1.29 supply exactly the cardinal arithmetic invoked by the Henkin and Löwenheim–Skolem proofs; no claim about arbitrary infinite products is being made.

Definition 1.30 (Ultrafilter Lemma)**.** The *Ultrafilter Lemma* states the following. Let I be a set and let $\mathcal{F} \subseteq \mathcal{P}(I)$ satisfy

$$I \in \mathcal{F}, \quad \varnothing \notin \mathcal{F}, \quad X, Y \in \mathcal{F} \Rightarrow X \cap Y \in \mathcal{F},$$

and

$$X \in \mathcal{F} \text{ and } X \subseteq Y \subseteq I \Rightarrow Y \in \mathcal{F}.$$

Then there is $\mathcal{U} \subseteq \mathcal{P}(I)$ containing $\mathcal{F}$ that satisfies the same conditions and, for every $X \subseteq I$, contains exactly one of X and $I \setminus X$. This is a metatheoretic choice principle about families of subsets, not a formula of a first-order theory under study. It is strictly weaker over ZF than the Axiom of Choice and therefore is not a second name for that axiom. The later filter chapter gives the short technical terms for the two displayed families before applying this principle.

Some model-theoretic constructions use the Ultrafilter Lemma, and some cardinal arguments use the Axiom of Choice. Later theorems state the required principle rather than silently treating either principle as part of pure first-order logic.

Exercise 1.31. Prove that the image of a countable set under a function is countable.

Exercise 1.32. Let E be congruence modulo 5 on $\mathbb{Z}$. Define addition on $\mathbb{Z}/E$ using representatives and prove that it is well-defined.

Exercise 1.33. Find the exact point at which the proof that a countable union of countable sets is countable requires a simultaneous choice of enumerations.

Dependency summary

The later syntax chapters use finite strings, functions, induction, and countability. Semantics uses relations, products, and power sets. Quotient constructions use equivalence classes and well-definedness. Model theory and set theory use cardinal comparison, Cantor's theorem, and stated choice principles.

Chapter 2

Levels, Objects, and Consequence

2.1 Formal logic and a logical system

Definition 2.1 (Formal language)**.** A *formal language* is a set of finite expressions determined from a specified alphabet by a specified grammar. A formal language is an abstract syntactic object. The grammar determines which finite strings count as expressions of each grammatical category.

Definition 2.2 (Calculus)**.** A *calculus* is a specified collection of initial derivations and inference rules that inductively determines a derivability relation. The standing of a calculus is syntactic: it concerns expressions and rules, not their interpretations.

Definition 2.3 (Semantics)**.** A *semantics* for a formal language specifies the complete semantic data permitted for that language and a satisfaction relation between those data and formulas. The data may be a propositional valuation, a first-order structure with a variable assignment, a modal structure with a world, or another tuple declared explicitly by the semantics. The satisfaction relation supplies the truth conditions of the language.

Definition 2.4 (Logical system)**.** A *logical system* is an ordered triple

$$\langle \mathcal{L}, \mathcal{C}, \mathcal{S} \rangle,$$

where $\mathcal{L}$ is a formal language, $\mathcal{C}$ is a calculus for $\mathcal{L}$, and $\mathcal{S}$ is a semantics for $\mathcal{L}$. The term is used only when all three components have been identified.

The ordered triple prevents a common error. A symbol such as $\vdash$ does not name one universal relation: it abbreviates derivability in a particular calculus. Likewise, $\models$ depends on a language and its specified semantic data. When more than one calculus or semantics is active, this document writes subscripts explicitly.

Example 2.5. For classical propositional logic, $\mathcal{L}$ contains propositional formulas, $\mathcal{C}$ may be a natural-deduction calculus, and $\mathcal{S}$ uses two-valued valuations. Replacing natural deduction by a sequent calculus changes $\mathcal{C}$ but not necessarily $\mathcal{L}$ or $\mathcal{S}$. Replacing two-valued valuations by three-valued matrices changes $\mathcal{S}$, and the resulting semantic consequence relation must be defined again.

2.2 Object language and metalanguage

Definition 2.6 (Object language)**.** The *object language* is the formal language currently being studied. Its symbols and formulas are objects of formal-logical investigation.

Definition 2.7 (Metalanguage)**.** The *metalanguage* is the language in which the syntax, semantics, and derivations of the object language are defined and studied. In this document the metalanguage is precise academic English supplemented by ordinary set theory.

The distinction is functional, not typographical. A Greek letter such as φ is normally a metavariable ranging over object-language formulas; it is not itself a symbol of the object language. Quotation marks or corner quotes may be used when an object-language inscription must be mentioned literally. Semantic predicates, including satisfaction, belong to the metalanguage unless the object language has been explicitly extended by a predicate with specified axioms.

Tarski's analysis of truth makes the separation of object language and metalanguage metalogically substantive, especially when a language can encode its own syntax [57]. The distinction will be required in the definition of satisfaction, in arithmetization, and in the undefinability theorem.

Example 2.8 (Use and mention)**.** The display

$$P \wedge Q$$

uses an object-language formula in a derivation. The sentence "$(P \wedge Q)$ contains five symbol occurrences, exactly one of which is a logical-connective occurrence" mentions that formula and makes a metalinguistic statement about its syntax. The mentioned formula carries the parentheses required by the grammar of chapter 5; displayed uses may omit the outermost pair under the typographical abbreviation declared there.

Warning 2.9. Do not write $\varphi \models \psi$ when φ and ψ are being used as object-language inscriptions unless the semantic consequence relation has been lifted to metavariables and the intended universal quantification is clear. A precise metalinguistic statement is:

$$\text{for all formulas } \varphi, \psi \text{ of the language}, \quad \{\varphi\} \models \psi \text{ iff } \cdots .$$

2.3 Syntax, semantics, and proof theory

Definition 2.10 (Syntax)**.** The *syntax* of a formal language consists of its alphabet, grammar, and operations on expressions defined without reference to interpretation.

Definition 2.11 (Proof theory)**.** The *proof theory* of a calculus studies derivations and transformations of derivations by syntactic methods.

Syntax and proof theory are related but distinct. Syntax determines which formulas exist. A calculus determines which derivations exist. Semantics determines when formulas are satisfied. Metatheorems connect these domains.

Definition 2.12 (Semantic consequence)**.** Let Γ be a set of formulas and let φ be a formula in a language with a specified semantics. Then φ is a *semantic consequence* of Γ, written

$$\Gamma \models \varphi,$$

iff, for every complete tuple of semantic data permitted by that semantics, satisfaction of every member of Γ implies satisfaction of φ. The permitted data may be a propositional valuation, a first-order structure with a variable assignment, a modal structure with a world, or another tuple explicitly declared by the active semantics.

Definition 2.13 (Derivability)**.** Let $\mathcal{C}$ be a specified calculus. A formula φ is *derivable* from a set of formulas Γ, written

$$\Gamma \vdash_{\mathcal{C}} \varphi,$$

iff there exists a derivation in $\mathcal{C}$ whose conclusion is φ and whose undischarged assumptions are members of Γ.

Definition 2.14 (Validity)**.** A formula φ is *valid* for a specified semantics iff $\varnothing \models \varphi$. We then write $\models \varphi$.

Definition 2.15 (Theorem of a calculus)**.** A formula φ is a *theorem* of a specified calculus $\mathcal{C}$ iff $\varnothing \vdash_{\mathcal{C}} \varphi$. We then write $\vdash_{\mathcal{C}} \varphi$.

Definition 2.16 (Soundness)**.** A calculus $\mathcal{C}$ is *sound* for a specified semantics iff, for every set of formulas Γ and formula φ,

$$\Gamma \vdash_{\mathcal{C}} \varphi \quad \Longrightarrow \quad \Gamma \models \varphi.$$

Definition 2.17 (Completeness)**.** A calculus $\mathcal{C}$ is *complete* for a specified semantics iff, for every set of formulas Γ and formula φ,

$$\Gamma \models \varphi \quad \Longrightarrow \quad \Gamma \vdash_{\mathcal{C}} \varphi.$$

Warning 2.18 (Direction check)*.* Soundness begins with a derivation. Completeness begins with semantic consequence. Writing the two displayed implications before beginning a proof prevents reversal of their roles.

2.4 Truth, satisfaction, validity, and models

The notions of structure, variable assignment, sentence, and first-order formula used in this section receive their exact inductive definitions in chapter 8 and chapter 9. The definitions below are stated here schematically, to fix the registered senses of the semantic vocabulary; nothing before those chapters depends on more than the registered senses.

Definition 2.19 (Satisfaction)**.** *Satisfaction* is the relation defined by the semantic clauses of the language. In first-order logic it has the form $\mathcal{M}, s \models \varphi$, where $\mathcal{M}$ is a structure, s is a variable assignment, and φ is a formula.

Definition 2.20 (Truth in a structure)**.** A sentence σ is *true in* a structure $\mathcal{M}$ iff $\mathcal{M}, s \models \sigma$ for every variable assignment s. Because a sentence has no free variable occurrence, the choice of s will later be proved irrelevant. We then write $\mathcal{M} \models \sigma$.

Definition 2.21 (Model)**.** Let T be a set of sentences. A structure $\mathcal{M}$ is a *model of* T iff $\mathcal{M} \models \sigma$ for every $\sigma \in T$. The class of models of T is

$$\mathrm{Mod}(T) := \{\mathcal{M} : \mathcal{M} \models T\},$$

where $\mathcal{M} \models T$ abbreviates: $\mathcal{M} \models \sigma$ for every $\sigma \in T$.

Thus a structure is not called a model without an accompanying theory or sentence. The phrase "the model $\mathcal{M}$" is permitted only after the relevant theory has been fixed.

2.5 Arguments and formalization

Definition 2.22 (Argument)**.** An *argument* is an ordered pair $\langle \Gamma, \varphi \rangle$, where Γ is a set of formulas called the premises and φ is a formula called the conclusion.

An argument $\langle \Gamma, \varphi \rangle$ is *semantically valid* iff $\Gamma \models \varphi$; for $\Gamma = \varnothing$ this agrees with validity of the formula φ, and the phrase names no relation beyond semantic consequence. A proof of the conclusion from the premises is a derivation witnessing $\Gamma \vdash_{\mathcal{C}} \varphi$. If every premise and the conclusion are sentences, an invalid argument is refuted by a countermodel. For formulas with free variable occurrences, the refuting data are stated explicitly as a structure and a variable assignment.

Definition 2.23 (Countermodel)**.** Let every member of $\Gamma \cup \{\varphi\}$ be a sentence. A *countermodel* to $\langle \Gamma, \varphi \rangle$ is a model of Γ that does not satisfy φ. The restriction to sentences preserves the fixed standing of a model as a structure satisfying a specified set of sentences.

Method 2.24 (Formalizing an argument)**.** Perform the following steps in order.

1. State the intended domain and the conditions under which an expression counts as a premise or conclusion.
2. Declare every nonlogical symbol, including its arity and intended interpretation.
3. Resolve pronouns, tense, modality, plurality, restricted quantification, and uniqueness before translating.
4. Translate each premise separately. Preserve quantifier scope with parentheses.
5. Translate the conclusion separately.
6. Test the translation against intended positive and negative cases.
7. Only then determine derivability. For a sentence argument, search for a countermodel when semantic consequence is in doubt; for formulas with free variable occurrences, search for an explicit structure and variable assignment that refute semantic consequence.

Example 2.25 (Quantifier order)**.** Over a domain of people, let $A(x, y)$ mean that x admires y. The sentence

$$\forall x \, \exists y \, A(x, y)$$

allows the admired person to depend on x. The sentence

$$\exists y \, \forall x \, A(x, y)$$

requires one person admired by everyone. Neither sentence may be substituted for the other without changing the claim.

Exercise 2.26. Classify each expression as object-language or metalanguage after fixing the object language to be classical propositional logic: P, φ, $P \wedge Q$, "is valid," $\models$, and the left parenthesis occurring in $P \wedge Q$.

Exercise 2.27. Give a valuation that satisfies both premises and falsifies the conclusion of the argument with premises $P \to Q$ and Q and conclusion P. State explicitly which formulas are true and which formula is false under the valuation.

Chapter 3

Finite Strings, Induction, and Recursion

Formal languages are usually built from finite strings. This chapter states the set-theoretic machinery used throughout the reference.

3.1 Finite sequences and strings

Definition 3.1 (Finite sequence)**.** Let A be a set and let $n \in \mathbb{N}$. A *finite sequence in A of length n* is a function $w : \{0, \dots, n-1\} \to A$. Its length is $\mathrm{len}(w) = n$. The unique sequence of length 0 is denoted by ϵ.

Definition 3.2 (String)**.** Let A be an alphabet. A *string over A* is a finite sequence in A. The set of all strings over A is

$$A^* := \bigcup_{n \in \mathbb{N}} A^n.$$

The proof below requires one injection that records two natural numbers in one natural number. Fix the metalinguistic abbreviation

$$\mathrm{pair}_{\mathbb{N}}(a, b) := \frac{(a+b)(a+b+1)}{2} + b \qquad (a, b \in \mathbb{N}).$$

This map is a bijection $\mathbb{N} \times \mathbb{N} \to \mathbb{N}$. Indeed, put $t_s = s(s+1)/2$. For each $z \in \mathbb{N}$, there is a unique s with $t_s \le z < t_{s+1}$, because the consecutive finite intervals $[t_s, t_{s+1})$ partition $\mathbb{N}$. Set

$$b = z - t_s, \qquad a = s - b.$$

The upper inequality gives $0 \le b \le s$, so $a, b \in \mathbb{N}$, and direct substitution gives $\mathrm{pair}_{\mathbb{N}}(a, b) = z$. Conversely, if $z = \mathrm{pair}_{\mathbb{N}}(a, b)$, then $s = a + b$ is the unique interval index and the displayed inverse recovers b and then a. Thus both surjectivity and injectivity hold. The abbreviation is used only to encode pairs; it does not add a new kind of sequence.

Lemma 3.3 (Countability of finite strings)**.** *If A is countable, then A^* is countable.*

Proof. Fix an injection $e : A \to \mathbb{N}$ and use the numerical pairing bijection just defined. Define injections $e_n : A^n \to \mathbb{N}$ by induction: $e_0(\epsilon) = 0$, and for $w \in A^{n+1}$, let

$$e_{n+1}(w) = \mathrm{pair}_{\mathbb{N}}(e_n(w \restriction n), e(w(n))).$$

Induction on n proves that every e_n is injective. The map sending $w \in A^n$ to $\mathrm{pair}_{\mathbb{N}}(n, e_n(w))$ is therefore an injection $A^* \to \mathbb{N}$. □

Corollary 3.4. *The set of formulas of a language with a countable alphabet is countable.*

Proof. Every formula is a finite string over the alphabet, so the formula set is a subset of the countable set A^*. □

If $u \in A^m$ and $v \in A^n$, their concatenation $uv \in A^{m+n}$ is the sequence defined by

$$(uv)(i) = \begin{cases} u(i), & i < m, \\ v(i-m), & m \le i < m+n. \end{cases}$$

Concatenation is associative and ϵ is its identity.

Lemma 3.5. *For all strings $u, v, w \in A^*$, $(uv)w = u(vw)$, $\epsilon u = u$, and $u\epsilon = u$.*

Proof. Two finite sequences are equal iff they have the same length and the same value at every argument. Write $m = \mathrm{len}(u)$, $n = \mathrm{len}(v)$, and $p = \mathrm{len}(w)$.

For associativity, first compare lengths. By the definition of concatenation, $\mathrm{len}(uv) = m + n$, so $\mathrm{len}((uv)w) = (m+n)+p$; likewise $\mathrm{len}(vw) = n+p$, so $\mathrm{len}(u(vw)) = m+(n+p)$. Addition on $\mathbb{N}$ is associative, so both strings are functions on $\{0, \ldots, m+n+p-1\}$.

Now fix an argument $i < m+n+p$ and distinguish the three exhaustive and pairwise disjoint intervals $i < m$, $m \le i < m+n$, and $m+n \le i < m+n+p$.

If $i < m$, then also $i < m+n$, so $((uv)w)(i) = (uv)(i) = u(i)$; and $(u(vw))(i) = u(i)$ directly, since $i < m$.

If $m \le i < m+n$, then $((uv)w)(i) = (uv)(i) = v(i-m)$, by the second clause for uv applied at i. On the other side, $i \ge m$ gives $(u(vw))(i) = (vw)(i-m)$, and $0 \le i-m < n$ puts $i-m$ in the first interval for vw, so $(vw)(i-m) = v(i-m)$.

If $m+n \le i < m+n+p$, then $i \ge m+n$ gives $((uv)w)(i) = w(i-(m+n))$. On the other side, $i \ge m$ gives $(u(vw))(i) = (vw)(i-m)$, and $i-m \ge n$ puts $i-m$ in the second interval for vw, so $(vw)(i-m) = w((i-m)-n) = w(i-(m+n))$.

In each case the two strings agree at i, so $(uv)w = u(vw)$.

For the identity equations, $\mathrm{len}(\epsilon u) = 0 + m = m = \mathrm{len}(u)$, and every argument $i < m$ satisfies $0 \le i < 0 + m$, so the second clause gives $(\epsilon u)(i) = u(i-0) = u(i)$; hence $\epsilon u = u$. Similarly $\mathrm{len}(u\epsilon) = m + 0 = m$, and every argument $i < m$ falls in the first interval, so $(u\epsilon)(i) = u(i)$; hence $u\epsilon = u$. □

3.2 Inductive definitions

Definition 3.6 (Inductively generated set)**.** Let U be a set. Let $B \subseteq U$ be a set of base elements, and let F be a set of finitary operations on U. The *set inductively generated by B under F* is the intersection of all subsets X of U such that $B \subseteq X$ and X is closed under every operation in F.

This definition is minimal: it includes the base elements and required closures, and nothing else. A grammar for formulas is an inductive definition of this form.

Theorem 3.7 (Structural induction)**.** *Let G be generated from base elements B by finitary constructors F. Let $P(x)$ be a metalinguistic property of members of G. Suppose that:*
1. $P(b)$ holds for every $b \in B$; and

2. for every k-ary constructor $f \in F$ and all $x_1, \dots, x_k \in G$, if $P(x_i)$ holds for every $1 \le i \le k$, then $P(f(x_1, \dots, x_k))$ holds.
Then $P(x)$ holds for every $x \in G$.

Proof. Let $X := \{x \in G : P(x)\}$. Condition 1 gives $B \subseteq X$, and Condition 2 says that X is closed under every constructor in F. By the minimality clause in the definition of G, $G \subseteq X$. Hence $P(x)$ holds for every $x \in G$. □

Warning 3.8. A proof by structural induction must contain one case for every base clause and one case for every constructor clause in the grammar actually in force. Cases from a familiar but different grammar do not establish the result.

3.3 Structural recursion

Theorem 3.9 (Structural recursion)**.** *Assume that every member of an inductively generated set G has a unique construction from the stated constructors. To define a function $h : G \to Y$, it is enough to specify:*
1. a value $h(b) \in Y$ for every base element b; and
2. for every constructor f of arity k, a function $H_f : Y^k \to Y$ such that

$$h(f(x_1, \dots, x_k)) = H_f(h(x_1), \dots, h(x_k)).$$

Then there exists a unique function h satisfying those equations.

Proof. Define G_0 as the set of base elements and G_{m+1} as G_m together with every value of a constructor applied to members of G_m; since G is generated by the constructors from the base elements, $G = \bigcup_{m \in \mathbb{N}} G_m$. Assign to each member x of G the least m with $x \in G_m$, called its *stage*. Define h by recursion on the stage. At stage 0, use the specified value for the base element. If $x = f(x_1, \dots, x_k)$ has positive stage, each x_i has smaller stage, so its value has already been defined; set

$$h(x) = H_f(h(x_1), \dots, h(x_k)).$$

Unique construction makes this value independent of any alternative clause, so the recursion defines a function on all of G. The defining equations hold by construction. If h' satisfies the same equations, induction on the stage gives $h'(b) = h(b)$ for every base element and then $h'(f(x_1, \dots, x_k)) = h(f(x_1, \dots, x_k))$; hence $h' = h$. □

The unique-construction hypothesis matters. Without it, two construction histories for the same expression could impose incompatible values. Prefix notation or fully parenthesized infix notation provides an immediate route to unique readability; conventional precedence rules provide another only after they have been formally fixed.

Example 3.10 (Formula complexity)**.** For a propositional grammar with variables, negation, and conjunction, define the complexity $c : \mathsf{Fm} \to \mathbb{N}$ by

$$c(p) = 0 \text{ for every propositional variable } p, \qquad c(\neg\varphi) = c(\varphi) + 1, \qquad c((\varphi \wedge \psi)) = c(\varphi) + c(\psi) + 1.$$

This is a structural recursion. It is not an induction proof.

3.4 Induction on derivations

A derivation is itself inductively generated: initial derivations form the base cases, and inference rules form the constructors. Consequently, a proof that every derivation has a property must inspect its final inference.

Method 3.11 (Proof by induction on a derivation)**.** To prove that every derivation D has property $P(D)$:

1. state the induction measure or invoke structural induction on the inductive definition of derivations;
2. prove the property for every initial derivation;
3. for each inference rule, assume the property for the immediate subderivations and prove it for the derivation formed by that rule; and
4. verify every side condition, including freshness and discharged assumptions.

Soundness proofs will repeatedly use this method. The local semantic preservation of each rule becomes the constructor case.

3.5 Relations and closures

Definition 3.12 (Reflexive transitive closure)**.** Let $R \subseteq X \times X$. The *reflexive transitive closure* of R, written R^*, is the least reflexive and transitive relation on X that contains R.

Lemma 3.13 (Path characterization of the reflexive transitive closure)**.** *Let $R \subseteq X \times X$ and let $x, y \in X$. Then xR^*y iff there are $n \in \mathbb{N}$ and $x_0, \dots, x_n \in X$ such that $x_0 = x$, $x_n = y$, and $x_i R x_{i+1}$ for every $i < n$. Status: complete proof.*

Proof. Call a finite sequence $x_0, \dots, x_n$ with the displayed three properties a path from x to y along R, and let $S \subseteq X \times X$ be the relation holding between x and y iff such a path exists. The claim is $R^* = S$, and each inclusion is proved separately.

For $R^* \subseteq S$, it suffices to show that S is reflexive and transitive and contains R, since R^* is the least such relation. Reflexivity: for any x, take $n = 0$ and $x_0 = x$; the condition $x_i R x_{i+1}$ for every $i < 0$ holds vacuously. Containment of R: if xRy, take $n = 1$, $x_0 = x$, $x_1 = y$. Transitivity: let $x_0, \dots, x_m$ be a path from x to y and $y_0, \dots, y_n$ a path from y to z. Define $u_0, \dots, u_{m+n}$ by $u_i = x_i$ for $i \le m$ and $u_{m+j} = y_j$ for $j \le n$; the two clauses agree at the one shared index m, since $x_m = y = y_0$. Then $u_0 = x$, $u_{m+n} = z$, and for every $i < m + n$: if $i < m$, then $u_i R u_{i+1}$ by the first path; if $m \le i < m + n$, write $i = m + j$ with $j < n$, and then $u_i = y_j R y_{j+1} = u_{i+1}$ by the second path. So $u_0, \dots, u_{m+n}$ is a path from x to z.

For $S \subseteq R^*$, induction on n proves that every path $x_0, \dots, x_n$ from x to y along R gives xR^*y. Base case $n = 0$: then $y = x_0 = x$, and xR^*x because R^* is reflexive. Inductive step: assume the claim for n, and let $x_0, \dots, x_{n+1}$ be a path from x to y. The initial segment $x_0, \dots, x_n$ is a path from x to x_n, so xR^*x_n by the induction hypothesis. Also $x_n R x_{n+1}$ and $R \subseteq R^*$, so $x_n R^* y$. Transitivity of R^* gives xR^*y. □

Definition 3.14 (Equivalence closure)**.** The *equivalence closure* of a relation $R \subseteq X \times X$ is the least equivalence relation on X containing R.

These closures will be used for reduction sequences, proof normalization, and quotient constructions. They must not be confused: a reflexive transitive closure need not be symmetric.

Exercise 3.15. Give an inductive definition of balanced-parenthesis strings and prove by structural induction that every generated string contains equally many left and right parentheses.

Exercise 3.16. Define the height of a derivation by structural recursion. Then prove that every proper subderivation has strictly smaller height.

Chapter 4

Writing Formal Definitions and Proofs

4.1 The dependency order of a formal development

A formal development should normally introduce material in the following order:
1. ambient sets and metalinguistic conventions;
2. signature and arities;
3. grammar of terms and formulas;
4. free variable occurrences and substitution;
5. structures and variable assignments;
6. term evaluation and satisfaction;
7. calculus and derivations;
8. semantic and proof-theoretic properties; and
9. connections between the calculus and semantics.

An author may reorder material for a stated pedagogical reason, but no definition may depend on an object that has not been defined.

4.2 Definitions

Method 4.1 (Definition audit)**.** For every definition, answer the following questions in the definition or its immediate context.
1. What is the type or standing of the defined item?
2. Over which previously defined objects does it range?
3. Are all quantifiers explicit?
4. Are all functions typed by domain and codomain?
5. Are all arities and binding positions specified?
6. Is the definition total, or is its domain of definition stated?
7. If the definition is recursive, what is the well-founded measure or inductive grammar?
8. If the definition uses a representative of an equivalence class, why is the result independent of that representative?
9. If a new term is used, what previously unavailable distinction requires it?

Example 4.2 (Typed definition)**.** Let X be a set and let $R \subseteq X \times X$. Define the successor set function $R[-] : X \to \mathcal{P}(X)$ by

$$R[x] := \{y \in X : xRy\}.$$

The set X, relation type, function type, argument, and value are all explicit.

Warning 4.3 (Circular definition). A clause of the form "a formula is an expression that is a formula according to the usual rules" does not define a set. A reference to prior literature may identify a convention, but a self-contained formal development must state the grammar it uses.

4.3 Theorem statements

A theorem statement should be readable independently of its proof. It must identify:

- every object quantified over;
- every hypothesis, including finiteness, countability, consistency, and effectiveness assumptions;
- the exact language, calculus, and semantic class;
- the direction of each implication; and
- whether the result concerns formulas, sentences, theories, structures, or derivations.

The words "clearly," "obviously," and "routine" do not discharge a proof obligation. A short proof may cite a previous theorem and verify its hypotheses; a missing proof may be marked as omitted, but it must not be presented as established.

Method 4.4 (Proof plan before proof)**.** Before writing details, record:

1. the target statement with all quantifiers;
2. the governing definition or theorem;
3. the proof method;
4. the induction parameter or minimal counterexample when applicable;
5. the object to construct when the proof is existential; and
6. the invariant maintained at each step.

Delete the plan from a polished proof only if the proof remains easy to audit.

4.4 Proof methods

4.4.1 Direct proof

To prove $A \to B$, assume A and derive B. In a metatheorem, the assumption and conclusion must be stated at the same level. Do not replace a semantic hypothesis by a syntactic one unless an already proved theorem licenses the replacement.

4.4.2 Proof by contrapositive

In classical metalogic, proving $A \to B$ by proving $\neg B \to \neg A$ is legitimate. In intuitionistic object logic, the formula $(\neg B \to \neg A) \to (A \to B)$ is not generally derivable. The ambient logic must therefore be stated when this distinction matters.

4.4.3 Proof by contradiction

To prove A by contradiction in classical logic, assume $\neg A$ and derive $\bot$. In intuitionistic logic this establishes $\neg\neg A$, and an additional principle is required to conclude A.

4.4.4 Induction

State the induction domain, induction predicate, base cases, and induction step. Structural induction follows the grammar; induction on derivations follows the final rule; induction on formula complexity

follows the defined complexity measure. These methods may coincide extensionally in a particular case, but the proof must identify the one used.

4.4.5 Construction

For an existence theorem, define the proposed witness and verify every required property. For a model-existence theorem, this includes a domain, an interpretation for each symbol, and a satisfaction proof. Naming a desired object without constructing it or invoking an existence theorem is not a construction.

4.4.6 Counterexample

To refute a universal statement, provide an object in the quantified domain and verify that the predicate fails. To refute semantic consequence for a sentence argument, provide a countermodel. For formulas with free variable occurrences, provide a structure and variable assignment that satisfy the premises and falsify the conclusion. Either form of semantic refutation is sufficient to refute derivability only after soundness of the stated calculus has been proved.

4.5 Displayed formulas and prose

Displayed formulas are parts of sentences and receive punctuation. Each symbol must be introduced before the display in which it first appears. Long transformations should use an aligned environment with a justification at each nontrivial step:

$$\begin{aligned} \neg(P \to Q) &\equiv \neg(\neg P \vee Q) && \text{equivalence of } P \to Q \text{ with } \neg P \vee Q \\ &\equiv P \wedge \neg Q && \text{De Morgan and double-negation equivalences.} \end{aligned}$$

The symbol $\equiv$ in this example is a metalinguistic equivalence sign; it is not an object-language connective. The object-language biconditional will be written $\leftrightarrow$.

Halmos, Knuth, Larrabee, and Roberts emphasize that rigorous prose must remain grammatical and that notation should reduce, not replace, explanation [26, 35]. The AMS author guidance provides the corresponding publication and LaTeX conventions [1].

4.6 Inference-rule typography

An inference rule has premises above a horizontal line and a conclusion below it. A rule name appears to the right, and side conditions appear explicitly:

$$\frac{\Gamma \vdash \varphi}{\Gamma \vdash \forall x\, \varphi}\ \forall I \quad \text{provided } x \text{ is not free in any member of } \Gamma.$$

The displayed rule is only illustrative here. It does not become part of the active calculus until that calculus is defined in chapter 11.

4.7 Error taxonomy

Error	Diagnostic question	Repair
Level error	Is an object-language formula being confused with a metalinguistic statement?	Mention the formula by quotation, or quantify over it with a metavariable, and state the metalinguistic relation.
Grammar error	Does the expression belong to the inductively defined set of formulas?	Parse it from the grammar or correct it.
Binding error	Which quantifier binds each variable occurrence?	Rename bound variables and restore parentheses.
Capture error	Does substitution change a free occurrence into a bound occurrence?	Rename the conflicting bound variable before substitution.
Semantic error	Is truth being asserted without a structure or required assignment?	Supply the semantic parameters.
Rule error	Is the cited inference rule part of the active calculus, and are its side conditions met?	State the rule instance and verify every side condition.
Scope error	Does a connective or quantifier govern the intended subformula?	Insert parentheses or rewrite with explicit restricted quantification.
Metatheorem error	Have soundness, completeness, consistency, or decidability been used in the wrong direction?	Write the theorem as a quantified implication and match its hypotheses.
Strength error	Does the proof establish a weaker statement than the theorem states?	Strengthen the proof or weaken and relabel the theorem statement.

Exercise 4.5. Rewrite the following statement so that its quantifiers, calculus, and semantic class are explicit: “If it is true, it is provable.” Give one corrected statement that is a completeness theorem and one corrected statement that is false.

Exercise 4.6. Audit the definition “A theory is complete when it proves everything that is true.” Identify at least three undefined or ambiguous components and replace the sentence with the definition of deductive completeness used in chapter 10; then state the separate semantic property and the hypothesis under which the two properties are equivalent.

Part II

Classical Propositional Logic

Classical propositional logic is the first complete instance of the general pattern fixed in chapter 2: an inductively generated language, a two-valued semantics, and calculi connected to that semantics by soundness and completeness. The treatment follows the standard separation of syntax, semantics, and derivation in [18, 47].

Chapter 5

Propositional Syntax and Semantics

5.1 Alphabet and grammar

Definition 5.1 (Propositional alphabet)**.** The *propositional alphabet* consists of:
1. propositional variables $\mathsf{Prop} := \{p_n : n \in \mathbb{N}\}$;
2. the nullary connective $\bot$;
3. the unary connective $\neg$;
4. the binary connectives $\wedge, \vee, \to$; and
5. the punctuation symbols (and).

All listed symbols are distinct. A propositional variable is an object-language symbol. The letters φ, ψ, χ used below are metavariables ranging over formulas. For readable examples, P, Q, R, S denote specified members of Prop; the indexed names p_n are used when the identity of a variable matters.

Definition 5.2 (Propositional formula)**.** The set $\mathsf{Fm_{CPL}}$ of *propositional formulas* is the least set of strings satisfying the following formation clauses:
1. $p_n \in \mathsf{Fm_{CPL}}$ for every $n \in \mathbb{N}$;
2. $\bot \in \mathsf{Fm_{CPL}}$;
3. if $\varphi \in \mathsf{Fm_{CPL}}$, then $\neg\varphi \in \mathsf{Fm_{CPL}}$; and
4. if $\varphi, \psi \in \mathsf{Fm_{CPL}}$ and $\circ \in \{\wedge, \vee, \to\}$, then $(\varphi \circ \psi) \in \mathsf{Fm_{CPL}}$.

No other string is a propositional formula.

The displayed grammar is

$$\varphi ::= p_n \mid \bot \mid \neg\varphi \mid (\varphi \wedge \varphi) \mid (\varphi \vee \varphi) \mid (\varphi \to \varphi).$$

Parentheses required by the grammar are part of the object language. In prose and displays, outermost parentheses may be omitted, and the precedence order is $\neg$, $\wedge$, $\vee$, $\to$. A chain of repeated $\wedge$, repeated $\vee$, or repeated $\to$ associates to the right. These are typographical abbreviations, not additional formation clauses.

Definition 5.3 (Defined connectives)**.** The symbols $\top$ and $\leftrightarrow$ are metalinguistic abbreviations:

$$\top := \neg\bot, \qquad (\varphi \leftrightarrow \psi) := ((\varphi \to \psi) \wedge (\psi \to \varphi)).$$

They do not add constructors to $\mathsf{Fm_{CPL}}$.

For an ordered finite sequence $\varphi_1, \ldots, \varphi_n$, define its finite conjunction and disjunction as right-associated formulas. When $n > 0$, put $C_n = \varphi_n$, $D_n = \varphi_n$, and recursively put

$$C_i = (\varphi_i \wedge C_{i+1}), \qquad D_i = (\varphi_i \vee D_{i+1}) \quad (1 \leq i < n);$$

then

$$\bigwedge_{i=1}^{n} \varphi_i := C_1, \qquad \bigvee_{i=1}^{n} \varphi_i := D_1.$$

For $n = 0$, define the finite conjunction as $\top$ and the finite disjunction as $\bot$. An indexed finite conjunction or disjunction is used only after its index order has been stated. This convention applies to both propositional and first-order formulas and gives every later finite connective display one formula-string value.

Lemma 5.4 (Parenthesis counting)**.** *Every member of* $\mathsf{Fm}_{\mathsf{CPL}}$ *contains equally many left and right parenthesis occurrences. Moreover, for every formula and every binary-connective occurrence in it, the number of left parenthesis occurrences strictly preceding that occurrence exceeds the number of right parenthesis occurrences strictly preceding it.*

Proof. Both claims proceed by structural induction on the formula. A propositional variable and $\bot$ contain no parentheses and no binary-connective occurrences. The negation clause prefixes one token and adds no parentheses, so both claims pass through it unchanged. For $(\varphi \circ \psi)$, balance holds because the clause adds one left and one right parenthesis to balanced parts. A binary-connective occurrence in $(\varphi \circ \psi)$ is the displayed occurrence of $\circ$, an occurrence in φ, or an occurrence in ψ. The tokens preceding the displayed $\circ$ are the initial left parenthesis followed by the balanced string φ, so the excess of left over right parentheses there is one. For an occurrence in φ, the initial left parenthesis raises the positive excess supplied by the induction hypothesis by one. For an occurrence in ψ, the preceding tokens are the initial left parenthesis, the balanced string φ, the connective, and a prefix of ψ, so the excess is one more than the positive excess supplied by the induction hypothesis within ψ. □

Theorem 5.5 (Unique readability)**.** *Every member of* $\mathsf{Fm}_{\mathsf{CPL}}$ *satisfies exactly one formation clause. In the negation case its immediate subformula is unique. In a binary case its main connective and its two immediate subformulas are unique.*

Proof. A propositional variable begins with a variable token, $\bot$ is a distinct one-token string, a negation begins with $\neg$, and every binary formula begins with a left parenthesis. Hence the four kinds of formation clause have disjoint ranges.

For a binary formula $(\varphi \circ \psi)$, scan from immediately after its first left parenthesis while maintaining the number of left minus right parenthesis occurrences already scanned. By the balance clause of lemma 5.4, this number is zero at the displayed occurrence of $\circ$, because the tokens scanned before it form exactly the balanced string φ. By the excess clause of lemma 5.4, applied to φ and to ψ, every binary connective inside either immediate subformula occurs while the number is positive. Hence the count-zero binary-connective position is unique. The first formula ends exactly before that unique position, and the second begins immediately after it and ends immediately before the final right parenthesis. Thus the split, the main connective, and the two immediate subformulas are unique. Removing an initial $\neg$ uniquely determines the immediate subformula in the unary case. Therefore every formula has one and only one outermost construction. □

Definition 5.6 (Complexity and subformula)**.** The *complexity* $c(\varphi) \in \mathbb{N}$ is defined by

$$\begin{aligned} c(p_n) &= 0, & c(\bot) &= 0, \\ c(\neg\varphi) &= c(\varphi) + 1, & c((\varphi \circ \psi)) &= c(\varphi) + c(\psi) + 1. \end{aligned}$$

The set $\mathrm{Sub}(\varphi)$ of *subformulas* is defined by

$$\begin{aligned} \mathrm{Sub}(p_n) &= \{p_n\}, & \mathrm{Sub}(\bot) &= \{\bot\}, \\ \mathrm{Sub}(\neg\varphi) &= \{\neg\varphi\} \cup \mathrm{Sub}(\varphi), \\ \mathrm{Sub}(\varphi \circ \psi) &= \{\varphi \circ \psi\} \cup \mathrm{Sub}(\varphi) \cup \mathrm{Sub}(\psi). \end{aligned}$$

Here and below $\circ \in \{\wedge, \vee, \to\}$.

5.2 Valuations and recursive truth clauses

Definition 5.7 (Truth values and valuation)**.** Let $\mathbb{B} := \{\mathbf{F}, \mathbf{T}\}$. A *valuation* is a function $v : \mathsf{Prop} \to \mathbb{B}$. Its unique extension $\widehat{v} : \mathsf{Fm_{CPL}} \to \mathbb{B}$ is defined by structural recursion:

$$\begin{aligned} \widehat{v}(p_n) &= v(p_n), \\ \widehat{v}(\bot) &= \mathbf{F}, \\ \widehat{v}(\neg\varphi) &= \mathbf{T} \quad \text{iff} \quad \widehat{v}(\varphi) = \mathbf{F}, \\ \widehat{v}(\varphi \wedge \psi) &= \mathbf{T} \quad \text{iff} \quad \widehat{v}(\varphi) = \mathbf{T} \text{ and } \widehat{v}(\psi) = \mathbf{T}, \\ \widehat{v}(\varphi \vee \psi) &= \mathbf{T} \quad \text{iff} \quad \widehat{v}(\varphi) = \mathbf{T} \text{ or } \widehat{v}(\psi) = \mathbf{T}, \\ \widehat{v}(\varphi \to \psi) &= \mathbf{F} \quad \text{iff} \quad \widehat{v}(\varphi) = \mathbf{T} \text{ and } \widehat{v}(\psi) = \mathbf{F}. \end{aligned}$$

We write $v \models \varphi$ iff $\widehat{v}(\varphi) = \mathbf{T}$.

Lemma 5.8 (Coincidence for propositional formulas)**.** *If valuations v and w agree on every propositional variable occurring in φ, then $\widehat{v}(\varphi) = \widehat{w}(\varphi)$.*

Proof. Proceed by structural induction on φ. The variable case is the hypothesis, and the $\bot$ case is immediate. Each connective case follows by applying the induction hypotheses to the immediate subformulas and then applying the corresponding recursive truth clause. □

Definition 5.9 (Semantic notions)**.** Let $\Gamma \subseteq \mathsf{Fm_{CPL}}$ and $\varphi, \psi \in \mathsf{Fm_{CPL}}$.

1. A valuation v *satisfies* Γ, written $v \models \Gamma$, iff $v \models \gamma$ for every $\gamma \in \Gamma$.
2. The set Γ is *satisfiable* iff some valuation satisfies it.
3. The formula φ is *valid*, written $\models \varphi$, iff every valuation satisfies it.
4. The formula φ is a *semantic consequence* of Γ, written $\Gamma \models \varphi$, iff every valuation satisfying Γ satisfies φ.
5. The formulas φ and ψ are *logically equivalent*, written $\varphi \equiv \psi$, iff every valuation gives them the same truth value.

The empty set is satisfied by every valuation. Consequently, $\varnothing \models \varphi$ iff $\models \varphi$. A countervaluation to $\Gamma \models \varphi$ is a valuation satisfying Γ and falsifying φ.

Example 5.10 (Truth table)**.** The formula $(P \to Q) \leftrightarrow (\neg P \vee Q)$ is valid:

P	Q	$P \to Q$	$\neg P \vee Q$
T	**T**	**T**	**T**
T	**F**	**F**	**F**
F	**T**	**T**	**T**
F	**F**	**T**	**T**

The table checks all valuations of the two variables occurring in the formula; lemma 5.8 shows that values assigned to other variables are irrelevant.

5.3 Translation from controlled prose

A translation requires a key assigning one propositional variable to each atomic statement. The key is part of the translation; a formula without its key does not preserve the content of the prose.

Example 5.11 (Necessary and sufficient conditions)**.** Use the key

$$P : \text{the archive is readable}, \qquad Q : \text{the checksum is valid}.$$

Then:

Controlled prose	Formula
If the archive is readable, then the checksum is valid.	$P \to Q$
The archive is readable only if the checksum is valid.	$P \to Q$
The archive is readable if the checksum is valid.	$Q \to P$
The archive is readable iff the checksum is valid.	$P \leftrightarrow Q$
The archive is readable unless the checksum is invalid.	$\neg\neg Q \to P$

The word "only" marks a necessary condition, and it reverses the effect of a bare "if": the "if" of the third row introduces the antecedent, while the "only if" of the second row introduces the consequent, so the main clause of the second row supplies the antecedent and the first two rows share one formula. For the final row, "P unless R" is $\neg R \to P$. Here R is "the checksum is invalid," namely $\neg Q$, so the translation is $\neg\neg Q \to P$, and that formula is what the table records. Simplifying it to $Q \to P$ is a further step, justified by an equivalence of classical propositional logic and not part of the translation.

Example 5.12 (Exclusive disjunction)**.** "Exactly one of P and Q" is translated by

$$(P \vee Q) \wedge \neg(P \wedge Q).$$

The formula $P \vee Q$ alone is inclusive: it is true when both disjuncts are true.

5.4 Normal forms

Definition 5.13 (Propositional literal, clause, and formula interpretation)**.** A *propositional literal* is a propositional variable or the negation of a propositional variable. A *clause* is a finite set of propositional literals. Thus clause equality is set equality: order and repeated occurrences are not part of a clause.

To map this set object back to the formula language, put

$$\operatorname{code}(p_n) = 2n, \qquad \operatorname{code}(\neg p_n) = 2n + 1.$$

For a clause C, let $k = |C|$; when $k > 0$, list its members as $L_1, \dots, L_k$ in strictly increasing code order and define, by downward recursion,

$$D_k := L_k, \qquad D_i := (L_i \vee D_{i+1}) \quad (1 \le i < k).$$

Then define

$$\mathrm{form}_{\mathsf{CPL}}(C) := \begin{cases} \bot, & C = \varnothing, \\ D_1, & C \neq \varnothing. \end{cases}$$

This is an exact metalinguistic function from clauses to $\mathsf{Fm}_{\mathsf{CPL}}$, not a new formula constructor. A valuation v *satisfies* C iff $v \models \mathrm{form}_{\mathsf{CPL}}(C)$. In particular, the empty clause is false under every valuation.

Definition 5.14 (Negation, conjunctive, and disjunctive normal form)**.** A formula is in *negation normal form* iff it is built from literals and $\bot, \top$ using only $\wedge$ and $\vee$. It is in *conjunctive normal form* iff it is $\top$, the empty conjunction, or there are clauses $C_1, \ldots, C_m$, with $m > 0$, such that it is N_1, where the formula sequence $N_m, \ldots, N_1$ is defined by

$$N_m := \mathrm{form}_{\mathsf{CPL}}(C_m), \qquad N_i := \big(\mathrm{form}_{\mathsf{CPL}}(C_i) \wedge N_{i+1}\big) \quad (1 \leq i < m).$$

Thus a conjunctive-normal-form formula is a right-associated finite conjunction of the formula interpretations of clauses. In every finite conjunction and finite disjunction used in this chapter, the empty conjunction is $\top$ and the empty disjunction is $\bot$. A formula is in *disjunctive normal form* iff it is a finite disjunction of finite conjunctions of literals under these conventions.

Lemma 5.15 (Satisfaction of clauses and of clause conjunctions)**.** *Let v be a valuation.*

1. *For every clause C: $v \models \mathrm{form}_{\mathsf{CPL}}(C)$ iff $v \models L$ for at least one literal $L \in C$.*
2. *For all clauses $C_1, \ldots, C_m$ with $m > 0$, and the formulas $N_m, \ldots, N_1$ built from them as in the definition of conjunctive normal form: $v \models N_1$ iff $v \models \mathrm{form}_{\mathsf{CPL}}(C_j)$ for every j with $1 \leq j \leq m$.*

Proof. For 1, if $C = \varnothing$, then $\mathrm{form}_{\mathsf{CPL}}(C) = \bot$, which no valuation satisfies, and C has no member, so both sides fail. Otherwise let $k = |C| > 0$, list C in strictly increasing code order as $L_1, \ldots, L_k$, and let $D_k, \ldots, D_1$ be the formulas of the clause-interpretation recursion. By downward induction along that recursion, $v \models D_i$ iff $v \models L_j$ for some j with $i \leq j \leq k$: the base case $D_k = L_k$ is immediate, and the case $D_i = (L_i \vee D_{i+1})$ follows from the recursive truth clause for $\vee$ together with the induction hypothesis. Taking $i = 1$ and $\mathrm{form}_{\mathsf{CPL}}(C) = D_1$ proves 1.

For 2, by downward induction along the recursion defining $N_m, \ldots, N_1$, $v \models N_i$ iff $v \models \mathrm{form}_{\mathsf{CPL}}(C_j)$ for every j with $i \leq j \leq m$: the base case $N_m = \mathrm{form}_{\mathsf{CPL}}(C_m)$ is immediate, and the case $N_i = (\mathrm{form}_{\mathsf{CPL}}(C_i) \wedge N_{i+1})$ follows from the recursive truth clause for $\wedge$ together with the induction hypothesis. Taking $i = 1$ proves 2. □

Theorem 5.16 (Negation normal form)**.** *Every propositional formula is logically equivalent to a formula in negation normal form.*

Proof. First eliminate every occurrence of $\to$ using $\varphi \to \psi \equiv \neg\varphi \vee \psi$. Proceed by structural induction through the resulting formula. Double negations are removed, and a negation immediately above a binary connective is moved inward by

$$\neg(\varphi \wedge \psi) \equiv \neg\varphi \vee \neg\psi, \qquad \neg(\varphi \vee \psi) \equiv \neg\varphi \wedge \neg\psi.$$

The base cases are variables, their negations, $\bot$, and $\top$. Each displayed replacement decreases the number of connectives lying below a negation, so the process terminates. Every replacement is checked by the recursive truth clauses and therefore preserves logical equivalence. □

Theorem 5.17 (Conjunctive and disjunctive normal forms)**.** *Every propositional formula is logically equivalent to a formula in disjunctive normal form and to a formula in conjunctive normal form.*

Proof. Let $p_1, \dots, p_n$ be the variables occurring in φ, listed in increasing index order. In this proof only, a *valuation on* a finite variable set is a function from that set to $\mathbb{B}$; the recursive truth clauses apply verbatim to formulas whose variables lie in the set, and lemma 5.8 identifies satisfaction under such a function with satisfaction under any total valuation extending it. For each valuation v on these variables define

$$\delta_v := \bigwedge_{i=1}^{n} \ell_i^v, \qquad \ell_i^v := \begin{cases} p_i, & v(p_i) = \mathbf{T}, \\ \neg p_i, & v(p_i) = \mathbf{F}. \end{cases}$$

A valuation on the empty variable set is the empty function, and when $n = 0$ the displayed conjunction is $\top$ by the preceding convention. A valuation w satisfies δ_v iff $w(p_i) = v(p_i)$ for every i. Let V_φ be the finite set of valuations on $p_1, \dots, p_n$ that satisfy φ. Order these valuations lexicographically by $(v(p_1), \dots, v(p_n))$, with $\mathbf{F} < \mathbf{T}$, enumerate the resulting order as $v_1, \dots, v_m$, and put

$$D_\varphi := \bigvee_{j=1}^{m} \delta_{v_j}.$$

If $V_\varphi = \varnothing$, then $m = 0$ and this disjunction is $\bot$. By the coincidence lemma, an arbitrary valuation satisfies D_φ iff it satisfies φ. Hence D_φ is the required disjunctive normal form.

For the conjunctive normal form, construct dually. For each valuation v on $p_1, \dots, p_n$ put

$$\bar{\ell}_i^v := \begin{cases} \neg p_i, & v(p_i) = \mathbf{T}, \\ p_i, & v(p_i) = \mathbf{F}, \end{cases} \qquad K_v := \{\bar{\ell}_1^v, \dots, \bar{\ell}_n^v\}.$$

The coding in the clause-interpretation definition gives the variable of index i the code $2i$ and its negation the code $2i + 1$, both strictly increasing in i. The enumeration $p_1, \dots, p_n$ is in increasing index order—the indices need not form an initial segment—so any literal on p_j has smaller code than any literal on p_k for $j < k$: if their indices are $i_j < i_k$, then $2i_j + 1 < 2i_k$. The listing of K_v in strictly increasing code order is therefore exactly $\bar{\ell}_1^v, \dots, \bar{\ell}_n^v$, and $\gamma_v := \mathrm{form}_{\mathsf{CPL}}(K_v)$ requires no re-ordering. A valuation w falsifies $\bar{\ell}_i^v$ iff $w(p_i) = v(p_i)$, so lemma 5.15(1) gives: w falsifies γ_v iff $w(p_i) = v(p_i)$ for every $i \le n$.

Let W_φ be the set of valuations on $p_1, \dots, p_n$ that falsify φ, enumerated in the lexicographic order fixed above as $v_1, \dots, v_m$. Put $C_\varphi := \top$ if $m = 0$; otherwise put $C_\varphi := N_1$, where $N_m, \dots, N_1$ are built from the clauses $C_j := K_{v_j}$ by the recursion in the definition of conjunctive normal form. In every case C_φ satisfies that definition verbatim, the degenerate cases included: for valid φ, $m = 0$ and $C_\varphi = \top$; for variable-free true φ, $n = m = 0$ and again $C_\varphi = \top$; for variable-free false φ, $n = 0$ and $m = 1$ with v_1 the empty valuation, so $K_{v_1} = \varnothing$ and $C_\varphi = \mathrm{form}_{\mathsf{CPL}}(\varnothing) = \bot$, the conjunctive normal form whose single clause is empty.

Let w be an arbitrary valuation and w_0 its restriction to $\{p_1, \dots, p_n\}$. Suppose $m > 0$. By lemma 5.15(2) and (1), w satisfies C_φ iff for every $j \le m$ it satisfies some literal of K_{v_j}, iff $w_0 \ne v_j$ for every $j \le m$, iff $w_0 \notin W_\varphi$, iff w_0 satisfies φ, iff, by the coincidence lemma, w satisfies φ. If $m = 0$, every valuation on the variable set satisfies φ, so by the coincidence lemma every valuation satisfies both φ and $C_\varphi = \top$. Hence C_φ is the required conjunctive normal form, obtained from the definitional recursions, lemma 5.15, and the coincidence lemma alone. □

An n-ary *truth-functional connective* is a connective symbol interpreted by a fixed function $\mathbb{B}^n \to \mathbb{B}$. For a set C of truth-functional connectives, the *formulas over* C are generated by the propositional variables together with one formation clause per member of C, and the recursive truth clause of each member applies its interpreting function to the values of the immediate subformulas. For a formula φ whose variables are among $p_1, \dots, p_n$, the *truth function of* φ *in those variables* is

the function $f_\varphi : \mathbb{B}^n \to \mathbb{B}$ with $f_\varphi(a_1, \ldots, a_n) := \widehat{v}(\varphi)$ for any valuation v with $v(p_i) = a_i$ for each i; this is well defined by lemma 5.8.

Definition 5.18 (Functional completeness)**.** A set C of truth-functional connectives is *functionally complete* iff, for every $n \in \mathbb{N}$ and every function $f : \mathbb{B}^n \to \mathbb{B}$, some formula built from variables $p_1, \ldots, p_n$ and connectives in C has truth function f in those variables.

Theorem 5.19 (A functionally complete basis)**.** *The set* $\{\neg, \wedge, \bot\}$ *is functionally complete.*

Proof. Given $f : \mathbb{B}^n \to \mathbb{B}$, form the disjunctive normal form whose conjunctions of literals correspond exactly to the inputs $a \in \mathbb{B}^n$ for which $f(a) = \mathbf{T}$, as in theorem 5.17. Replace each occurrence of $\alpha \vee \beta$ by $\neg(\neg\alpha \wedge \neg\beta)$. The empty disjunction is $\bot$. The resulting formula uses only $\neg, \wedge, \bot$ and computes f. □

Chapter 6

Natural Deduction and Sequents

6.1 Classical natural deduction

Definition 6.1 (Natural-deduction derivation)**.** A *natural-deduction derivation* is a finite rooted tree generated by the rules displayed below. Leaves may be marked assumptions. An application of an introduction or elimination rule may discharge the marked assumptions indicated in brackets. An assumption is *open* iff no rule on the path from that assumption to the root discharges it.

Discharge labels are metalinguistic markers drawn from $\mathbb{N}$. A consistent injective renaming of the labels on assumptions and of the matching labels on rules preserves a derivation and which assumptions it discharges. Since a derivation is finite, copies of derivations can always be given pairwise disjoint label sets fresh from every label in a fixed receiving derivation.

An application of $\to I$, $\neg I$, or RAA carries a label i and discharges exactly the open occurrences marked i of the bracketed assumption above its premise; an application of $\vee E$ carries two labels i, j, as displayed, and discharges exactly the open occurrences of the first disjunct marked i above its second premise and the open occurrences of the second disjunct marked j above its third premise. Every label carried by such an application discharges at least one occurrence. Marking any set of open occurrences of one formula with a common label fresh from every label in the derivation preserves the derivation and the occurrences discharged by each of its rule applications, since no rule application carries the fresh label; this merging relabeling is distinct from the injective renamings above, and is the operation used when a newly added root inference is to discharge several open occurrences at once. There is no repetition step: where an informal presentation repeats a formula, the tree reuses the node already present.

The active calculus $\mathsf{ND}_{\mathsf{CPL}}$ has the following rules.

$$\frac{\varphi \quad \psi}{\varphi \wedge \psi}\,\wedge I \qquad \frac{\varphi \wedge \psi}{\varphi}\,\wedge E_1 \qquad \frac{\varphi \wedge \psi}{\psi}\,\wedge E_2$$

$$\frac{\varphi}{\varphi \vee \psi}\,\vee I_1 \qquad \frac{\psi}{\varphi \vee \psi}\,\vee I_2 \qquad \frac{\varphi \quad \varphi \to \psi}{\psi}\,\to E$$

$$\frac{\varphi \vee \psi \qquad \begin{array}{c}[\varphi]^i \\ \vdots \\ \chi\end{array} \qquad \begin{array}{c}[\psi]^j \\ \vdots \\ \chi\end{array}}{\chi}\,\vee E$$

discharging all assumptions marked i and j.

$$\frac{\begin{array}{c}[\varphi]^i\\ \vdots\\ \psi\end{array}}{\varphi \to \psi}\ {\to}I \qquad \frac{\begin{array}{c}[\varphi]^i\\ \vdots\\ \bot\end{array}}{\neg\varphi}\ {\neg}I \qquad \frac{\varphi \quad \neg\varphi}{\bot}\ {\neg}E$$

$$\frac{\bot}{\varphi}\ \bot E \qquad \frac{\begin{array}{c}[\neg\varphi]^i\\ \vdots\\ \bot\end{array}}{\varphi}\ \mathsf{RAA}$$

The rules $\to I$, $\neg I$, and RAA discharge the indicated assumptions. The last rule is the explicitly classical rule.

Definition 6.2 (Propositional derivability)**.** For $\Gamma \subseteq \mathsf{Fm}_{\mathsf{CPL}}$, write $\Gamma \vdash_{\mathsf{ND}_{\mathsf{CPL}}} \varphi$ iff there is a $\mathsf{ND}_{\mathsf{CPL}}$-derivation of φ whose open assumptions all belong to Γ. A *proof* is such a derivation together with its stated set of permitted open assumptions.

Example 6.3 (Commutativity of conjunction)**.** The following derivation proves $\vdash_{\mathsf{ND}_{\mathsf{CPL}}} (P \wedge Q) \to (Q \wedge P)$:

1.	$[P \wedge Q]^1$	assumption
2.	Q	$\wedge E_2, 1$
3.	P	$\wedge E_1, 1$
4.	$Q \wedge P$	$\wedge I, 2, 3$
5.	$(P \wedge Q) \to (Q \wedge P)$	$\to I$, 1–4.

The superscript marks the assumption discharged at line 5.

Example 6.4 (Excluded middle)**.** The formula $P \vee \neg P$ has a classical proof:

1.	$[\neg(P \vee \neg P)]^1$	assumption
2.	$\quad [P]^2$	assumption
3.	$\quad P \vee \neg P$	$\vee I_1, 2$
4.	$\quad \bot$	$\neg E, 1, 3$
5.	$\neg P$	$\neg I$, 2–4
6.	$P \vee \neg P$	$\vee I_2, 5$
7.	$\bot$	$\neg E, 1, 6$
8.	$P \vee \neg P$	RAA, 1–7.

The proof uses RAA. The calculus obtained by omitting RAA is intuitionistic natural deduction, made exact in chapter 31; no derivation in that calculus concludes $P \vee \neg P$ from no open assumptions.

Lemma 6.5 (Substitution for open assumptions)**.** *If D derives ψ with an open assumption φ, and E derives φ, then replacing each designated open occurrence of φ in D by a copy of E, after consistently renaming every label in every copy to be globally fresh from D and from all other copies, produces a derivation of ψ. Its open assumptions are those of D other than the replaced occurrences, together with the correspondingly renamed open assumptions of the inserted copies of E.*

Proof. First reserve for each designated occurrence a set of labels disjoint from the labels of D and from the sets reserved for the other occurrences, and consistently rename its copy of E into that set. Strengthen the induction invariant by requiring every inserted copy to use its reserved globally fresh labels. Induct on the height of D. If D is the designated assumption, use its renamed copy of

E. If it is another assumption, leave it unchanged. For an inference, apply the induction hypothesis to each immediate subderivation and then reapply the same rule with the original discharge labels of D. Consistent renaming preserves every discharge internal to a copy of E, while global freshness makes every label of that copy different from every label on a rule of D. Hence no rule above a graft point can accidentally discharge an open assumption of the inserted copy, and the stated open-assumption accounting follows. □

6.2 Classical sequent calculus

Definition 6.6 (Sequent)**.** A *finite multiset of formulas* is a function $\Gamma : \mathsf{Fm}_{\mathsf{CPL}} \to \mathbb{N}$ whose support

$$\operatorname{supp}(\Gamma) := \{\varphi \in \mathsf{Fm}_{\mathsf{CPL}} : \Gamma(\varphi) > 0\}$$

is finite. The empty multiset is the constant-zero function. If Γ and Π are finite multisets, then the comma Γ, Π denotes their pointwise sum; a displayed formula φ in a comma list denotes the multiset having value 1 at φ and 0 elsewhere. Thus Γ, φ, φ increases the multiplicity of φ by two. In the first-order sequent chapter the same definition is used with $\mathsf{Fm}(\mathcal{L})$ in place of $\mathsf{Fm}_{\mathsf{CPL}}$.

A *sequent* is an expression $\Gamma \Rightarrow \Delta$, where Γ and Δ are finite multisets of formulas. It is satisfied by a valuation iff either some formula in $\operatorname{supp}(\Gamma)$ is false or some formula in $\operatorname{supp}(\Delta)$ is true. With respect to satisfaction, an empty antecedent therefore behaves as $\top$ and an empty succedent as $\bot$.

When a finite multiset occurs to the left of $\models$ or $\vdash$, this document defines

$$\Gamma \models \varphi \iff \operatorname{supp}(\Gamma) \models \varphi, \qquad \Gamma \vdash_{\mathcal{C}} \varphi \iff \operatorname{supp}(\Gamma) \vdash_{\mathcal{C}} \varphi.$$

Multiplicity is therefore proof-structural data in a sequent, while semantic consequence and natural-deduction assumptions depend only on the support.

The calculus $\mathsf{LK}_{\mathsf{CPL}}$ has two forms of *initial sequent*: identity sequents $\Gamma, \varphi \Rightarrow \varphi, \Delta$ and $\bot$-sequents $\Gamma, \bot \Rightarrow \Delta$. It also has the following rules. Exchange is implicit because contexts are multisets.

$$\frac{\Gamma \Rightarrow \Delta}{\Gamma, \varphi \Rightarrow \Delta}\, W_L \qquad \frac{\Gamma \Rightarrow \Delta}{\Gamma \Rightarrow \Delta, \varphi}\, W_R$$

$$\frac{\Gamma, \varphi, \varphi \Rightarrow \Delta}{\Gamma, \varphi \Rightarrow \Delta}\, C_L \qquad \frac{\Gamma \Rightarrow \Delta, \varphi, \varphi}{\Gamma \Rightarrow \Delta, \varphi}\, C_R$$

$$\frac{\Gamma \Rightarrow \Delta, \varphi}{\Gamma, \neg\varphi \Rightarrow \Delta}\, \neg L \qquad \frac{\Gamma, \varphi \Rightarrow \Delta}{\Gamma \Rightarrow \Delta, \neg\varphi}\, \neg R$$

$$\frac{\Gamma, \varphi, \psi \Rightarrow \Delta}{\Gamma, \varphi \wedge \psi \Rightarrow \Delta}\, \wedge L \qquad \frac{\Gamma \Rightarrow \Delta, \varphi \qquad \Gamma \Rightarrow \Delta, \psi}{\Gamma \Rightarrow \Delta, \varphi \wedge \psi}\, \wedge R$$

$$\frac{\Gamma, \varphi \Rightarrow \Delta \qquad \Gamma, \psi \Rightarrow \Delta}{\Gamma, \varphi \vee \psi \Rightarrow \Delta}\, \vee L \qquad \frac{\Gamma \Rightarrow \Delta, \varphi, \psi}{\Gamma \Rightarrow \Delta, \varphi \vee \psi}\, \vee R$$

$$\frac{\Gamma \Rightarrow \Delta, \varphi \qquad \Gamma, \psi \Rightarrow \Delta}{\Gamma, \varphi \to \psi \Rightarrow \Delta}\, \to L \qquad \frac{\Gamma, \varphi \Rightarrow \Delta, \psi}{\Gamma \Rightarrow \Delta, \varphi \to \psi}\, \to R.$$

The cut rule is

$$\frac{\Gamma \Rightarrow \Delta, \varphi \qquad \Sigma, \varphi \Rightarrow \Pi}{\Gamma, \Sigma \Rightarrow \Delta, \Pi}\, \textsc{Cut}.$$

A $\mathsf{LK}_{\mathsf{CPL}}$-derivation is a finite tree whose leaves are initial sequents and whose internal nodes are licensed rule instances.

Example 6.7 (A sequent derivation)**.** The sequent $\Rightarrow P \to (P \vee Q)$ has the cut-free derivation

$$\frac{\dfrac{P \Rightarrow P, Q}{P \Rightarrow P \vee Q}\, \vee R}{\Rightarrow P \to (P \vee Q)}\, \to R.$$

The upper sequent is an identity sequent.

Chapter 7

Soundness, Completeness, Compactness, and Resolution

7.1 Soundness

Theorem 7.1 (Soundness of classical propositional natural deduction)**.** *For all* $\Gamma \subseteq \mathsf{Fm}_{\mathsf{CPL}}$ *and* $\varphi \in \mathsf{Fm}_{\mathsf{CPL}}$*,*

$$\Gamma \vdash_{\mathsf{ND}_{\mathsf{CPL}}} \varphi \quad \Longrightarrow \quad \Gamma \models \varphi.$$

Proof. Induct on the construction of the derivation. The induction statement is stronger than the theorem: every valuation satisfying all open assumptions of a subderivation satisfies its conclusion.

An assumption case is immediate. The conjunction introduction and elimination cases, disjunction introduction cases, implication elimination case, negation elimination case, and bottom elimination case follow directly from their recursive truth clauses. For $\vee E$, let v satisfy the open assumptions of the conclusion. It satisfies $\varphi \vee \psi$, so it satisfies φ or ψ. In the first case the induction hypothesis for the left subsidiary derivation gives $v \models \chi$; in the second case the right subsidiary derivation gives the same conclusion.

For $\to I$, let v satisfy the open assumptions. If $v \not\models \varphi$, then $v \models \varphi \to \psi$. If $v \models \varphi$, it satisfies every open assumption of the subsidiary derivation, so the induction hypothesis gives $v \models \psi$. Thus again $v \models \varphi \to \psi$. The $\neg I$ case is analogous: if $v \models \varphi$, the induction hypothesis would give $v \models \bot$, which is impossible; hence $v \models \neg\varphi$.

For RAA, suppose v satisfies the open assumptions and falsifies φ. Then it satisfies $\neg\varphi$, so the induction hypothesis for the subsidiary derivation gives $v \models \bot$, impossible. Therefore $v \models \varphi$. Every rule preserves truth from open assumptions to conclusion, completing the induction. □

Theorem 7.2 (Soundness of the classical propositional sequent calculus)**.** *Every sequent derivable in* $\mathsf{LK}_{\mathsf{CPL}}$ *is satisfied by every valuation.*

Proof. Induct on the sequent derivation, showing that every derivable sequent is satisfied by every valuation in the sense of the sequent-satisfaction definition of section 6.2. Both forms of initial sequent are satisfied by every valuation v. In an identity sequent $\Gamma, \varphi \Rightarrow \varphi, \Delta$, the formula φ belongs to the support of the antecedent and to the support of the succedent, so whichever truth value v gives φ, either some antecedent formula is false under v or some succedent formula is true under v. In a $\bot$-sequent $\Gamma, \bot \Rightarrow \Delta$, the antecedent support contains $\bot$, which is false under v.

For the induction step, suppose toward a contradiction that some valuation v does not satisfy the conclusion of a rule each of whose premises is satisfied by every valuation. By the sequent-satisfaction

definition, v makes every formula in the support of the conclusion's antecedent true and every formula in the support of its succedent false. For weakening, the supports of the premise are included in those of the conclusion, so v does not satisfy the premise. For contraction, premise and conclusion have the same supports, so exactly the same valuations satisfy both. For cut, v makes all formulas in Γ, Σ true and all formulas in Δ, Π false; because v satisfies the left premise, φ is true under v, and because v satisfies the right premise, φ is false under v, which is impossible.

For each logical rule, use the truth clause for the principal connective to produce a premise that v does not satisfy. For example, if v does not satisfy the conclusion of $\wedge R$, then v makes every formula in Γ true, every formula in Δ false, and $\varphi \wedge \psi$ false; hence φ or ψ is false under v, so v does not satisfy the corresponding premise. If v does not satisfy the conclusion of $\to L$, then v makes $\varphi \to \psi$ true. If φ is false under v, then v does not satisfy the left premise; if ψ is true under v, then v does not satisfy the right premise. The cases for $\neg L, \neg R, \wedge L, \vee L, \vee R, \to R$ follow by the same direct application of their displayed truth clauses. Thus every rule whose premises are satisfied by every valuation has a conclusion satisfied by every valuation, completing the induction. □

7.2 Maximal consistent sets

Definition 7.3 (Consistency)**.** A set $\Gamma \subseteq \mathsf{Fm}_{\mathsf{CPL}}$ is *consistent* iff $\Gamma \nvdash_{\mathsf{ND}_{\mathsf{CPL}}} \bot$.

Definition 7.4 (Maximal consistent set)**.** A *maximal consistent set* is a consistent set $\Delta \subseteq \mathsf{Fm}_{\mathsf{CPL}}$ such that $\Delta \cup \{\varphi\}$ is inconsistent for every $\varphi \notin \Delta$.

Lemma 7.5 (Closure properties)**.** *If Δ is maximal consistent, then:*
1. if $\Delta \vdash_{\mathsf{ND}_{\mathsf{CPL}}} \varphi$, then $\varphi \in \Delta$;
2. exactly one of φ and $\neg\varphi$ belongs to Δ;
3. $\varphi \wedge \psi \in \Delta$ iff $\varphi \in \Delta$ and $\psi \in \Delta$;
4. $\varphi \vee \psi \in \Delta$ iff $\varphi \in \Delta$ or $\psi \in \Delta$; and
5. $\varphi \to \psi \in \Delta$ iff $\varphi \notin \Delta$ or $\psi \in \Delta$.

Proof. For 1, suppose $\Delta \vdash \varphi$ but $\varphi \notin \Delta$. Maximality makes $\Delta \cup \{\varphi\}$ inconsistent. Substitute the derivation of φ for every open use of the added assumption by lemma 6.5; this yields a derivation of $\bot$ from Δ, contrary to consistency.

The two formulas in 2 cannot both belong to Δ, by $\neg E$. If $\varphi \notin \Delta$, maximality yields a derivation D of $\bot$ whose open assumptions lie in $\Delta \cup \{\varphi\}$. Because Δ is consistent, at least one open occurrence of φ remains in D. Mark all open occurrences of φ in D with a common fresh label i, by the merging relabeling of the derivation definition, and apply $\neg I$ with label i at the root, discharging every such occurrence, to obtain $\Delta \vdash \neg\varphi$; then 1 gives $\neg\varphi \in \Delta$. This proves 2.

For 3, conjunction elimination proves the forward implication and conjunction introduction proves the reverse implication, using 1. For 4, disjunction introduction proves the reverse implication. If $\varphi \vee \psi \in \Delta$ while neither disjunct belongs to Δ, then 2 gives $\neg\varphi, \neg\psi \in \Delta$. Disjunction elimination, using each negation to derive $\bot$ in its corresponding branch, derives $\bot$ from Δ, a contradiction.

For 5, if $\varphi \to \psi \in \Delta$ and $\varphi \in \Delta$, implication elimination and 1 give $\psi \in \Delta$. Conversely, if $\psi \in \Delta$, assume φ with label i and take ψ as an open assumption; $\wedge I$ gives $\varphi \wedge \psi$, and $\wedge E_2$ gives ψ; then $\to I$ discharges the single open occurrence marked i, concluding $\varphi \to \psi$ with open assumptions in Δ. The discharged occurrence is genuinely present, and only displayed rules occur. If $\varphi \notin \Delta$, then $\neg\varphi \in \Delta$ by 2; from an assumed φ, derive $\bot$, then ψ, and discharge the assumption, which occurs open in that derivation. In either case 1 places $\varphi \to \psi$ in Δ. □

Lemma 7.6 (Lindenbaum lemma)**.** *Every consistent subset of $\mathsf{Fm}_{\mathsf{CPL}}$ is contained in a maximal consistent set.*

Proof. The set $\mathsf{Fm}_{\mathsf{CPL}}$ is countable; enumerate it as $\varphi_0, \varphi_1, \ldots$. Starting with a consistent set $\Gamma_0 = \Gamma$, define

$$\Gamma_{n+1} = \begin{cases} \Gamma_n \cup \{\varphi_n\}, & \text{if this set is consistent,} \\ \Gamma_n, & \text{otherwise.} \end{cases}$$

Let $\Delta = \bigcup_{n \in \mathbb{N}} \Gamma_n$. If Δ were inconsistent, a finite derivation of $\bot$ would use only finitely many assumptions, all lying in some Γ_n, contradicting its consistency. Thus Δ is consistent. If $\varphi_n \notin \Delta$, it was rejected at stage $n+1$, so $\Gamma_n \cup \{\varphi_n\}$, and hence $\Delta \cup \{\varphi_n\}$, is inconsistent. Therefore Δ is maximal consistent. □

Lemma 7.7 (Truth lemma)**.** *Let Δ be maximal consistent and define a valuation v_Δ by*

$$v_\Delta(p_n) = \mathbf{T} \quad \textit{iff} \quad p_n \in \Delta.$$

Then, for every propositional formula φ,

$$v_\Delta \models \varphi \quad \textit{iff} \quad \varphi \in \Delta.$$

Proof. Proceed by structural induction. The variable case is the definition of v_Δ, and $\bot \notin \Delta$ by consistency. The negation case uses lemma 7.5(2) and the truth clause for $\neg$. The conjunction, disjunction, and implication cases use respectively clauses 3, 4, and 5 of that lemma together with the corresponding recursive truth clauses. These are all constructors in the grammar, so the induction is complete. □

7.3 Completeness and compactness

Theorem 7.8 (Strong completeness of classical propositional natural deduction)**.** *For every $\Gamma \subseteq \mathsf{Fm}_{\mathsf{CPL}}$ and every $\varphi \in \mathsf{Fm}_{\mathsf{CPL}}$,*

$$\Gamma \models \varphi \quad \Longrightarrow \quad \Gamma \vdash_{\mathsf{ND}_{\mathsf{CPL}}} \varphi.$$

Proof. We prove the contrapositive. Suppose $\Gamma \nvdash_{\mathsf{ND}_{\mathsf{CPL}}} \varphi$. Then $\Gamma \cup \{\neg\varphi\}$ is consistent. For suppose a derivation D of $\bot$ has open assumptions in $\Gamma \cup \{\neg\varphi\}$. If no open occurrence of $\neg\varphi$ appeared in D, then D would witness $\Gamma \vdash_{\mathsf{ND}_{\mathsf{CPL}}} \bot$, and $\bot E$ would give $\Gamma \vdash_{\mathsf{ND}_{\mathsf{CPL}}} \varphi$, contrary to the supposition. So at least one open occurrence remains; mark all open occurrences of $\neg\varphi$ in D with a common fresh label i, by the merging relabeling of the derivation definition, and apply RAA with label i at the root, discharging every such occurrence, to obtain $\Gamma \vdash_{\mathsf{ND}_{\mathsf{CPL}}} \varphi$—again contrary to the supposition. Hence $\Gamma \cup \{\neg\varphi\}$ is consistent. By lemma 7.6, extend this set to a maximal consistent set Δ. The truth lemma gives $v_\Delta \models \Gamma$ and $v_\Delta \models \neg\varphi$. Hence $v_\Delta \not\models \varphi$, so $\Gamma \not\models \varphi$. □

Combining theorems 7.1 and 7.8 yields

$$\Gamma \vdash_{\mathsf{ND}_{\mathsf{CPL}}} \varphi \quad \text{iff} \quad \Gamma \models \varphi.$$

Theorem 7.9 (Propositional compactness)**.** *For every $\Gamma \subseteq \mathsf{Fm}_{\mathsf{CPL}}$ and every $\varphi \in \mathsf{Fm}_{\mathsf{CPL}}$:*
1. if $\Gamma \models \varphi$, then some finite $\Gamma_0 \subseteq \Gamma$ satisfies $\Gamma_0 \models \varphi$; and
2. Γ is satisfiable iff every finite subset of Γ is satisfiable.

Proof. If $\Gamma \models \varphi$, completeness gives a finite derivation from Γ. Only finitely many members of Γ occur as open assumptions; call their set Γ_0. Soundness gives $\Gamma_0 \models \varphi$, proving 1.

The forward implication in 2 is immediate. For the converse, if Γ were not satisfiable, then $\Gamma \models \bot$. Clause 1 would give a finite $\Gamma_0 \subseteq \Gamma$ with $\Gamma_0 \models \bot$, so Γ_0 would not be satisfiable. □

Theorem 7.10 (Completeness of the sequent calculus)**.** *For all finite multisets* Γ, Δ *of propositional formulas, the sequent* $\Gamma \Rightarrow \Delta$ *is derivable in* $\mathsf{LK}_{\mathsf{CPL}}$ *iff every valuation making every formula of* $\mathrm{supp}(\Gamma)$ *true makes some formula of* $\mathrm{supp}(\Delta)$ *true. Status: proof sketch.*

Proof sketch. The exact scope is the displayed cut-admitting calculus with finite multiset contexts. Left to right is theorem 7.2, whose semantic condition on a sequent is exactly the one displayed here.

For right to left, first record the translation subroutine. Induction on a $\mathsf{ND}_{\mathsf{CPL}}$-derivation translates every introduction rule to the corresponding right rule, every elimination rule to the corresponding left rule followed when necessary by cut, and assumption discharge to a right rule. Weakening and contraction align the finite contexts. The induction establishes that if $\Gamma_0 \vdash_{\mathsf{ND}_{\mathsf{CPL}}} \varphi$ for a finite set Γ_0, then the sequent $\Gamma_0 \Rightarrow \varphi$ is derivable in the displayed calculus. The individual rule-by-rule translation trees are omitted, and the argument does not establish cut elimination.

Now suppose every valuation making every formula of $\mathrm{supp}(\Gamma)$ true makes some formula of $\mathrm{supp}(\Delta)$ true, and list $\mathrm{supp}(\Delta) = \{\delta_1, \ldots, \delta_m\}$ with $m \geq 0$.

1. Let S be the set of members of $\mathrm{supp}(\Gamma)$ together with $\neg\delta_1, \ldots, \neg\delta_m$. A valuation satisfying all of S would make every member of $\mathrm{supp}(\Gamma)$ true, hence some δ_j true together with $\neg\delta_j$. So no valuation satisfies S, and $S \models \bot$.
2. Theorem 7.8 gives $S \vdash_{\mathsf{ND}_{\mathsf{CPL}}} \bot$. The open assumptions of that derivation form a finite subset of S, say $\gamma_1, \ldots, \gamma_k \in \mathrm{supp}(\Gamma)$ and $\neg\delta_{j_1}, \ldots, \neg\delta_{j_r}$. The translation subroutine turns it into an $\mathsf{LK}_{\mathsf{CPL}}$-derivation of $\gamma_1, \ldots, \gamma_k, \neg\delta_{j_1}, \ldots, \neg\delta_{j_r} \Rightarrow \bot$.
3. Cut with the initial $\bot$-sequent $\bot \Rightarrow$, the instance of $\Gamma, \bot \Rightarrow \Delta$ with both contexts empty, empties the succedent.
4. For each j, the sequent $\Rightarrow \delta_j, \neg\delta_j$ is derivable: start from the identity sequent $\delta_j \Rightarrow \delta_j$ and apply $\neg R$. Cutting it against the current sequent, with cut formula $\neg\delta_j$,

$$\frac{\Rightarrow \delta_j, \neg\delta_j \qquad \Gamma', \neg\delta_j \Rightarrow \Pi}{\Gamma' \Rightarrow \delta_j, \Pi}\ \textsc{Cut},$$

   moves δ_j into the succedent; after r such cuts the derived sequent is $\gamma_1, \ldots, \gamma_k \Rightarrow \delta_{j_1}, \ldots, \delta_{j_r}$.
5. Weakening (W_L, W_R) adds the unused members of Γ and Δ, and contraction (C_L, C_R) merges duplicates, giving exactly the multiset sequent $\Gamma \Rightarrow \Delta$.

When $m = 0$, the hypothesis asserts that no valuation makes every member of $\mathrm{supp}(\Gamma)$ true, step 4 performs no cuts, and the same route derives $\Gamma \Rightarrow \Delta$ with empty succedent; an empty Γ is likewise covered. The route uses cut, which is primitive in the displayed calculus, and makes no cut-free claim; no choice principle enters, because the propositional Lindenbaum lemma runs on a countable enumeration. The status is a sketch because the individual translation trees of the subroutine remain omitted. □

7.4 Propositional resolution

The clause object in this section is exactly the finite set defined in section 5.4. A valuation satisfies a clause set S iff it satisfies every $C \in S$, so the set is interpreted conjunctively. The complement of p is $\neg p$, and the complement of $\neg p$ is p. A clause containing both a literal and its complement is *tautological*; its formula interpretation is valid, so deleting it from a clause set preserves the satisfying valuations of that set.

Definition 7.11 (Resolvent)**.** Let E and F be clauses, let $L \in E$, and let its complement $\overline{L}$ belong to F. Their *resolvent on* L is the clause

$$(E \setminus \{L\}) \cup (F \setminus \{\overline{L}\}).$$

A *resolution derivation* from a clause set S is a finite sequence whose members are clauses in S or resolvents of earlier members.

Theorem 7.12 (Soundness of propositional resolution)**.** *Every valuation satisfying both parent clauses satisfies their resolvent. Consequently, deriving the empty clause proves that the initial clause set is not satisfiable.*

Proof. Let E, F be the parents and let $R = (E \setminus \{L\}) \cup (F \setminus \{\overline{L}\})$. Suppose a valuation falsifies R. It therefore falsifies every nonpivot literal in both parents. To satisfy E, it must satisfy L; to satisfy F, it must satisfy $\overline{L}$. No valuation satisfies complementary literals. Therefore no valuation can satisfy both parents while falsifying the resolvent. Induction on a resolution derivation preserves satisfaction. The empty clause is false under every valuation. □

Theorem 7.13 (Refutational completeness of propositional resolution)**.** *Let S be a finite set of clauses. Then S is not satisfiable iff the empty clause has a resolution derivation from S.*

Proof. Soundness proves the reverse implication. For the forward implication, delete tautological clauses. Induct on the number of variables occurring in S. If there are no variables, an unsatisfiable clause set contains the empty clause.

Choose a variable p. Form S_p from all clauses not containing p or $\neg p$, together with every non-tautological resolvent on p of a clause containing p and a clause containing $\neg p$. We claim that S is satisfiable iff S_p is satisfiable. Restricting a valuation satisfying S to the other variables satisfies all retained clauses and all resolvents by soundness. Conversely, let w satisfy S_p. A clause $\{p\} \cup A$, where $p \notin A$, whose remainder A is false under w requires $p = \mathbf{T}$; a clause $\{\neg p\} \cup B$, where $\neg p \notin B$, whose remainder B is false requires $p = \mathbf{F}$. Both requirements cannot occur: otherwise the resolvent $A \cup B$ on p would be false under w; being false under w, it contains no complementary pair of literals, so it is not tautological, belongs to S_p, and contradicts the assumption that w satisfies S_p. Choose either permitted value for p. The resulting extension satisfies S, proving the claim.

If S is not satisfiable, neither is S_p. It contains fewer variables, so the induction hypothesis gives a resolution derivation of the empty clause from S_p. Every noninitial clause of S_p is itself a resolvent of clauses in S. Splicing those one-step derivations into the derivation from S_p gives a resolution derivation from S. □

Remark 7.14 (Attribution and scope of the finiteness hypothesis). The variable-elimination step in the proof of theorem 7.13—passing from S to S_p and discarding the variable p—is the elimination rule of Davis and Putnam [16]. The finiteness hypothesis on S is consumed by the induction, not by the result: an infinite clause set may contain infinitely many variables, and the induction on the number of variables then has no measure to decrease. The restriction is removed by propositional compactness. Suppose an arbitrary clause set S is not satisfiable. A valuation satisfies a clause C iff it satisfies the formula $\mathrm{form}_{\mathsf{CPL}}(C)$ of section 5.4, so the formula set $\{\mathrm{form}_{\mathsf{CPL}}(C) : C \in S\}$ is not satisfiable, and theorem 7.9 gives a finite subset of it that is not satisfiable. Selecting for each of its finitely many members one clause of S interpreting it yields a finite $S_0 \subseteq S$ that is not satisfiable. Theorem 7.13 applied to S_0 gives a resolution derivation of the empty clause from S_0, and every resolution derivation from S_0 is one from S. Since the soundness theorem above is stated without finiteness of the clause set, the equivalence of theorem 7.13 therefore holds for every clause set, finite or not.

Resolution is a refutation calculus: it derives the empty clause from an unsatisfiable clause set. Conversion to conjunctive normal form and resolution must therefore be kept distinct. The former supplies clauses; the latter derives clauses. The original resolution method is due to Robinson [49]; normal-form transformations and their preservation properties are treated systematically in [4].

7.5 Common errors

Warning 7.15 (Propositional ambiguity). The following replacements are invalid unless separately justified:
1. replacing $\varphi \to \psi$ by $\psi \to \varphi$;
2. treating $\varphi \vee \psi$ as exclusive;
3. inferring either disjunct from a disjunction;
4. inferring φ from $\varphi \to \psi$ and ψ;
5. using a truth table for some variables while ignoring another variable occurring in the formula;
6. citing soundness in the completeness direction; and
7. calling a clause set satisfiable because resolution search has not yet found the empty clause.

7.6 Exercises

Exercise 7.16. Parse $\neg P \wedge Q \to R \vee S$ using the declared precedence and associativity conventions. Restore every parenthesis required by the grammar and list all subformulas.

Exercise 7.17. Prove by structural induction that, in the syntax tree of every formula, the number of leaves—occurrences of propositional variables or $\bot$—is one greater than the number of binary-connective occurrences.

Exercise 7.18. Construct a countervaluation to $P \to Q, Q \models P$. Explain why this refutes semantic consequence but does not by itself inspect any derivation.

Exercise 7.19. Translate "The process terminates only if either the input is finite or the search is bounded, but not both." Supply a key and state whether "or" is inclusive before adding the final restriction.

Exercise 7.20. Find conjunctive and disjunctive normal forms for $(P \to Q) \wedge (Q \to R)$. Verify both by a truth table.

Exercise 7.21. Give a $\mathsf{ND}_{\mathsf{CPL}}$-derivation of $(P \to R) \to ((Q \to R) \to (P \vee Q \to R))$. Mark every discharged assumption.

Exercise 7.22. Derive $\neg(P \wedge \neg P)$ without using RAA. Then identify the discharging rule used in place of RAA, and explain why every rule in the derivation is available in both classical and intuitionistic natural deduction.

Exercise 7.23. Give a cut-free $\mathsf{LK}_{\mathsf{CPL}}$-derivation of $P \to Q, Q \to R \Rightarrow P \to R$.

Exercise 7.24. Prove that logical equivalence is an equivalence relation and a congruence for every primitive connective.

Exercise 7.25. Use the maximal-consistent-set construction to find a countervaluation to a chosen underivable formula. Show explicitly how membership in the set determines the valuation.

Exercise 7.26. Convert

$$(P \vee Q) \wedge (\neg P \vee R) \wedge (\neg Q \vee R) \wedge \neg R$$

to a clause set and give a resolution refutation.

Exercise 7.27. Show that deleting a tautological clause preserves the set of satisfying valuations. Show by example that deleting a clause that some valuation satisfies, but that is not tautological, can change the set of valuations satisfying the clause set.

Part III

Classical First-Order Logic

Classical first-order logic adds variables ranging over a nonempty domain, symbols for functions and relations, and quantifiers. That additional expressive power makes the syntax of binding and substitution part of the formal-logical development. The definitions in this part follow the set-based semantics of [18, 30]; equality is logical identity, not an unspecified binary relation.

Chapter 8

First-Order Syntax

Parallel textbook developments of the material in this part are [12] and [18].

8.1 Signatures and terms

Fix the countably infinite set $\mathsf{Var} := \{x_n : n \in \mathbb{N}\}$ of object-language variables. The letters x, y, z denote members of Var, while r, t, u are metavariables ranging over terms. The letter s is reserved for variable assignments.

Definition 8.1 (First-order signature)**.** A *first-order signature* is a tuple

$$\mathcal{L} = \langle \mathcal{F}, \mathcal{R}, \mathrm{ar}_{\mathcal{F}}, \mathrm{ar}_{\mathcal{R}} \rangle,$$

where $\mathcal{F}$ and $\mathcal{R}$ are sets of function and relation symbols and

$$\mathrm{ar}_{\mathcal{F}} : \mathcal{F} \to \mathbb{N}, \qquad \mathrm{ar}_{\mathcal{R}} : \mathcal{R} \to \mathbb{N}$$

assign each symbol its arity. A function symbol of arity zero is a *constant symbol*. The equality sign $=$ is logical and is not a member of $\mathcal{R}$. The sets $\mathcal{F}$, $\mathcal{R}$, Var, and the set of reserved logical and punctuation tokens

$$\{=, \bot, \neg, \wedge, \vee, \to, \forall, \exists, \mathsf{(}, \mathsf{)}, \mathsf{,}\}$$

are pairwise disjoint. Consequently no signature symbol can also be a variable, logical symbol, parenthesis, or comma.

The cardinality of the signature is $|\mathcal{L}| := |\mathcal{F} \cup \mathcal{R}|$. Arity data add no new symbols.

The signature is a syntactic object. It assigns arities but assigns no functions, relations, or elements. A structure supplies those semantic objects later.

Definition 8.2 (Term)**.** For a signature $\mathcal{L}$, the set $\mathsf{Term}(\mathcal{L})$ of *terms* is the least set satisfying:
1. every $x \in \mathsf{Var}$ belongs to $\mathsf{Term}(\mathcal{L})$; and
2. if $f \in \mathcal{F}$ has arity n and $t_1, \ldots, t_n \in \mathsf{Term}(\mathcal{L})$, then $f(t_1, \ldots, t_n) \in \mathsf{Term}(\mathcal{L})$.

For $n = 0$, the expression is written f, not $f()$. No other string is a term.

Example 8.3. Let $\mathcal{L}_{\mathrm{ar}}$ have function symbols $\{0, S, +, \cdot\}$ of arities $0, 1, 2, 2$, respectively, and relation symbol $<$ of arity 2. The strings

$$x_0, \qquad S(0), \qquad +(S(0), x_2), \qquad \cdot(+(x_0, x_1), S(x_2))$$

are terms. The string $<(x_0, x_1)$ is not a term because $<$ is a relation symbol. Conventional infix displays such as $x_0 + S(0)$ are typographical abbreviations for the prefix grammar.

8.2 Formulas and unique readability

Definition 8.4 (First-order formula)**.** For a signature $\mathcal{L}$, the set $\mathsf{Fm}(\mathcal{L})$ of *formulas* is the least set satisfying:
1. if $r, t \in \mathsf{Term}(\mathcal{L})$, then $r = t \in \mathsf{Fm}(\mathcal{L})$;
2. if $R \in \mathcal{R}$ has arity n and $t_1, \ldots, t_n \in \mathsf{Term}(\mathcal{L})$, then $R(t_1, \ldots, t_n) \in \mathsf{Fm}(\mathcal{L})$;
3. $\bot \in \mathsf{Fm}(\mathcal{L})$;
4. if $\varphi \in \mathsf{Fm}(\mathcal{L})$, then $\neg\varphi \in \mathsf{Fm}(\mathcal{L})$;
5. if $\varphi, \psi \in \mathsf{Fm}(\mathcal{L})$ and $\circ \in \{\wedge, \vee, \to\}$, then $(\varphi \circ \psi) \in \mathsf{Fm}(\mathcal{L})$; and
6. if $x \in \mathsf{Var}$ and $\varphi \in \mathsf{Fm}(\mathcal{L})$, then $\forall x\, \varphi$ and $\exists x\, \varphi$ belong to $\mathsf{Fm}(\mathcal{L})$.

No other string is a formula. A formula introduced by clause 1 or 2 is *atomic*. When a relation symbol has arity zero, its atomic formula is written R, not $R()$.

The scope of a quantifier in the official grammar is the formula immediately following its variable. In abbreviated displays, this document gives a quantifier tighter binding than every binary connective; parentheses extend its scope. Thus $\forall x\, P(x) \to Q(x)$ abbreviates $(\forall x\, P(x)) \to Q(x)$ under the declared connective precedence, whereas $\forall x\, (P(x) \to Q(x))$ has the implication inside the quantifier's scope.

Theorem 8.5 (Unique readability for first-order expressions)**.** *Every $\mathcal{L}$-term has exactly one outermost formation clause, and its immediate subterms are uniquely determined. Every $\mathcal{L}$-formula has exactly one outermost formation clause, and its principal symbol, bound variable when present, and immediate subformulas are uniquely determined.*

Proof. Function symbols and relation symbols are disjoint and carry fixed arities; variables, logical symbols, punctuation, and signature symbols are pairwise distinguishable tokens. A variable term is one token. A compound term begins with its unique function symbol, and its fixed arity determines the number of comma-separated argument positions. Balanced-parenthesis scanning uniquely separates those arguments. Induction on term length therefore gives unique term construction.

An atomic formula either has the uniquely located equality sign between two terms or begins with a relation symbol whose arity uniquely determines its term arguments. The remaining formula constructors have distinguishable initial forms: $\bot$, $\neg$, an initial left parenthesis for a binary connective, and $\forall$ or $\exists$ followed by one variable. The depth-zero scan used in theorem 5.5 uniquely separates a binary formula. Removing a unary logical symbol leaves its unique immediate formula. Induction on formula length completes the proof. □

8.3 Free variables and sentences

Definition 8.6 (Variables occurring in a term)**.** The sans-serif symbol Var without an argument denotes the fixed set of all object-language variables; the roman operator Var with an argument denotes an occurrence set. The two roles are distinguished by the presence of the argument, and both are registered here. The set $\mathrm{Var}(t)$ of variables occurring in a term is defined by

$$\mathrm{Var}(x) = \{x\}, \qquad \mathrm{Var}(f(t_1, \ldots, t_n)) = \bigcup_{i=1}^{n} \mathrm{Var}(t_i).$$

Definition 8.7 (Free variables)**.** The set $\mathrm{FV}(\varphi)$ of *free variables* of a formula is defined by

$$\begin{aligned} \mathrm{FV}(r = t) &= \mathrm{Var}(r) \cup \mathrm{Var}(t), \\ \mathrm{FV}(R(t_1, \dots, t_n)) &= \bigcup_{i=1}^{n} \mathrm{Var}(t_i), \\ \mathrm{FV}(\bot) &= \varnothing, \\ \mathrm{FV}(\neg\varphi) &= \mathrm{FV}(\varphi), \\ \mathrm{FV}(\varphi \circ \psi) &= \mathrm{FV}(\varphi) \cup \mathrm{FV}(\psi), \\ \mathrm{FV}(\forall x\, \varphi) &= \mathrm{FV}(\varphi) \setminus \{x\}, \\ \mathrm{FV}(\exists x\, \varphi) &= \mathrm{FV}(\varphi) \setminus \{x\}. \end{aligned}$$

A formula with no free variable is a *sentence*; the set of $\mathcal{L}$-sentences is $\mathsf{Sent}(\mathcal{L})$.

Definition 8.8 (First-order formula complexity)**.** For $\varphi \in \mathsf{Fm}(\mathcal{L})$, its *complexity* $c(\varphi) \in \mathbb{N}$ is defined by

$$\begin{aligned} c(r = t) &= 0, & c(R(t_1, \dots, t_n)) &= 0, & c(\bot) &= 0, \\ c(\neg\varphi) &= c(\varphi) + 1, & c(\varphi \circ \psi) &= c(\varphi) + c(\psi) + 1, & & \\ c(Qx\, \varphi) &= c(\varphi) + 1, & & & & \end{aligned}$$

where $\circ \in \{\wedge, \vee, \to\}$ and $Q \in \{\forall, \exists\}$. Every immediate subformula has smaller complexity. This is the measure used by every later induction or recursion described as being on first-order formula complexity.

For a set of formulas Γ, extend the notation by

$$\mathrm{FV}(\Gamma) := \bigcup_{\gamma \in \Gamma} \mathrm{FV}(\gamma).$$

Thus $x \notin \mathrm{FV}(\Gamma)$ means exactly that x is free in no member of Γ.

Let $\mathrm{Var}(\varphi)$ denote the set of all variables having a free or bound occurrence in φ. It is defined by the same recursive clauses as FV, except that a quantifier clause adds its bound variable instead of removing it.

An occurrence of x is *bound* when it lies in the scope of a quantifier $\forall x$ or $\exists x$ that governs that occurrence. The quantifier binds occurrences, not the variable symbol throughout the formula. In

$$P(x) \wedge \exists x\, Q(x, y),$$

the first occurrence of x is free, the second is bound, and the occurrence of y is free.

8.4 Capture-avoiding substitution

Definition 8.9 (Substitution in a term)**.** For terms r, t and a variable x, the term $r[t/x]$ is defined by

$$x[t/x] = t, \qquad y[t/x] = y \ (y \neq x),$$

and

$$f(r_1, \dots, r_n)[t/x] = f(r_1[t/x], \dots, r_n[t/x]).$$

Only occurrences of x are replaced.

Fix the order $x_0, x_1, \ldots$ on Var. This order makes every choice of a fresh variable below deterministic.

Definition 8.10 (Capture-avoiding substitution in a formula)**.** For a formula φ, term t, and variable x, the formula $\varphi[t/x]$ is defined by recursion on formula complexity. In atomic formulas, substitute in every term. Substitution commutes with $\bot, \neg, \wedge, \vee, \to$. For $Q \in \{\forall, \exists\}$, define

$$(Qy\,\psi)[t/x] = \begin{cases} Qy\,\psi, & y = x, \\ Qy\,(\psi[t/x]), & y \neq x \text{ and } y \notin \mathrm{Var}(t), \\ Qz\,((\psi[z/y])[t/x]), & y \neq x \text{ and } y \in \mathrm{Var}(t), \end{cases}$$

where in the last clause z is the least variable not in

$$\mathrm{Var}(\psi) \cup \mathrm{Var}(t) \cup \{x, y\}.$$

The recursion is on complexity, and it is well founded by a simultaneous strong induction on complexity: the calls on ψ concern a formula of lower complexity than $Qy\,\psi$, and variable-for-variable substitution preserves complexity—each clause preserves the outer constructor—so in the last clause $c(\psi[z/y]) = c(\psi) < c(Qy\,\psi)$ and the outer call is also at lower complexity. Complexity preservation for substitution by an arbitrary term is recorded as lemma 8.11 below.

The last clause first renames the binder and then substitutes. Its purpose is not stylistic: it prevents a free variable of t from becoming bound.

Lemma 8.11 (Substitution preserves formula complexity)**.** *For every formula φ, term t, and variable x,*

$$c(\varphi[t/x]) = c(\varphi).$$

Proof. Induct on $c(\varphi)$. Term substitution changes no logical constructor in an atomic formula. The connective clauses preserve the displayed outer constructor and use the induction hypotheses. At a quantifier, substitution either stops, commutes with the quantifier, or renames its bound variable before recurring on the matrix. Bound-variable renaming changes no logical constructor, so every quantifier case also preserves complexity. □

Definition 8.12 (Alpha-equivalence)**.** The relation of *alpha-equivalence*, written $\equiv_\alpha$, is the least equivalence relation on formulas that is a congruence for every formula constructor and contains

$$Qx\,\varphi \equiv_\alpha Qy\,(\varphi[y/x])$$

for $Q \in \{\forall, \exists\}$ whenever $y \notin \mathrm{Var}(\varphi)$. Thus a generating step renames one bound variable and exactly the occurrences governed by its quantifier.

Alpha-equivalence is syntactic and is not literal identity of strings. It will be shown below to preserve satisfaction.

Example 8.13 (Avoiding capture)**.** Let $\varphi = \forall y\, R(x, y)$ and substitute the term y for x. Direct textual replacement would produce $\forall y\, R(y, y)$, incorrectly binding the inserted occurrence. If z is the least permitted fresh variable, the defined result is

$$\varphi[y/x] = \forall z\, R(y, z).$$

The free occurrence introduced by substitution remains free.

Lemma 8.14 (Free variables under substitution)**.** *For all terms r, t and every variable x: if $x \notin \mathrm{Var}(r)$, then $r[t/x] = r$; and*

$$\mathrm{Var}(r[t/x]) \subseteq (\mathrm{Var}(r) \setminus \{x\}) \cup \mathrm{Var}(t).$$

For every formula φ, term t, and variables x, y:
1. $\mathrm{FV}(\varphi[t/x]) \subseteq (\mathrm{FV}(\varphi) \setminus \{x\}) \cup \mathrm{Var}(t)$*; and*
2. *if $y \notin \mathrm{Var}(\varphi)$, then $\mathrm{FV}(\varphi[y/x]) = (\mathrm{FV}(\varphi) \setminus \{x\}) \cup \{y\}$ when $x \in \mathrm{FV}(\varphi)$, and $\mathrm{FV}(\varphi[y/x]) = \mathrm{FV}(\varphi)$ when $x \notin \mathrm{FV}(\varphi)$.*

A term's variables are measured by Var*, exactly as in the* FV *clauses for atomic formulas; no separate free-variable operator on terms is defined in this document. Status: complete proof.*

Proof. For the term claims, induct on r. A variable other than x is unchanged, the variable x itself is replaced by t, and the compound clause applies the induction hypotheses argumentwise.

For the formula claims, proceed by strong induction on formula complexity c. The atomic cases follow from the term claims and the FV clauses for atomic formulas; the $\bot$ case is immediate, and the connective cases combine the induction hypotheses through the FV clauses.

For clause 2, the renaming clause of the substitution definition never fires: every binder w of a subformula of φ lies in $\mathrm{Var}(\varphi)$, so $w \neq y$, and the substitution is textual replacement of the free occurrences of x by y. At a quantifier $Qw\,\psi$: if $w = x$, substitution stops, x is not free, and both sides equal $\mathrm{FV}(Qw\,\psi)$; if $w \neq x$, the substitution commutes with the quantifier, and the induction hypothesis for ψ passes through the clause $\mathrm{FV}(Qw\,(\psi[y/x])) = \mathrm{FV}(\psi[y/x]) \setminus \{w\}$, using $w \neq y$ and the fact that $x \in \mathrm{FV}(Qw\,\psi)$ iff $x \in \mathrm{FV}(\psi)$.

For clause 1 at a quantifier $Qy\,\psi$: if $y = x$, substitution stops and $x \notin \mathrm{FV}(Qy\,\psi)$; if $y \neq x$ and $y \notin \mathrm{Var}(t)$, apply the induction hypothesis to ψ and remove y. In the renaming case $(Qy\,\psi)[t/x] = Qz\,((\psi[z/y])[t/x])$, apply the induction hypothesis to ψ with $[z/y]$—clause 2—and to $\psi[z/y]$ with $[t/x]$; the latter application is at the equal complexity $c(\psi[z/y]) = c(\psi) < c(Qy\,\psi)$, licensed by lemma 8.11. Since $z \notin \mathrm{Var}(t) \cup \{x\}$, computing with the FV clauses gives

$$\mathrm{FV}(Qz\,((\psi[z/y])[t/x])) \subseteq (\mathrm{FV}(Qy\,\psi) \setminus \{x\}) \cup \mathrm{Var}(t). \qquad \square$$

Lemma 8.15. *If $x \notin \mathrm{FV}(\varphi)$, then $\varphi[t/x] \equiv_\alpha \varphi$. Status: complete proof.*

Proof. Proceed by strong induction on formula complexity c. In the atomic case, x occurs in no term of φ, so every term is unchanged by the term clause of lemma 8.14 and $\varphi[t/x] = \varphi$; the $\bot$ case is likewise immediate. In the connective cases, x is free in no immediate subformula; the induction hypotheses and the congruence property of $\equiv_\alpha$ conclude.

At a quantifier $Qy\,\psi$ with $y = x$, substitution stops, and reflexivity concludes. If $y \neq x$ and $y \notin \mathrm{Var}(t)$, then $x \notin \mathrm{FV}(\psi)$; the induction hypothesis gives $\psi[t/x] \equiv_\alpha \psi$, and the quantifier congruence concludes. If $y \neq x$ and $y \in \mathrm{Var}(t)$, the definition returns $Qz\,((\psi[z/y])[t/x])$, where z is the least variable outside $\mathrm{Var}(\psi) \cup \mathrm{Var}(t) \cup \{x, y\}$. By lemma 8.14(2), $x \notin \mathrm{FV}(\psi[z/y])$, because $x \notin \mathrm{FV}(\psi)$ and $x \neq z$. By lemma 8.11, $c(\psi[z/y]) = c(\psi) < c(Qy\,\psi)$, so the strong induction hypothesis applies to $\psi[z/y]$ and gives $(\psi[z/y])[t/x] \equiv_\alpha \psi[z/y]$. The quantifier congruence gives $Qz\,((\psi[z/y])[t/x]) \equiv_\alpha Qz\,(\psi[z/y])$; the generating clause, licensed by $z \notin \mathrm{Var}(\psi)$, gives $Qz\,(\psi[z/y]) \equiv_\alpha Qy\,\psi$; transitivity concludes. $\square$

Chapter 9

Structures, Satisfaction, and Translation

9.1 Structures and variable assignments

Definition 9.1 (First-order structure)**.** Let $\mathcal{L}$ be a signature. A *$\mathcal{L}$-structure* $\mathcal{M}$ consists of:
1. a nonempty set $|\mathcal{M}|$, called its domain;
2. for each n-ary function symbol f, a function $f^{\mathcal{M}} : |\mathcal{M}|^n \to |\mathcal{M}|$; and
3. for each n-ary relation symbol R, a relation $R^{\mathcal{M}} \subseteq |\mathcal{M}|^n$.

For a constant symbol c, the zero-ary function is identified with its value $c^{\mathcal{M}} \in |\mathcal{M}|$. The equality sign is always interpreted as actual identity on $|\mathcal{M}|$.

Definition 9.2 (Signature extension, expansion, and reduct)**.** Let $\mathcal{L} = \langle \mathcal{F}, \mathcal{R}, \mathrm{ar}_{\mathcal{F}}, \mathrm{ar}_{\mathcal{R}} \rangle$ and $\mathcal{L}' = \langle \mathcal{F}', \mathcal{R}', \mathrm{ar}_{\mathcal{F}'}, \mathrm{ar}_{\mathcal{R}'} \rangle$. The signature $\mathcal{L}'$ *extends* $\mathcal{L}$ iff $\mathcal{F} \subseteq \mathcal{F}'$, $\mathcal{R} \subseteq \mathcal{R}'$, and both arity maps of $\mathcal{L}'$ restrict to the corresponding arity maps of $\mathcal{L}$.

If $\mathcal{M}$ is a $\mathcal{L}$-structure, a $\mathcal{L}'$-structure $\mathcal{M}'$ is an *expansion* of $\mathcal{M}$ iff it has the same domain and gives every symbol of $\mathcal{L}$ the same interpretation. If $\mathcal{M}'$ is a $\mathcal{L}'$-structure, its *reduct* to $\mathcal{L}$, written $\mathcal{M}' \restriction \mathcal{L}$, is the $\mathcal{L}$-structure with the same domain and the interpretations of the symbols in $\mathcal{L}$. Expansion adds interpretations and reduct forgets them; neither operation changes the domain, so neither term means substructure. These operations are required in the Henkin, Löwenheim–Skolem, and Skolemization arguments below.

Definition 9.3 (Variable assignment and update)**.** A *variable assignment* in $\mathcal{M}$ is a function $s : \mathsf{Var} \to |\mathcal{M}|$. For $a \in |\mathcal{M}|$, define the updated assignment $s[x \mapsto a]$ by

$$s[x \mapsto a](y) = \begin{cases} a, & y = x, \\ s(y), & y \neq x. \end{cases}$$

Definition 9.4 (Term value)**.** The *value* of a term t in $\mathcal{M}$ under s, written $[\![t]\!]^{\mathcal{M}}_s$, is defined by

$$[\![x]\!]^{\mathcal{M}}_s = s(x), \qquad [\![f(t_1, \ldots, t_n)]\!]^{\mathcal{M}}_s = f^{\mathcal{M}}([\![t_1]\!]^{\mathcal{M}}_s, \ldots, [\![t_n]\!]^{\mathcal{M}}_s).$$

Lemma 9.5 (Term coincidence)**.** *If assignments s and s' agree on* $\mathrm{Var}(t)$*, then* $[\![t]\!]^{\mathcal{M}}_s = [\![t]\!]^{\mathcal{M}}_{s'}$.

Proof. Induct on the construction of t. The variable case is the hypothesis. In a function case, apply the induction hypothesis to each argument and then apply the function $f^{\mathcal{M}}$. □

Lemma 9.6 (Term-substitution lemma)**.** *For all terms r, t, assignments s, and variables x,*

$$[\![r[t/x]]\!]^{\mathcal{M}}_s = [\![r]\!]^{\mathcal{M}}_{s[x \mapsto [\![t]\!]^{\mathcal{M}}_s]}.$$

Proof. Induct on r. If $r = x$, both sides equal $[\![t]\!]^{\mathcal{M}}_s$. If $r = y \neq x$, both sides equal $s(y)$. The function case follows componentwise from the induction hypotheses and the definition of term value. □

9.2 Satisfaction

Definition 9.7 (Satisfaction)**.** For a structure $\mathcal{M}$, assignment s, and formula φ, the relation $\mathcal{M}, s \models \varphi$ is defined by structural recursion:

$$\begin{aligned}
\mathcal{M}, s \models r = t &\quad \text{iff} \quad [\![r]\!]^{\mathcal{M}}_s = [\![t]\!]^{\mathcal{M}}_s, \\
\mathcal{M}, s \models R(t_1, \ldots, t_n) &\quad \text{iff} \quad ([\![t_1]\!]^{\mathcal{M}}_s, \ldots, [\![t_n]\!]^{\mathcal{M}}_s) \in R^{\mathcal{M}}, \\
\mathcal{M}, s \not\models \bot, & \\
\mathcal{M}, s \models \neg\varphi &\quad \text{iff} \quad \mathcal{M}, s \not\models \varphi, \\
\mathcal{M}, s \models \varphi \wedge \psi &\quad \text{iff} \quad \mathcal{M}, s \models \varphi \text{ and } \mathcal{M}, s \models \psi, \\
\mathcal{M}, s \models \varphi \vee \psi &\quad \text{iff} \quad \mathcal{M}, s \models \varphi \text{ or } \mathcal{M}, s \models \psi, \\
\mathcal{M}, s \models \varphi \to \psi &\quad \text{iff} \quad \mathcal{M}, s \not\models \varphi \text{ or } \mathcal{M}, s \models \psi, \\
\mathcal{M}, s \models \forall x\, \varphi &\quad \text{iff} \quad \mathcal{M}, s[x \mapsto a] \models \varphi \text{ for every } a \in |\mathcal{M}|, \\
\mathcal{M}, s \models \exists x\, \varphi &\quad \text{iff} \quad \mathcal{M}, s[x \mapsto a] \models \varphi \text{ for some } a \in |\mathcal{M}|.
\end{aligned}$$

Lemma 9.8 (Formula coincidence)**.** *If assignments s and s' agree on $\mathrm{FV}(\varphi)$, then*

$$\mathcal{M}, s \models \varphi \quad \textit{iff} \quad \mathcal{M}, s' \models \varphi.$$

Proof. Proceed by structural induction. Atomic cases use term coincidence. Connective cases follow from the induction hypotheses. For $\forall x\, \psi$, the assignments $s[x \mapsto a]$ and $s'[x \mapsto a]$ agree on $\mathrm{FV}(\psi)$ for each a, because s and s' agree on $\mathrm{FV}(\psi) \setminus \{x\}$. Apply the induction hypothesis for every a. The existential case is identical with "some" in place of "every." □

Theorem 9.9 (Substitution lemma)**.** *For every formula φ, term t, variable x, structure $\mathcal{M}$, and assignment s,*

$$\mathcal{M}, s \models \varphi[t/x] \quad \textit{iff} \quad \mathcal{M}, s[x \mapsto [\![t]\!]^{\mathcal{M}}_s] \models \varphi.$$

Proof. Proceed by induction on formula complexity. Atomic cases follow from lemma 9.6. Connective cases follow immediately from the induction hypotheses.

Let $\varphi = Qy\, \psi$, where $Q \in \{\forall, \exists\}$. If $y = x$, substitution stops at the quantifier. Updating x before evaluating $Qx\, \psi$ is immaterial, because the quantified update of x overrides it. If $y \neq x$ and $y \notin \mathrm{Var}(t)$, then $[\![t]\!]^{\mathcal{M}}_{s[y \mapsto a]} = [\![t]\!]^{\mathcal{M}}_s$ for every a. Apply the induction hypothesis to ψ under each updated assignment and then apply the truth clause for Q.

If $y \neq x$ and $y \in \mathrm{Var}(t)$, the definition first replaces y by a fresh z. The variable z occurs neither in t nor free in ψ. The induction hypothesis applied to the renamed body— legitimate because renaming preserves complexity, by lemma 8.11—together with formula coincidence, shows directly that $Qy\, \psi$ and $Qz\, (\psi[z/y])$ have the same satisfaction condition. The preceding quantifier argument then applies to the renamed body, whose complexity equals $c(\psi)$ by lemma 8.11. Thus the equivalence holds in all cases. □

Corollary 9.10 (Alpha-invariance)**.** *If $\varphi \equiv_\alpha \psi$, then $\mathcal{M}, s \models \varphi$ iff $\mathcal{M}, s \models \psi$ for every structure $\mathcal{M}$ and assignment s.*

Proof. Define a relation R on formulas by $\varphi R \psi$ iff, for every structure $\mathcal{M}$ and assignment s, $\mathcal{M}, s \models \varphi$ iff $\mathcal{M}, s \models \psi$. The relation R is reflexive, symmetric, and transitive. It is a congruence for $\bot, \neg, \wedge, \vee, \to$ by the corresponding satisfaction clauses. It is also a congruence for each quantifier: if $\varphi R \psi$, then the equivalence holds under every assignment $s[x \mapsto a]$, and the universal or existential satisfaction clause transfers it to $Qx\,\varphi$ and $Qx\,\psi$.

It remains to check a generating renaming. Suppose $y \notin \mathrm{Var}(\varphi)$. By theorem 9.9, satisfaction of $\varphi[y/x]$ under $s[y \mapsto a]$ is equivalent to satisfaction of φ under $s[y \mapsto a][x \mapsto a]$. Since $y \notin \mathrm{FV}(\varphi)$, formula coincidence identifies the latter satisfaction condition with that of φ under $s[x \mapsto a]$. Applying the satisfaction clause for $Q \in \{\forall, \exists\}$ gives $Qx\,\varphi \; R \; Qy\,(\varphi[y/x])$. Thus R is an equivalence relation and a congruence containing every generating renaming. By the leastness clause in the definition of $\equiv_\alpha$, alpha-equivalence is contained in R. □

Corollary 9.11 (Assignment independence for sentences)**.** *If $\sigma \in \mathsf{Sent}(\mathcal{L})$, then $\mathcal{M}, s \models \sigma$ iff $\mathcal{M}, s' \models \sigma$ for all assignments s, s' in $\mathcal{M}$.*

Proof. The assignments agree on $\mathrm{FV}(\sigma) = \varnothing$; apply lemma 9.8. □

Definition 9.12 (Truth, validity, and semantic consequence)**.** For a sentence σ, write $\mathcal{M} \models \sigma$ iff $\mathcal{M}, s \models \sigma$ for one, equivalently every, assignment s. For $\Gamma \subseteq \mathsf{Fm}(\mathcal{L})$, write $\Gamma \models \varphi$ iff, for every $\mathcal{L}$-structure $\mathcal{M}$ and assignment s,

$$(\mathcal{M}, s \models \gamma \text{ for every } \gamma \in \Gamma) \quad \Longrightarrow \quad \mathcal{M}, s \models \varphi.$$

The formula φ is *valid* iff $\varnothing \models \varphi$. Two $\mathcal{L}$-formulas φ and ψ are *logically equivalent*, written $\varphi \equiv \psi$, iff, for every $\mathcal{L}$-structure $\mathcal{M}$ and every assignment s,

$$\mathcal{M}, s \models \varphi \quad \Longleftrightarrow \quad \mathcal{M}, s \models \psi.$$

This is logical equivalence over all $\mathcal{L}$-structures, not alpha-equivalence and not literal identity of formula strings.

9.3 Translation with quantifiers

Every first-order translation must state a signature and the intended range of the variables. Restricted quantification is an abbreviation:

$$\forall x \in A\,\varphi \quad \text{means} \quad \forall x\,(A(x) \to \varphi),$$

and

$$\exists x \in A\,\varphi \quad \text{means} \quad \exists x\,(A(x) \wedge \varphi).$$

Example 9.13 (Quantifier order)**.** Let the variables range over people, with unary relation symbols S and P for student and professor, and binary relation symbol A for admires. Then

$$\forall x\big(S(x) \to \exists y(P(y) \wedge A(x, y))\big)$$

says that every student admires some professor. The formula

$$\exists y(P(y) \wedge \forall x(S(x) \to A(x, y)))$$

says that one professor is admired by every student. The second formula implies the first: its existentially quantified professor serves as a witness for each student. The first formula does not in general imply the second. Quantifier order records dependence of a witness on earlier universally quantified objects.

Example 9.14 (Negated universal)**.** In the same signature, "No professor admires every student" is

$$\forall y(P(y) \to \neg\forall x(S(x) \to A(y, x))).$$

By classical quantifier duality, this is equivalent to $\forall y(P(y) \to \exists x(S(x) \land \neg A(y, x)))$. If there are no students, the inner universal in the first formula is vacuously true and the existential in the second formula is false; both formulas therefore require that there be no professors.

Chapter 10

Theories and Axiomatization

Definition 10.1 (Theory, model, satisfiability, and structure theory)**.** A *theory* in $\mathcal{L}$ is a set $T \subseteq \mathsf{Sent}(\mathcal{L})$. A $\mathcal{L}$-structure $\mathcal{M}$ is a *model* of T, written $\mathcal{M} \models T$, iff $\mathcal{M} \models \sigma$ for every $\sigma \in T$. The class of models of T is $\mathrm{Mod}(T)$.

For any $\Gamma \subseteq \mathsf{Fm}(\mathcal{L})$, say that Γ is *satisfiable* iff there are a $\mathcal{L}$-structure $\mathcal{M}$ and an assignment s such that $\mathcal{M}, s \models \gamma$ for every $\gamma \in \Gamma$. Because sentences are assignment-independent, a theory is satisfiable iff it has a model.

For a $\mathcal{L}$-structure $\mathcal{M}$, define its *structure theory* by

$$\mathrm{Th}_{\mathcal{L}}(\mathcal{M}) := \{\sigma \in \mathsf{Sent}(\mathcal{L}) : \mathcal{M} \models \sigma\}.$$

When the signature is fixed by the surrounding statement, $\mathrm{Th}(\mathcal{M})$ is the metalinguistic abbreviation for $\mathrm{Th}_{\mathcal{L}}(\mathcal{M})$. A theory is an arbitrary set of sentences, whereas a structure theory contains exactly all sentences true in one specified structure. Both notions are required below: the first axiomatizes classes, and the second states elementary equivalence and forms elementary diagrams.

A theory is not required to be closed under semantic consequence. Its semantic closure is

$$\mathrm{Cn}(T) := \{\sigma \in \mathsf{Sent}(\mathcal{L}) : T \models \sigma\}.$$

Definition 10.2 (Axiomatization)**.** A theory T *axiomatizes* a class $\mathcal{K}$ of $\mathcal{L}$-structures iff $\mathrm{Mod}(T) = \mathcal{K}$. A class is *finitely axiomatizable* iff some finite theory axiomatizes it.

Example 10.3 (Groups)**.** Let $\mathcal{L}_{\mathrm{grp}}$ contain a binary function symbol $\cdot$, a unary function symbol i, and a constant symbol e. The following finite theory axiomatizes groups:

$$\forall x \forall y \forall z\, ((x \cdot y) \cdot z = x \cdot (y \cdot z)),$$
$$\forall x\, (e \cdot x = x \wedge x \cdot e = x),$$
$$\forall x\, (i(x) \cdot x = e \wedge x \cdot i(x) = e).$$

The symbols are interpreted by the structure; the equations constrain those interpretations. Equality itself remains actual identity.

Example 10.4 (Dense linear orders without endpoints)**.** In the signature containing one binary relation symbol $<$, the following sentences axiomatize dense linear orders without endpoints:

$$\forall x\, \neg(x < x),$$
$$\forall x \forall y \forall z\, ((x < y \wedge y < z) \to x < z),$$
$$\forall x \forall y\, (x < y \vee x = y \vee y < x),$$
$$\forall x \forall y\, (x < y \to \exists z (x < z \wedge z < y)),$$
$$\forall x\, \exists y \exists z\, (y < x \wedge x < z).$$

Each sentence contributes a distinct condition: irreflexivity, transitivity, linearity, density, and absence of endpoints.

Example 10.5 (Infinite structures)**.** For $n \geq 2$, let

$$\iota_n := \exists x_0 \cdots \exists x_{n-1} \bigwedge_{0 \leq i < j < n} x_i \neq x_j,$$

where the pairs (i, j) are ordered lexicographically and the finite conjunction uses the global right-association convention. The notation $r \neq t$ abbreviates $\neg(r = t)$. A structure satisfies every member of $T_\infty = \{\iota_n : n \geq 2\}$ iff its domain is infinite. This is an axiomatization by a theory, not by one sentence.

Example 10.6 (Definitions inside a theory)**.** Suppose T is a theory in $\mathcal{L}$, R is an n-ary relation symbol not in $\mathcal{L}$, and $\varphi(x_1, \ldots, x_n)$ is an $\mathcal{L}$-formula whose free variables are among the n distinct displayed variables. Add the *defining axiom*

$$\delta_R := \forall x_1 \cdots \forall x_n \big(R(x_1, \ldots, x_n) \leftrightarrow \varphi(x_1, \ldots, x_n)\big).$$

The passage from T to $T \cup \{\delta_R\}$ is a *definitional extension*, not a new axiom about the old signature: by theorem 10.7 below, every $\mathcal{L}$-model of T has exactly one expansion satisfying δ_R, and the extended theory has exactly the old $\mathcal{L}$-consequences.

Theorem 10.7 (Conservativity of definitional extensions)**.** *Let T, R, φ, and δ_R be as in the preceding example, and let $\mathcal{L}_R$ denote the signature extending $\mathcal{L}$ by the single relation symbol R.*

1. *Every $\mathcal{L}$-model $\mathcal{M}$ of T has exactly one $\mathcal{L}_R$-expansion satisfying δ_R, and that expansion is a model of $T \cup \{\delta_R\}$.*
2. *The extension is* conservative*: for every $\sigma \in \mathsf{Sent}(\mathcal{L})$, $T \cup \{\delta_R\} \models \sigma$ iff $T \models \sigma$.*

Status: complete proof.

Proof. Both clauses use invariance under reduct (lemma 11.2); that lemma is stated in chapter 11, and its proof uses only the satisfaction clauses, not any material of this chapter. Every member of $T \cup \{\delta_R, \sigma\}$ is a sentence, so satisfaction is assignment-independent and it suffices to reason with models.

For clause 1, let $\mathcal{M}$ be an $\mathcal{L}$-model of T. Since $\mathrm{FV}(\varphi) \subseteq \{x_1, \ldots, x_n\}$, formula coincidence (lemma 9.8) shows that whether $\mathcal{M}, s[x_1 \mapsto a_1] \cdots [x_n \mapsto a_n] \models \varphi$ holds depends only on the tuple $(a_1, \ldots, a_n)$, not on s; define $R^* \subseteq |\mathcal{M}|^n$ as the set of tuples for which it holds. Now let $\mathcal{M}'$ be any $\mathcal{L}_R$-expansion of $\mathcal{M}$, and let s be any assignment. Unwinding the satisfaction clause for $\forall$ at each of the n distinct quantified variables and then the clauses for $\leftrightarrow$, $\mathcal{M}'$ satisfies δ_R iff for every tuple $(a_1, \ldots, a_n) \in |\mathcal{M}|^n$, writing $s' = s[x_1 \mapsto a_1] \cdots [x_n \mapsto a_n]$,

$$\mathcal{M}', s' \models R(x_1, \ldots, x_n) \quad \text{iff} \quad \mathcal{M}', s' \models \varphi.$$

The atomic clause evaluates the left side as $(a_1, \ldots, a_n) \in R^{\mathcal{M}'}$, and because φ is an $\mathcal{L}$-formula, invariance under reduct evaluates the right side as $(a_1, \ldots, a_n) \in R^*$. Hence $\mathcal{M}'$ satisfies δ_R iff $R^{\mathcal{M}'} = R^*$: the expansion interpreting R as R^* is the unique one satisfying δ_R. It also satisfies every member of T, because each is an $\mathcal{L}$-sentence true in its reduct $\mathcal{M}$.

For clause 2, first suppose $T \models \sigma$, and let $\mathcal{N}$ be any model of $T \cup \{\delta_R\}$ in the signature $\mathcal{L}_R$. By invariance under reduct, $\mathcal{N} \restriction \mathcal{L}$ is an $\mathcal{L}$-model of T; hence it satisfies σ, and invariance under reduct returns $\mathcal{N} \models \sigma$. Thus $T \cup \{\delta_R\} \models \sigma$. Conversely, suppose $T \cup \{\delta_R\} \models \sigma$, and let $\mathcal{M}$ be any $\mathcal{L}$-model of T. Clause 1 expands $\mathcal{M}$ to a model of $T \cup \{\delta_R\}$, which therefore satisfies σ; since σ is an $\mathcal{L}$-sentence, invariance under reduct gives $\mathcal{M} \models \sigma$. Thus $T \models \sigma$. □

Once soundness (theorem 11.6) and completeness (theorem 12.14, under its Axiom-of-Choice hypothesis) are available, clause 2 transfers to derivability: for every $\sigma \in \mathsf{Sent}(\mathcal{L})$, $T \cup \{\delta_R\} \vdash_{\mathsf{ND}_{\mathsf{FOL}}} \sigma$ iff $T \vdash_{\mathsf{ND}_{\mathsf{FOL}}} \sigma$, since the left-to-right direction composes soundness, clause 2, and completeness, and the converse holds because a derivation from a set is a derivation from any superset.

10.1 Axiomatization errors

Warning 10.8. When writing an axiomatization, check all of the following:

1. every displayed symbol belongs to the declared signature;
2. the arity of every symbol is respected;
3. every axiom is a sentence;
4. the nonempty-domain convention is accounted for;
5. intended restrictions are written with implications for universal quantification and conjunctions for existential quantification;
6. a schema is identified as a set of sentences, not printed as though it were one first-order sentence; and
7. both directions of the claim $\mathrm{Mod}(T) = \mathcal{K}$ are proved or explained.

Chapter 11

First-Order Deduction

11.1 Natural-deduction rules for quantifiers and equality

The calculus $\mathsf{ND}_{\mathsf{FOL}}$ contains every rule of $\mathsf{ND}_{\mathsf{CPL}}$ and the following rules. Every displayed substitution is the capture-avoiding substitution of chapter 8.

$$\frac{\forall x\,\varphi}{\varphi[t/x]}\;\forall E \qquad \frac{\varphi[t/x]}{\exists x\,\varphi}\;\exists I$$

$$\frac{\varphi}{\forall x\,\varphi}\;\forall I \quad \text{provided } x \text{ is not free in any open assumption.}$$

$$\frac{\exists x\,\varphi \qquad \begin{array}{c}[\varphi[y/x]]^{i}\\ \vdots\\ \psi\end{array}}{\psi}\;\exists E \quad y \text{ is absent from } \psi\text{, from every other open assumption, and from } \exists x\,\varphi.$$

A variable subject to a freshness side condition in a rule instance is the *eigenvariable* of that instance; in $\exists E$ the eigenvariable is y, and in $\forall I$ it is the generalized variable x. The rule $\exists E$ discharges all assumptions marked i. Its freshness condition prevents the temporary witness from appearing in the conclusion or surviving assumptions.

Because formulas are strings rather than alpha-equivalence classes, the calculus also contains the explicit conversion rule

$$\frac{\varphi}{\psi}\;\alpha \quad \text{provided } \varphi \equiv_{\alpha} \psi.$$

The rule's premise and conclusion are formulas, and its side condition is the syntactic alpha-equivalence relation of section 8.4. It changes only bound-variable names, so it is neither identity elimination nor replacement by an arbitrary logically equivalent formula. The rule is required below when a fresh eigenvariable of $\forall I$ produces an alpha-equivalent, but not literally identical, quantified formula.

Equality is governed by identity introduction and substitutivity:

$$\frac{}{t=t}\;=I \qquad \frac{r=t \qquad \varphi[r/x]}{\varphi[t/x]}\;=E.$$

The formula φ may contain zero or more free occurrences of x. The rule uses actual identity in the semantics; it does not merely postulate that an arbitrary relation is reflexive.

Definition 11.1 (First-order derivability)**.** For $\Gamma \subseteq \mathsf{Fm}(\mathcal{L})$, write $\Gamma \vdash_{\mathsf{ND}_{\mathsf{FOL}}} \varphi$ iff there is a $\mathsf{ND}_{\mathsf{FOL}}$-derivation of φ whose open assumptions all belong to Γ.

Lemma 11.2 (Invariance under reduct)**.** *Let $\mathcal{L}'$ extend $\mathcal{L}$, let $\mathcal{M}'$ be a $\mathcal{L}'$-structure, and let s be a variable assignment in its domain. For every $\mathcal{L}$-term t and $\mathcal{L}$-formula φ,*

$$[\![t]\!]_s^{\mathcal{M}'} = [\![t]\!]_s^{\mathcal{M}' \restriction \mathcal{L}}, \qquad \mathcal{M}', s \models \varphi \iff \mathcal{M}' \restriction \mathcal{L}, s \models \varphi.$$

Proof. Induct on terms for the first equality. Every function symbol in the term has the same interpretation in the expansion and reduct. Then induct on formulas. The atomic cases use the term result and equality of the old relation-symbol interpretations. The clauses for $\bot$ and the connectives are immediate from the induction hypotheses. In a quantifier case the two structures have the same domain, so they range over the same elements; apply the induction hypothesis under each updated assignment. □

Example 11.3 (Universal reasoning)**.** The premises $\forall x(P(x) \to Q(x))$ and $\forall x P(x)$ derive $\forall x Q(x)$:

1.	$\forall x(P(x) \to Q(x))$	premise
2.	$\forall x P(x)$	premise
3.	$P(y) \to Q(y)$	$\forall E, 1$
4.	$P(y)$	$\forall E, 2$
5.	$Q(y)$	$\to E, 3, 4$
6.	$\forall y Q(y)$	$\forall I, 5$
7.	$\forall x Q(x)$	$\alpha, 6.$

The variable y is not free in either premise, so the side condition at line 6 is met. Line 7 applies the α rule, because $\forall y Q(y)$ and $\forall x Q(x)$ are alpha-equivalent but not identical strings.

Example 11.4 (Existential elimination)**.** From $\exists x(P(x) \wedge Q(x))$ derive $\exists x P(x)$:

1.	$\exists x(P(x) \wedge Q(x))$	premise
2.	$\quad [P(y) \wedge Q(y)]^1$	assumption, y fresh
3.	$\quad P(y)$	$\wedge E_1, 2$
4.	$\quad \exists x P(x)$	$\exists I, 3$
5.	$\exists x P(x)$	$\exists E, 1, 2$–$4.$

The eigenvariable y does not occur in the conclusion.

Example 11.5 (Symmetry of equality)**.** The rule $= E$ derives $t = r$ from $r = t$. Choose a variable x with $x \notin \mathrm{Var}(r)$; this is possible because Var is infinite and r is a finite string. From $= I$ obtain $r = r$. In the formula $\varphi := x = r$, the instance $\varphi[r/x]$ is $r = r$ and the instance $\varphi[t/x]$ is $t = r$. Apply $= E$ to $r = t$ and $r = r$.

11.2 First-order sequent rules

The first-order sequent calculus $\mathsf{LK_{FOL}}$ extends $\mathsf{LK_{CPL}}$. A sequent is satisfied by $\mathcal{M}, s$ under the same left-conjunctive and right-disjunctive reading used for propositional sequents. Its additional rules are

$$\frac{\Gamma, \varphi[t/x] \Rightarrow \Delta}{\Gamma, \forall x\, \varphi \Rightarrow \Delta}\ \forall L \qquad \frac{\Gamma \Rightarrow \Delta, \varphi[t/x]}{\Gamma \Rightarrow \Delta, \exists x\, \varphi}\ \exists R$$

$$\frac{\Gamma \Rightarrow \Delta, \varphi[y/x]}{\Gamma \Rightarrow \Delta, \forall x\, \varphi}\ \forall R \qquad \frac{\Gamma, \varphi[y/x] \Rightarrow \Delta}{\Gamma, \exists x\, \varphi \Rightarrow \Delta}\ \exists L.$$

In $\forall R$, the eigenvariable y occurs free in no formula of $\Gamma, \Delta, \forall x\, \varphi$. In $\exists L$, it occurs free in no formula of $\Gamma, \Delta, \exists x\, \varphi$.

Equality may be included by the initial sequent $\Rightarrow t = t$ and the replacement rule

$$\frac{\Gamma, \varphi[r/x] \Rightarrow \Delta}{\Gamma, r = t, \varphi[t/x] \Rightarrow \Delta} = L.$$

The symmetric replacement instance—from $\Gamma, \varphi[t/x] \Rightarrow \Delta$ infer $\Gamma, r = t, \varphi[r/x] \Rightarrow \Delta$—is derivable. First, $r = t \Rightarrow t = r$ is derivable: apply $= L$ with the auxiliary formula $t = x$, for a variable x absent from r and t, to the identity sequent $t = r \Rightarrow t = r$, obtaining $r = t$, $t = t \Rightarrow t = r$, and cut with the initial sequent $\Rightarrow t = t$. Second, the displayed rule with the roles of the two terms exchanged turns the premise into $\Gamma, t = r, \varphi[r/x] \Rightarrow \Delta$, and a cut with $r = t \Rightarrow t = r$ yields the instance. A first-order sequent derivation is a finite tree generated by the propositional and displayed first-order rules.

11.3 Soundness

Theorem 11.6 (Soundness of first-order natural deduction)**.** *For every set $\Gamma \cup \{\varphi\} \subseteq \mathsf{Fm}(\mathcal{L})$,*

$$\Gamma \vdash_{\mathsf{ND}_{\mathsf{FOL}}} \varphi \quad \Longrightarrow \quad \Gamma \models \varphi.$$

Proof. Induct on the construction of the derivation, using the strengthened statement that any structure and assignment satisfying all open assumptions satisfies the conclusion. The propositional cases are those of theorem 7.1.

For $\forall E$, if $\mathcal{M}, s \models \forall x\, \varphi$, then it satisfies φ under $s[x \mapsto [\![t]\!]^{\mathcal{M}}_s]$. The substitution lemma gives $\mathcal{M}, s \models \varphi[t/x]$. The $\exists I$ case reverses this argument: a satisfying value $[\![t]\!]^{\mathcal{M}}_s$ supplies the existential witness.

For $\forall I$, let $\mathcal{M}, s$ satisfy all open assumptions and choose an arbitrary $a \in |\mathcal{M}|$. Because x is free in no open assumption, formula coincidence shows that $s[x \mapsto a]$ also satisfies every open assumption. The induction hypothesis gives $\mathcal{M}, s[x \mapsto a] \models \varphi$. Since a was arbitrary, $\mathcal{M}, s \models \forall x\, \varphi$.

For $\exists E$, let $\mathcal{M}, s$ satisfy the surviving open assumptions and $\exists x\, \varphi$. Choose a such that $\mathcal{M}, s[x \mapsto a] \models \varphi$, and put $s' = s[y \mapsto a]$. Freshness of y, formula coincidence, and the substitution lemma give $\mathcal{M}, s' \models \varphi[y/x]$, while freshness also ensures that s' satisfies every other open assumption. The induction hypothesis for the subsidiary derivation yields $\mathcal{M}, s' \models \psi$. Because $y \notin \mathrm{FV}(\psi)$, formula coincidence gives $\mathcal{M}, s \models \psi$.

The α rule is sound by alpha-invariance. For $= I$, actual identity is reflexive. For $= E$, satisfaction of $r = t$ gives $[\![r]\!]^{\mathcal{M}}_s = [\![t]\!]^{\mathcal{M}}_s$. The substitution lemma says that the two instances of φ are evaluated under updates of s by these equal elements, so satisfaction of one instance implies satisfaction of the other. This treats every rule of $\mathsf{ND}_{\mathsf{FOL}}$ that is not a rule of $\mathsf{ND}_{\mathsf{CPL}}$, completing the induction. □

Theorem 11.7 (Soundness of the first-order sequent calculus)**.** *Every sequent derivable in $\mathsf{LK}_{\mathsf{FOL}}$ is satisfied by every $\mathcal{L}$-structure and assignment.*

Proof. Extend the induction in theorem 7.2: for each rule, if every premise is satisfied by every structure and assignment, so is the conclusion. The rules $\forall L$ and $\exists R$ use the term value and the substitution lemma.

For $\forall R$, suppose toward a contradiction that some $\mathcal{M}, s$ does not satisfy the conclusion $\Gamma \Rightarrow \Delta, \forall x\, \varphi$ while the premise $\Gamma \Rightarrow \Delta, \varphi[y/x]$ is satisfied by every structure and assignment. Then $\mathcal{M}, s$ satisfies every formula in $\mathrm{supp}(\Gamma)$, no formula in $\mathrm{supp}(\Delta)$, and not $\forall x\, \varphi$. The universal satisfaction clause supplies an element $a \in |\mathcal{M}|$ with $\mathcal{M}, s[x \mapsto a] \not\models \varphi$. Put $s' = s[y \mapsto a]$, where y is the eigenvariable.

First, $\mathcal{M}, s' \not\models \varphi[y/x]$. The substitution lemma (theorem 9.9) gives $\mathcal{M}, s' \models \varphi[y/x]$ iff $\mathcal{M}, s'[x \mapsto [\![y]\!]^{\mathcal{M}}_{s'}] \models \varphi$, and $[\![y]\!]^{\mathcal{M}}_{s'} = s'(y) = a$. If $y = x$, then $s'[x \mapsto a] = s[x \mapsto a]$ and the right side fails directly. If $y \neq x$, then the eigenvariable condition—y occurs free in no formula of $\Gamma, \Delta, \forall x\, \varphi$—gives $y \notin \mathrm{FV}(\varphi)$, so $s'[x \mapsto a]$ and $s[x \mapsto a]$ agree on $\mathrm{FV}(\varphi)$, and formula coincidence (lemma 9.8) reduces the right side to $\mathcal{M}, s[x \mapsto a] \models \varphi$, which fails.

Second, the same eigenvariable condition puts y outside the free variables of every formula of Γ and Δ, so s and s' agree on those free variables, and formula coincidence transfers the pattern above to s': $\mathcal{M}, s'$ satisfies every formula in $\mathrm{supp}(\Gamma)$ and no formula in $\mathrm{supp}(\Delta)$. Together with $\mathcal{M}, s' \not\models \varphi[y/x]$, this means $\mathcal{M}, s'$ does not satisfy the premise, a contradiction. The eigenvariable condition is what licenses both uses of formula coincidence; without it, updating y could change the truth values of the contexts or of φ itself.

The $\exists L$ case is dual. The equality initial sequent $\Rightarrow t = t$ is satisfied by every structure and assignment because actual identity is reflexive, and $= L$ preserves satisfaction by every structure and assignment by the same equal-update argument used for $= E$ in theorem 11.6. □

11.4 Quantifier-rule errors

Warning 11.8. None of the following inference steps is licensed by the rules of $\mathsf{ND}_{\mathsf{FOL}}$, and each associated semantic-consequence claim fails:

1. from the open assumption $P(x)$, infer $\forall x P(x)$;
2. from $\exists x P(x)$, introduce a permanent constant c and conclude $P(c)$;
3. in $\exists E$, allow the eigenvariable to occur in the conclusion;
4. from $\forall x \exists y R(x, y)$, infer $\exists y \forall x R(x, y)$; and
5. replace a free variable by a term using textual substitution that permits capture.

The first and third violate explicit side conditions. The second treats an existential witness as a named element outside its subsidiary derivation. The fourth reverses witness dependence. The fifth changes formula meaning.

11.5 Exercises

Exercise 11.9. Compute $\mathrm{FV}(\varphi)$ for

$$\varphi = P(x, z) \wedge \forall x\big(Q(x, y) \rightarrow \exists y R(x, y, z)\big).$$

Identify the quantifier binding each bound occurrence.

Exercise 11.10. Compute capture-avoiding substitutions

$$(\exists y R(x, y))[y/x] \quad \text{and} \quad (\forall x(P(x) \rightarrow Q(y)))[f(x)/y].$$

Record every bound-variable renaming.

Exercise 11.11. Prove term coincidence and formula coincidence simultaneously for a language with one binary function and one binary relation, then explain why the same proof covers arbitrary signatures.

Exercise 11.12. Give a $\mathsf{ND}_{\mathsf{FOL}}$-derivation of

$$\forall x(P(x) \rightarrow Q(x)),\ \exists x P(x) \vdash_{\mathsf{ND}_{\mathsf{FOL}}} \exists x Q(x).$$

Mark the eigenvariable and every discharged assumption.

Exercise 11.13. Derive transitivity of equality from $= I$ and $= E$.

Exercise 11.14. Translate each sentence using one declared signature: “Every reviewer read exactly one paper”; “One paper was read by every reviewer”; “No reviewer read only papers that cited themselves.” State the domain.

Exercise 11.15. Prove both directions of the claim that the displayed group theory axiomatizes groups in the signature $\mathcal{L}_{\mathrm{grp}}$.

Exercise 11.16. Write a theory whose models are simple undirected graphs. Treat equality as logical identity, and state whether loops are permitted.

Exercise 11.17. Give a cut-free $\mathsf{LK}_{\mathsf{FOL}}$-derivation of $\forall x P(x) \Rightarrow \exists x P(x)$. Identify where the nonempty-domain convention is used.

Chapter 12

Henkin Completeness and Compactness

The completeness theorem for classical first-order logic is Gödel's [24]. This chapter develops the construction introduced by Henkin [27], which extracts a term structure from a maximal consistent, witness-saturated set.

12.1 Syntactic preparation

Definition 12.1 (First-order consistency)**.** A set $\Gamma \subseteq \mathsf{Fm}(\mathcal{L})$ is *consistent* iff $\Gamma \nvdash_{\mathsf{ND}_{\mathsf{FOL}}} \bot$. It is *maximal consistent* iff it is consistent and $\Gamma \cup \{\varphi\}$ is inconsistent for every $\varphi \notin \Gamma$.

The closure properties in lemma 7.5 also hold for a maximal consistent $\Delta \subseteq \mathsf{Fm}(\mathcal{L})$, but their first-order proof does not silently graft derivations across eigenvariable rules. For deductive closure, suppose $\Delta \vdash_{\mathsf{ND}_{\mathsf{FOL}}} \varphi$ and $\varphi \notin \Delta$. Maximality supplies a derivation of $\bot$ from $\Delta \cup \{\varphi\}$. Because Δ is consistent, at least one open occurrence of the added assumption remains in that derivation. Apply $\neg I$ at its root, discharging every such occurrence, to obtain $\Delta \vdash_{\mathsf{ND}_{\mathsf{FOL}}} \neg\varphi$. Combining this derivation with the given derivation of φ by $\neg E$ derives $\bot$ from Δ, contradicting consistency. Clauses 2–5 of lemma 7.5 then use this deductive-closure result and only the displayed propositional rules, so their proofs transfer without an eigenvariable-renaming step.

Lemma 12.2 (Classical quantifier duality)**.** *The calculus* $\mathsf{ND}_{\mathsf{FOL}}$ *derives*

$$\neg\forall x\,\varphi \leftrightarrow \exists x\,\neg\varphi \quad \textit{and} \quad \neg\exists x\,\varphi \leftrightarrow \forall x\,\neg\varphi.$$

Proof. For $\neg\exists x\,\varphi \to \forall x\neg\varphi$, take a variable y absent from φ and free in no open assumption. Assume $\varphi[y/x]$; existential introduction contradicts $\neg\exists x\varphi$. Negation introduction gives $\neg\varphi[y/x]$, and universal introduction gives $\forall y\,\neg\varphi[y/x]$. Since y is fresh for φ, the α rule gives $\forall x\neg\varphi$. The reverse implication follows by existential elimination: a witness to φ contradicts the corresponding instance of $\forall x\neg\varphi$.

For $\exists x\neg\varphi \to \neg\forall x\varphi$, eliminate the existential and contradict the corresponding universal instance. For the remaining direction, assume $\neg\forall x\varphi$ and, for reductio, $\neg\exists x\neg\varphi$. The already proved implication gives $\forall x\neg\neg\varphi$. For a variable y absent from φ and free in no open assumption, double-negation elimination gives $\varphi[y/x]$; universal introduction gives $\forall y\,\varphi[y/x]$, and the α rule gives $\forall x\varphi$, contradicting the first assumption. Reductio yields $\exists x\neg\varphi$. Every eigenvariable is chosen absent from the surviving assumptions and conclusion. □

Lemma 12.3 (Fresh-constant lemma)**.** *Let c be a constant symbol occurring in neither Γ nor φ. If $\Gamma \vdash_{\mathsf{ND_{FOL}}} \varphi[c/x]$, then $\Gamma \vdash_{\mathsf{ND_{FOL}}} \forall x\varphi$.*

Proof. Fix a derivation witnessing the premise. Because the derivation is finite, only a finite set $\Gamma_0 \subseteq \Gamma$ occurs among its open assumptions. Choose a variable y absent from Γ_0, from φ, and from the finite derivation. Uniformly replace c by y in that derivation. Induction on the derivation shows that every rule instance remains an instance: c has no binding behavior, it occurs in no open assumption, and y occurred in no original rule instance, so every eigenvariable condition is preserved. Hence $\Gamma_0 \vdash_{\mathsf{ND_{FOL}}} \varphi[y/x]$. Since y is free in no member of Γ_0, $\forall I$ gives $\Gamma_0 \vdash_{\mathsf{ND_{FOL}}} \forall y\, \varphi[y/x]$. Because $y \notin \mathrm{Var}(\varphi)$, this conclusion is alpha-equivalent to $\forall x\varphi$; the α rule therefore gives the latter. The same finite derivation also witnesses $\Gamma \vdash_{\mathsf{ND_{FOL}}} \forall x\varphi$. □

Definition 12.4 (Witness property)**.** A set $\Delta \subseteq \mathsf{Fm}(\mathcal{L})$ has the *witness property* iff, for every formula $\exists x\varphi \in \Delta$, there is a constant symbol c of $\mathcal{L}$ such that $\varphi[c/x] \in \Delta$.

The witness property is necessary in the term-structure construction: membership of an existential formula must supply a term naming a witness.

Lemma 12.5 (Witness-extension lemma)**.** *Let Σ be consistent, let c occur in neither Σ nor $\exists x\varphi$, and put*

$$H_{x,\varphi,c} := \exists x\varphi \to \varphi[c/x].$$

Then $\Sigma \cup \{H_{x,\varphi,c}\}$ is consistent.

Proof. Suppose the extension is inconsistent. Negation introduction gives $\Sigma \vdash \neg H_{x,\varphi,c}$. Classical propositional derivations then give

$$\Sigma \vdash \exists x\varphi \quad \text{and} \quad \Sigma \vdash \neg\varphi[c/x],$$

because $\neg(A \to B) \vdash A \wedge \neg B$. The fresh-constant lemma applied to the second derivation gives $\Sigma \vdash \forall x\neg\varphi$. Quantifier duality gives $\Sigma \vdash \neg\exists x\varphi$, contradicting the first derivation. Thus the extension is consistent. □

Lemma 12.6 (Eliminating newly added constants)**.** *Let $\mathcal{L}'$ extend $\mathcal{L}$ only by constant symbols, and let $\Gamma \cup \{\varphi\} \subseteq \mathsf{Fm}(\mathcal{L})$. If there is an $\mathsf{ND_{FOL}}$-derivation of φ from Γ in the expanded signature $\mathcal{L}'$, then there is such a derivation in $\mathcal{L}$.*

Proof. Fix an expanded-signature derivation. It is finite, so only finitely many constants from $\mathcal{L}' \setminus \mathcal{L}$, say $d_1, \ldots, d_n$, occur in it. Choose distinct variables $z_1, \ldots, z_n$ absent from every formula and rule annotation in the derivation. Simultaneously replace each d_i by z_i throughout the derivation.

Induction on the derivation verifies that every inference remains an instance of the same rule. Replacement commutes with the propositional constructors and with term formation. Because the z_i occurred in no original binder, replacement causes no capture, and for every quantifier instance it commutes with capture-avoiding substitution. The eigenvariables of $\forall I$ and of $\exists E$ are original variables, distinct from all z_i; replacing constants therefore preserves their absence conditions. Replacement also preserves alpha-equivalence and every identity-rule instance. The end formula and open assumptions contain none of the d_i, so they are unchanged. The transformed tree is consequently a $\mathcal{L}$-derivation of φ from Γ. □

Lemma 12.7 (Transfinite Lindenbaum lemma)**.** *Let $\mathcal{L}'$ be a signature, let λ be an ordinal, and let $\langle \varphi_\alpha : \alpha < \lambda \rangle$ be an enumeration of $\mathsf{Fm}(\mathcal{L}')$ in which every formula occurs at least once. Then every set $\Sigma \subseteq \mathsf{Fm}(\mathcal{L}')$ that is consistent for $\mathsf{ND_{FOL}}$ is contained in a maximal consistent $\Delta \subseteq \mathsf{Fm}(\mathcal{L}')$. The construction is choice-free given the enumeration; the Axiom of Choice enters only when such an enumeration must first be produced. Status: complete proof.*

Proof. By theorem 1.24 with domain $\lambda + 1$, define a sequence $\langle \Gamma_\alpha : \alpha \leq \lambda \rangle$ by

$$\Gamma_0 = \Sigma, \qquad \Gamma_{\alpha+1} = \begin{cases} \Gamma_\alpha \cup \{\varphi_\alpha\}, & \text{if this set is consistent,} \\ \Gamma_\alpha, & \text{otherwise,} \end{cases} \qquad \Gamma_\beta = \bigcup_{\alpha<\beta} \Gamma_\alpha \text{ for limit } \beta.$$

Chain claim: if $\alpha \leq \beta \leq \lambda$, then $\Gamma_\alpha \subseteq \Gamma_\beta$. Proceed by transfinite induction on β: the claim is trivial at $\beta = \alpha$, both successor clauses extend the preceding stage, and a limit stage is the union of all earlier stages.

Consistency: every Γ_α is consistent, by transfinite induction on α. Stage zero is the hypothesis on Σ, and the successor clause preserves consistency by its case split. At a limit β, suppose $\Gamma_\beta \vdash_{\mathsf{ND}_{\mathsf{FOL}}} \bot$. The derivation is finite, so its open assumptions form a finite $F \subseteq \Gamma_\beta$. Each $\psi \in F$ lies in some Γ_{α_ψ} with $\alpha_\psi < \beta$. If $F \neq \varnothing$, put $\alpha^* = \max_{\psi \in F} \alpha_\psi < \beta$, which exists because a finite nonempty set of ordinals has a maximum; if $F = \varnothing$, put $\alpha^* = 0 < \beta$. The chain claim locates $F \subseteq \Gamma_{\alpha^*}$, so Γ_{α^*} is inconsistent, contradicting the induction hypothesis at α^*.

Maximality: put $\Delta = \Gamma_\lambda$; it is consistent, and it contains Σ by the chain claim. Suppose $\varphi \notin \Delta$ and pick $\alpha < \lambda$ with $\varphi_\alpha = \varphi$. Rejection at stage $\alpha + 1$ means that $\Gamma_\alpha \cup \{\varphi\}$ derives $\bot$. A derivation from a set is a derivation from any superset, because derivability from a set requires only that every open assumption belong to it; hence $\Delta \cup \{\varphi\}$ derives $\bot$. Therefore Δ is maximal consistent. □

Lemma 12.8 (Henkin–Lindenbaum lemma)**.** *Let $\mathcal{L}$ be a signature and assume the Axiom of Choice in the ambient set theory. Every consistent $\Sigma \subseteq \mathsf{Fm}(\mathcal{L})$ is contained in a maximal consistent set Δ in an expanded signature $\mathcal{L}^H$, and Δ has the witness property. Status: complete proof.*

Proof. Let $\kappa = \max(|\mathcal{L}|, \aleph_0)$. Add new constant symbols $\langle d_\xi : \xi < \kappa \rangle$. Regard Σ as a set of formulas in the expanded signature. It remains consistent there: otherwise lemma 12.6 would transform an expanded-signature derivation of $\bot$ into an original-signature derivation from Σ. By corollary 1.29, the expanded language has at most κ formulas. The κ formulas $\exists x_0 (x_0 = d_\xi)$, for $\xi < \kappa$, are pairwise distinct, so it has exactly κ formulas and exactly κ existential formulas. Since a cardinal is an ordinal by definition, fix a bijective enumeration

$$\langle E_\alpha : \alpha < \kappa \rangle, \qquad E_\alpha = \exists x_{n_\alpha} \varphi_\alpha \text{ with } n_\alpha \in \mathbb{N},$$

of its existential formulas. By the transfinite-recursion theorem theorem 1.24, at stage $\alpha < \kappa$ add a witness implication $H_{x_{n_\alpha}, \varphi_\alpha, c_\alpha}$, where c_α is the least d_ξ, in the displayed index order, that is absent from the current theory and from E_α. This least-available convention makes the recursive clause functional. Before stage α, fewer than κ finite witness formulas have been added. By theorem 1.28, their union contains fewer than κ constants; E_α contains only finitely many constants. Therefore one of the κ new constants remains available. The witness-extension lemma preserves consistency at successor stages. At a limit stage take the union; any derivation of $\bot$ would use finitely many assumptions and would already occur at an earlier stage. Let Σ^H be the resulting consistent set.

The set $\mathsf{Fm}(\mathcal{L}^H)$ has cardinality exactly κ, as established above from corollary 1.29 and the formulas $\exists x_0 (x_0 = d_\xi)$. By corollary 1.27, under the Axiom of Choice already assumed, fix a bijective enumeration $\langle \varphi_\alpha : \alpha < \kappa \rangle$ of $\mathsf{Fm}(\mathcal{L}^H)$. Apply lemma 12.7 to Σ^H under this enumeration to obtain a maximal consistent $\Delta \supseteq \Sigma^H \supseteq \Sigma$. Every witness implication remains in Δ. If $\exists x \varphi \in \Delta$, implication elimination and deductive closure put its designated instance $\varphi[c/x]$ in Δ. Hence Δ has the witness property. □

12.2 The term structure

Fix a maximal consistent $\Delta \subseteq \mathsf{Fm}(\mathcal{L}^H)$ with the witness property. Define a relation on $\mathsf{Term}(\mathcal{L}^H)$ by

$$r \sim_\Delta t \quad \text{iff} \quad \Delta \vdash_{\mathsf{ND}_{\mathsf{FOL}}} r = t.$$

Lemma 12.9 (Equality congruence)**.** *The relation $\sim_\Delta$ is an equivalence relation. If $n \in \mathbb{N}$, $r_i, t_i \in \mathsf{Term}(\mathcal{L}^H)$, and $r_i \sim_\Delta t_i$ for $1 \le i \le n$, then for every n-ary $f \in \mathcal{F}$,*

$$f(r_1, \ldots, r_n) \sim_\Delta f(t_1, \ldots, t_n),$$

and for every n-ary $R \in \mathcal{R}$,

$$R(r_1, \ldots, r_n) \in \Delta \quad \textit{iff} \quad R(t_1, \ldots, t_n) \in \Delta.$$

Proof. Reflexivity is $= I$. Symmetry was derived in chapter 11; transitivity follows by applying $= E$ to the formula $r = x$. Repeated applications of $= E$, one argument at a time, give compatibility with every function and relation symbol. Maximal consistent sets are deductively closed, so each derived replacement belongs to Δ. Symmetry supplies the reverse relational implication. □

Definition 12.10 (Term structure)**.** The *term structure* $\mathcal{M}_\Delta$ has domain

$$|\mathcal{M}_\Delta| = \mathsf{Term}(\mathcal{L}^H)/{\sim_\Delta}.$$

Write $[t]$ for the class of t. Interpret symbols by

$$f^{\mathcal{M}_\Delta}([t_1], \ldots, [t_n]) = [f(t_1, \ldots, t_n)]$$

and

$$R^{\mathcal{M}_\Delta}([t_1], \ldots, [t_n]) \quad \text{iff} \quad R(t_1, \ldots, t_n) \in \Delta.$$

Let s_0 be the variable assignment $s_0(x) = [x]$.

Lemma 12.9 proves that these interpretations are independent of the chosen representatives. The domain is nonempty because $\mathcal{L}^H$ has constant symbols.

Lemma 12.11 (Term evaluation in the term structure)**.** *For every term t,*

$$[\![t]\!]^{\mathcal{M}_\Delta}_{s_0} = [t].$$

Proof. Induct on t. For a variable this is the definition of s_0; the function case is the definition of $f^{\mathcal{M}_\Delta}$ applied to the induction hypotheses. □

Lemma 12.12 (Quantifier membership)**.** *For every formula φ and variable x:*
1. $\exists x\varphi \in \Delta$ iff $\varphi[t/x] \in \Delta$ for some term t; and
2. $\forall x\varphi \in \Delta$ iff $\varphi[t/x] \in \Delta$ for every term t.

Proof. If an existential formula belongs to Δ, the witness property supplies a constant instance. Conversely, $\exists I$ derives the existential from any term instance, and deductive closure places it in Δ.

Universal elimination proves the forward implication in 2. Conversely, suppose $\forall x\varphi \notin \Delta$. Maximal consistency gives $\neg\forall x\varphi \in \Delta$; quantifier duality and deductive closure give $\exists x\neg\varphi \in \Delta$. By the witness property, $\neg\varphi[c/x] \in \Delta$ for some constant c, so consistency gives $\varphi[c/x] \notin \Delta$. Therefore not every term instance belongs to Δ. □

Lemma 12.13 (First-order truth lemma)**.** *For every $\mathcal{L}^H$-formula φ,*

$$\mathcal{M}_\Delta, s_0 \models \varphi \quad \textit{iff} \quad \varphi \in \Delta.$$

Proof. Proceed by strong induction on $c(\varphi)$. For equality, term evaluation gives

$$\mathcal{M}_\Delta, s_0 \models r = t \quad \text{iff} \quad [r] = [t] \quad \text{iff} \quad \Delta \vdash r = t \quad \text{iff} \quad r = t \in \Delta,$$

where the last equivalence is deductive closure and the definition of $\sim_\Delta$. The relation case follows directly from the interpretation of $R^{\mathcal{M}_\Delta}$, and $\bot \notin \Delta$ by consistency. The Boolean cases use the propositional closure properties of maximal consistent sets exactly as in lemma 7.7.

For the existential case, term-structure evaluation and the substitution lemma (theorem 9.9) give, for every term t,

$$\mathcal{M}_\Delta, s_0[x \mapsto [t]] \models \varphi \quad \text{iff} \quad \mathcal{M}_\Delta, s_0 \models \varphi[t/x].$$

By lemma 8.11, every instance $\varphi[t/x]$ has complexity $c(\varphi)$, which is smaller than the complexity of $\exists x\varphi$. The strong induction hypothesis and lemma 12.12(1) therefore show that some class $[t]$ satisfies the matrix iff $\exists x\varphi \in \Delta$. The universal case uses "every" and lemma 12.12(2). Every element of the term structure has the form $[t]$, so all semantic quantifier cases are covered. □

12.3 Completeness

Theorem 12.14 (Strong completeness of classical first-order logic)**.** *Let $\mathcal{L}$ be a signature and assume the Axiom of Choice in the ambient set theory. For every $\Gamma \cup \{\varphi\} \subseteq \mathsf{Fm}(\mathcal{L})$,*

$$\Gamma \models \varphi \quad \Longrightarrow \quad \Gamma \vdash_{\mathsf{ND}_{\mathsf{FOL}}} \varphi.$$

Proof. We prove the contrapositive. If $\Gamma \nvdash \varphi$, then $\Sigma = \Gamma \cup \{\neg\varphi\}$ is consistent. For suppose a derivation D of $\bot$ has open assumptions in Σ. If no open occurrence of $\neg\varphi$ appeared in D, then D would witness $\Gamma \vdash \bot$, and $\bot E$ would give $\Gamma \vdash \varphi$; otherwise mark all open occurrences of $\neg\varphi$ in D with a common fresh label and close D by RAA at the root, discharging every such occurrence, to derive φ from Γ. Either way $\Gamma \vdash \varphi$, contrary to supposition. Apply the Henkin–Lindenbaum lemma to obtain a maximal consistent witness set $\Delta \supseteq \Sigma$. Its term structure and canonical assignment satisfy every member of Δ by the truth lemma. In particular, $\mathcal{M}_\Delta, s_0 \models \Gamma$ and $\mathcal{M}_\Delta, s_0 \models \neg\varphi$. Thus the reduct $\mathcal{M}_\Delta \restriction \mathcal{L}$, with the same assignment s_0, satisfies Γ and falsifies φ by lemma 11.2. It is a $\mathcal{L}$-structure, so $\Gamma \not\models \varphi$ under the semantic consequence relation stated in the theorem. □

Together with soundness, completeness gives

$$\Gamma \vdash_{\mathsf{ND}_{\mathsf{FOL}}} \varphi \quad \text{iff} \quad \Gamma \models \varphi.$$

This equivalence concerns the specified calculus and the set-based semantics with nonempty domains. It does not identify syntactic derivability with semantic truth as objects.

Corollary 12.15 (Completeness for countable signatures, without choice)**.** *Let $\mathcal{L}$ be a signature whose set of function and relation symbols is countable in the sense of section 1.5, finite signatures included. Then, for every $\Gamma \cup \{\varphi\} \subseteq \mathsf{Fm}(\mathcal{L})$,*

$$\Gamma \models \varphi \quad \Longrightarrow \quad \Gamma \vdash_{\mathsf{ND}_{\mathsf{FOL}}} \varphi,$$

and no use is made of the Axiom of Choice. This is the countable-language version of the completeness theorem promised in section 1.5. Status: complete proof.

Proof. Run the proof of theorem 12.14 unchanged, replacing each input that the Axiom of Choice supplied with an explicit one. Choice enters that proof only through lemma 12.8, at exactly two points: the cardinal bookkeeping at $\kappa = \max(|\mathcal{L}|, \aleph_0)$ through corollary 1.27, theorem 1.28, and corollary 1.29; and the bijective enumeration of $\mathsf{Fm}(\mathcal{L}^H)$ handed to lemma 12.7, whose own construction is choice-free given the enumeration, as its statement records. The enumeration of the existential formulas $\langle E_\alpha \rangle$ is an instance of the same second point.

Both inputs are explicit here. Fix an injection ι of the symbol set of $\mathcal{L}$ into $\mathbb{N}$; one exists by the definition of countable set, and fixing a single witness of one existential statement is not an application of the Axiom of Choice. Take $\kappa = \aleph_0$ and add the witness constants $\langle d_k : k \in \mathbb{N} \rangle$. The alphabet of $\mathcal{L}^H$ is the union of three explicitly enumerated families: the finitely many logical symbols and punctuation marks together with the variables in the fixed order $x_0, x_1, \ldots$ declared in chapter 8; the symbols of $\mathcal{L}$ in the order given by ι; and the constants d_k in index order. Sending the kth member of the three families to $3k$, $3k+1$, and $3k+2$ respectively injects the alphabet into $\mathbb{N}$, and the string coding of chapter 3—iterating $\mathrm{pair}_{\mathbb{N}}$ as in the countability-of-finite-strings lemma—then assigns every formula of $\mathcal{L}^H$ a distinct natural number code. Define φ_n to be the formula whose code is the nth smallest member of the set of formula codes; recursion on $\mathbb{N}$ with the least-number principle produces this bijective enumeration of $\mathsf{Fm}(\mathcal{L}^H)$ without choice, and its restriction to the existential formulas enumerates them likewise. The transfinite recursions of lemma 12.8 and lemma 12.7 then run along ω and $\omega + 1$ with functional clauses. At $\kappa = \aleph_0$ the cardinal-arithmetic appeals reduce to finite counting and to the displayed coding: before any stage n of the witness recursion, only finitely many witness implications have been added, so they contain only finitely many of the d_k and a least available one exists; and the code injection directly witnesses that $\mathsf{Fm}(\mathcal{L}^H)$ is countably infinite.

No other ingredient of the completeness proof invokes a choice principle: lemma 12.6 and lemma 12.5 manipulate single finite derivations; the term structure interprets symbols on equivalence classes with representative independence proved by lemma 12.9; the canonical assignment is $s_0(x) = [x]$; and the truth lemma and lemma 11.2 are inductions all of whose clauses are explicit. The contrapositive argument of theorem 12.14 therefore goes through verbatim for countable $\mathcal{L}$ with no appeal to the Axiom of Choice. □

Theorem 12.16 (Completeness of the first-order sequent calculus)**.** *Assume the Axiom of Choice in the ambient set theory. For all finite multisets Γ, Δ of $\mathcal{L}$-formulas, the sequent $\Gamma \Rightarrow \Delta$ is derivable in $\mathsf{LK}_{\mathsf{FOL}}$ iff for every $\mathcal{L}$-structure $\mathcal{M}$ and assignment s: if $\mathcal{M}, s \models \gamma$ for every $\gamma \in \mathrm{supp}(\Gamma)$, then $\mathcal{M}, s \models \delta$ for some $\delta \in \mathrm{supp}(\Delta)$. Status: proof sketch.*

Proof sketch. The exact scope is the cut-admitting calculus of chapter 11. Left to right is theorem 11.7, whose satisfaction condition on a sequent is exactly the one displayed here; at first order the semantic data are a structure together with an assignment.

For right to left, first record the translation subroutine. Induction on a $\mathsf{ND}_{\mathsf{FOL}}$-derivation translates the propositional rules as in the propositional completeness proof, translates $\forall I$ and $\exists E$ using the corresponding eigenvariable rules, translates $\forall E$, $\exists I$, and equality rules directly, and translates the α rule by an induction on the generating alpha-equivalence steps using the sequent quantifier rules. Cut composes translated elimination steps. Bound-variable alpha-equivalence and the separate eigenvariable freshness conditions are preserved by the relevant subinductions. The induction establishes that if $\Gamma_0 \vdash_{\mathsf{ND}_{\mathsf{FOL}}} \varphi$ for a finite set Γ_0, then the sequent $\Gamma_0 \Rightarrow \varphi$ is derivable in $\mathsf{LK}_{\mathsf{FOL}}$. The individual translation trees and those subinductions are omitted.

Now suppose the semantic condition holds, and list $\mathrm{supp}(\Delta) = \{\delta_1, \ldots, \delta_m\}$ with $m \geq 0$.

1. Let S be the set of members of $\mathrm{supp}(\Gamma)$ together with $\neg\delta_1, \ldots, \neg\delta_m$. A structure and assignment

satisfying all of S would satisfy every member of $\mathrm{supp}(\Gamma)$, hence some δ_j together with $\neg\delta_j$. So no structure and assignment satisfies S, and $S \models \bot$.

2. Theorem 12.14 gives $S \vdash_{\mathsf{ND}_{\mathsf{FOL}}} \bot$. The open assumptions of that derivation form a finite subset of S, say $\gamma_1, \ldots, \gamma_k \in \mathrm{supp}(\Gamma)$ and $\neg\delta_{j_1}, \ldots, \neg\delta_{j_r}$. The translation subroutine turns it into an $\mathsf{LK}_{\mathsf{FOL}}$-derivation of $\gamma_1, \ldots, \gamma_k, \neg\delta_{j_1}, \ldots, \neg\delta_{j_r} \Rightarrow \bot$.
3. Cut with the initial $\bot$-sequent $\bot \Rightarrow$, the instance of $\Gamma, \bot \Rightarrow \Delta$ with both contexts empty, empties the succedent.
4. For each j, the sequent $\Rightarrow \delta_j, \neg\delta_j$ is derivable: start from the identity sequent $\delta_j \Rightarrow \delta_j$ and apply $\neg R$. Cutting it against the current sequent, with cut formula $\neg\delta_j$,

$$\frac{\Rightarrow \delta_j, \neg\delta_j \qquad \Gamma', \neg\delta_j \Rightarrow \Pi}{\Gamma' \Rightarrow \delta_j, \Pi}\ \textsc{Cut},$$

   moves δ_j into the succedent; after r such cuts the derived sequent is $\gamma_1, \ldots, \gamma_k \Rightarrow \delta_{j_1}, \ldots, \delta_{j_r}$.
5. Weakening (W_L, W_R) adds the unused members of Γ and Δ, and contraction (C_L, C_R) merges duplicates, giving exactly the multiset sequent $\Gamma \Rightarrow \Delta$.

When $m = 0$, the hypothesis asserts that no structure and assignment satisfies every member of $\mathrm{supp}(\Gamma)$, step 4 performs no cuts, and the same route derives $\Gamma \Rightarrow \Delta$ with empty succedent; an empty Γ is likewise covered. The route uses cut, which is primitive in the displayed calculus, and makes no cut-free claim; extending the subroutine's translation induction to multi-succedent targets is unnecessary, because the reduction reuses it verbatim. The status is a sketch because the individual translation trees and subinductions of the subroutine remain omitted; the argument establishes completeness but not cut elimination. □

12.4 Compactness

Theorem 12.17 (First-order compactness)**.** *Assume the Axiom of Choice in the ambient set theory. For every $T \subseteq \mathsf{Sent}(\mathcal{L})$ and every $\sigma \in \mathsf{Sent}(\mathcal{L})$:*
1. T has a model iff every finite subset of T has a model; and
2. if $T \models \sigma$, then some finite $T_0 \subseteq T$ satisfies $T_0 \models \sigma$.
Clause 1 is the model form of compactness and clause 2 is the consequence form; they parallel the two clauses of theorem 7.9.

Proof. For clause 1, left to right is immediate: a model of T is a model of each of its finite subsets. For right to left, suppose every finite subset of T has a model. If T were inconsistent, a finite derivation of $\bot$ would use only a finite set $T_0 \subseteq T$ of assumptions. Soundness (theorem 11.6) would make T_0 unsatisfiable, contrary to hypothesis. Thus T is consistent. The Henkin term-structure construction gives an $\mathcal{L}^H$-structure satisfying T. Its $\mathcal{L}$-reduct is a model of T by lemma 11.2.

For clause 2, suppose $T \models \sigma$. Completeness (theorem 12.14) gives a finite derivation of σ from T. Let T_0 be the finite set of assumptions used; soundness gives $T_0 \models \sigma$. □

Corollary 12.18. *Assume the Axiom of Choice in the ambient set theory. The class of finite structures in the empty signature is not axiomatizable by a first-order theory.*

Proof. Suppose T had exactly the finite nonempty structures as models. Add the sentences ι_n from chapter 10. Every finite subset of $T \cup T_\infty$ has a sufficiently large finite model of T. By compactness, the union has a model. Its satisfaction of every ι_n makes it infinite, while its satisfaction of T makes it finite, a contradiction. □

Chapter 13

Cardinality, Normal Forms, and Resolution

13.1 Elementary substructures

Definition 13.1 (Substructure)**.** Let $\mathcal{M}$ and $\mathcal{N}$ be $\mathcal{L}$-structures. Then $\mathcal{N}$ is a *substructure* of $\mathcal{M}$, written $\mathcal{N} \subseteq \mathcal{M}$, iff:

1. $|\mathcal{N}| \subseteq |\mathcal{M}|$;
2. every constant has the same value in both structures;
3. $|\mathcal{N}|$ is closed under every function of $\mathcal{M}$, and each function of $\mathcal{N}$ is the corresponding restriction; and
4. every relation of $\mathcal{N}$ is the restriction of the corresponding relation of $\mathcal{M}$ to $|\mathcal{N}|$.

Definition 13.2 (Elementary substructure)**.** A substructure $\mathcal{N} \subseteq \mathcal{M}$ is an *elementary substructure*, written $\mathcal{N} \preccurlyeq \mathcal{M}$, iff for every formula $\varphi(x_1, \ldots, x_n)$ and every $a_1, \ldots, a_n \in |\mathcal{N}|$,

$$\mathcal{N} \models \varphi(a_1, \ldots, a_n) \quad \text{iff} \quad \mathcal{M} \models \varphi(a_1, \ldots, a_n).$$

The notation with parameters abbreviates satisfaction under assignments that map the displayed variables to the displayed elements.

Theorem 13.3 (Tarski–Vaught test)**.** *Let $\mathcal{N} \subseteq \mathcal{M}$. Then $\mathcal{N} \preccurlyeq \mathcal{M}$ iff, for every formula $\varphi(y, x_1, \ldots, x_n)$ and every tuple $\bar{a} \in |\mathcal{N}|^n$,*

$$\mathcal{M} \models \exists y\, \varphi(y, \bar{a}) \quad \Longrightarrow \quad \mathcal{M} \models \varphi(b, \bar{a}) \text{ for some } b \in |\mathcal{N}|.$$

Proof. If $\mathcal{N} \preccurlyeq \mathcal{M}$ and the existential formula with parameters is satisfied in $\mathcal{M}$, it is satisfied in $\mathcal{N}$; its witness belongs to $|\mathcal{N}|$.

Conversely, assume the witness condition and prove elementarity by structural induction on formulas. Term values of tuples from $\mathcal{N}$ agree in the two structures because $\mathcal{N}$ is closed under the functions. Atomic formulas therefore agree because relations are restricted. The Boolean cases follow from the induction hypotheses. If an existential formula with parameters from $\mathcal{N}$ is true in $\mathcal{N}$, its witness is also in $\mathcal{M}$, and the induction hypothesis transfers the matrix. If it is true in $\mathcal{M}$, the assumed condition supplies a witness in $\mathcal{N}$, and the induction hypothesis transfers the matrix back. For a universal formula $\forall y\, \varphi$ with parameters from $\mathcal{N}$: if it is true in $\mathcal{M}$, then every instance at an element of $|\mathcal{N}|$ is true in $\mathcal{M}$, and the induction hypothesis for φ transfers each instance to $\mathcal{N}$. If it is false in $\mathcal{M}$, the satisfaction clauses give $\mathcal{M} \models \exists y\, \neg\varphi$ at the same parameters; the witness condition supplies a witness in $|\mathcal{N}|$, the induction hypothesis for φ transfers the negated instance to $\mathcal{N}$, and $\forall y\, \varphi$ is false in $\mathcal{N}$. □

13.2 The Löwenheim–Skolem theorems

Theorem 13.4 (Downward Löwenheim–Skolem)**.** *Assume the Axiom of Choice in the ambient set theory. Let $\mathcal{M}$ be a $\mathcal{L}$-structure and let $A \subseteq |\mathcal{M}|$. There is $\mathcal{N} \preccurlyeq \mathcal{M}$ such that $A \subseteq |\mathcal{N}|$ and*

$$||\mathcal{N}|| \leq \max(|A|, |\mathcal{L}|, \aleph_0).$$

Proof. Let $\mu = \max(|A|, |\mathcal{L}|, \aleph_0)$, and choose $m_0 \in |\mathcal{M}|$. For each formula $\exists y \varphi(y, \bar{x})$, where $\bar{x} = (x_1, \ldots, x_k)$ and $k \in \mathbb{N}$, choose a function $F_\varphi : |\mathcal{M}|^k \to |\mathcal{M}|$ such that, whenever $\mathcal{M} \models \exists y \varphi(y, \bar{a})$, $F_\varphi(\bar{a})$ is a witness; otherwise let its value be m_0.

Define

$$A_0 = A \cup \{m_0\} \cup \{c^{\mathcal{M}} : c \text{ is a constant symbol of } \mathcal{L}\}.$$

Given A_n, define A_{n+1} to be exactly

$$A_n \cup \{f^{\mathcal{M}}(\bar{a}) : k \geq 1, f \text{ is a } k\text{-ary function symbol}, \ \bar{a} \in A_n^k\}$$
$$\cup \{F_\varphi(\bar{a}) : k \in \mathbb{N}, \ \exists y\, \varphi(y, x_1, \ldots, x_k) \text{ is a formula}, \ \bar{a} \in A_n^k\}.$$

By corollary 1.29, there are at most μ function symbols, formulas, and finite tuples at each stage, so $|A_n| \leq \mu$ by induction. Put $N = \bigcup_{n \in \mathbb{N}} A_n$. Theorem 1.28 gives $|N| \leq \mu$, and N is nonempty, contains A, and is closed under every signature function. Let $\mathcal{N}$ be the induced substructure on N.

If $\mathcal{M} \models \exists y \varphi(y, \bar{a})$ with $\bar{a} \in N^k$, all coordinates of $\bar{a}$ lie together in some A_n. The chosen witness $F_\varphi(\bar{a})$ lies in $A_{n+1} \subseteq N$. The Tarski–Vaught test therefore gives $\mathcal{N} \preccurlyeq \mathcal{M}$. □

Theorem 13.5 (Upward Löwenheim–Skolem)**.** *Assume the Axiom of Choice in the ambient set theory. Let T be a $\mathcal{L}$-theory with an infinite model. For every cardinal*

$$\kappa \geq \max(|\mathcal{L}|, \aleph_0),$$

the theory T has a model of cardinality exactly κ. In particular, if $\mathcal{M}$ is infinite, then for each such κ there is a structure $\mathcal{N}$ of cardinality κ with $\mathcal{N} \equiv \mathcal{M}$, where elementary equivalence means satisfaction of the same $\mathcal{L}$-sentences.

Proof. Add constants c_α for $\alpha < \kappa$ and add all sentences $c_\alpha \neq c_\beta$ for $\alpha < \beta < \kappa$. Every finite subset of the expanded theory has a model: expand a fixed infinite model of T by assigning distinct elements to the finitely many constants involved. Compactness gives a model $\mathcal{M}^*$ of the entire expanded theory, so the set $A = \{c_\alpha^{\mathcal{M}^*} : \alpha < \kappa\}$ has cardinality κ.

Apply downward Löwenheim–Skolem in the expanded signature to obtain $\mathcal{N}^* \preccurlyeq \mathcal{M}^*$ containing A and of cardinality at most κ. Containment of A gives cardinality at least κ, so $||\mathcal{N}^*|| = \kappa$. Its reduct to $\mathcal{L}$ is a model of T. For the final assertion take $T = \mathrm{Th}_{\mathcal{L}}(\mathcal{M})$, as defined in definition 10.1. □

The downward theorem constructs a small elementary substructure of a given structure. The upward theorem constructs a new model of the same theory. It does not assert that the larger model contains the original structure as an elementary substructure.

13.3 Prenex form and Skolemization

Definition 13.6 (First-order literal and negation normal form)**.** A *first-order literal* is an atomic formula or the negation of an atomic formula. A first-order formula is in *negation normal form* iff it is generated from first-order literals, $\bot$, and $\neg\bot$ by $\wedge, \vee, \forall, \exists$. Thus every negation in such a formula occurs only in a first-order literal or immediately before $\bot$. This extends the earlier propositional definition by adding the two quantifier constructors and supplies the induction class used in the prenex theorem.

Definition 13.7 (Prenex formula)**.** A formula is in *prenex form* iff it has the shape

$$Q_1 x_1 \cdots Q_n x_n \, \theta,$$

where each Q_i is $\forall$ or $\exists$, and θ contains no quantifier. The formula is *rectified* iff no variable has both a free and a bound occurrence and distinct quantifiers bind distinct variables.

Lemma 13.8 (First-order negation normal form)**.** *There exist functions* $\mathrm{nnf}, \mathrm{nnf}' : \mathsf{Fm}(\mathcal{L}) \to \mathsf{Fm}(\mathcal{L})$ *such that, for every formula* φ*:*
1. $\mathrm{nnf}(\varphi)$ *and* $\mathrm{nnf}'(\varphi)$ *are in negation normal form;*
2. $\mathrm{nnf}(\varphi) \equiv \varphi$ *and* $\mathrm{nnf}'(\varphi) \equiv \neg\varphi$*; and*
3. FV *and* Var *of* $\mathrm{nnf}(\varphi)$ *and of* $\mathrm{nnf}'(\varphi)$ *equal* $\mathrm{FV}(\varphi)$ *and* $\mathrm{Var}(\varphi)$*.*
Status: complete proof.

Proof. Define the two functions by simultaneous recursion. For α atomic and $Q \in \{\forall, \exists\}$,

$$\begin{aligned}
\mathrm{nnf}(\alpha) &= \alpha, & \mathrm{nnf}'(\alpha) &= \neg\alpha,\\
\mathrm{nnf}(\bot) &= \bot, & \mathrm{nnf}'(\bot) &= \neg\bot,\\
\mathrm{nnf}(\neg\varphi) &= \mathrm{nnf}'(\varphi), & \mathrm{nnf}'(\neg\varphi) &= \mathrm{nnf}(\varphi),\\
\mathrm{nnf}(\varphi \wedge \psi) &= \mathrm{nnf}(\varphi) \wedge \mathrm{nnf}(\psi), & \mathrm{nnf}'(\varphi \wedge \psi) &= \mathrm{nnf}'(\varphi) \vee \mathrm{nnf}'(\psi),\\
\mathrm{nnf}(\varphi \vee \psi) &= \mathrm{nnf}(\varphi) \vee \mathrm{nnf}(\psi), & \mathrm{nnf}'(\varphi \vee \psi) &= \mathrm{nnf}'(\varphi) \wedge \mathrm{nnf}'(\psi),\\
\mathrm{nnf}(\varphi \to \psi) &= \mathrm{nnf}'(\varphi) \vee \mathrm{nnf}(\psi), & \mathrm{nnf}'(\varphi \to \psi) &= \mathrm{nnf}(\varphi) \wedge \mathrm{nnf}'(\psi),\\
\mathrm{nnf}(Qx\,\varphi) &= Qx\ \mathrm{nnf}(\varphi), & \mathrm{nnf}'(\forall x\,\varphi) &= \exists x\ \mathrm{nnf}'(\varphi),\\
& & \mathrm{nnf}'(\exists x\,\varphi) &= \forall x\ \mathrm{nnf}'(\varphi).
\end{aligned}$$

Every recursive call is on an immediate subformula, so the complexity c strictly decreases and the recursion is legitimate.

The three claims follow by simultaneous strong induction on $c(\varphi)$. For claim 1, every clause of nnf and nnf′ produces a first-order literal, $\bot$, or $\neg\bot$—each admitted by the definition of negation normal form—or combines values of the two functions by $\wedge, \vee, \forall, \exists$; the induction hypotheses close each case.

For claim 2, check each case against the satisfaction clauses. The atomic, $\bot$, and negation cases are immediate; the binary cases for nnf′ are the recursive truth clauses for $\neg(\varphi \wedge \psi)$, $\neg(\varphi \vee \psi)$, and $\neg(\varphi \to \psi)$, and the $\to$ case of nnf is the truth clause for $\to$. The quantifier cases of nnf′ are the pointwise computations $\mathcal{M}, s \models \neg\forall x\,\varphi$ iff some $a \in |\mathcal{M}|$ has $\mathcal{M}, s[x \mapsto a] \models \neg\varphi$ iff $\mathcal{M}, s \models \exists x\,\neg\varphi$, and dually for $\exists$—computed directly from the satisfaction clauses, with the induction hypothesis applied under each updated assignment.

For claim 3, every clause preserves the multiset of atomic subformulas and the quantifier occurrences with their variables, so the recursive FV and Var clauses compute the same sets on both sides; the induction hypotheses close each case. □

Lemma 13.9 (Rectified variant)**.** *For every formula* φ *and every finite set* $V \supseteq \mathrm{Var}(\varphi)$ *of variables there is a formula* $r(\varphi, V)$ *such that:*
1. $r(\varphi, V) \equiv_\alpha \varphi$*, hence* $r(\varphi, V) \equiv \varphi$*;*
2. $r(\varphi, V)$ *is rectified, its binders are pairwise distinct and lie outside* V*, and* $\mathrm{FV}(r(\varphi, V)) = \mathrm{FV}(\varphi)$*;*
3. alpha-equivalent formulas have equal free-variable sets; and
4. r *preserves the constructor tree; in particular, if* φ *is in negation normal form, then so is* $r(\varphi, V)$*.*
Status: complete proof.

Proof. Define r by recursion: atomic formulas and $\bot$ are unchanged; $r(\neg\varphi, V) = \neg r(\varphi, V)$; $r(\varphi \circ \psi, V) = r(\varphi, V) \circ r(\psi,\, V \cup \mathrm{Var}(r(\varphi, V)))$ for $\circ \in \{\wedge, \vee, \to\}$; and $r(Qx\,\varphi, V) = Qz\, r(\varphi[z/x],\, V \cup \{z\})$, where z is the least variable outside V. The recursion is well founded by the pattern of the substitution definition: since $\mathrm{Var}(\varphi) \subseteq V$ and $z \notin V$, we have $z \notin \mathrm{Var}(\varphi) \cup \{x\}$, so no binder of φ equals z, the substitution $\varphi[z/x]$ runs on the first two quantifier clauses only and is textual replacement—whence $\mathrm{Var}(\varphi[z/x]) \subseteq \mathrm{Var}(\varphi) \cup \{z\}$, so the invariant $\mathrm{Var} \subseteq V$ persists into every recursive call—and $c(\varphi[z/x]) = c(\varphi) < c(Qx\,\varphi)$ by lemma 8.11.

Prove claim 3 first. The relation "has the same free-variable set" is an equivalence relation and a congruence for every formula constructor, by the recursive FV clauses. It contains each generating pair $Qx\,\varphi \equiv_\alpha Qy\,(\varphi[y/x])$ with $y \notin \mathrm{Var}(\varphi)$: both cases of lemma 8.14(2) give $\mathrm{FV}(Qy\,(\varphi[y/x])) = \mathrm{FV}(Qx\,\varphi)$, because when $x \in \mathrm{FV}(\varphi)$ the sets $(\mathrm{FV}(\varphi)\setminus\{x\})\cup\{y\}$ minus $\{y\}$ and $\mathrm{FV}(\varphi)\setminus\{x\}$ coincide ($y \notin \mathrm{FV}(\varphi)$), and when $x \notin \mathrm{FV}(\varphi)$ both sides equal $\mathrm{FV}(\varphi)$. By leastness of $\equiv_\alpha$, alpha-equivalence is contained in the relation.

Claims 1, 2, and 4 follow by strong induction on $c(\varphi)$ under the invariant $\mathrm{Var}(\varphi) \subseteq V$. For claim 1, the atomic and $\bot$ cases are reflexivity, and the connective cases are the congruence property of $\equiv_\alpha$. In the quantifier case, the induction hypothesis applies to $\varphi[z/x]$ at complexity $c(\varphi[z/x]) = c(\varphi) < c(Qx\,\varphi)$—licensed by lemma 8.11—and gives $r(\varphi[z/x], V \cup \{z\}) \equiv_\alpha \varphi[z/x]$; the quantifier congruence gives $Qz\, r(\varphi[z/x], V \cup \{z\}) \equiv_\alpha Qz\,(\varphi[z/x])$; the generating clause $Qx\,\varphi \equiv_\alpha Qz\,(\varphi[z/x])$ is licensed by $z \notin \mathrm{Var}(\varphi)$; transitivity and symmetry conclude. Then $r(\varphi, V) \equiv \varphi$ by corollary 9.10.

For claim 2, the binders of $r(\varphi, V)$ lie outside V and are pairwise distinct: the quantifier clause chooses $z \notin V$ and recurses with $V \cup \{z\}$, so all inner binders avoid z and V, and the binary clause forbids every variable of the left output to the right recursion, so the right binders avoid the left binders. By claims 1 and 3, $\mathrm{FV}(r(\varphi, V)) = \mathrm{FV}(\varphi) \subseteq \mathrm{Var}(\varphi) \subseteq V$, so no binder is also free and $r(\varphi, V)$ is rectified.

For claim 4, every clause of r preserves the outer constructor, and in the quantifier case the textual replacement $\varphi[z/x]$ changes only variables inside atomic formulas; induction on the clauses shows that the constructor tree, and hence membership in the negation-normal-form class, is preserved. □

Lemma 13.10 (Prenex extraction)**.** *For every rectified formula φ in negation normal form there is a formula $p(\varphi)$ such that:*

1. *$p(\varphi) \equiv \varphi$;*
2. *$p(\varphi)$ is rectified and in prenex form, and its matrix is quantifier-free and in negation normal form; and*
3. (invariant) FV *and* Var *are unchanged, and the prefix of $p(\varphi)$ consists of exactly the quantifier occurrences of φ, with the same quantifier and the same variable; no new free or bound variables occur.*

Status: complete proof.

Proof. Record two facts. (C1) If $y \notin \mathrm{FV}(\chi)$, then $(Qy\,\pi) \circ \chi \equiv Qy\,(\pi \circ \chi)$ and $\chi \circ (Qy\,\pi) \equiv Qy\,(\chi \circ \pi)$ for $Q \in \{\forall, \exists\}$ and $\circ \in \{\wedge, \vee\}$. For the $\forall/\vee$ case: if $\mathcal{M}, s \models (\forall y\,\pi) \vee \chi$, then either $\mathcal{M}, s[y \mapsto a] \models \pi$ for every a, or $\mathcal{M}, s \models \chi$ and, by lemma 9.8, $\mathcal{M}, s[y \mapsto a] \models \chi$ for every a; in both cases $\mathcal{M}, s \models \forall y\,(\pi \vee \chi)$. Conversely, if $\mathcal{M}, s \models \forall y\,(\pi \vee \chi)$ and $\mathcal{M}, s \not\models \chi$, then lemma 9.8 gives $\mathcal{M}, s[y \mapsto a] \not\models \chi$ for every a, so every $s[y \mapsto a]$ satisfies π, and $\mathcal{M}, s \models \forall y\,\pi$. The remaining seven cases follow by the same computation from the satisfaction clauses and lemma 9.8. (C2) $\equiv$ is a congruence for $\wedge, \vee$ and for each quantifier, immediately from the satisfaction clauses.

Define p by structural induction on the negation-normal generation of φ; every subformula of a rectified formula is rectified, so the induction hypotheses apply. On first-order literals, $\bot$, and $\neg\bot$,

let p be the identity. For a quantifier case $Qx\,\alpha$, put $p(Qx\,\alpha) = Qx\,p(\alpha)$: claim 1 is (C2) with the induction hypothesis, and claims 2 and 3 hold because the prefix of $p(Qx\,\alpha)$ consists of Qx followed by the prefix of $p(\alpha)$, whose entries are exactly the quantifier occurrences of α, pairwise distinct from x by rectifiedness.

For a binary case $\alpha \circ \beta$ with $\circ \in \{\wedge, \vee\}$, let $p(\alpha) = Q_1 y_1 \cdots Q_k y_k\,\alpha_0$ and $p(\beta) = R_1 z_1 \cdots R_m z_m\,\beta_0$ with quantifier-free α_0, β_0, and put

$$p(\alpha \circ \beta) = Q_1 y_1 \cdots Q_k y_k\, R_1 z_1 \cdots R_m z_m\,(\alpha_0 \circ \beta_0).$$

The required side conditions hold by rectifiedness of $\alpha \circ \beta$ and the invariant: each y_i is bound in α, so $y_i \notin \mathrm{FV}(\beta) = \mathrm{FV}(p(\beta))$; and each z_j is bound in β, so $z_j \notin \mathrm{FV}(\alpha_0) \subseteq \mathrm{FV}(\alpha) \cup \{y_1, \ldots, y_k\}$, using binder distinctness for $z_j \neq y_i$. An inner induction on $k + m$ then proves $p(\alpha \circ \beta) \equiv p(\alpha) \circ p(\beta)$: while a quantifier heads either side of $\circ$, (C1) moves it outward across $\circ$—its variable is not free in the other side by the side conditions just checked—and (C2) transports the equivalence under the already-extracted prefix. With the induction hypotheses $p(\alpha) \equiv \alpha$ and $p(\beta) \equiv \beta$ and (C2), claim 1 follows. The output is prenex with quantifier-free negation-normal matrix $\alpha_0 \circ \beta_0$; it is rectified and satisfies the invariant because nothing is renamed or introduced: its prefix lists exactly the quantifier occurrences of α followed by those of β, which are pairwise distinct and disjoint from the free variables by rectifiedness of $\alpha \circ \beta$. The base, quantifier, and binary cases exhaust the constructors of a first-order formula in negation normal form. □

Theorem 13.11 (Prenex normal form)**.** *Every first-order formula is logically equivalent to a rectified formula in prenex form in the same signature. Status: complete proof.*

Proof. Let $\varphi \in \mathsf{Fm}(\mathcal{L})$. Put $\psi = r\big(\mathrm{nnf}(\varphi), \mathrm{Var}(\mathrm{nnf}(\varphi))\big)$ and $\chi = p(\psi)$, with the three operations supplied by lemmas 13.8 to 13.10. By lemma 13.8, $\mathrm{nnf}(\varphi) \equiv \varphi$ and $\mathrm{nnf}(\varphi)$ is in negation normal form. By lemma 13.9, ψ is rectified, remains in negation normal form, and satisfies $\psi \equiv \mathrm{nnf}(\varphi)$. By lemma 13.10, χ is rectified and in prenex form and satisfies $\chi \equiv \psi$. Hence $\chi \equiv \varphi$, and all three operations stay inside $\mathsf{Fm}(\mathcal{L})$, so χ is in the same signature. □

Definition 13.12 (Skolemization)**.** Let

$$\sigma = Q_1 x_1 \cdots Q_n x_n\,\theta$$

be a rectified prenex sentence. Traverse its quantifier prefix from left to right. At an existential quantifier $\exists x_i$, let $u_1, \ldots, u_m$ be the variables governed by preceding universal quantifiers, introduce a new m-ary function symbol f_i, replace the occurrences bound by $\exists x_i$ with $f_i(u_1, \ldots, u_m)$, and delete that existential quantifier. Retain every universal quantifier. The resulting universal sentence in the expanded signature is the *Skolemization* $\mathrm{Sk}(\sigma)$.

An existential quantifier with no preceding universal quantifier produces a new constant symbol. A different new function symbol is used for each existential-quantifier occurrence.

Example 13.13. Skolemization transforms

$$\forall x \exists y \forall z \exists w\, R(x, y, z, w)$$

into

$$\forall x \forall z\, R(x, f(x), z, g(x, z)).$$

The second witness may depend on x and z; its indirect dependence on the first witness is already determined by $f(x)$.

Theorem 13.14 (Equisatisfiability of Skolemization)**.** *Let σ be a rectified prenex $\mathcal{L}$-sentence.*
1. (Axiom of Choice.) *Every $\mathcal{L}$-structure satisfying σ has an expansion to the Skolem signature satisfying* $\mathrm{Sk}(\sigma)$*.*
2. (No choice.) *The $\mathcal{L}$-reduct of every model of* $\mathrm{Sk}(\sigma)$ *satisfies σ.*
Hence, assuming the Axiom of Choice in the ambient set theory, σ has a $\mathcal{L}$-model iff $\mathrm{Sk}(\sigma)$ *has a model in its expanded signature; the right-to-left implication holds without choice. Status: complete proof.*

Proof. Write $\sigma = Q_1 x_1 \cdots Q_n x_n\, \theta$ with θ quantifier-free. For $0 \le i \le n$, let $\sigma_i = Q_{i+1} x_{i+1} \cdots Q_n x_n\, \theta$, so that $\sigma_0 = \sigma$, $\sigma_n = \theta$, and $\mathrm{FV}(\sigma_i) \subseteq \{x_1, \dots, x_i\}$. Let $q(i)$ be the number of universal positions among the first i, let $\bar{u}(j)$ list the variables of the universal positions before position j, and, for each existential position j, let $t_j = f_j(\bar{u}(j))$ be its Skolem term. Then $\mathrm{Sk}(\sigma) = \forall u_1 \cdots \forall u_p\, \theta^*$, where $u_1, \dots, u_p$ are the universal variables in prefix order, $p = q(n)$, and $\theta^* = \theta[t_j/x_j]$ simultaneously for all existential positions j. Because θ is quantifier-free, each substitution is plain replacement, and because no t_j contains an existential variable, the simultaneous substitution equals the composition of the single substitutions in any order.

Given a structure and a tuple $\bar{a}$ of values for the universal variables among $x_1, \dots, x_i$, the *Skolem-term assignment* $s_{\bar{a}}$ assigns to a universal x_j its entry of $\bar{a}$ and to an existential x_j the value v_j of t_j computed from the entries for $\bar{u}(j)$; the values of all other variables are irrelevant by lemma 9.8.

Clause 1. Let $\mathcal{M} \models \sigma$. By finite recursion on i, choose functions $F_j : |\mathcal{M}|^{q(j-1)} \to |\mathcal{M}|$ for the existential positions $j \le i$ while proving the invariant

$$\mathrm{Inv}(i): \quad \mathcal{M}, s_{\bar{a}} \models \sigma_i \quad \text{for every } \bar{a} \in |\mathcal{M}|^{q(i)},$$

where $s_{\bar{a}}$ computes each v_j as F_j applied to the entries of $\bar{a}$ for $\bar{u}(j)$. $\mathrm{Inv}(0)$ is the hypothesis $\mathcal{M} \models \sigma$.

Universal step ($Q_{i+1} = \forall$): from $\mathcal{M}, s_{\bar{a}} \models \forall x_{i+1}\, \sigma_{i+1}$, the universal satisfaction clause gives $\mathcal{M}, s_{\bar{a}}[x_{i+1} \mapsto b] \models \sigma_{i+1}$ for every $b \in |\mathcal{M}|$, which is $\mathrm{Inv}(i+1)$ for the tuple $(\bar{a}, b)$.

Existential step ($Q_{i+1} = \exists$): by $\mathrm{Inv}(i)$ and the existential satisfaction clause, each witness set

$$W(\bar{a}) = \{b \in |\mathcal{M}| : \mathcal{M}, s_{\bar{a}}[x_{i+1} \mapsto b] \models \sigma_{i+1}\}$$

is nonempty. Apply the Axiom of Choice to the set $\{\{\bar{a}\} \times W(\bar{a}) : \bar{a} \in |\mathcal{M}|^{q(i)}\}$ of nonempty sets to obtain a function F_{i+1} with $F_{i+1}(\bar{a}) \in W(\bar{a})$ for every $\bar{a}$. This is one application of choice per existential quantifier, and choice is used nowhere else. With $v_{i+1} = F_{i+1}(\bar{a})$, the Skolem-term assignment satisfies $\mathrm{Inv}(i+1)$.

Define the expansion $\mathcal{M}^*$ of $\mathcal{M}$ to the Skolem signature by $f_j^{\mathcal{M}^*} = F_j$ for every existential position j. Fix $\bar{a} \in |\mathcal{M}|^p$. By $\mathrm{Inv}(n)$, $\mathcal{M}, s_{\bar{a}} \models \theta$, hence $\mathcal{M}^*, s_{\bar{a}} \models \theta$ by lemma 11.2. The term-value clauses give each t_j the value v_j under $[\bar{u} \mapsto \bar{a}]$ in $\mathcal{M}^*$, so iterating theorem 9.9 once per existential position turns $\mathcal{M}^*, s_{\bar{a}} \models \theta$ into $\mathcal{M}^*, [\bar{u} \mapsto \bar{a}] \models \theta^*$. Since $\bar{a}$ was arbitrary, the p-fold universal satisfaction clause yields $\mathcal{M}^* \models \mathrm{Sk}(\sigma)$.

Clause 2. Let $\mathcal{N} \models \mathrm{Sk}(\sigma)$ in the expanded signature; now v_j is the value of the term t_j in $\mathcal{N}$ at the entries for $\bar{u}(j)$, and no choice is needed to produce it. By downward induction from $i = n$ to $i = 0$, prove

$$\mathrm{Inv}'(i): \quad \mathcal{N}, s_{\bar{a}} \models \sigma_i \quad \text{for every } \bar{a} \in |\mathcal{N}|^{q(i)}.$$

Base $i = n$: instantiating the universal prefix of $\mathrm{Sk}(\sigma)$ at $\bar{a}$ gives $\mathcal{N}, [\bar{u} \mapsto \bar{a}] \models \theta^*$, and reversing the iterated substitution-lemma computation of clause 1, again by theorem 9.9, gives $\mathcal{N}, s_{\bar{a}} \models \theta$. Universal step ($Q_{i+1} = \forall$): for every b, $\mathrm{Inv}'(i+1)$ at $(\bar{a}, b)$ gives $\mathcal{N}, s_{\bar{a}}[x_{i+1} \mapsto b] \models \sigma_{i+1}$—the values

v_j for the existential $j \le i$ depend only on the entries of $\bar{a}$—and the universal satisfaction clause gives $\mathrm{Inv}'(i)$. Existential step ($Q_{i+1} = \exists$): $q(i+1) = q(i)$, and $\mathrm{Inv}'(i+1)$ at $\bar{a}$ exhibits the witness v_{i+1}, so the existential satisfaction clause gives $\mathrm{Inv}'(i)$. Then $\mathrm{Inv}'(0)$ says $\mathcal{N} \models \sigma$ in the expanded signature, and lemma 11.2 yields $\mathcal{N} \restriction \mathcal{L} \models \sigma$. No choice principle is used. □

Skolemization generally does not preserve logical equivalence: its output is in an expanded signature and fixes witness functions that the original sentence did not name. The theorem states equisatisfiability and the precise expansion–reduct relation, not equivalence of formulas.

13.4 First-order resolution

Definition 13.15 (First-order clause and formula interpretation)**.** Using the first-order literal defined before the prenex theorem, a *clause* is a finite set of first-order literals, with set equality as its equality relation. Thus literal order and multiplicity are not clause data.

For an exact map back to formulas, take as presentation data a well-order $\prec_{\mathcal{L}}$ of the symbol tokens of $\mathcal{L}$. Supplying this order is an explicit well-ordering assumption; for an arbitrary signature, its existence follows from the well-ordering theorem and hence from the Axiom of Choice. The semantic results below do not depend on which supplied well-order is used. Combine $\prec_{\mathcal{L}}$ with the order $x_0, x_1, \ldots$ and a fixed order of the finitely many logical and punctuation tokens, and order formula strings first by length and then lexicographically. For a clause C, let $k = |C|$; when $k > 0$, list its members as $L_1 \prec \cdots \prec L_k$ in the induced order and define, by downward recursion,

$$D_k := L_k, \qquad D_i := (L_i \vee D_{i+1}) \quad (1 \le i < k).$$

Then define

$$\mathrm{form}_{\mathcal{L}}(C) := \begin{cases} \bot, & C = \varnothing, \\ D_1, & C \neq \varnothing. \end{cases}$$

Let $x_{i_1}, \ldots, x_{i_m}$, with $i_1 < \cdots < i_m$, be exactly the variables occurring in C. Its formula interpretation is

$$\mathrm{ucl}_{\mathcal{L}}(C) := \forall x_{i_1} \cdots \forall x_{i_m}\, \mathrm{form}_{\mathcal{L}}(C),$$

with the quantifier prefix empty when $m = 0$. The functions $\mathrm{form}_{\mathcal{L}}$ and $\mathrm{ucl}_{\mathcal{L}}$ are metalinguistic, not new object-language constructors. They are required because a clause is a set rather than a formula; $\mathrm{ucl}_{\mathcal{L}}$ differs from $\mathrm{form}_{\mathcal{L}}$ by closing every clause variable. A structure satisfies C iff it satisfies $\mathrm{ucl}_{\mathcal{L}}(C)$, and it satisfies a clause set iff it satisfies every member.

A clause is *equality-free* iff no literal in it contains $=$; that is, the atom underlying each of its literals has the form $R(t_1, \ldots, t_n)$, not $r = t$. A clause set is equality-free iff each of its clauses is equality-free. This condition concerns the literals in the clauses, not the signature, because logical equality is not a signature member.

Definition 13.16 (Finite-support simultaneous term substitution)**.** A *term substitution* is a total function $\theta : \mathsf{Var} \to \mathsf{Term}(\mathcal{L})$ whose support

$$\mathrm{supp}(\theta) := \{x \in \mathsf{Var} : \theta(x) \neq x\}$$

is finite. A finite display such as $\{x_1 \mapsto t_1, \ldots, x_n \mapsto t_n\}$ denotes the substitution shown on those variables and the identity on every other variable.

Its simultaneous homomorphic action is defined by

$$x\theta = \theta(x), \qquad f(t_1, \ldots, t_n)\theta = f(t_1\theta, \ldots, t_n\theta),$$

$$\begin{aligned} R(t_1,\dots,t_n)\theta &:= R(t_1\theta,\dots,t_n\theta),\\ (r=t)\theta &:= (r\theta = t\theta),\\ (\neg A)\theta &:= \neg(A\theta),\\ C\theta &:= \{L\theta : L \in C\}. \end{aligned}$$

Here A is atomic, L is a literal, and C is a clause. The application is simultaneous: when $\theta(x)$ is inserted for x, the same application does not apply θ again inside that inserted term. The clause action may identify two literals, in which case set equality removes the duplicate. This quantifier-free action is the one needed by resolution; it neither replaces nor restricts the total capture-avoiding single-variable formula substitution $\varphi[t/x]$ defined earlier.

Definition 13.17 (Composition and instance preorder)**.** The identity substitution id maps every variable to itself. For substitutions θ and η, their composition $\theta\eta$ is oriented by

$$(\theta\eta)(x) := (\theta(x))\eta.$$

Its support is contained in $\operatorname{supp}(\theta) \cup \operatorname{supp}(\eta)$, so it is again a finite-support term substitution. Thus $t(\theta\eta) = (t\theta)\eta$, and the same equation holds for atoms, literals, and clauses. Define the *instance preorder* on term substitutions by

$$\theta \preceq_{\text{inst}} \rho \quad\Longleftrightarrow\quad \text{there is a term substitution } \eta \text{ such that } \rho = \theta\eta.$$

This is a relation among substitutions, not equality: it records that ρ results from further simultaneous instantiation of θ. It is reflexive by id and transitive by associativity of the displayed composition, so it is a preorder. Its use below is precisely to state what "most general" means.

Lemma 13.18 (Composition of simultaneous substitutions)**.** *For every term t, atom A, literal L, and clause C,*

$$\begin{aligned} t(\theta\eta) &= (t\theta)\eta, \qquad & A(\theta\eta) &= (A\theta)\eta,\\ L(\theta\eta) &= (L\theta)\eta, \qquad & C(\theta\eta) &= (C\theta)\eta. \end{aligned}$$

Proof. Induct on terms for the first equation. The atomic and literal equations then follow from their displayed homomorphic clauses. Taking the pointwise image of a finite set proves the clause equation. The term equation also proves associativity of substitution composition by extensionality. □

Lemma 13.19 (Satisfaction under simultaneous term substitution)**.** *If a $\mathcal{L}$-structure $\mathcal{M}$ satisfies a clause C, then it satisfies $C\theta$ for every term substitution θ.*

Proof. For an assignment s, define another assignment by

$$s^\theta(x) := [\![\theta(x)]\!]_s^{\mathcal{M}}.$$

Induction on terms gives $[\![t\theta]\!]_s^{\mathcal{M}} = [\![t]\!]_{s^\theta}^{\mathcal{M}}$, and the atomic truth clauses extend this equality to literals. Because $\mathcal{M}$ satisfies the universal closure of C, the assignment s^θ satisfies some literal $L \in C$. Hence s satisfies $L\theta \in C\theta$. Since s was arbitrary, $\mathcal{M}$ satisfies the universal closure of $C\theta$. □

Definition 13.20 (Renaming apart)**.** A *variable renaming* is a finite-support term substitution induced by a bijection $\pi : \mathsf{Var} \to \mathsf{Var}$; hence every $\pi(x)$ is a variable. For a clause C, put

$$\operatorname{Var}(C) := \bigcup_{L\in C} \operatorname{Var}(L).$$

Clauses $C\pi$ and $D\rho$ are *renamed apart* iff $\mathrm{Var}(C\pi) \cap \mathrm{Var}(D\rho) = \varnothing$. Before a resolution inference, each parent is replaced by such a renamed copy. The required renamings exist because both variable sets are finite and Var is infinite. Universal closure makes a renamed copy equivalent to the original clause; the operation changes bound variable names in its formula interpretation, not its satisfaction conditions.

Definition 13.21 (Unifier and most general unifier)**.** A term substitution θ is a *unifier* of atomic formulas A and B iff $A\theta$ and $B\theta$ are literally identical. A unifier θ is a *most general unifier* iff $\theta \preceq_{\text{inst}} \rho$ for every unifier ρ; equivalently, every unifier ρ has the form $\theta\eta$ for some term substitution η.

Definition 13.22 (First-order unification procedure)**.** Below, *atom* abbreviates *atomic formula*. To unify atoms A and B, first fail if their outer atomic symbols or arities differ. Otherwise form a finite set E of metalinguistic term equations $r \doteq t$ by pairing corresponding arguments (the two ordered arguments when the common atomic symbol is $=$), and initialize θ to id. Repeatedly apply the following transformations to a selected equation:

1. delete $t \doteq t$;
2. replace $f(r_1, \ldots, r_n) \doteq f(t_1, \ldots, t_n)$ by the equations $r_i \doteq t_i$ for $1 \le i \le n$;
3. fail on $f(r_1, \ldots, r_m) \doteq g(t_1, \ldots, t_n)$ when $f \neq g$ or $m \neq n$;
4. orient $t \doteq x$ as $x \doteq t$ when x is a variable and t is not;
5. for $x \doteq t$ with $x \neq t$, perform the *occurs check*: if $x \in \mathrm{Var}(t)$, fail; otherwise put $\delta = \{x \mapsto t\}$, apply δ simultaneously to every remaining equation, and replace the accumulator by $\theta\delta$.

If the equation set becomes empty, return θ. The occurs check is part of this procedure rather than part of the definition of a term substitution: it rejects the impossible demand that a finite term equal a proper term containing itself.

Theorem 13.23 (First-order unification)**.** *The unification procedure terminates. It returns failure iff its input atoms are not unifiable; otherwise it returns a most general unifier.*

Proof sketch. Maintain the invariant that the unifiers of the original problem are exactly the substitutions $\theta\eta$ for substitutions η solving every equation still in E. Deletion, decomposition, and orientation preserve this invariant directly. Elimination preserves it by the composition lemma. The clash rule has no solution. If x occurs properly in t, no finite term can satisfy $x\eta = t\eta$, which justifies occurs-check failure.

An elimination removes its variable from the remaining problem; a decomposition removes matching outer function symbols; deletion removes an equation, and orientation is immediately followed by one of these steps. The corresponding lexicographic measure therefore decreases, proving termination. At success the remaining equation set is empty, so the invariant says both that θ unifies the original atoms and that every other unifier is an instance $\theta\eta$. At either failure rule the invariant says that no unifier exists. The full lexicographic-measure check and the transformation-by-transformation invariant calculations are omitted. □

After the two parent clauses E and F have been renamed apart, choose $A \in E$ and $\neg B \in F$, and let θ be a most general unifier of the atoms A and B. The first-order resolution rule is

$$\frac{E \qquad F}{((E \setminus \{A\}) \cup (F \setminus \{\neg B\}))\theta}\ \textsc{Resolution}.$$

Factoring applies to two distinct literals of the same polarity in one clause E. If their underlying atoms are A and B, and θ is a most general unifier of those atoms, its rule is

$$\frac{E}{E\theta}\ \textsc{Factoring}.$$

Because clauses are sets, the two selected literals become one member of $E\theta$. A *first-order resolution derivation* from a clause set S is a finite sequence in which each clause belongs to S or follows by resolution or factoring from earlier clauses, using renamed-apart copies of two parents before every resolution step.

Theorem 13.24 (Soundness of first-order resolution and factoring)**.** *Every structure satisfying the parent clauses of a resolution or factoring inference satisfies its conclusion. Consequently, if a first-order resolution derivation from S contains the empty clause, then S is not satisfiable.*

Proof. Renaming apart preserves the universally closed formula interpretations: one direction follows from lemma 13.19, and the other follows by applying the inverse variable renaming. By the same lemma, a structure satisfying E and F satisfies $E\theta$ and $F\theta$. If an assignment falsified every literal in the displayed resolvent, satisfaction of the substituted first parent would force $A\theta$, while satisfaction of the substituted second parent would force $\neg(B\theta)$. But $A\theta = B\theta$ literally, a contradiction. Factoring is sound directly by lemma 13.19. Induction on a derivation now preserves satisfaction, whereas the formula interpretation of the empty clause is $\bot$. □

Definition 13.25 (Ground term and ground instance)**.** A term t is *ground* iff $\mathrm{Var}(t) = \varnothing$. Let C be a first-order clause. A *ground instance* of C is a clause $C\theta$ where θ maps every variable occurring in C to a ground term. These terms classify terms and substitution instances, respectively; neither means a closed formula. They are required in the two cited results and the resolution-completeness proof below.

Theorem 13.26 (Herbrand theorem for clauses)**.** *Let S be a finite equality-free set of first-order clauses in a signature having at least one ground term. Then S is not satisfiable iff some finite set of ground instances of members of S, regarded as propositional clauses whose distinct ground atoms are propositional variables, is not satisfiable. Status: cited theorem.*

Theorem 13.27 (Lifting lemma)**.** *Let C, D be first-order clauses with disjoint variable sets, and let G be the ground resolvent produced by one ground resolution step from ground instances of C and D. Factor each parent literal family that its ground substitution identifies, and let R be the first-order resolvent of the resulting parents obtained with a most general unifier. Then there is a substitution η that maps every variable occurring in R to a ground term and satisfies $R\eta = G$. The analogous equality holds for a ground factoring step and its lifted first-order factoring step. Consequently, for every clause set S, every finite ground resolution refutation of ground instances of S lifts, step by step, to a first-order resolution derivation with factoring from S. Status: cited theorem.*

See [49, 5] for complete proofs of theorems 13.26 and 13.27.

Example 13.28 (Unification and resolution)**.** Resolve

$$\{P(f(x)), Q(x)\} \quad \text{and} \quad \{\neg P(y), R(y)\}.$$

The substitution $\theta = \{y \mapsto f(x)\}$ is a most general unifier of $P(f(x))$ and $P(y)$. The resolvent is

$$\{Q(x), R(f(x))\}.$$

Theorem 13.29 (Refutational completeness of first-order resolution)**.** *Let S be a finite equality-free set of first-order clauses. After adding a constant if the signature has no ground term, S is not satisfiable iff resolution with factoring derives the empty clause.*

Proof sketch. The preceding soundness theorem proves the reverse implication. For the forward implication, theorem 13.26 yields a finite propositionally unsatisfiable set of ground instances of clauses in S. Propositional refutational completeness gives a ground resolution refutation. Theorem 13.27 replaces each ground resolution step by a first-order resolution or factoring step whose instance is that ground step. Induction through the finite ground refutation yields a first-order derivation of the empty clause. This proof depends on the two cited theorems, the existence of most general unifiers for unifiable first-order terms, and propositional refutational completeness; it is the precise equality-free scope of the classical result in [49, 5]. The cited complete proofs and the step-by-step lifted derivation are not reproduced here. □

Bare resolution is not complete for logical equality. One must add an equality-complete treatment, such as equality axioms together with resolution or a sound and complete paramodulation or superposition calculus. The theorem above deliberately excludes every clause having a literal that contains logical equality.

13.5 Common errors

Error	Invalid move	Required correction
Capture	Replace x by y in $\forall y\, R(x, y)$ textually.	Rename the binder, then perform capture-avoiding substitution.
Structure/model confusion	Call every $\mathcal{L}$-structure a model without naming a theory.	Use "structure" until a set of sentences has been stated and satisfied.
Formula/sentence confusion	Write $\mathcal{M} \models \varphi(x)$ without an assignment or parameter convention.	Supply an assignment, or close the formula.
Witness dependence	Reverse $\forall x \exists y$ to $\exists y \forall x$.	Preserve quantifier order or prove a uniform witness exists.
Skolem equivalence	Claim that a sentence and its Skolemization are equivalent in one signature.	State equisatisfiability and the expansion–reduct directions.
Cardinality omission	State a Löwenheim–Skolem theorem without a language size or infinite-model hypothesis.	State the exact cardinal bound and the direction of the theorem.
Resolution with equality	Apply bare resolution and claim completeness for logical equality.	Add an equality-complete calculus or restrict the theorem.

13.6 Exercises

Exercise 13.30. Prove the Tarski–Vaught test in full detail, handling both quantifiers directly in the structural induction and justifying every appeal to the induction hypothesis.

Exercise 13.31. Let $\mathcal{M}$ be a structure in a countable signature and let $A \subseteq |\mathcal{M}|$ be countable. Write out the first three stages A_0, A_1, A_2 of the downward Löwenheim–Skolem construction and prove that the union $\bigcup_{n \in \mathbb{N}} A_n$ of all stages is closed under every signature function.

Exercise 13.32. Use compactness to prove that if a theory has arbitrarily large finite models, then it has an infinite model. State why this conclusion alone does not say that any finite model embeds into the infinite model.

Exercise 13.33. Prove that no first-order sentence in the empty signature is true exactly in the finite nonempty structures.

Exercise 13.34. Convert

$$\neg \forall x \big(P(x) \to \exists y (Q(y) \wedge R(x, y)) \big)$$

to rectified prenex normal form. Justify each equivalence and side condition.

Exercise 13.35. Skolemize

$$\exists u \forall x \exists y \forall z \exists w \, R(u, x, y, z, w).$$

Give the arity and argument list of every new symbol.

Exercise 13.36. Give two structures showing why a sentence and its Skolemization cannot be compared for logical equivalence before an expanded signature and a reduct map have been specified.

Exercise 13.37. Find a most general unifier, or prove none exists, for each pair:

$$P(f(x), x), P(y, g(y)); \qquad P(f(x), x), P(f(y), y); \qquad P(x, x), P(y, f(y)).$$

The first and final pairs require the occurs check.

Exercise 13.38. Clausify and refute by first-order resolution:

$$\forall x (P(x) \to Q(x)), \qquad \exists x P(x), \qquad \forall x \neg Q(x).$$

Record the Skolem constant, every substitution, and the empty clause.

Exercise 13.39. Explain exactly why failure to find a resolution refutation within a fixed time bound does not establish satisfiability.

Part IV

Model Theory

Chapter 14

Maps, Substructures, and Elementarity

This part assumes the Axiom of Choice in the ambient set theory. A signature means a single-sorted first-order signature with equality, and all structures have a nonempty domain. The definitions extend to many-sorted signatures by requiring each map to preserve sorts. The main references are [11, 30, 44].

Convention 14.1 (Status of results)**.** Each major result in this part carries one of three status statements. "Complete proof" means that all substantive steps are supplied here. "Proof sketch" identifies the construction and the missing verification. "Cited theorem" records a theorem with exact hypotheses but refers to the given source for its proof. These descriptions concern this text only; they do not rank the formal-logical importance of the results.

14.1 Homomorphisms, embeddings, and isomorphisms

Let $\mathcal{L}$ be a signature, and let $\mathcal{M}$ and $\mathcal{N}$ be $\mathcal{L}$-structures with domains M and N.

Definition 14.2 (Homomorphism)**.** A *homomorphism* from $\mathcal{M}$ to $\mathcal{N}$ is a function $h : M \to N$ such that:
1. $h(c^{\mathcal{M}}) = c^{\mathcal{N}}$ for every constant symbol c;
2. $$h(f^{\mathcal{M}}(a_1, \dots, a_n)) = f^{\mathcal{N}}(h(a_1), \dots, h(a_n))$$
for every n-ary function symbol f and all $a_1, \dots, a_n \in M$; and
3. if $\mathcal{M} \models R(a_1, \dots, a_n)$, then $\mathcal{N} \models R(h(a_1), \dots, h(a_n))$, for every relation symbol R.

The third clause preserves relations only in the forward direction.

Definition 14.3 (Embedding)**.** An *embedding* $h : \mathcal{M} \to \mathcal{N}$ is an injective homomorphism that also reflects every relation symbol:

$$\mathcal{M} \models R(\bar{a}) \quad \Longleftrightarrow \quad \mathcal{N} \models R(h\bar{a}).$$

Here $h\bar{a}$ denotes the coordinatewise image of the finite tuple $\bar{a}$. Because equality is logical identity, injectivity is exactly the reflection condition for equality.

Definition 14.4 (Isomorphism and automorphism)**.** An *isomorphism* is a surjective embedding. An *automorphism* of $\mathcal{M}$ is an isomorphism from $\mathcal{M}$ to itself.

Lemma 14.5 (Preservation of terms)**.** *If $h : \mathcal{M} \to \mathcal{N}$ is a homomorphism, $t(\bar{x})$ is a $\mathcal{L}$-term, and $\bar{a} \in M$, then*

$$h(t^{\mathcal{M}}(\bar{a})) = t^{\mathcal{N}}(h\bar{a}).$$

Status: complete proof.

Proof. Induct on the construction of t. A variable is immediate, and a constant uses the constant clause in the definition of homomorphism. If $t = f(t_1, \ldots, t_n)$, apply the induction hypothesis to each t_i, then apply preservation of f. □

Warning 14.6. A homomorphism need not preserve negated atomic formulas, and an embedding need not preserve formulas with quantifiers. Elementarity is the additional condition that preserves all first-order formulas.

14.2 Substructures

Substructure, elementary substructure, and the Tarski–Vaught test were introduced for the first-order metatheory in section 13.1. This chapter restates them once in its map-based framework; the definitions agree, and the restatement is recorded here rather than at each item.

Definition 14.7 (Substructure)**.** The structure $\mathcal{M}$ is a *substructure* of $\mathcal{N}$, written $\mathcal{M} \subseteq \mathcal{N}$, iff $M \subseteq N$, the inclusion map is an embedding, and therefore:
1. every constant of $\mathcal{N}$ lies in M;
2. M is closed under every function of $\mathcal{N}$; and
3. every relation of $\mathcal{M}$ is the restriction to M of the corresponding relation of $\mathcal{N}$.

Suppose that $A \subseteq N$ is nonempty, or that $\mathcal{L}$ has a constant symbol. The *substructure generated by* A is the intersection of all substructures of $\mathcal{N}$ whose domains contain A. Its domain consists exactly of values $t^{\mathcal{N}}(\bar{a})$, where t is a term and $\bar{a}$ is a finite tuple from A, including the values of constant terms. If structures are required to be nonempty, $A = \varnothing$, and the signature has no constant, a least substructure need not exist; the phrase "generated by $\varnothing$" then requires a separate convention.

Example 14.8 (A substructure that is not elementary)**.** In the relational signature $\mathcal{L} = \{<\}$, the standard order $(\mathbb{N}, <)$ is a substructure of $(\mathbb{Z}, <)$. It is not elementary. With the parameter $0 \in \mathbb{N}$,

$$(\mathbb{Z}, <) \models \exists y\, (y < 0), \qquad (\mathbb{N}, <) \not\models \exists y\, (y < 0).$$

This example also shows why agreement on atomic formulas does not imply agreement on quantified formulas.

Lemma 14.9 (Isomorphism invariance of satisfaction)**.** *Let $h : \mathcal{M} \to \mathcal{N}$ be an isomorphism. Then, for every formula $\varphi(x_1, \ldots, x_k)$ and all $a_1, \ldots, a_k \in M$,*

$$\mathcal{M} \models \varphi(\bar{a}) \iff \mathcal{N} \models \varphi(h\bar{a}).$$

In particular, isomorphic structures satisfy the same sentences. Status: complete proof.

Proof. Induct on φ. For an equality atom $t_1 = t_2$, lemma 14.5 gives $h(t_i^{\mathcal{M}}(\bar{a})) = t_i^{\mathcal{N}}(h\bar{a})$ for both terms, and injectivity of h yields $t_1^{\mathcal{M}}(\bar{a}) = t_2^{\mathcal{M}}(\bar{a})$ iff $t_1^{\mathcal{N}}(h\bar{a}) = t_2^{\mathcal{N}}(h\bar{a})$. For a relation atom $R(t_1, \ldots, t_l)$, lemma 14.5 reduces the claim to the preservation and reflection of R required of an embedding. The Boolean cases follow from the induction hypotheses and the satisfaction

clauses. For $\exists y\,\psi$: a witness $a' \in M$ on the left maps to the witness $h(a')$ on the right by the induction hypothesis; conversely, a witness $b \in N$ equals $h(a')$ for some $a' \in M$ by surjectivity, and the induction hypothesis returns the witness a'. The universal case is dual, using the same surjectivity. □

Lemma 14.10 (Image of an embedding)**.** *Let $h : \mathcal{M} \to \mathcal{N}$ be an embedding. Then $h(M)$ contains $c^{\mathcal{N}}$ for every constant symbol c and is closed under $f^{\mathcal{N}}$ for every function symbol f. The structure $h[\mathcal{M}]$ with domain $h(M)$, with the constants $c^{\mathcal{N}}$, with the restrictions of the functions $f^{\mathcal{N}}$, and with the relations $R^{\mathcal{N}}$ restricted to $h(M)$ is a substructure of $\mathcal{N}$, and h, taken with codomain $h(M)$, is an isomorphism from $\mathcal{M}$ to $h[\mathcal{M}]$. Status: complete proof.*

Proof. By the homomorphism clauses, $c^{\mathcal{N}} = h(c^{\mathcal{M}}) \in h(M)$, and $f^{\mathcal{N}}(h\bar{a}) = h(f^{\mathcal{M}}(\bar{a})) \in h(M)$ for every tuple $h\bar{a}$ from $h(M)$; so $h(M)$ is closed as claimed and the definition of $h[\mathcal{M}]$ is legitimate. Its interpretations are by construction the restrictions of those of $\mathcal{N}$, so the inclusion map preserves and reflects atomic formulas, and $h[\mathcal{M}] \subseteq \mathcal{N}$ in the sense of the substructure definition above. The map h with codomain $h(M)$ is injective, because h is, and surjective by construction. It is a homomorphism into $h[\mathcal{M}]$, since the interpretations there are the restrictions of those of $\mathcal{N}$ to a set containing every required value, and it reflects every relation symbol R because the relation of $h[\mathcal{M}]$ interpreting R is the restriction of $R^{\mathcal{N}}$ to tuples from $h(M)$ while h reflects $R^{\mathcal{N}}$. Hence the corestricted map is a surjective embedding, which is an isomorphism by definition. □

14.3 Elementary maps and elementary substructures

Definition 14.11 (Elementary map)**.** Let $A \subseteq M$. A map $h : A \to N$ is *elementary* iff, for every $\mathcal{L}$-formula $\varphi(x_1, \dots, x_n)$ and every $a_1, \dots, a_n \in A$,

$$\mathcal{M} \models \varphi(a_1, \dots, a_n) \quad \Longleftrightarrow \quad \mathcal{N} \models \varphi(h(a_1), \dots, h(a_n)).$$

When $A = M$, the map is an *elementary embedding*.

Definition 14.12 (Elementary substructure)**.** If $\mathcal{M} \subseteq \mathcal{N}$ and the inclusion map is elementary, then $\mathcal{M}$ is an *elementary substructure* of $\mathcal{N}$, written $\mathcal{M} \preccurlyeq \mathcal{N}$. The structure $\mathcal{N}$ is then an *elementary extension* of $\mathcal{M}$.

Remark 14.13. The symbols $\subseteq$ and $\preccurlyeq$ express different claims. The first concerns the interpretations of the signature symbols; the second concerns all formulas with parameters from the smaller domain.

Theorem 14.14 (Tarski–Vaught test)**.** *Let $\mathcal{M} \subseteq \mathcal{N}$. Then $\mathcal{M} \preccurlyeq \mathcal{N}$ iff, for every formula $\varphi(y, \bar{x})$ and every finite tuple $\bar{a}$ from M,*

$$\mathcal{N} \models \exists y\,\varphi(y, \bar{a}) \quad \Longrightarrow \quad \mathcal{N} \models \varphi(b, \bar{a}) \text{ for some } b \in M.$$

Status: complete proof.

This restates theorem 13.3 in the map-based framework of this chapter; the proof is repeated so that the present part remains self-contained.

Proof. Suppose first that $\mathcal{M} \preccurlyeq \mathcal{N}$ and that the displayed existential formula is satisfied in $\mathcal{N}$. By elementarity it is satisfied in $\mathcal{M}$, so a witness $b \in M$ exists. Elementarity again gives $\mathcal{N} \models \varphi(b, \bar{a})$.

Conversely, assume the witness condition. We prove by induction on formulas $\psi(\bar{x})$ that, for every $\bar{a}$ from M,

$$\mathcal{M} \models \psi(\bar{a}) \quad \Longleftrightarrow \quad \mathcal{N} \models \psi(\bar{a}).$$

Atomic formulas agree because $\mathcal{M} \subseteq \mathcal{N}$, using lemma 14.5. Boolean connectives follow from the induction hypothesis. For the existential step, if $\mathcal{M} \models \exists y\, \varphi(y, \bar{a})$, a witness in M and the induction hypothesis give the same existential statement in $\mathcal{N}$. In the reverse direction, the assumed witness condition supplies $b \in M$ with $\mathcal{N} \models \varphi(b, \bar{a})$; the induction hypothesis yields $\mathcal{M} \models \varphi(b, \bar{a})$. For a universal formula $\forall y\, \varphi(y, \bar{x})$: if $\mathcal{N} \models \forall y\, \varphi(y, \bar{a})$, then for each $b \in M$ the instance $\varphi(b, \bar{a})$ is satisfied in $\mathcal{N}$, and the induction hypothesis transfers it to $\mathcal{M}$. Conversely, if $\mathcal{N} \not\models \forall y\, \varphi(y, \bar{a})$, the satisfaction clauses give $\mathcal{N} \models \exists y\, \neg\varphi(y, \bar{a})$; the witness condition supplies $b \in M$ with $\mathcal{N} \models \neg\varphi(b, \bar{a})$, the induction hypothesis for φ gives $\mathcal{M} \models \neg\varphi(b, \bar{a})$, and hence $\mathcal{M} \not\models \forall y\, \varphi(y, \bar{a})$. □

Example 14.15 (Applying the test)**.** Let $K \subseteq L$ be algebraically closed fields in the language of rings. The later quantifier-elimination theorem for algebraically closed fields implies $K \preccurlyeq L$. Without that theorem, closure under the ring operations establishes only $K \subseteq L$; algebraic closure alone does not yield the Tarski–Vaught witness condition without further argument.

14.4 Diagrams and their embedding tests

Definition 14.16 (Constants for parameters)**.** If $A \subseteq M$, let

$$\mathcal{L}(A) = \mathcal{L} \cup \{c_a : a \in A\},$$

where the c_a are pairwise distinct new constant symbols, and let $\mathcal{M}_A$ be the $\mathcal{L}(A)$-expansion of $\mathcal{M}$ in which $c_a^{\mathcal{M}_A} = a$. The notation is needed to distinguish formulas that may name members of A from formulas in the original signature. It names a signature expansion and an interpreted structure, not a theory, and it will be used both for diagrams and for types over parameters.

Definition 14.17 (Atomic and elementary diagrams)**.** For a $\mathcal{L}$-structure $\mathcal{M}$, its *atomic diagram* $\mathrm{Diag}(\mathcal{M})$ is the set of all atomic and negated atomic $\mathcal{L}(M)$-sentences true in $\mathcal{M}_M$. Its *elementary diagram* is

$$\mathrm{ElDiag}(\mathcal{M}) = \mathrm{Th}_{\mathcal{L}(M)}(\mathcal{M}_M),$$

the set of all $\mathcal{L}(M)$-sentences true in $\mathcal{M}_M$. Both objects are sets of sentences. The atomic diagram records exactly the quantifier-free atomic information needed to test embeddings, whereas the elementary diagram records every formula needed to test elementary embeddings; the two names are therefore not interchangeable, and the two tests below use the distinction.

Theorem 14.18 (Diagram tests for embeddings)**.** *Let $\mathcal{M}$ and $\mathcal{N}$ be $\mathcal{L}$-structures.*

1. *There is an embedding $h : \mathcal{M} \to \mathcal{N}$ iff $\mathcal{N}$ has a $\mathcal{L}(M)$-expansion $\mathcal{N}'$ such that $\mathcal{N}' \models \mathrm{Diag}(\mathcal{M})$.*
2. *There is an elementary embedding $h : \mathcal{M} \to \mathcal{N}$ iff $\mathcal{N}$ has a $\mathcal{L}(M)$-expansion $\mathcal{N}'$ such that $\mathcal{N}' \models \mathrm{ElDiag}(\mathcal{M})$.*

Status: complete proof.

Proof. Given an embedding h, expand $\mathcal{N}$ by interpreting $c_a^{\mathcal{N}'} = h(a)$. Preservation and reflection of atomic formulas then make every sentence in $\mathrm{Diag}(\mathcal{M})$ true in $\mathcal{N}'$. Conversely, suppose such an expansion is given and define $h(a) = c_a^{\mathcal{N}'}$. For $a \neq b$, the sentence $c_a \neq c_b$ belongs to the atomic diagram, so h is injective. The atomic sentences specifying constants and function values show that h preserves those symbols, and the positive and negated relation instances show that it preserves and reflects relations. Hence h is an embedding.

The same expansion constructed from an elementary embedding satisfies every sentence in the elementary diagram. Conversely, if $\mathcal{N}' \models \mathrm{ElDiag}(\mathcal{M})$, the map just defined is an embedding by the

first part. For every formula $\varphi(\bar{x})$ and tuple $\bar{a}$ from M, exactly one of $\varphi(\bar{c}_{\bar{a}})$ and $\neg\varphi(\bar{c}_{\bar{a}})$ belongs to the elementary diagram. Satisfaction of that sentence by $\mathcal{N}'$ gives

$$\mathcal{M} \models \varphi(\bar{a}) \quad \Longleftrightarrow \quad \mathcal{N} \models \varphi(h\bar{a}),$$

so h is elementary. □

For diagrams and the two embedding tests, see [30, 44].

Example 14.19 (The two diagrams make different tests)**.** For the inclusion $(\mathbb{N}, <) \subseteq (\mathbb{Z}, <)$, interpret each new constant c_n by the integer n. The resulting expansion of $(\mathbb{Z}, <)$ satisfies the atomic diagram of $(\mathbb{N}, <)$, in agreement with the fact that the inclusion is an embedding. It does not satisfy the elementary diagram: $\neg\exists y\,(y < c_0)$ is true in $(\mathbb{N}, <)_{\mathbb{N}}$ and false in the expanded integer order.

Exercise 14.20. Let $\mathcal{M}$ be a substructure of $\mathcal{N}$. Prove directly that the canonical $\mathcal{L}(M)$-expansion of $\mathcal{N}$ satisfies $\mathrm{Diag}(\mathcal{M})$. Show that it satisfies $\mathrm{ElDiag}(\mathcal{M})$ iff $\mathcal{M} \preccurlyeq \mathcal{N}$.

14.5 Elementary chains

Definition 14.21 (Elementary chain)**.** Let δ be a limit ordinal. A sequence $\langle \mathcal{M}_i : i < \delta \rangle$ is an *elementary chain* iff $\mathcal{M}_i \preccurlyeq \mathcal{M}_j$ whenever $i < j < \delta$. Its union $\bigcup_{i<\delta} \mathcal{M}_i$ has domain $\bigcup_{i<\delta} M_i$, with each symbol interpreted by the compatible union of its interpretations in the chain.

Theorem 14.22 (Elementary chain theorem)**.** *If $\langle \mathcal{M}_i : i < \delta \rangle$ is an elementary chain and $\mathcal{M} = \bigcup_{i<\delta} \mathcal{M}_i$, then $\mathcal{M}_i \preccurlyeq \mathcal{M}$ for every $i < \delta$. Status: complete proof.*

Proof. We prove simultaneously, by induction on formulas $\psi(\bar{x})$, that for every $i < \delta$ and every tuple $\bar{a}$ from M_i,

$$\mathcal{M}_i \models \psi(\bar{a}) \quad \Longleftrightarrow \quad \mathcal{M} \models \psi(\bar{a}).$$

Atomic formulas agree because all interpretations are compatible; Boolean cases follow from the induction hypothesis. For the existential case, a witness in M_i remains a witness in the union by the induction hypothesis. Conversely, suppose $\mathcal{M} \models \exists y\,\varphi(y, \bar{a})$, and choose a witness $b \in M$. Some $j \geq i$ contains b. Applying the induction hypothesis to the strictly simpler formula φ, now at stage j, gives $\mathcal{M}_j \models \varphi(b, \bar{a})$. Therefore $\mathcal{M}_j \models \exists y\,\varphi(y, \bar{a})$, and $\mathcal{M}_i \preccurlyeq \mathcal{M}_j$ yields the existential statement in $\mathcal{M}_i$.

For a universal formula $\forall y\,\varphi(y, \bar{x})$: if $\mathcal{M} \models \forall y\,\varphi(y, \bar{a})$, then for every $b \in M_i$ the induction hypothesis at stage i transfers the instance $\varphi(b, \bar{a})$ from $\mathcal{M}$ to $\mathcal{M}_i$, so $\mathcal{M}_i \models \forall y\,\varphi(y, \bar{a})$. If $\mathcal{M} \not\models \forall y\,\varphi(y, \bar{a})$, choose $b \in M$ with $\mathcal{M} \not\models \varphi(b, \bar{a})$ and $j \geq i$ with $b \in M_j$; the induction hypothesis at stage j gives $\mathcal{M}_j \not\models \varphi(b, \bar{a})$, so $\mathcal{M}_j \not\models \forall y\,\varphi(y, \bar{a})$, and $\mathcal{M}_i \preccurlyeq \mathcal{M}_j$ transfers this failure to $\mathcal{M}_i$. This completes the induction and proves elementarity of every stage in the union. □

Warning 14.23 (A frequent chain error). The union of an increasing chain of substructures is a substructure, but it need not be an elementary extension of every stage. Elementarity of the transition maps is the hypothesis that supports theorem 14.22.

Exercise 14.24. Let $\mathcal{M} \preccurlyeq \mathcal{N}$ and let $A \subseteq M$. Prove that the expansion $(\mathcal{M}, a)_{a\in A}$ is an elementary substructure of $(\mathcal{N}, a)_{a\in A}$ in the language with a constant symbol for every member of A.

Exercise 14.25. Give an increasing chain of substructures $\mathcal{M}_0 \subseteq \mathcal{M}_1 \subseteq \cdots$ such that $\mathcal{M}_0 \not\preccurlyeq \bigcup_n \mathcal{M}_n$. Identify a formula and parameters witnessing failure of elementarity.

Chapter 15

Types, Saturation, and Ultraproducts

15.1 Types over parameter sets

Let $\mathcal{M}$ be a $\mathcal{L}$-structure and $A \subseteq M$. Use the parameter expansion $\mathcal{L}(A)$ and $\mathcal{M}_A$ fixed in definition 14.16.

Definition 15.1 (Type)**.** Let $\bar{x} = (x_1, \ldots, x_n)$ be a finite tuple of distinct variables. An *n-type over A consistent with $\mathcal{M}$* is a set $p(\bar{x})$ of $\mathcal{L}(A)$-formulas whose free variables are among $\bar{x}$ such that

$$\mathrm{Th}_{\mathcal{L}(A)}(\mathcal{M}_A) \cup \{\varphi(\bar{c}) : \varphi(\bar{x}) \in p\}$$

is satisfiable, where $\bar{c}$ is a new tuple of constants. The type is *complete* iff, for every $\mathcal{L}(A)$-formula $\varphi(\bar{x})$, exactly one of φ and $\neg\varphi$ belongs to p. The phrase *consistent with* is the standard model-theoretic term and is defined here semantically, by satisfiability; by soundness and completeness of the fixed first-order calculus, this sense agrees with consistency in the registered derivability sense. That departure is declared once here and governs types over parameter sets and over theories.

Definition 15.2 (Types over a theory and type spaces)**.** Let T be a satisfiable $\mathcal{L}$-theory. An *n-type consistent with T* is a set $p(\bar{x})$ of $\mathcal{L}$-formulas whose free variables are among the distinct variables in $\bar{x}$, such that

$$T \cup \{\varphi(\bar{c}) : \varphi(\bar{x}) \in p\}$$

is satisfiable for a new n-tuple of constants $\bar{c}$. It is complete when it contains exactly one of φ and $\neg\varphi$ for every $\mathcal{L}$-formula $\varphi(\bar{x})$. Let $S_n(T)$ be the set of all complete n-types consistent with T. For each formula $\varphi(\bar{x})$, put

$$[\varphi] = \{p \in S_n(T) : \varphi \in p\}.$$

The sets $[\varphi]$ form a basis of clopen sets because $[\varphi] \cap [\psi] = [\varphi \wedge \psi]$ and $S_n(T) \setminus [\varphi] = [\neg\varphi]$. The set $S_n(T)$ with this topology is the *n-type space of T*. The topology is required to express isolation and compactness of complete descriptions; it is not the set of tuples realized in one chosen model. This distinction is used below in the omitting-types theorem and the Ryll–Nardzewski theorem.

Theorem 15.3 (Compactness of type spaces)**.** *Assume the Axiom of Choice in the ambient set theory. If T is satisfiable, then $S_n(T)$ is compact, Hausdorff, and has a basis of clopen sets. Status: complete proof.*

Proof. The displayed basis in definition 15.2 consists of clopen sets. If $p \neq q$, choose $\varphi \in p \setminus q$; completeness gives $\neg\varphi \in q$, so the disjoint neighborhoods $[\varphi]$ and $[\neg\varphi]$ separate them.

For compactness it suffices to consider a cover by basic open sets: an arbitrary open cover refines to one by basic sets, because every point of an open set lies in a basic set contained in that open set, and a finite subfamily of the basic refinement yields a finite subcover of the original cover upon replacing each basic set by a member of the original cover containing it.

So let $\{[\varphi_i] : i \in I\}$ cover $S_n(T)$ and suppose, toward a contradiction, that no finite subfamily covers $S_n(T)$. Let $\bar{c}$ be a new n-tuple of constants and consider the set

$$T \cup \{\neg\varphi_i(\bar{c}) : i \in I\}.$$

Every finite subset of the displayed set is contained in $T \cup \{\neg\varphi_j(\bar{c}) : j \in J\}$ for some finite $J \subseteq I$. Since $\{[\varphi_j] : j \in J\}$ is a finite subfamily, it does not cover $S_n(T)$, so there is a complete type $p \in S_n(T)$ with $\varphi_j \notin p$ for every $j \in J$; completeness of p gives $\neg\varphi_j \in p$. By definition 15.2, $T \cup \{\psi(\bar{c}) : \psi \in p\}$ is satisfiable, and any model of it is a model of $T \cup \{\neg\varphi_j(\bar{c}) : j \in J\}$.

By first-order compactness (theorem 12.17, clause 1, applied in the signature expanded by the constants $\bar{c}$), the displayed set has a model $\mathcal{N}$; let $\bar{b}$ be the interpretation of $\bar{c}$. The complete-type step, which here plays the role of the Lindenbaum lemma, extracts a point of $S_n(T)$ from $\mathcal{N}$: let q be the set of all $\mathcal{L}$-formulas $\psi(\bar{x})$ with free variables among $\bar{x}$ such that $\mathcal{N} \models \psi(\bar{b})$. For every $\mathcal{L}$-formula $\psi(\bar{x})$, exactly one of ψ and $\neg\psi$ is satisfied by $\bar{b}$ in $\mathcal{N}$, so q contains exactly one of them; and $\mathcal{N}$ itself is a model of $T \cup \{\psi(\bar{c}) : \psi \in q\}$, because it satisfies T and interprets $\bar{c}$ as $\bar{b}$. Hence q is a complete n-type consistent with T, that is, $q \in S_n(T)$. For every $i \in I$, the model $\mathcal{N}$ satisfies $\neg\varphi_i(\bar{b})$, so $\varphi_i \notin q$ and $q \notin [\varphi_i]$. This contradicts the assumption that $\{[\varphi_i] : i \in I\}$ covers $S_n(T)$. □

Example 15.4 (A finite type space)**.** Let $T = \mathrm{Th}(\mathcal{M})$, where $\mathcal{M}$ is a two-element structure in the equality-only signature. Every model of T has exactly two elements and is isomorphic to $\mathcal{M}$, so every complete type consistent with T is realized in $\mathcal{M}$. Its one-type space has one point, since the two elements are exchanged by an automorphism. Its two-type space has exactly two points: the complete type containing $x_1 = x_2$ and the complete type containing $x_1 \neq x_2$. Both points are clopen, so this type space is discrete.

Exercise 15.5. For formulas $\varphi(\bar{x})$ and $\psi(\bar{x})$, verify directly that $[\varphi] \cap [\psi] = [\varphi \wedge \psi]$ and that $S_n(T) \setminus [\varphi] = [\neg\varphi]$. Compute the one- and two-type spaces in the preceding example and exhibit every singleton as a basic open set.

Definition 15.6 (Realization)**.** A tuple $\bar{b} \in M^n$ *realizes* $p(\bar{x})$ in $\mathcal{M}$ iff $\mathcal{M} \models \varphi(\bar{b})$ for every $\varphi(\bar{x}) \in p$. The structure $\mathcal{M}$ *omits* p iff no tuple in M^n realizes p in $\mathcal{M}$.

Example 15.7 (A finitely satisfiable type not realized)**.** Work in the parameter expansion of $(\mathbb{N}, <)$ having the previously defined constant c_n for each $n \in \mathbb{N}$, and let

$$p(x) = \{c_n < x : n \in \mathbb{N}\}.$$

Thus every displayed member is a formula of the fixed parameter language. Every finite subset is realized in $\mathbb{N}$, but the whole type is not. Assuming the Axiom of Choice in the ambient set theory, compactness shows that p is realized in an elementary extension of $(\mathbb{N}, <)$. Such a realization is greater than every standard natural number.

Definition 15.8 (Saturation)**.** Let κ be an infinite cardinal. The structure $\mathcal{M}$ is *κ-saturated* iff, whenever $A \subseteq M$ has $|A| < \kappa$, every complete type in finitely many variables over A consistent with $\mathcal{M}$ is realized in $\mathcal{M}$. An infinite structure is *saturated* iff it is $|M|$-saturated. This reference does not apply the unqualified term "saturated" to a finite structure, because κ-saturation has just been defined only for infinite κ.

Remark 15.9. Some sources allow tuples of length less than κ in the definition of κ-saturation. This reference uses finite tuples. The convention must be restated before applying a theorem quoted from a source using the stronger version.

Theorem 15.10 (Existence of saturated elementary extensions)**.** *Assume the Axiom of Choice in the ambient set theory. Let $\mathcal{M}$ be a structure and let κ be an infinite cardinal. There is a κ-saturated elementary extension $\mathcal{N} \succcurlyeq \mathcal{M}$. Status: cited theorem.*

See [44] for a complete proof. The construction iterates elementary extensions realizing all types over small parameter sets, and takes unions at limit stages using theorem 14.22. Producing a specified cardinal bound requires additional cardinal-arithmetic bookkeeping and is not asserted here.

15.2 Homogeneity

Definition 15.11 (Homogeneity)**.** Let κ be an infinite cardinal. A structure $\mathcal{M}$ is *κ-homogeneous* iff, whenever $A \subseteq M$ has $|A| < \kappa$, $h : A \to M$ is an elementary map, and $b \in M$, there is an elementary map $h' : A \cup \{b\} \to M$ extending h. This is a property of a structure, not a new kind of map. It records extension of partial elementary self-maps, whereas saturation records realization of types; the notions are therefore not definitionally redundant. The theorem below relates them, and the later back-and-forth method uses the extension property in both directions.

Theorem 15.12 (Saturation supplies homogeneity)**.** *Assume the Axiom of Choice in the ambient set theory. Every κ-saturated structure is κ-homogeneous. Status: complete proof.*

Proof. Let $h : A \to M$ be elementary with $|A| < \kappa$ and let $b \in M$. If $b \in A$, take $h' = h$. So suppose $b \notin A$, and let $p(x)$ be the complete type of b over A: the set of $\mathcal{L}(A)$-formulas $\varphi(x)$ with $\mathcal{M}_A \models \varphi(b)$. For an $\mathcal{L}(A)$-formula φ, let $h(\varphi)$ be the $\mathcal{L}(h[A])$-formula obtained by replacing each parameter constant c_a by $c_{h(a)}$, and put $q(x) = \{h(\varphi) : \varphi \in p\}$. Every $\mathcal{L}(h[A])$-formula $\psi(x)$ has the form $h(\varphi)$, and $h(\neg\varphi) = \neg h(\varphi)$, so q contains ψ or $\neg\psi$ for every such ψ.

The set q is a type over $h[A]$ consistent with $\mathcal{M}$; this is the compactness appeal. Fix a new constant c. By the type definition, the claim is that

$$\mathrm{Th}_{\mathcal{L}(h[A])}(\mathcal{M}_{h[A]}) \cup \{h(\varphi)(c) : \varphi \in p\}$$

is satisfiable. Every finite subset of the displayed set is contained in $\mathrm{Th}_{\mathcal{L}(h[A])}(\mathcal{M}_{h[A]}) \cup \{h(\varphi_1)(c), \dots, h(\varphi_k)(c)\}$ for some $\varphi_1, \dots, \varphi_k \in p$. Since b satisfies each φ_i, the structure $\mathcal{M}_A$ satisfies $\exists x(\varphi_1 \wedge \cdots \wedge \varphi_k)$. Write this sentence as $\exists x\, \chi(x, c_{a_1}, \dots, c_{a_m})$ with χ an $\mathcal{L}$-formula and $a_1, \dots, a_m \in A$; then $\mathcal{M} \models \exists x\, \chi(x, a_1, \dots, a_m)$, and elementarity of h gives $\mathcal{M} \models \exists x\, \chi(x, h(a_1), \dots, h(a_m))$, that is, $\mathcal{M}_{h[A]} \models \exists x(h(\varphi_1) \wedge \cdots \wedge h(\varphi_k))$. Interpreting c as such a witness expands $\mathcal{M}_{h[A]}$ to a model of the finite subset. By first-order compactness (theorem 12.17, clause 1, applied in the signature $\mathcal{L}(h[A])$ expanded by c), the displayed set is satisfiable; this appeal is the step that uses the Axiom of Choice assumed in the statement. In particular q never contains both ψ and $\neg\psi$, so with the previous paragraph q is a complete type over $h[A]$ consistent with $\mathcal{M}$.

Since $|h[A]| \leq |A| < \kappa$, saturation supplies a realization $d \in M$ of q. Define $h'(b) = d$ and let h' agree with h on A. For every $\mathcal{L}$-formula $\chi(x, \bar{y})$ and tuple $\bar{a}$ from A, exactly one of $\chi(x, c_{a_1}, \dots, c_{a_m})$ and its negation belongs to p, and d satisfies its image under h; hence $\mathcal{M} \models \chi(b, \bar{a})$ iff $\mathcal{M} \models \chi(d, h(a_1), \dots, h(a_m))$. A tuple from $A \cup \{b\}$ arises from a tuple from A by inserting b into some coordinates, and replacing those coordinates by one fresh variable reduces the elementarity condition for h' to the displayed equivalence. Thus h' is elementary. □

See [44] for the general saturation–homogeneity development.

Example 15.13 (What homogeneity adds)**.** The cited saturated-extension theorem produces, for any structure $\mathcal{M}$ and infinite κ, an elementary extension $\mathcal{N}$ in which every type over fewer than κ parameters is realized. By theorem 15.12, every elementary map from fewer than κ elements of N can then be extended to include any one further domain element. This extension claim is the homogeneity content; it is not part of the definition of saturation.

Exercise 15.14. Let $\mathcal{M}$ be κ-homogeneous, let $h : A \to M$ be elementary with $|A| < \kappa$, and let $d \in M$. Apply homogeneity to the inverse map on $h[A]$ to prove that h has an elementary extension whose range contains d.

15.3 Omitting types

Definition 15.15 (Principal and nonprincipal types)**.** Let $p(\bar{x})$ be a type consistent with a theory T. The type is *principal over* T iff some formula $\theta(\bar{x})$ satisfies
1. $T \cup \{\exists \bar{x}\, \theta(\bar{x})\}$ is satisfiable; and
2. for every $\varphi(\bar{x}) \in p$, $T \models \forall \bar{x}(\theta \to \varphi)$.
If no such formula exists, p is *nonprincipal over* T. This is a property of a type relative to a theory, not a claim that a selected model realizes or omits it. It distinguishes a type forced by one consistent formula from a merely finitely satisfiable collection, and that distinction is the hypothesis of the theorem below. When p is complete, it is principal iff its point is isolated in $S_n(T)$: an isolating formula θ gives $\{p\} = [\theta]$, and a basic open singleton gives such a formula.

Theorem 15.16 (Omitting types theorem)**.** *Let T be a satisfiable theory in a countable signature, and let $p_0, p_1, \ldots$ be a countable family of nonprincipal types consistent with T, each in a finite tuple of variables. Then T has a countable model that omits every p_n. Status: cited theorem.*

See [30, 44] for complete proofs. The Henkin construction enumerates both witness requirements and pairs consisting of a term tuple and a type p_n. At the latter requirement, nonprincipality permits adding the negation of some formula in p_n at that tuple without destroying finite satisfiability. The term model therefore witnesses the theory while no tuple realizes every formula of any listed type.

Example 15.17 (A type that a countable model omits)**.** Take a signature with constants $c_0, c_1, \ldots$, and let

$$T = \{c_m \neq c_n : m \neq n\}, \qquad p(x) = \{x \neq c_n : n \in \mathbb{N}\}.$$

The type is consistent and nonprincipal over T. The model whose domain consists exactly of the distinct interpretations of the constants satisfies T and omits p, while a model with one additional unnamed element realizes p. Thus omission is a property of a model, whereas nonprincipality is the syntactic-semantic condition on the type used by theorem 15.16.

Exercise 15.18. Prove the two assertions in the example. For nonprincipality, use the fact that a single formula contains only finitely many of the constant symbols; for omission, show that every element of the displayed countable model is the interpretation of some c_n.

15.4 Filters and ultrafilters

Definition 15.19 (Filter)**.** Let I be a nonempty set. A *filter* on I is a set $\mathcal{F} \subseteq \mathcal{P}(I)$ such that:
1. $I \in \mathcal{F}$ and $\varnothing \notin \mathcal{F}$;

2. if $X, Y \in \mathcal{F}$, then $X \cap Y \in \mathcal{F}$; and
3. if $X \in \mathcal{F}$ and $X \subseteq Y \subseteq I$, then $Y \in \mathcal{F}$.

An *ultrafilter* is a filter $\mathcal{U}$ such that, for every $X \subseteq I$, exactly one of X and $I \setminus X$ belongs to $\mathcal{U}$. The requirement that exactly one of X and $I \setminus X$ belong to $\mathcal{U}$ is cited below as the *ultrafilter alternative*.

An ultrafilter is *principal* iff it has the form $\{X \subseteq I : i_0 \in X\}$ for some $i_0 \in I$. A subset $X \subseteq I$ is *cofinite* iff $I \setminus X$ is finite.

Lemma 15.20 (Existence of nonprincipal ultrafilters)**.** *Assume the Ultrafilter Lemma in the ambient set theory. Let I be an infinite set.*
1. The cofinite subsets of I form a filter on I, the Fréchet filter *on I.*
2. Some ultrafilter on I contains the Fréchet filter, and every ultrafilter containing it is nonprincipal.
3. An ultrafilter on I is nonprincipal iff it contains every cofinite subset of I.
In particular, there is a nonprincipal ultrafilter on $\mathbb{N}$. Status: complete proof.

Proof. For clause 1, check the three filter conditions. $I \setminus I = \varnothing$ is finite, so I is cofinite, while $I \setminus \varnothing = I$ is infinite, so $\varnothing$ is not. If X and Y are cofinite, then $I \setminus (X \cap Y) = (I \setminus X) \cup (I \setminus Y)$ is a union of two finite sets, hence finite, so $X \cap Y$ is cofinite. If X is cofinite and $X \subseteq Y \subseteq I$, then $I \setminus Y \subseteq I \setminus X$ is finite, so Y is cofinite.

For clause 2, the Fréchet filter satisfies the two displayed conditions of the Ultrafilter Lemma (definition 1.30); these are exactly the filter conditions just verified. The assumed principle therefore supplies an ultrafilter containing the Fréchet filter. Now let $\mathcal{U}$ be any ultrafilter containing the Fréchet filter and let $i_0 \in I$. The set $I \setminus \{i_0\}$ is cofinite, so it belongs to $\mathcal{U}$; it omits i_0, so $\mathcal{U} \neq \{X \subseteq I : i_0 \in X\}$. Since i_0 was arbitrary, $\mathcal{U}$ is nonprincipal.

For clause 3, right to left, an ultrafilter containing every cofinite subset contains the Fréchet filter and is nonprincipal by clause 2. Left to right, argue contrapositively in three steps that an ultrafilter $\mathcal{U}$ containing some finite set is principal. First, if $X \cup Y \in \mathcal{U}$, then $X \in \mathcal{U}$ or $Y \in \mathcal{U}$: otherwise the ultrafilter alternative would put $I \setminus X$ and $I \setminus Y$ in $\mathcal{U}$, hence also $(I \setminus X) \cap (I \setminus Y) = I \setminus (X \cup Y)$, and the alternative would then exclude $X \cup Y$, contrary to assumption. Second, by induction on $m \geq 1$: if $F \in \mathcal{U}$ is finite with $|F| = m$, then $\{i\} \in \mathcal{U}$ for some $i \in F$. For $m = 1$ the set F is itself a singleton. For the step from m to $m + 1$, choose $i \in F$ and write $F = (F \setminus \{i\}) \cup \{i\}$; the first step gives $\{i\} \in \mathcal{U}$, in which case the claim holds, or $F \setminus \{i\} \in \mathcal{U}$, a finite member of size m to which the induction hypothesis applies. Third, if $\{i_0\} \in \mathcal{U}$, then $\mathcal{U} = \{X \subseteq I : i_0 \in X\}$: every X with $i_0 \in X$ includes $\{i_0\}$ and belongs to $\mathcal{U}$ by upward closure, and no X with $i_0 \notin X$ belongs to $\mathcal{U}$, since otherwise $X \cap \{i_0\} = \varnothing$ would lie in $\mathcal{U}$. Contrapositively, a nonprincipal ultrafilter contains no finite set, and the ultrafilter alternative then puts the complement of every finite set, that is every cofinite set, into it.

The final claim is clause 2 applied to the infinite set $\mathbb{N}$. □

15.5 Ultraproducts

Throughout this section, assume the Axiom of Choice in the ambient metatheory. It supplies an element of every Cartesian product of the nonempty domains below and supplies the coordinatewise witness functions used in the quantifier cases of Łoś's theorem. This is a metatheoretic assumption, not a first-order axiom of any factor structure.

Let $\langle \mathcal{M}_i : i \in I \rangle$ be $\mathcal{L}$-structures and let $\mathcal{U}$ be an ultrafilter on I. On the Cartesian product $\prod_{i \in I} M_i$, define

$$a \sim_{\mathcal{U}} b \quad \Longleftrightarrow \quad \{i \in I : a(i) = b(i)\} \in \mathcal{U}.$$

Definition 15.21 (Ultraproduct)**.** The *ultraproduct*

$$\prod_{i\in I} \mathcal{M}_i/\mathcal{U}$$

has as its domain the set of $\sim_{\mathcal{U}}$-equivalence classes $[a]_{\mathcal{U}}$. Constants and functions are interpreted coordinatewise. For an n-ary relation symbol R, define

$$R([a_1]_{\mathcal{U}},\dots,[a_n]_{\mathcal{U}}) \quad\Longleftrightarrow\quad \{i\in I : \mathcal{M}_i \models R(a_1(i),\dots,a_n(i))\}\in\mathcal{U}.$$

When all $\mathcal{M}_i$ are the same structure $\mathcal{M}$, the result is the *ultrapower* $\mathcal{M}^I/\mathcal{U}$.

Lemma 15.22. *The relation $\sim_{\mathcal{U}}$ is an equivalence relation, and the coordinatewise interpretations in the ultraproduct do not depend on the chosen representatives. Status: complete proof.*

Proof. Reflexivity and symmetry are immediate. For transitivity, intersect the two $\mathcal{U}$-large equality sets and use upward closure. For well-definedness, if representatives agree on $\mathcal{U}$-large sets, then their common intersection is $\mathcal{U}$-large. On that intersection the corresponding function values agree, and the truth values of each relation instance agree. Upward closure completes both verifications. □

Lemma 15.23 (Term evaluation in an ultraproduct)**.** *Let $t(x_1,\dots,x_n)$ be a $\mathcal{L}$-term and let $a_1,\dots,a_n\in\prod_{i\in I} M_i$. Define $b_t\in\prod_{i\in I} M_i$ by*

$$b_t(i) = t^{\mathcal{M}_i}(a_1(i),\dots,a_n(i)).$$

Then

$$t^{\prod_i \mathcal{M}_i/\mathcal{U}}([a_1]_{\mathcal{U}},\dots,[a_n]_{\mathcal{U}}) = [b_t]_{\mathcal{U}}.$$

Status: complete proof.

Proof. Induct on t. For a variable x_j, both sides are $[a_j]_{\mathcal{U}}$. For a constant, the claim is the coordinatewise definition of its ultraproduct interpretation. If $t = f(t_1,\dots,t_m)$, apply the induction hypothesis to every t_j and then use the coordinatewise definition of f in the ultraproduct. □

Theorem 15.24 (Łoś's theorem)**.** *Assume the Axiom of Choice in the ambient set theory, as fixed for this section. Let $\varphi(x_1,\dots,x_n)$ be a first-order $\mathcal{L}$-formula and let $a_1,\dots,a_n\in\prod_{i\in I} M_i$. Then*

$$\prod_{i\in I}\mathcal{M}_i/\mathcal{U} \models \varphi([a_1]_{\mathcal{U}},\dots,[a_n]_{\mathcal{U}})$$

iff

$$\{i\in I : \mathcal{M}_i\models\varphi(a_1(i),\dots,a_n(i))\}\in\mathcal{U}.$$

Status: complete proof.

The original source is [42].

Proof. We prove the displayed equivalence by structural induction for every formula and every tuple of representatives for its free variables. Brackets on a tuple are coordinatewise, so $[\bar a]_{\mathcal{U}} = ([a_1]_{\mathcal{U}},\dots,[a_n]_{\mathcal{U}})$. For such a formula θ, write

$$X_\theta(\bar a) = \{i\in I : \mathcal{M}_i\models\theta(\bar a(i))\}.$$

If θ is $t = s$, lemma 15.23 and the definition of the quotient give

$$\prod_i \mathcal{M}_i/\mathcal{U} \models t([\bar{a}]_{\mathcal{U}}) = s([\bar{a}]_{\mathcal{U}}) \quad \Longleftrightarrow \quad \{i : t^{\mathcal{M}_i}(\bar{a}(i)) = s^{\mathcal{M}_i}(\bar{a}(i))\} \in \mathcal{U}.$$

For $R(t_1, \dots, t_m)$, the term lemma followed by the definition of the ultraproduct relation gives the required equivalence. For $\bot$, both sides are false because $X_\bot = \varnothing \notin \mathcal{U}$.

For negation, $X_{\neg\alpha} = I \setminus X_\alpha$, and the ultrafilter alternative says that this complement is large iff X_α is not large. For conjunction, $X_{\alpha\wedge\beta} = X_\alpha \cap X_\beta$; a filter contains this intersection iff it contains both factors. For disjunction, $X_{\alpha\vee\beta} = X_\alpha \cup X_\beta$, and an ultrafilter contains a finite union iff it contains at least one member. Indeed, if it contained neither, it would contain both complements and hence the complement of the union. Finally,

$$X_{\alpha\to\beta} = (I \setminus X_\alpha) \cup X_\beta.$$

The union property and the ultrafilter alternative make this set large iff X_α is not large or X_β is large. In each of these four cases, the induction hypotheses and the corresponding truth clause in the ultraproduct give the desired equivalence.

Suppose $\theta(\bar{x}) = \exists y\, \psi(y, \bar{x})$. If the ultraproduct has a witness $[b]_{\mathcal{U}}$, the induction hypothesis makes $X_\psi(b, \bar{a})$ large. This set is contained in $X_\theta(\bar{a})$, so the latter is large. Conversely, if $X_\theta(\bar{a}) \in \mathcal{U}$, choose for each index in that set a witness $b(i) \in M_i$, and choose any $b(i) \in M_i$ at every remaining index. Then $X_\psi(b, \bar{a})$ contains $X_\theta(\bar{a})$ and is large; the induction hypothesis makes $[b]_{\mathcal{U}}$ a witness in the ultraproduct.

Suppose $\theta(\bar{x}) = \forall y\, \psi(y, \bar{x})$. If $X_\theta(\bar{a})$ is large, then for every $b \in \prod_i M_i$ it is contained in $X_\psi(b, \bar{a})$. Upward closure and the induction hypothesis show that every $[b]_{\mathcal{U}}$ satisfies ψ, so the ultraproduct satisfies θ. Conversely, suppose the ultraproduct satisfies θ but $X_\theta(\bar{a}) \notin \mathcal{U}$. Its complement is large. At each index in that complement choose $b(i)$ for which $\psi(b(i), \bar{a}(i))$ is false, and choose any value elsewhere. If $X_\psi(b, \bar{a})$ were large, its intersection with the large complement of $X_\theta(\bar{a})$ would be empty and large, a contradiction. Thus the induction hypothesis says that $\psi([b]_{\mathcal{U}}, [\bar{a}]_{\mathcal{U}})$ is false in the ultraproduct, contradicting its universal statement. The two quantifier cases use the stated Axiom of Choice and the stipulated nonemptiness of every M_i. □

Corollary 15.25 (Diagonal elementary embedding)**.** *The map $d : M \to M^I/\mathcal{U}$ sending a to the class of the constant function with value a is an elementary embedding. Status: complete proof.*

Proof. For a formula with parameters from M, its truth set under constant representatives is either all of I or empty. Apply theorem 15.24. □

Example 15.26 (An infinite ultraproduct of finite structures)**.** Let $\mathcal{M}_n$ be the finite linear order $\{0, \dots, n-1\}$, and let $\mathcal{U}$ be a nonprincipal ultrafilter on $\mathbb{N} \setminus \{0\}$. The ultraproduct is infinite: for each k, the sentence asserting that at least k distinct elements exist holds in all $\mathcal{M}_n$ with $n \geq k$, a cofinite set of indices. By theorem 15.24, it holds in the ultraproduct.

Warning 15.27 (First-order scope)*.* Łoś's theorem applies to first-order formulas. The property "the domain is finite" is not first-order definable over all structures, and the example above must not be read as a contradiction.

Exercise 15.28. Let $\mathcal{U}$ be principal at i_0. Construct an explicit isomorphism

$$\prod_{i \in I} \mathcal{M}_i/\mathcal{U} \cong \mathcal{M}_{i_0}.$$

Exercise 15.29. Use an ultrapower of $(\mathbb{N}, 0, S, +, \times, <)$ by a nonprincipal ultrafilter to produce an element greater than every diagonal image $d(n)$. Verify the claim with theorem 15.24.

Chapter 16

Quantifier Elimination and Categoricity

16.1 Quantifier elimination

Definition 16.1 (Quantifier elimination)**.** A theory T in a signature $\mathcal{L}$ *eliminates quantifiers* iff, for every $\mathcal{L}$-formula $\varphi(\bar{x})$, there is a quantifier-free $\mathcal{L}$-formula $\psi(\bar{x})$ such that

$$T \models \forall \bar{x}\, (\varphi(\bar{x}) \leftrightarrow \psi(\bar{x})).$$

The equivalence is relative to the models of T; it is not asserted in all $\mathcal{L}$-structures.

Method 16.2 (One-quantifier reduction)**.** To prove quantifier elimination, it suffices to give an explicit reduction of every formula $\exists y\, \theta(y, \bar{x})$, with θ quantifier-free, to a quantifier-free formula modulo T. Boolean induction, replacement of $\forall y\, \theta$ by $\neg \exists y\, \neg \theta$, and renaming of bound variables then remove all quantifiers from arbitrary formulas.

Theorem 16.3 (Dense linear orders)**.** *Let* DLO *be the first-order theory of nonempty dense linear orders without endpoints in the signature* $\{<\}$*. Then* DLO *eliminates quantifiers. Status: cited theorem.*

Remark 16.4 (Construction underlying the cited theorem). Put a quantifier-free matrix in disjunctive normal form and distribute the existential quantifier over the finite disjunction. In one conjunction, separate literals involving y into lower bounds $t_i < y$, upper bounds $y < r_j$, equalities $y = u_k$, and disequalities. An equality substitutes its term for y. Without an equality, density and the absence of endpoints show that a witness exists iff every required lower bound is below every required upper bound, subject to the finitely many equalities and disequalities among parameters. Writing those finite compatibility conditions gives a quantifier-free formula. A complete proof verifies all inconsistent literal combinations and then applies the one-quantifier reduction method; see [44].

Example 16.5 (Eliminating one quantifier in DLO)**.** Modulo DLO,

$$\exists y\, (x < y \wedge y < z) \quad \longleftrightarrow \quad x < z.$$

The forward implication uses transitivity. The reverse implication is exactly density.

Definition 16.6 (Deductive completeness, semantic completeness, and model completeness)**.** A theory T is *deductively complete* iff, for every sentence σ in its signature, either $T \vdash \sigma$ or $T \vdash \neg\sigma$. It

is *semantically complete* iff, for every such σ, either $T \models \sigma$ or $T \models \neg\sigma$. Because derivability in this reference is taken in the first-order calculus fixed by theorems 11.6 and 12.14,

$$T \vdash \sigma \quad \Longleftrightarrow \quad T \models \sigma,$$

so these two properties are equivalent here. The fixed term for the derivability property is "deductively complete," and the fixed term for the semantic property is "semantically complete."

A theory is *model complete* iff every embedding between its models is elementary. This is not either decision property above: it concerns formulas with parameters along embeddings, and later results require that distinction.

Lemma 16.7 (Reduction to substructure inclusions)**.** *Let T be a theory such that $\mathcal{M} \preccurlyeq \mathcal{N}$ whenever $\mathcal{M} \subseteq \mathcal{N}$ are both models of T. Then T is model complete. Status: complete proof.*

Proof. Let $h : \mathcal{M} \to \mathcal{N}$ be an embedding between models of T. By lemma 14.10, h factors as the isomorphism $h_0 : \mathcal{M} \to h[\mathcal{M}]$ obtained by corestriction, followed by the inclusion $h[\mathcal{M}] \subseteq \mathcal{N}$. By lemma 14.9, $h[\mathcal{M}]$ satisfies the same sentences as $\mathcal{M}$, so $h[\mathcal{M}]$ is a model of T. The hypothesis applied to $h[\mathcal{M}] \subseteq \mathcal{N}$ gives $h[\mathcal{M}] \preccurlyeq \mathcal{N}$. Then, for every formula $\varphi(x_1, \ldots, x_k)$ and every tuple $\bar{a}$ from M,

$$\mathcal{M} \models \varphi(\bar{a}) \quad \Longleftrightarrow \quad h[\mathcal{M}] \models \varphi(h\bar{a}) \quad \Longleftrightarrow \quad \mathcal{N} \models \varphi(h\bar{a}),$$

the first equivalence by lemma 14.9 applied to h_0 and the second because the inclusion is elementary. Hence h is elementary, and T is model complete. □

Corollary 16.8. *The theory* DLO *is deductively complete (equivalently, semantically complete) and model complete. Status: proof sketch.*

Proof sketch. A sentence is equivalent modulo DLO to a quantifier-free sentence. The signature has no constant symbols, so there are no closed atomic formulas; every quantifier-free sentence is a truth-functional combination of $\bot$, and its truth value is fixed by the semantics of the connectives alone. Hence any two models of DLO satisfy the same sentences. Thus the theory is semantically complete and therefore deductively complete by definition 16.6. If $\mathcal{M} \subseteq \mathcal{N}$ are models of DLO, they agree on quantifier-free formulas with parameters from M. Quantifier elimination therefore gives agreement on all formulas, so $\mathcal{M} \preccurlyeq \mathcal{N}$. By lemma 16.7, every embedding between models of DLO is elementary; this is model completeness. The omitted literal-case verification is the one described in the remark following theorem 16.3.

Theorem 16.9 (Algebraically closed fields)**.** *For each characteristic p, where $p = 0$ or p is prime, the theory ACF_p of algebraically closed fields of characteristic p eliminates quantifiers in the language of rings. Status: cited theorem.*

See [44] for a complete proof.

16.2 Categoricity

Definition 16.10 (Categoricity)**.** Let T be a theory and let κ be an infinite cardinal. The theory T is *κ-categorical* iff T has a model of cardinality κ and any two models of T of cardinality κ are isomorphic.

Warning 16.11. Categoricity at one cardinal is not categoricity at every cardinal. The Löwenheim–Skolem theorems imply that a first-order theory with an infinite model has models in multiple infinite cardinalities, subject to the language bound. They do not preclude uniqueness up to isomorphism separately within each cardinality. For example, the equality-only theory asserting that there are at least n elements for every $n \geq 1$ is κ-categorical for every infinite cardinal κ: any two pure equality structures of size κ are isomorphic. A transfer theorem such as Morley's theorem requires additional hypotheses before categoricity at one cardinal implies categoricity at others.

Definition 16.12 (Finite partial isomorphism)**.** A *finite partial isomorphism* from $\mathcal{M}$ to $\mathcal{N}$ is a finite injective map $f : A \to N$, with $A \subseteq M$, that preserves and reflects all atomic formulas on tuples from A.

Method 16.13 (Back-and-forth)**.** For countable structures, build an isomorphism as the union of an increasing sequence of finite partial isomorphisms. At each "forth" stage extend the domain to contain the next enumerated element of the first structure; at each "back" stage extend the range to contain the next enumerated element of the second. The name denotes this alternating proof method, not a new map or relation. It differs from the one-sided extension property in definition 15.11 because it enforces totality and surjectivity together. The next theorem gives the exact hypotheses, and the dense-order proof then uses them.

Theorem 16.14 (Back-and-forth test)**.** *Let $\mathcal{M}$ and $\mathcal{N}$ be countable structures. Suppose $\mathcal{F}$ is a nonempty set of finite partial isomorphisms from $\mathcal{M}$ to $\mathcal{N}$ such that, for every $f \in \mathcal{F}$:*
1. for every $a \in M$, some $g \in \mathcal{F}$ extends f and has a in its domain; and
2. for every $b \in N$, some $g \in \mathcal{F}$ extends f and has b in its range.
Then, for every $f_0 \in \mathcal{F}$, there is an isomorphism $h : \mathcal{M} \to \mathcal{N}$ with $f_0 \subseteq h$, extension taken as sets of ordered pairs; in particular $\mathcal{M} \cong \mathcal{N}$. Status: complete proof.

Proof. Enumerate $M = \{a_0, a_1, \ldots\}$ and $N = \{b_0, b_1, \ldots\}$, and let f_0 be the given member of $\mathcal{F}$. Recursively use clause 1 at stage $2n$ to put a_n into the domain and clause 2 at stage $2n + 1$ to put b_n into the range, obtaining an increasing sequence $f_0 \subseteq f_1 \subseteq f_2 \subseteq \cdots$ in $\mathcal{F}$. The union $h = \bigcup_{k \in \mathbb{N}} f_k$ of the resulting increasing sequence is a function because the maps extend one another, and it is injective because every stage is. The alternating requirements make it total and surjective. Every atomic instance involves only finitely many arguments and is therefore already preserved and reflected at some stage. The union is an isomorphism. Since $f_0 \subseteq f_1 \subseteq \cdots$ and $h = \bigcup_{k \in \mathbb{N}} f_k$, the isomorphism extends f_0. □

See [30, 44] for further back-and-forth applications.

Corollary 16.15 (Countable saturated structures)**.** *Assume the Axiom of Choice in the ambient set theory. If $\mathcal{M}$ is countable and $\aleph_0$-saturated, every finite elementary map from $\mathcal{M}$ to itself extends to an automorphism of $\mathcal{M}$. Status: complete proof.*

Proof. Let f be the given finite elementary map, and let $\mathcal{F}$ be the set of finite elementary maps from $\mathcal{M}$ to itself that extend f. The set is nonempty, because $f \in \mathcal{F}$. Every member is a finite partial isomorphism from $\mathcal{M}$ to $\mathcal{M}$: elementary maps preserve and reflect all atomic formulas, and they reflect equality, hence are injective. Clause 1 of theorem 16.14 holds within $\mathcal{F}$: given $g \in \mathcal{F}$ and $a \in M$, theorem 15.12 extends g to an elementary map whose domain also contains a, and this extension still extends f. Clause 2 holds by applying the same theorem to g^{-1}, which is elementary because the inverse of a finite elementary self-map is elementary: extend g^{-1} to an elementary map whose domain contains the prescribed element b and invert the result, obtaining a member of $\mathcal{F}$ that extends $g \supseteq f$ and has b in its range. Theorem 16.14, applied with $f_0 := f$, yields an isomorphism $h : \mathcal{M} \to \mathcal{M}$ with $f \subseteq h$: an automorphism extending f. □

Theorem 16.16 (Countable categoricity of dense linear order)**.** *Any two countable dense linear orders without endpoints are isomorphic. Consequently,* DLO *is* $\aleph_0$*-categorical. Status: complete proof.*

Proof. Let $\mathcal{M}$ and $\mathcal{N}$ be countable models, with enumerations $M = \{m_0, m_1, \ldots\}$ and $N = \{n_0, n_1, \ldots\}$. Construct a chain of finite order-preserving bijections

$$f_0 \subseteq f_1 \subseteq f_2 \subseteq \cdots .$$

This is the back-and-forth method. At an even stage, take the first m_i not in the current domain. Its order relations to the finitely many domain points determine a cut in the finite range. Density and the absence of endpoints supply an unused point of N in the corresponding interval; extend the map by that pair. At an odd stage, perform the symmetric construction for the first n_i not in the range. The union $f = \bigcup_{n\in\mathbb{N}} f_n$ is total and surjective by the alternating construction, and it preserves and reflects order. Hence it is an isomorphism. □

Example 16.17 (Uncountable categoricity of algebraically closed fields)**.** Fix $p = 0$ or a prime. For every uncountable cardinal κ, any algebraically closed field of characteristic p and cardinality κ has transcendence degree κ over its prime field. Transcendence bases of the same cardinality yield isomorphic algebraic closures. Thus ACF_p is κ-categorical for every uncountable κ. It is not $\aleph_0$-categorical: countable models can have different finite or countable transcendence degree.

Theorem 16.18 (Ryll–Nardzewski)**.** *Let T be a deductively complete theory in a countable signature with an infinite model. The following are equivalent:*

1. *T is $\aleph_0$-categorical;*
2. *for every $n \geq 1$, T has only finitely many complete n-types over $\varnothing$; and*
3. *for every $n \geq 1$, there are, up to equivalence modulo T, only finitely many formulas whose free variables are among $x_1, \ldots, x_n$.*

Status: cited theorem.

See [30] for a complete proof.

Theorem 16.19 (Morley's categoricity theorem)**.** *Let T be a deductively complete theory in a countable signature. If T is categorical in one uncountable cardinal, then T is categorical in every uncountable cardinal. Status: cited theorem.*

See [44] for a complete proof.

Warning 16.20 (Completeness versus categoricity). A deductively complete theory may have many nonisomorphic models of the same cardinality. By definition 16.6, its equivalent semantic condition says that its models agree on sentences; categoricity says that models of a specified size are isomorphic.

Exercise 16.21. Prove directly that every countable model of DLO is isomorphic to $(\mathbb{Q}, <)$.

Exercise 16.22. Let T eliminate quantifiers, and let $\mathcal{M} \subseteq \mathcal{N}$ be models of T. Prove that $\mathcal{M} \preccurlyeq \mathcal{N}$. Explain why the assumption that both structures model T cannot be omitted.

Exercise 16.23. Show that the theory of an equivalence relation with infinitely many classes, all infinite—axiomatized by the equivalence axioms together with, for every n, the sentences asserting that there are at least n pairwise inequivalent elements and that every element is equivalent to at least n distinct elements—is $\aleph_0$-categorical. Give an explicit back-and-forth argument rather than invoking Ryll–Nardzewski.

Part V

Computability

Chapter 17

Machines and Effective Procedures

This part uses deterministic single-tape Turing machines as the primary formalism for computation. Other standard formalisms are mentioned only after the primary definitions have been fixed. Historical sources are Church and Turing [13, 63]; systematic references include [55, 9].

17.1 Deterministic single-tape Turing machines

Definition 17.1 (Turing machine)**.** A *deterministic single-tape Turing machine* is a tuple

$$\mathcal{T} = (Q, \mathcal{A}_{\text{in}}, \mathcal{A}_{\text{tape}}, \delta, q_0, q_{\text{acc}}, q_{\text{rej}})$$

such that:
1. Q is a finite set of states;
2. $\mathcal{A}_{\text{in}}$ is a finite input alphabet not containing the blank symbol $\sqcup$;
3. $\mathcal{A}_{\text{tape}}$ is a finite tape alphabet with $\mathcal{A}_{\text{in}} \cup \{\sqcup\} \subseteq \mathcal{A}_{\text{tape}}$;
4. $q_0, q_{\text{acc}}, q_{\text{rej}} \in Q$, with $q_{\text{acc}} \neq q_{\text{rej}}$; and
5. the transition function is

$$\delta : (Q \setminus \{q_{\text{acc}}, q_{\text{rej}}\}) \times \mathcal{A}_{\text{tape}} \longrightarrow Q \times \mathcal{A}_{\text{tape}} \times \{L, R\}.$$

The two $\mathcal{A}$-symbols name components of this machine tuple only; they are not formula contexts or arithmetical-hierarchy class names. The tape is infinite in both directions, has only finitely many nonblank cells at every finite time, and the head scans one cell.

Definition 17.2 (Configuration and computation)**.** A *configuration* of $\mathcal{T}$ is a tuple

$$C = (q, T, h),$$

where $q \in Q$, $h \in \mathbb{Z}$, and $T : \mathbb{Z} \to \mathcal{A}_{\text{tape}}$ has finite nonblank support:

$$\{i \in \mathbb{Z} : T(i) \neq \sqcup\} \quad \text{is finite.}$$

Suppose $q \notin \{q_{\text{acc}}, q_{\text{rej}}\}$ and

$$\delta(q, T(h)) = (q', a, d).$$

Let $T' : \mathbb{Z} \to \mathcal{A}_{\text{tape}}$ be given by

$$T'(i) = \begin{cases} a, & i = h, \\ T(i), & i \neq h, \end{cases}$$

and let $h' = h - 1$ when $d = L$, while $h' = h + 1$ when $d = R$. Then, and only then,

$$(q, T, h) \longrightarrow_{\mathcal{T}} (q', T', h').$$

A configuration in either halting state has no successor. Write $\longrightarrow_{\mathcal{T}}^{*}$ for the reflexive transitive closure of this one-step relation.

For an input word $w \in \mathcal{A}_{\text{in}}^{*}$ of length n, define

$$T_w(i) = \begin{cases} w(i), & 0 \le i < n, \\ \sqcup, & \text{otherwise.} \end{cases}$$

The initial configuration is $(q_0, T_w, 0)$. Thus on empty input the head is at cell 0, which contains a blank. A computation on w is the unique maximal finite or infinite sequence beginning with this initial configuration and following $\longrightarrow_{\mathcal{T}}$ at each successor step. It *halts* iff the sequence reaches either halting state, and it *accepts* or *rejects* according to which halting state it reaches.

Fix the canonical binary word $\text{bin}(n)$ for each $n \in \mathbb{N}$: $\text{bin}(0)$ is the one-symbol word 0, and for $n > 0$ it is the ordinary base-two expansion without leading zeros, with the most significant bit first. Use the numerical pairing map fixed in the finite-string chapter:

$$\text{pair}_{\mathbb{N}}(a, b) = \frac{(a + b)(a + b + 1)}{2} + b.$$

Its arguments and value are natural numbers; it is distinct in notation and type from the set-theoretic ordered pair $\langle a, b \rangle$. For each fixed arity k, define an injective tuple code by

$$\text{code}_0() := 0, \qquad \text{code}_1(n) := n,$$

and, for $k \ge 2$,

$$\text{code}_k(n_0, \ldots, n_{k-1}) := \text{pair}_{\mathbb{N}}(n_0, \text{pair}_{\mathbb{N}}(n_1, \ldots, \text{pair}_{\mathbb{N}}(n_{k-2}, n_{k-1}) \ldots)).$$

The arity is fixed by the type $\mathbb{N}^k$, so it need not be stored inside the code. A numerical input for a k-ary function is the word $\text{bin}(\text{code}_k(\bar{n}))$, placed with its most significant bit in cell 0.

For machines used to compute numerical functions, the input alphabet contains the symbols $0, 1$. If a halting configuration has tape function T, let

$$m := \min\{i \in \mathbb{N} : T(i) = \sqcup\}.$$

Finite nonblank support makes m defined. The tape prefix of length m is a *valid numerical output* iff it is exactly $\text{bin}(y)$ for a unique $y \in \mathbb{N}$. Thus the empty word, a word containing a symbol other than $0, 1$, and a nonzero binary word with a leading zero are invalid. Cells outside this prefix and the final head position do not affect the output. A machine used to compute a numerical function is required to halt in q_{acc} with a valid numerical output.

Definition 17.3 (Partial computable function)**.** A partial function $f : \mathbb{N}^k \rightharpoonup \mathbb{N}$ is *partial computable* iff some Turing machine satisfies both of the following conditions for every $\bar{n} \in \mathbb{N}^k$: when $f(\bar{n})$ is defined, its computation on $\text{bin}(\text{code}_k(\bar{n}))$ reaches q_{acc} with output word $\text{bin}(f(\bar{n}))$; and when $f(\bar{n})$ is undefined, that computation is infinite. A total partial computable function is called *computable*.

The arrow $\rightharpoonup$ permits undefined values. Divergence is not a third numerical output.

The fixed binary and tuple encodings have computable encoding and decoding maps. For pairing, given z, boundedly search for the greatest $s \leq z$ with $s(s+1)/2 \leq z$, put $b = z - s(s+1)/2$, and put $a = s - b$. When the arity k is supplied, iterating these projections decodes the k tuple entries; the code alone does not encode k. Repeated division by two produces the binary word, and the left-to-right recurrence $r \mapsto 2r + b$ decodes it. Each description is a terminating Turing-machine procedure, so the uses of "code" in the definition above do not add an unexamined computation model.

Example 17.4 (Unary successor)**.** A machine computing $S(n) = n + 1$ reads the arity-one input word $\mathrm{bin}(n)$, since $\mathrm{code}_1(n) = n$. On that block it scans right to the end, propagates a carry left through consecutive 1s, and writes 1 at the first 0 encountered. If the carry passes the most significant bit of an m-bit input, all old bits are now 0. The machine shifts that m-cell zero block one cell to the right, writes 1 in cell 0, and leaves a blank in cell $m + 1$. Thus the output prefix in cells $0, \ldots, m$ changes 111 to 1000. It then treats that prefix as the output word, not as another tuple code, because the output convention requires the binary word for the numerical value. The loop invariant is that every processed suffix is the correct low-order suffix of $n + 1$, while the state records whether a carry remains. Decoding and block relocation are justified by the computable coding operations established above.

Warning 17.5 (Machine formalism versus complexity measure). Changing from one tape to several tapes does not change which partial functions are computable, but it can change running time by a polynomial factor. A computability claim and a complexity claim therefore require different invariance arguments; see [9] for the multi-tape invariance argument.

17.2 Primitive recursion

Definition 17.6 (Primitive recursive functions)**.** The class of *primitive recursive functions* is the least class of total functions on finite powers of $\mathbb{N}$ containing:
1. for every $k \in \mathbb{N}$, the zero function $Z_k : \mathbb{N}^k \to \mathbb{N}$ given by $Z_k(\bar{n}) = 0$, including $Z_0() = 0$;
2. the successor function $S(n) = n + 1$; and
3. every projection $P_i^k(n_1, \ldots, n_k) = n_i$, for $k \geq 1$ and $1 \leq i \leq k$,

and closed under the following two operations. Composition forms $f : \mathbb{N}^k \to \mathbb{N}$ from $g : \mathbb{N}^m \to \mathbb{N}$ and $h_1, \ldots, h_m : \mathbb{N}^k \to \mathbb{N}$ by

$$f(\bar{x}) = g(h_1(\bar{x}), \ldots, h_m(\bar{x})).$$

For every $k \in \mathbb{N}$, primitive recursion forms $f : \mathbb{N}^{k+1} \to \mathbb{N}$ from $g : \mathbb{N}^k \to \mathbb{N}$ and $h : \mathbb{N}^{k+2} \to \mathbb{N}$ by

$$f(\bar{x}, 0) = g(\bar{x}), \qquad f(\bar{x}, n+1) = h(\bar{x}, n, f(\bar{x}, n)).$$

Definition 17.7 (Primitive recursive relation)**.** A relation $R \subseteq \mathbb{N}^k$ is *primitive recursive* iff its characteristic function $\chi_R : \mathbb{N}^k \to \{0, 1\}$, with value 1 exactly on tuples in R, is primitive recursive. This term classifies a relation, not a representing arithmetic formula. It is required below because the coding predicates for formulas and derivations are relations, whereas the preceding definition applies only to functions.

Theorem 17.8. *Every primitive recursive function is computable and total. Status: proof sketch.*

Proof sketch. The initial functions have direct machines. Turing machines are closed under composition by running machines sequentially while retaining the required inputs. For primitive

recursion, store $\bar{x}$, a counter i, and the current value $f(\bar{x}, i)$. Initialize with the machine for g, and run the machine for h exactly n times while incrementing i. The loop has a prescribed finite number of iterations, so it halts. Induction on the generation of the primitive recursive class proves the claim. The omitted verification gives concrete transition tables and invariants for the closure constructions.

Example 17.9 (Addition and multiplication)**.** Addition and multiplication are primitive recursive because

$$x + 0 = x, \quad x + (y + 1) = S(x + y),$$
$$x \times 0 = 0, \quad x \times (y + 1) = (x \times y) + x.$$

The second definition legitimately uses addition because addition was generated at an earlier stage.

Remark 17.10. The Ackermann function is computable and total but not primitive recursive. Thus primitive recursion is a proper subclass of total computability. The proof requires a domination argument and is omitted here; see [9].

17.3 Decidable and computably enumerable sets

Definition 17.11 (Decidable set)**.** A set $A \subseteq \mathbb{N}$ is *decidable* iff its characteristic function $\chi_A : \mathbb{N} \to \{0, 1\}$ is computable.

Definition 17.12 (Computably enumerable set)**.** A set $A \subseteq \mathbb{N}$ is *computably enumerable* iff it is the domain of a partial computable function.

Definition 17.13 (Decidable and computably enumerable relations)**.** For $R \subseteq \mathbb{N}^k$, the relation R is *decidable* iff its characteristic function is computable. It is *computably enumerable* iff

$$\{\mathrm{code}_k(\bar{n}) : \bar{n} \in R\}$$

is a computably enumerable subset of $\mathbb{N}$. These terms classify relations; for $k = 1$ they reproduce the preceding definitions for sets. The extension to relations is needed by the computation-history and proof predicates below.

Definition 17.14 (Machine decision and recognition)**.** Let $A \subseteq \mathbb{N}$. A Turing machine *decides* A iff it halts on every unary numerical input $\mathrm{bin}(n)$, accepts exactly those inputs whose decoded number n belongs to A, and rejects exactly those whose decoded number lies outside A. A Turing machine *recognizes* A iff it accepts exactly the unary numerical inputs whose decoded numbers belong to A, with no halting requirement on inputs outside A. A machine with the first property is a *decider* for A; a machine with the second is a *recognizer* for A.

These terms classify machine behavior, not sets or numerical functions. They are required to state the machine forms of decidability and computable enumerability without ambiguity. Every decider is a recognizer, but a recognizer may diverge outside A, so the two terms are not redundant. The next lemma and theorem are their uses.

Lemma 17.15 (Machine form of decidability)**.** *A set $A \subseteq \mathbb{N}$ is decidable iff some Turing machine decides A. Status: complete proof.*

Proof. Given a machine computing χ_A, simulate it and accept when its output is 1, while rejecting when its output is 0. Totality makes this machine halt on every input. Conversely, simulate a decider and write 1 after an accepting halt and 0 after a rejecting halt. The resulting machine computes χ_A. □

Theorem 17.16 (Equivalent forms of computable enumerability)**.** *For $A \subseteq \mathbb{N}$, the following are equivalent:*
1. A is computably enumerable;
2. some Turing machine recognizes A;
3. either $A = \varnothing$, or A is the range of a computable function $e : \mathbb{N} \to \mathbb{N}$; and
4. there is a decidable relation $R(x, k)$ such that $x \in A$ iff $\exists k\, R(x, k)$.
Status: proof sketch.

Proof sketch. Convert a domain machine into a recognizer. Dovetail all computations to enumerate the accepted inputs. Record a finite accepting computation as the witness k for a decidable computation-history relation. Conversely, search successive k for a witness to $R(x, k)$. The empty-set exception in the range formulation is necessary because a total function on $\mathbb{N}$ has nonempty range.

Theorem 17.17. *A set $A \subseteq \mathbb{N}$ is decidable iff both A and $\mathbb{N} \setminus A$ are computably enumerable. Status: complete proof.*

Proof. If A is decidable, a decider recognizes A, and reversing its two halting answers gives a recognizer for the complement. Conversely, run a recognizer for A and a recognizer for its complement in dovetailing fashion. Exactly one eventually accepts, and its identity determines membership in A. □

Exercise 17.18. Prove that the decidable sets are closed under complement, finite union, and finite intersection. Prove that computably enumerable sets are closed under finite union and finite intersection, but do not use closure under complement.

Chapter 18

Universal Computation, Reductions, and Fixed Points

18.1 An effective numbering of programs

Normalize every machine description as follows. A description is *normalized* iff its tape symbols are the numbers $0, \ldots, |\mathcal{A}_{\text{tape}}| - 1$ with the designated assignments $\sqcup \mapsto 0$, digit $0 \mapsto 1$, digit $1 \mapsto 2$, and its states are the numbers $0, \ldots, |Q| - 1$ with the designated assignments $q_0 \mapsto 0$, $q_{\text{acc}} \mapsto 1$, $q_{\text{rej}} \mapsto 2$; the remaining numbers are assigned arbitrarily, and the row list is the lexicographic listing, by numbered key (q, a), of the full transition table over its non-halting domain. (A clause fixing an "order of first appearance" instead would be circular: the lexicographic row order presupposes the numbering of states and symbols. Without the designated assignments, "valid code" below would not be well defined.) Encode a table tuple of length k as $\text{pair}_{\mathbb{N}}(k, \text{code}_k(\bar{n}))$. No canonical machine-to-code map is fixed; none is needed, since indices are not required to be unique. A natural number that is not the code of a valid normalized numerical-machine description denotes a machine that diverges on every input. Such a description must in particular satisfy $\{0, 1\} \subseteq \mathcal{A}_{\text{in}}$, read at the designated digit numbers $1, 2$ together with the input and output conventions fixed above; a normalized description whose input alphabet omits either binary symbol is treated as invalid by this numbering. In a valid code the three designated states are pairwise distinct; a machine whose start state is halting, which the machine definition permits by requiring only $q_{\text{acc}} \neq q_{\text{rej}}$, is captured up to behavior by the surrogate-start step in the proof of theorem 18.2 below. For a valid code e, define $\varphi_e(x)$ to be the numerical output when that machine accepts with a valid numerical output, and leave $\varphi_e(x)$ undefined when it rejects, diverges, or halts without a valid numerical output. A fixed wrapper simulates the decoded machine, copies a canonical numerical output after a valid accepting halt, and enters an infinite loop after either rejection or an accepting halt without a valid numerical output.

The number e is an *index* for φ_e. Different indices may compute the same partial function. For a finite tuple $\bar{x}$, the notation $\varphi_e(\bar{x})$, with $\bar{x} \in \mathbb{N}^k$, denotes the behavior of indexed program e on the input word $\text{bin}(\text{code}_k(\bar{x}))$. This is arity-dependent application notation for the one fixed program numbering, not a second numbering and not a second application of the unary tuple code.

For expressions denoting possibly undefined values, $a \simeq b$ means that both sides are undefined or both are defined and equal. For partial functions $f, g : \mathbb{N} \rightharpoonup \mathbb{N}$, the notation $f \simeq g$ abbreviates: $f(x) \simeq g(x)$ for every $x \in \mathbb{N}$.

Lemma 18.1 (Renaming invariance)**.** *Let $\mathcal{T}$ be a Turing machine used to compute a numerical function and let $\iota = (\iota_Q, \iota_A)$ bijectively rename its states and tape symbols, preserving the roles of*

$q_0, q_{\text{acc}}, q_{\text{rej}}$ and of $\sqcup$ and the digits $0, 1$. Write $\mathcal{T}^\iota$ for the machine obtained by applying ι to every component of the machine tuple, and C^ι for the corresponding renaming of a configuration C. Then $C \longrightarrow_{\mathcal{T}} C'$ iff $C^\iota \longrightarrow_{\mathcal{T}^\iota} (C')^\iota$. Hence, on every pair of input words corresponding letter by letter under ι_A, the machines $\mathcal{T}$ and $\mathcal{T}^\iota$ accept, reject, or diverge together, with corresponding tapes at every step. Status: complete proof.

Proof. For the one-step claim, read the configuration definition on both sides: $\delta(q, T(h)) = (q', a, d)$ holds in $\mathcal{T}$ iff $\delta^\iota(\iota_Q(q), \iota_A(T(h))) = (\iota_Q(q'), \iota_A(a), d)$ holds in $\mathcal{T}^\iota$, since δ^ι is by construction the ι-conjugate of δ; the rewritten cell and the moved head then correspond position by position. The claim for whole computations follows by induction on the number of steps. Initial configurations correspond because ι_A preserves $\sqcup$, and halting, acceptance, and rejection correspond because ι_Q preserves the designated states. Valid numerical outputs correspond because ι_A preserves the digits. □

Theorem 18.2 (Adequacy of the numbering: enumeration theorem)**.** *(a) Every φ_e is partial computable.*

(b) For every $k \geq 1$ and every partial computable $f : \mathbb{N}^k \rightharpoonup \mathbb{N}$ there is an index e with $\varphi_e(\bar{x}) \simeq f(\bar{x})$ for all $\bar{x} \in \mathbb{N}^k$, under the arity-k application convention fixed above.

Status: complete proof.

Proof. (a) For a valid code e, the fixed wrapper described in the numbering paragraph simulates the decoded machine, accepts with the canonical output word exactly when that machine accepts with a valid numerical output, and otherwise runs forever; for an invalid code, φ_e is the everywhere-undefined function, which a machine with an immediate infinite loop computes. In both cases φ_e satisfies definition 17.3.

(b) Let $\mathcal{T}$ witness that f is partial computable in the sense of definition 17.3. First, $\{0, 1\} \subseteq \mathcal{A}_{\text{in}}$ holds automatically: code_1 is the identity and, for $k \geq 2$, code_k is built from the pairing bijection, so code_k is surjective for every $k \geq 1$ and the input words $\text{bin}(\text{code}_k(\bar{x}))$ exhaust all binary numerals. An initial configuration exists only for a word over $\mathcal{A}_{\text{in}}$, and definition 17.3 demands an accepting or infinite computation of $\mathcal{T}$ on each of these words; since some required numeral contains both digits, both digits lie in $\mathcal{A}_{\text{in}}$, and no alphabet extension is needed.

Second, since the machine definition requires only $q_{\text{acc}} \neq q_{\text{rej}}$, the start state of $\mathcal{T}$ may coincide with a halting state. In that case replace it by a fresh start state q_0' with $\delta(q_0', a) = (q_0, a, R)$ for every tape symbol a: the one extra step leaves the tape unchanged, and the output convention ignores the final head position and the cells outside the output prefix, so the accepted outputs and the divergence behavior are unchanged. After this step the three designated states are pairwise distinct, as are the three designated symbols $\sqcup, 0, 1$.

Now choose any bijective renaming ι of states and tape symbols onto the designated-slot numbering that preserves the six designated roles, and let $\mathcal{T}^*$ be the resulting normalized machine; lemma 18.1 makes $\mathcal{T}^*$ behave exactly as $\mathcal{T}$ on corresponding inputs, and the binary numerals correspond to themselves read at the designated digit numbers $1, 2$. List the rows of $\mathcal{T}^*$ lexicographically by numbered key (q, a); the row list is the full graph of the transition function, which is total on non-halting pairs by the machine definition. Encode the resulting description tuple of length k' as $\text{pair}_{\mathbb{N}}(k', \text{code}_{k'}(\bar{n}))$, obtaining a number e. This e is a valid code by construction. On each input word $\text{bin}(\text{code}_k(\bar{x}))$, the decoded machine $\mathcal{T}^*$ accepts with output word $\text{bin}(f(\bar{x}))$ when $f(\bar{x})$ is defined and runs forever when $f(\bar{x})$ is undefined, so $\varphi_e(\bar{x}) \simeq f(\bar{x})$ for all $\bar{x} \in \mathbb{N}^k$. □

Kleene's treatment of enumeration and normal form over standard numberings [34] is parallel reading; the in-document proof is primary because this numbering is bespoke: its normalization, its

validity constraint $\{0,1\} \subseteq \mathcal{A}_{\text{in}}$, and its wrapper are fixed here rather than taken from the cited source.

Theorem 18.3 (Universal computation)**.** *There is a partial computable function $U(e,x)$ such that*

$$U(e,x) \simeq \varphi_e(x)$$

for all $e, x \in \mathbb{N}$. Status: proof sketch.

Proof sketch. Decode e as a machine description. If decoding fails, diverge. Otherwise store an encoded configuration and repeatedly apply the machine's finite transition table. Return an output only when the simulated machine reaches its accepting state with a valid numerical output; copy that output into canonical form. If it rejects or accepts without a valid numerical output, enter an infinite loop. Each simulation step is effective, and every finite computation is eventually reproduced. Turing's original universal-machine construction is the historical source [63].

Theorem 18.4 (Parameter theorem)**.** *For every $m, n \geq 1$, there is a total computable function $\mathsf{par}_n^m : \mathbb{N}^{m+1} \to \mathbb{N}$ such that, for every index e, every $\bar{a} \in \mathbb{N}^m$, and every $\bar{x} \in \mathbb{N}^n$,*

$$\varphi_{\mathsf{par}_n^m(e,\bar{a})}(\bar{x}) \simeq \varphi_e(\bar{a}, \bar{x}).$$

Status: cited theorem.

From e and the constants $\bar{a}$, construct a normalized transition table for a machine that decodes its runtime input as the n-tuple $\bar{x}$, computes $\text{code}_{m+n}(\bar{a}, \bar{x})$, writes the binary word for that code, and then simulates program e. The established tuple encoding and decoding maps, state renumbering, and finite table concatenation are computable, so the map from $(e, \bar{a})$ to the new index is total and computable. The omitted verification writes the compiler's transition table and proves its simulation invariant. See [34] for a complete proof of the parameter theorem for effective program numberings.

18.2 Many-one reductions

Definition 18.5 (Many-one reduction)**.** For $A, B \subseteq \mathbb{N}$, a *many-one reduction* from A to B is a total computable function $f : \mathbb{N} \to \mathbb{N}$ satisfying

$$x \in A \iff f(x) \in B \qquad \text{for every } x \in \mathbb{N}.$$

Write $A \leq_m B$ when such an f exists.

Lemma 18.6. *If $A \leq_m B$ and B is decidable, then A is decidable. If $A \leq_m B$ and B is computably enumerable, then A is computably enumerable. Status: complete proof.*

Proof. Compute $f(x)$. In the first case run the decider for B; in the second case run the recognizer for B. The defining biconditional transfers the answer or acceptance condition to A. □

Warning 18.7 (Direction of a reduction). To prove that B is at least as hard as A, construct $A \leq_m B$. Reversing the reduction does not establish the desired lower bound.

18.3 The self-application domain problem

Define the *self-application domain set*

$$K = \{e \in \mathbb{N} : \varphi_e(e) \text{ is defined}\}.$$

Theorem 18.8 (Undecidability of the self-application domain set)**.** *The set K is computably enumerable but not decidable. Status: complete proof.*

Proof. The universal machine recognizes K: simulate $U(e, e)$ and accept if it returns a value. Suppose, for contradiction, that K is decidable, and let h be its total computable characteristic function. Define a partial computable function

$$d(e) \simeq \begin{cases} 0, & h(e) = 0, \\ \text{undefined}, & h(e) = 1. \end{cases}$$

By theorem 18.2, fix an index k with $\varphi_k \simeq d$. If $k \in K$, then $h(k) = 1$, so $\varphi_k(k)$ is undefined and $k \notin K$. If $k \notin K$, then $h(k) = 0$, so $\varphi_k(k) = 0$ and $k \in K$. Both alternatives are contradictions. □

Corollary 18.9. *The complement $\mathbb{N} \setminus K$ is not computably enumerable.*

Proof. If both K and its complement were computably enumerable, then K would be decidable. □

Example 18.10 (Reduction to the binary application domain set)**.** Let

$$H = \{\text{pair}_{\mathbb{N}}(e, x) : e, x \in \mathbb{N} \text{ and } \varphi_e(x) \text{ is defined}\}.$$

The map $e \mapsto \text{pair}_{\mathbb{N}}(e, e)$ reduces K to H, so H is undecidable. Conversely, decode an input to H as $\text{pair}_{\mathbb{N}}(e, x)$. Fix an index b satisfying

$$\varphi_b(e, x, y) \simeq \varphi_e(x) \qquad \text{for all } e, x, y \in \mathbb{N};$$

such an index exists because omission of the third input followed by universal simulation is partial computable. Put $q = \mathsf{par}_1^2(b, e, x)$. The parameter theorem makes $(e, x) \mapsto q$ total computable, and $\varphi_q(q) \simeq \varphi_e(x)$. Thus $\varphi_q(q)$ is defined iff $\varphi_e(x)$ is defined, so this map reduces H to K.

18.4 The recursion theorem

Theorem 18.11 (Kleene recursion theorem)**.** *If $f : \mathbb{N} \to \mathbb{N}$ is total computable, then there is an index e such that*

$$\varphi_e \simeq \varphi_{f(e)}.$$

Status: proof sketch.

Proof sketch. By universal computation and the parameter theorem, there is a total computable function p such that

$$\varphi_{p(x)}(y) \simeq \varphi_{\varphi_x(x)}(y),$$

with the right side undefined when $\varphi_x(x)$ is undefined. By theorem 18.2, let a be an index of the total computable function $x \mapsto f(p(x))$. Then $\varphi_a(a) = f(p(a))$. Set $e = p(a)$. We obtain

$$\varphi_e = \varphi_{p(a)} \simeq \varphi_{\varphi_a(a)} = \varphi_{f(p(a))} = \varphi_{f(e)}.$$

This completes the fixed-point calculation using the cited universal and parameter theorems. The direct construction of the program transformation p from the fixed encodings is omitted; this omission is why the recursion theorem retains proof-sketch status.

Example 18.12 (Self-reference without a self-inspection primitive)**.** Suppose a total computable transformation f adds a fixed output header to the behavior of a program whose index it receives. The recursion theorem supplies a program e whose behavior is exactly that of the transformed program $f(e)$. The theorem does not assert $e = f(e)$; it asserts equality of the computed partial functions.

18.5 Rice's theorem

Definition 18.13 (Extensional index property)**.** A set $A \subseteq \mathbb{N}$ is *extensional* iff

$$\varphi_e \simeq \varphi_d \quad \Longrightarrow \quad (e \in A \Longleftrightarrow d \in A) \qquad \text{for all } e, d \in \mathbb{N}.$$

It is *nontrivial* iff it is neither empty nor all of $\mathbb{N}$.

Theorem 18.14 (Rice)**.** *Every nontrivial extensional set of indices of unary partial computable functions is undecidable. Status: complete proof.*

Proof. Let A be nontrivial and extensional. First suppose an index of the everywhere-undefined function does not belong to A. Choose $a \in A$. Fix an index b for the binary partial computable function

$$F(e, x) \simeq \begin{cases} \varphi_a(x), & \text{if } \varphi_e(e) \text{ returns a value,} \\ \text{undefined}, & \text{otherwise.} \end{cases}$$

It is partial computable by first simulating $U(e, e)$ and, only after that simulation returns, simulating $U(a, x)$; theorem 18.2 then supplies the index b. Define the total computable map $g(e) = \mathsf{par}_1^1(b, e)$. The parameter theorem gives $\varphi_{g(e)}(x) \simeq F(e, x)$ for every x. If $e \notin K$, the new program is everywhere undefined, so $g(e) \notin A$. If $e \in K$, the new program computes φ_a, so $g(e) \in A$. Hence $K \leq_m A$, and A is undecidable.

If the everywhere-undefined function has an index in A, apply the first case to the nontrivial extensional complement $\mathbb{N} \setminus A$. Decidability of A would imply decidability of its complement, which is impossible. □

Warning 18.15. Rice's theorem concerns semantic properties of the partial function computed by a program. Syntactic properties such as "the source contains ten states" can be decidable and are not extensional.

Exercise 18.16. Use Rice's theorem to prove undecidable each of the following index sets: programs computing a total function; programs whose range contains 0; and programs computing a constant partial function, where this last phrase means a partial function with nonempty domain that takes one fixed value on every member of its domain. Verify nontriviality in each case.

Chapter 19

Logical Decision Problems and the Arithmetical Hierarchy

19.1 The classical decision problem

Fix an effective coding of first-order formulas and finite derivations.

Theorem 19.1 (Negative solution to the classical Entscheidungsproblem)**.** *Under the fixed effective coding, no Turing machine decides the set of codes of logically valid first-order sentences. Status: cited theorem.*

Church and Turing supply the historical negative solution [13, 63]. The theorem records only that historically sourced general decision problem.

Corollary 19.2 (Enumerability without decidability)**.** *Under the fixed effective coding, the set of logically valid first-order sentences is computably enumerable but not decidable. Status: complete proof.*

Proof. Enumerate all finite derivations in the fixed effective first-order calculus that have no open assumptions and whose conclusion is a sentence, and output their conclusions. Soundness for the empty premise set ensures that every output is valid, and completeness ensures that every valid sentence eventually appears as such a conclusion, so the set is computably enumerable. A decider for this set would solve the general decision problem forbidden by theorem 19.1. □

Warning 19.3 (Restricted signatures)*.* The theorem concerns the general decision problem. The monadic fragment without function symbols is decidable, so undecidability may not be transferred to an individual fixed signature or fragment without a theorem controlling that restriction.

Theorem 19.4 (Trakhtenbrot)**.** *For every finite relational signature containing a binary relation symbol, the set of sentences having a finite model is computably enumerable but undecidable, and the set of sentences true in every finite structure is not computably enumerable. Status: cited theorem.*

See [59] for a complete proof.

The contrast with unrestricted validity is important: unrestricted validities are computably enumerable by completeness of the fixed effective first-order calculus, while finite validities admit no effective calculus both sound and complete for truth in all finite structures.

19.2 Quantifier complexity over arithmetic

Definition 19.5 (Language and standard structure of arithmetic)**.** The *language of arithmetic* is

$$\mathcal{L}_A = \{0, S, +, \times\},$$

where 0 is a constant, S is unary, and $+$ and $\times$ are binary function symbols. The *standard structure of arithmetic* is

$$\mathcal{N} = (\mathbb{N}, 0, S, +, \times)$$

with the usual operations. These objects supply the syntax and semantics for every hierarchy classification below.

For $n \in \mathbb{N}$, the *numeral* for n is the closed term $\bar{n} = S^n 0$. For $\bar{a} = (a_1, \ldots, a_k)$, write $\bar{\bar{a}} = (\bar{a}_1, \ldots, \bar{a}_k)$. The bar is metalinguistic: it maps a natural number or finite tuple of natural numbers to an arithmetic term or tuple of arithmetic terms. When a bar sits over a single letter carrying a numeric subscript, as in $\bar{a}_1$, it denotes the numeral of that tuple entry; a bar is never applied to a subscripted tuple name.

For arithmetic terms r, t, write $r < t$ as the metalinguistic abbreviation $\exists z\,(r + Sz = t)$, where z is the least variable absent from r, t. If $x \notin \mathrm{Var}(t)$, the marked bounded quantifiers abbreviate

$$\exists x < t\,\varphi := \exists x\,(x < t \wedge \varphi),$$
$$\forall x < t\,\varphi := \forall x\,(x < t \to \varphi).$$

Their satisfaction clauses therefore quantify over exactly the natural numbers smaller than the value of t. The hierarchy counts each marked bounded quantifier as bounded before this eliminable expansion.

Alongside $<$, write $r \leq t$ as the metalinguistic abbreviation $\exists z\,(z + r = t)$, where z is the least variable absent from r, t. If $x \notin \mathrm{Var}(t)$, the marked bounded quantifiers

$$\exists x \leq t\,\varphi := \exists x\,(x \leq t \wedge \varphi),$$
$$\forall x \leq t\,\varphi := \forall x\,(x \leq t \to \varphi)$$

are counted as bounded exactly as their $<$-marked counterparts. In the standard structure, $\mathcal{N} \models x \leq t \leftrightarrow x < St$; provability of this equivalence in Q is not claimed and is not used below. The witness z sits to the *left* of $+$ in $z + r = t$; this choice is what lets the comparison lemmas proved for $\leq$ in the incompleteness part rest only on the right-recursion axioms $x + 0 = x$ and $x + Sy = S(x + y)$ for addition.

Definition 19.6 (Syntactic arithmetical hierarchy)**.** A *bounded arithmetic formula* is generated from atomic $\mathcal{L}_A$-formulas by $\bot, \neg, \wedge, \vee, \to$ and marked bounded quantifiers only.

To classify an arbitrary arithmetic formula, first eliminate implications and push negations to atomic formulas by the De Morgan and quantifier-duality rewrites, treating a marked bounded quantifier by the corresponding bounded duality. Rename bound variables by the fixed least-fresh-variable order when needed to avoid capture. This fixed conversion reaches negation normal form; it does not move an unbounded quantifier across a bounded quantifier or a connective.

At level zero, the Σ_0^0 formulas and the Π_0^0 formulas are exactly the bounded arithmetic formulas in negation normal form. Having defined both classes at level n, define the classes at level $n + 1$ simultaneously as follows:

- the Σ_{n+1}^0 formulas in negation normal form form the least class containing every Σ_n^0 and Π_n^0 formula and closed under $\wedge, \vee$, both marked bounded quantifiers, and unbounded existential quantification;

- the Π^0_{n+1} formulas in negation normal form form the least class containing every Σ^0_n and Π^0_n formula and closed under $\wedge, \vee$, both marked bounded quantifiers, and unbounded universal quantification.

An arbitrary formula is a Σ^0_n *formula* or a Π^0_n *formula* iff its fixed negation-normal-form conversion lies in the corresponding recursively generated class. The definition therefore handles a bounded quantifier outside an unbounded quantifier without an invalid quantifier exchange. It depends on syntax and the fixed conversion, not on whether the sentence happens to be true in $\mathcal{N}$.

Definition 19.7 (Definability classes of relations)**.** Let $k \in \mathbb{N}$. A relation $A \subseteq \mathbb{N}^k$ is Σ^0_n*-definable* iff there is a Σ^0_n formula $\varphi(\bar{x})$ such that, for every $\bar{a} \in \mathbb{N}^k$,

$$\bar{a} \in A \quad \Longleftrightarrow \quad \mathcal{N} \models \varphi(\bar{a}).$$

Define Π^0_n-definable analogously, and call A Δ^0_n*-definable* iff it is both Σ^0_n-definable and Π^0_n-definable. Write

$$\begin{aligned}
\mathbf{\Sigma}^0_n(k) &= \{A \subseteq \mathbb{N}^k : A \text{ is } \Sigma^0_n\text{-definable}\},\\
\mathbf{\Pi}^0_n(k) &= \{A \subseteq \mathbb{N}^k : A \text{ is } \Pi^0_n\text{-definable}\},\\
\mathbf{\Delta}^0_n(k) &= \mathbf{\Sigma}^0_n(k) \cap \mathbf{\Pi}^0_n(k).
\end{aligned}$$

For unary relations, the argument (1) may be omitted. The unbold symbols classify formulas syntactically; the bold symbols classify relations by definability in $\mathcal{N}$. This typography records different standing and prevents equivalence in the standard structure $\mathcal{N}$ from reclassifying every true or false sentence as bounded. Both standings are used in the results below.

The semantic fact that a relation is decidable does not by itself provide an object-language formula for that relation. The lemmas and coding definition below supply the particular formulas required in the hierarchy arguments. The coding of computation histories is built in-document from the numerical pairing map of chapter 3 together with the Gödel beta function; every projection used is defined by a bounded formula, proved as such here.

Lemma 19.8 (Pairing graph)**.** *Let*

$$\mathsf{P}(x, y, z) := (x + y) \times S(x + y) + (y + y) = z + z.$$

Then P *is a quantifier-free* $\mathcal{L}_A$*-formula; for all* $a, b, c \in \mathbb{N}$, $\mathcal{N} \models \mathsf{P}(\bar{a}, \bar{b}, \bar{c})$ *iff* $c = \mathrm{pair}_{\mathbb{N}}(a, b)$*; and both* $a \leq \mathrm{pair}_{\mathbb{N}}(a, b)$ *and* $b \leq \mathrm{pair}_{\mathbb{N}}(a, b)$ *hold for all* $a, b \in \mathbb{N}$*. Status: complete proof.*

Proof. The display is a single atomic equation, so P is quantifier-free. In $\mathcal{N}$ it evaluates to $(a+b)(a+b+1)+2b = 2c$, which holds iff $c = (a+b)(a+b+1)/2+b = \mathrm{pair}_{\mathbb{N}}(a, b)$; the doubled form avoids division, which $\mathcal{L}_A$ does not provide. For the monotonicity bounds: $b \leq (a+b)(a+b+1)/2+b$ is immediate, and $a \leq a + b \leq (a + b)(a + b + 1)/2 \leq \mathrm{pair}_{\mathbb{N}}(a, b)$, where the middle inequality holds because $a + b = 0$ forces $a = 0$, while $a + b \geq 1$ makes the factor $(a + b + 1)/2$ at least 1. □

In the next display, d abbreviates the term $S((Si) \times m)$, and u is a fresh variable with $u \notin \mathrm{Var}(d)$. The final conjunct $\exists u < d\,(v + Su = d)$ is the *marked-bounded* form of the comparison $v < d$: it is $\mathcal{N}$-equivalent to the plain abbreviation of $v < d$, because any witness of $v + Su = d$ satisfies $Su \leq d$, hence $u < d$. The marked form is required because the hierarchy definition above counts only marked bounded quantifiers as bounded, while the bare comparison $v < d$ expands to an unbounded existential.

Lemma 19.9 (Gödel beta function)**.** *Define*

$$\mathsf{B}(w,i,v) := \exists t < Sw\, \exists m < Sw\, \exists q < St\, \big(\mathsf{P}(t,m,w) \wedge t = d \times q + v \wedge \exists u < d\, (v + Su = d)\big).$$

Then B *is a bounded arithmetic formula, and:*

(a) for all $w, i \in \mathbb{N}$ *there is exactly one* $v \in \mathbb{N}$ *with* $\mathcal{N} \models \mathsf{B}(\bar{w}, \bar{i}, \bar{v})$*, and this* v *satisfies* $v \le w$*; write* $\beta(w,i)$ *for it; and*

(b) for every finite sequence $a_0, \dots, a_{k-1}$ *of natural numbers there is* $w \in \mathbb{N}$ *with* $\beta(w,i) = a_i$ *for every* $i < k$*.*

Status: complete proof.

Proof. Boundedness: every quantifier in the display is marked bounded, the final conjunct is the marked-bounded form of $v < d$ as recorded above, and the remaining conjuncts are atomic.

(a) Since $\mathrm{pair}_{\mathbb{N}}$ is a bijection, exactly one pair (t,m) satisfies $\mathrm{pair}_{\mathbb{N}}(t,m) = w$, and lemma 19.8 gives $t \le w$ and $m \le w$, so the displayed bounds $t < Sw$ and $m < Sw$ do not exclude this pair, while $\mathsf{P}(t,m,w)$ excludes every other. The term d evaluates to $1 + (i+1)m \ge 1$, so Euclidean division of t by that value determines q and v uniquely via $t = (1 + (i+1)m)\, q + v$ with $v < 1 + (i+1)m$; moreover $q \le t < St$ and $v \le t \le w$, so the bound on q is respected and $v \le w$.

(b) Given $a_0, \dots, a_{k-1}$, put $N := \max(k, a_0, \dots, a_{k-1}) + 1$ and $m := N!$. The moduli $1 + (i+1)m$ for $i < k$ are pairwise coprime: a prime p dividing both $1 + (i+1)m$ and $1 + (j+1)m$ with $i < j < k$ divides their difference $(j-i)m$ but not m, hence divides $j - i$; yet $0 < j - i < k \le N$ gives $p \le N$, so p divides $N! = m$, a contradiction. Each $a_i < N \le m < 1 + (i+1)m$. By the Chinese remainder theorem for pairwise coprime moduli there is t with $t \equiv a_i \pmod{1 + (i+1)m}$ for every $i < k$, and since $a_i < 1 + (i+1)m$, the remainder of t on division by $1 + (i+1)m$ is exactly a_i. Put $w := \mathrm{pair}_{\mathbb{N}}(t,m)$; then $\beta(w,i) = a_i$ for every $i < k$. □

Gödel's original beta-function argument [25] and the treatment in [9, ch. 16] are parallel reading for lemma 19.9.

Lemma 19.10 (Pairing contraction)**.** *Let* θ *be an* $\mathcal{L}_A$*-formula and let* z *be a variable not free in* θ *and distinct from* x, y*. Then* $\mathcal{N}$ *satisfies, under every variable assignment,*

$$\exists x\, \exists y\, \theta \;\leftrightarrow\; \exists z\, \exists x < Sz\, \exists y < Sz\, (\mathsf{P}(x,y,z) \wedge \theta),$$

and dually

$$\forall x\, \forall y\, \theta \;\leftrightarrow\; \forall z\, \forall x < Sz\, \forall y < Sz\, (\mathsf{P}(x,y,z) \to \theta).$$

Status: complete proof.

Proof. Fix an assignment. If $a, b \in \mathbb{N}$ satisfy θ at x, y, put $c := \mathrm{pair}_{\mathbb{N}}(a,b)$; then $\mathsf{P}(a,b,c)$ holds in $\mathcal{N}$ by lemma 19.8, and its monotonicity bounds give $a, b \le c < c + 1$, so a and b witness the two marked bounded quantifiers under $z \mapsto c$. Conversely, witnesses of the right side satisfy θ in particular. Since z is not free in θ and differs from x, y, no capture occurs and θ keeps its value. The dual equivalence is the first applied to $\neg\theta$, negating both sides and moving negations inward. □

Lemma 19.11 (Bounded-quantifier exchange)**.** *Let* θ *be an* $\mathcal{L}_A$*-formula, let* t *be an* $\mathcal{L}_A$*-term with* $y, w \notin \mathrm{Var}(t)$*, and let* w *be fresh. Then* $\mathcal{N}$ *satisfies, under every variable assignment,*

$$\forall y < t\, \exists x\, \theta \;\leftrightarrow\; \exists w\, \forall y < t\, \exists x < w\, \theta, \qquad \exists y < t\, \forall x\, \theta \;\leftrightarrow\; \forall w\, \exists y < t\, \forall x < w\, \theta.$$

Both are equivalences of satisfaction in the standard structure, used below for the boldface definability classes only; they are not derivability claims in any formal theory of arithmetic. Status: complete proof.

Proof. Fix an assignment and consider the first equivalence. Right to left is immediate: a witness below w is in particular a witness. Left to right: the assignment gives t a value n, and for each of the finitely many $b < n$ there is by hypothesis a witness $x_b \in \mathbb{N}$ for θ at $y \mapsto b$; let c be 0 when $n = 0$ and $\max_{b<n} x_b + 1$ otherwise. Then $w \mapsto c$ makes $\forall y < t\, \exists x < w\, \theta$ true. Freshness of w and $y, w \notin \mathrm{Var}(t)$ prevent capture and keep the value of t fixed. The second equivalence is the first applied to $\neg\theta$, negating both sides and moving negations inward. □

Call two $\mathcal{L}_A$-formulas *$\mathcal{N}$-equivalent* iff exactly the same variable assignments satisfy them in $\mathcal{N}$. Call a formula *prefix-form* Σ^0_n iff it consists of n alternating blocks of unbounded quantifiers, the first block existential, applied to a bounded arithmetic formula in negation normal form, and *prefix-form* Π^0_n dually, with the first block universal. Every prefix-form formula lies in the corresponding syntactic class, since the generating clauses allow each unbounded quantifier to be prepended at the appropriate level. The cited hierarchy sources define their classes by prefix forms; the next theorem identifies the two presentations.

Theorem 19.12 (Prefix normal form)**.** *For every $n \geq 0$:*

(a) the relations defined in $\mathcal{N}$ by prefix-form Σ^0_n formulas are closed, up to $\mathcal{N}$-equivalence of the defining formulas, under $\wedge$, $\vee$, and both marked bounded quantifiers, and for $n \geq 1$ also under unbounded existential quantification; dually for Π^0_n with unbounded universal quantification;

(b) every Σ^0_n formula is $\mathcal{N}$-equivalent to a prefix-form Σ^0_n formula, and every Π^0_n formula to a prefix-form Π^0_n formula; and

(c) the boldface classes $\mathbf{\Sigma}^0_n(k)$, $\mathbf{\Pi}^0_n(k)$, and $\mathbf{\Delta}^0_n(k)$ coincide with the classes of relations defined in $\mathcal{N}$ by prefix-form formulas of the corresponding kinds.

Status: complete proof.

Proof. Two preliminary observations. *Padding*: for $m < n$, every prefix-form formula of level m is $\mathcal{N}$-equivalent to a prefix-form Σ^0_n formula and to a prefix-form Π^0_n formula, obtained by prepending and interleaving quantifiers on fresh variables; each added quantifier is vacuous because the domain $\mathbb{N}$ is nonempty. *Normal-form bookkeeping*: an implication $\mathsf{P} \to \theta$ introduced by the dual contraction of lemma 19.10 is written $\neg\mathsf{P} \vee \theta$, which is in negation normal form because P is atomic.

(a) Induction on n, both classes at once; the Π^0_n case is dual throughout, exchanging $\exists$ with $\forall$, $\wedge$ with $\vee$ where the argument negates, and lemmas 19.10 and 19.11 with their dual halves.

Level $n = 0$: a prefix-form Σ^0_0 or Π^0_0 formula is a bounded formula in negation normal form, and $\wedge$, $\vee$, and marked bounded quantification produce bounded formulas in negation normal form syntactically; no unbounded closure is claimed at this level.

Level $n \geq 1$: a prefix-form Σ^0_n formula is $\mathcal{N}$-equivalent, by iterating the contraction of lemma 19.10 inside its first block, to one of the form $\exists x\, B$ with B prefix-form Π^0_{n-1}; the residual bounded quantifiers and the conjuncts P introduced by each contraction are absorbed into B by the induction hypothesis for level $n - 1$. It therefore suffices to treat formulas $\exists x\, B$ and $\exists x'\, B'$ with B, B' prefix-form Π^0_{n-1}.

(i) $\wedge$, $\vee$: rename the bound variables apart, so that $x \notin \mathrm{FV}(B')$ and $x' \notin \mathrm{FV}(B)$, and pull the existentials out: $\exists x\, B \wedge \exists x'\, B'$ is $\mathcal{N}$-equivalent to $\exists x\, \exists x'(B \wedge B')$, and $\exists x\, B \vee \exists x'\, B'$ to $\exists x\, \exists x'(B \vee B')$, the latter because the quantifier on the disjunct that did not supply the witness is vacuous over the nonempty domain. Contract the two existentials by lemma 19.10; the resulting matrix applies two marked bounded quantifiers and a conjunction with the quantifier-free P to $B \wedge B'$, respectively $B \vee B'$, each $\mathcal{N}$-equivalent to a prefix-form Π^0_{n-1} formula by the induction hypothesis; prepending $\exists z$ gives prefix-form Σ^0_n.

(ii) Marked bounded $\exists$: with x renamed so that $x \neq y$ and $x \notin \mathrm{Var}(t)$, $\exists y < t\, \exists x\, B$ is $\mathcal{N}$-equivalent to $\exists x\, \exists y < t\, B$ by pure logic; the induction hypothesis absorbs $\exists y < t$ into B, and prepending $\exists x$ gives prefix-form Σ^0_n.

(iii) Marked bounded $\forall$: by lemma 19.11, $\forall y < t\, \exists x\, B$ is $\mathcal{N}$-equivalent to $\exists w\, \forall y < t\, \exists x < w\, B$; the induction hypothesis absorbs the two marked bounded quantifiers into B, and prepending $\exists w$ gives prefix-form Σ^0_n.

(iv) Unbounded $\exists$: contract $\exists x'\, \exists x\, B$ by lemma 19.10 and absorb as in (i).

(b) By the hierarchy definition, an arbitrary formula is Σ^0_n or Π^0_n exactly when its fixed negation-normal-form conversion lies in the corresponding generated class, and each conversion step — implication elimination, the De Morgan and quantifier-duality rewrites, the bounded dualities, and capture-avoiding renaming — preserves satisfaction under every assignment, so every formula is $\mathcal{N}$-equivalent to its conversion. It therefore suffices to prove the claim for the members of the generated classes, by induction on n and, within each level, by induction on the generating clauses of the syntactic hierarchy definition. At level 0 the generated classes consist of the bounded formulas in negation normal form, which are exactly the prefix-form formulas of level 0. At level $n+1$, the base formulas are the Σ^0_n and Π^0_n formulas, which are $\mathcal{N}$-equivalent to prefix-form formulas of level n by the induction hypothesis and hence, by padding, to prefix-form Σ^0_{n+1} formulas; each closure clause — $\wedge$, $\vee$, either marked bounded quantifier, unbounded $\exists$ — preserves $\mathcal{N}$-equivalence to prefix form by (a) at level $n+1$. The Π^0_{n+1} half is dual.

(c) By (b), a relation definable by a Σ^0_n formula is definable by a prefix-form Σ^0_n formula, and conversely every prefix-form Σ^0_n formula is a Σ^0_n formula; likewise for Π^0_n, hence for the intersections $\mathbf{\Delta}^0_n(k)$. □

Definition 19.13 (History coding)**.** Fix an index scheme storing finitely many *entry families* in a single number s: the member j of the family with tag t is the entry $\beta(s, \mathrm{pair}_{\mathbb{N}}(t, j))$, with finitely many family tags fixed once for the clauses below. A number s is a *history code* for the pair (e, x) iff its entries comprise, under this scheme:

(h1) the binary digits $d_0, \ldots, d_{L-1}$ of x, most significant digit first, with certified expansion values $v_0, \ldots, v_{L-1}$ satisfying $v_0 = d_0$, $v_{j+1} = 2v_j + d_{j+1}$ for $j + 1 < L$, and $v_{L-1} = x$, together with the certificates $v_j \leq x$ and $d_j \leq 1$ for all $j < L$;

(h2) the transition table of e: a decoding chain consisting of $\mathsf{P}(k, c_0, e)$, $\mathsf{P}(r_i, c_{i+1}, c_i)$ for $i < k - 1$, and $r_{k-1} = c_{k-1}$, unpacking the table tuple of length k into its entries $r_0, \ldots, r_{k-1}$, each entry unpacked further by fixed-depth P-steps, together with the validity checks of the fixed numbering under its designated-slot normalization;

(h3) configurations $C_0, \ldots, C_T$ of the decoded machine, each tape coded in the tape-alphabet base with certified expansions as in (h1), the configuration C_0 initial for the input word certified by (h1), and each successor configuration matching its predecessor and the decoded table by bounded digit comparisons; and

(h4) a final configuration whose state is the accepting state and whose tape prefix before the first blank is a valid binary numeral in the sense of the fixed output convention.

Every auxiliary value occurring in these clauses is an entry of s, unique by lemma 19.9(a), or a component of a pairing code, bounded by the monotonicity clause of lemma 19.8.

Theorem 19.14 (Bounded verification of computations)**.** *There is a bounded $\mathcal{L}_A$-formula $\mathsf{Comp}(e, x, s)$ such that:*

(a) for all $e, x, s \in \mathbb{N}$, $\mathcal{N} \models \mathsf{Comp}(\bar{e}, \bar{x}, \bar{s})$ iff s is a history code for (e, x), so that s codes a valid accepting computation of program e on input x in the fixed machine numbering whose final tape has a valid numerical output; and

(b) $\varphi_e(x)$ *is defined iff* $\mathcal{N} \models \exists s\, \mathsf{Comp}(\bar{e}, \bar{x}, s)$.
The symbol Comp *stands for this arithmetic formula, not for the metalinguistic relation it defines; the distinction is used in the theorem and exercises immediately below. Status: proof sketch.*

Proof sketch. Render each clause of definition 19.13 as a conjunction of arithmetic conditions on the entries of s. Every auxiliary value is an entry of s or a component of a pairing code, so every quantifier can be bounded by a term in e, x, s: entry extraction is the bounded formula B of lemma 19.9, whose own witnesses are bounded by s, and component extraction is P of lemma 19.8 with its monotonicity bounds. Every comparison conjunct $r < t$ or $r \leq t$ is written in its marked-bounded form $\exists u < t\,(r + Su = t)$, respectively $\exists u \leq t\,(u + r = t)$, with u fresh, each $\mathcal{N}$-equivalent to the plain abbreviation; the digit recurrence of (h1) is rendered with a numeral constant as $v_{j+1} = \bar{2} \times v_j + d_{j+1}$. Under the counting rule for marked bounded quantifiers, the resulting matrix is a bounded formula, which gives boundedness. Clause (a) then restates the history-code conditions. For clause (b): if $\varphi_e(x)$ is defined, the finite accepting computation of the decoded machine, its table decoding, and the certified expansions form one finite sequence of numbers, and lemma 19.9(b) supplies an s whose entries realize it; if e is not a valid code, no s passes the validity checks of (h2), matching the convention that invalid codes diverge everywhere. The omitted substantive step is the transcription of the full clause list (h1)–(h4) into one displayed formula and the verification of the step-matching digit comparisons of (h3); lemma 19.8 and lemma 19.9 carry complete proofs.

See [55, 9] for complete coding proofs using bounded checks of coded configurations and finite computation histories; those sources state their hierarchy results for prefix-form classes, and theorem 19.12 licenses transferring them to the closure classes fixed here.

Theorem 19.15. *A set is computably enumerable iff it belongs to* $\mathbf{\Sigma}^0_1$. *A set is decidable iff it belongs to* $\mathbf{\Delta}^0_1$. *Status: proof sketch.*

Proof sketch. If A is computably enumerable, membership is equivalent to the existence of a finite accepting computation. The formula Comp gives its bounded arithmetic matrix, yielding a Σ^0_1 definition. Conversely, search successive witnesses for the bounded matrix of a Σ^0_1 definition. For the second claim, $A \in \mathbf{\Delta}^0_1$ means that both A and its complement are computably enumerable; apply the earlier characterization of decidability. The omitted substantive step constructs and verifies the numerical coding of finite computation histories.

Definition 19.16 (Many-one hardness and completeness for a class)**.** Let $\mathcal{C}$ be a class of subsets of $\mathbb{N}$. A set B is *$\mathcal{C}$-hard under many-one reductions* iff $A \leq_m B$ for every $A \in \mathcal{C}$. It is *$\mathcal{C}$-complete under many-one reductions* iff, in addition, $B \in \mathcal{C}$. This term is required to distinguish an upper-bound classification from the uniform lower bound used in the next theorem. It is a property of a set relative to $\mathcal{C}$ and $\leq_m$, and is not semantic completeness, deductive completeness, or the status of a proof. The theorem and the following exercise use exactly this distinction.

Theorem 19.17 (Hierarchy classifications of program domains and totality)**.** *The set* K *is* $\mathbf{\Sigma}^0_1$*-complete under many-one reductions, and its complement is* $\mathbf{\Pi}^0_1$*-complete under many-one reductions. The totality set*

$$\mathsf{TOT} = \{e : \forall x\, \exists k \textit{ program } e \textit{ accepts on } x \textit{ with a valid numerical output within } k \textit{ steps}\}$$

is $\mathbf{\Pi}^0_2$*-complete under many-one reductions. Status: cited theorem.*

See [55] for complete proofs. The displayed prefix gives the upper bound for TOT, read into the closure classes by theorem 19.12; the completeness conclusion additionally requires a uniform many-one reduction. The cited completeness proofs classify relations by prefix-form formulas, and theorem 19.12 licenses their transfer to the classes fixed here.

Theorem 19.18 (Strictness of the arithmetical hierarchy)**.** *For every $n \geq 1$,*

$$\mathbf{\Delta}^0_n \subsetneq \mathbf{\Sigma}^0_n, \qquad \mathbf{\Delta}^0_n \subsetneq \mathbf{\Pi}^0_n, \qquad \mathbf{\Sigma}^0_n \neq \mathbf{\Pi}^0_n,$$

and $\mathbf{\Sigma}^0_n \cup \mathbf{\Pi}^0_n \subsetneq \mathbf{\Delta}^0_{n+1}$, for unary relations. Status: cited theorem.

See [55] for a complete proof. The cited proof works with prefix-form classes; theorem 19.12 transfers the statement to the closure classes fixed here.

Warning 19.19 (Syntactic prefix versus semantic class). The syntactic classification of a displayed formula supplies an upper bound on the definability class of the relation it defines. A completeness reduction or a nondefinability argument is needed to show that the relation does not lie in a lower boldface class.

Exercise 19.20. Give a Σ^0_1 formula defining the set of indices whose domain is nonempty. Prove the set is $\mathbf{\Sigma}^0_1$-complete under many-one reductions by reducing K to it.

Exercise 19.21. Classify the following index sets by giving the best immediate upper bound in the arithmetical hierarchy: finite domain, infinite domain, and cofinite domain. Do not claim completeness without constructing reductions.

Exercise 19.22. Explain why first-order validity is computably enumerable even though validity is defined by a universal quantification over all structures. Identify exactly where the completeness theorem is used.

19.3 Relative computation and Post's theorem

Many-one reducibility compares sets by a total computable translation of questions. Relative computation compares them by permitting one computation to consult a fixed set during its run.

Definition 19.23 (Oracle Turing machine and Turing reducibility)**.** An *oracle Turing machine* with oracle $B \subseteq \mathbb{N}$ is a deterministic single-tape Turing machine augmented with a distinguished query operation: at a query, the machine computes a number n and control passes to one of two designated states according as $n \in B$ or $n \notin B$. The run is otherwise governed by the fixed machine definition. An oracle Turing machine *computes* a partial numerical function by the input and output discipline fixed for ordinary machines, read verbatim with the oracle present: on the input word $\mathrm{bin}(x)$, its value is the numerical output when the run halts accepting with a valid numerical output, and its value is undefined when the run rejects, diverges, or accepts without a valid numerical output. A set $A \subseteq \mathbb{N}$ is *Turing reducible* to B, written $A \leq_T B$, iff some oracle Turing machine with oracle B decides A. A set is *computably enumerable relative to B* iff it is the domain of a partial function computed by some oracle Turing machine with oracle B.

Every many-one reduction yields a Turing reduction: translate the input and query the oracle once. The converse fails: $\mathbb{N} \setminus K \leq_T K$ by a single query with the answer reversed, while $\mathbb{N} \setminus K \not\leq_m K$, because computable enumerability is preserved downward by many-one reductions, K is computably enumerable, and $\mathbb{N} \setminus K$ is not.

Definition 19.24 (Turing jump)**.** Fix an effective numbering of oracle Turing machines analogous to the fixed machine numbering: the same designated-slot normalization, with the query rows included in the row list, and with every number that is not a valid code denoting an oracle machine that diverges on every input. For a valid code e and an oracle $B \subseteq \mathbb{N}$, let $\varphi^B_e(x)$ be the numerical

output when the decoded machine with oracle B, on the input word $\mathrm{bin}(x)$, halts accepting with a valid numerical output, and leave $\varphi_e^B(x)$ undefined when that run rejects, diverges, or accepts without a valid numerical output. The *Turing jump* of $B \subseteq \mathbb{N}$ is

$$B' := \{e \in \mathbb{N} : \varphi_e^B(e) \text{ is defined}\}.$$

Define $B^{(0)} := B$ and $B^{(n+1)} := (B^{(n)})'$; in particular $\varnothing^{(n)}$ is the n-th jump of the empty set.

Lemma 19.25 (Relativized adequacy)**.** *For every oracle $B \subseteq \mathbb{N}$:*

(a) every partial function computed by an oracle Turing machine with oracle B occurs as φ_e^B for some valid code e;

(b) there is a fixed oracle machine computing a function $U^B(e,x) \simeq \varphi_e^B(x)$ for all $e, x \in \mathbb{N}$; and

(c) for every $m, n \geq 1$ there is a total computable function $\mathsf{par}_n^m : \mathbb{N}^{m+1} \to \mathbb{N}$, not depending on B, with $\varphi^B_{\mathsf{par}_n^m(e,\bar{a})}(\bar{x}) \simeq \varphi_e^B(\bar{a}, \bar{x})$ for every index e, all argument tuples, and every oracle B simultaneously.

Status: proof sketch.

Proof sketch. Clause (a) relativizes the proof of theorem 18.2: renaming commutes with the step relation with the query rows included, the surrogate-start and automatic-alphabet arguments are unchanged, and the designated-slot normalization applies to the extended row list. Clause (b) relativizes the universal simulation behind theorem 18.3, passing each query of the simulated machine through to the oracle. Clause (c) relativizes the compiler construction given for theorem 18.4, copying the query rows unchanged; the compiled table transformation does not consult the oracle, which gives the simultaneous claim. The omitted verifications are exactly those of the results being relativized.

By lemma 19.25(a), a set is computably enumerable relative to B iff it is the domain of some φ_e^B.

Theorem 19.26 (Jump theorem)**.** *For every $B \subseteq \mathbb{N}$:*

(a) B' is computably enumerable relative to B; and

(b) $B' \not\leq_T B$.

Status: proof sketch.

Proof sketch. (a) B' is the domain of the partial function $e \mapsto U^B(e,e)$, which an oracle machine with oracle B computes by lemma 19.25(b).

(b) Relativized diagonal. Suppose some oracle machine with oracle B decides B', and let h^B be its total characteristic function, with $h^B(e) = 1$ iff $e \in B'$. Define

$$d^B(e) \simeq \begin{cases} 0, & h^B(e) = 0, \\ \text{undefined}, & h^B(e) = 1. \end{cases}$$

Then d^B is computed by an oracle machine with oracle B: run the decider, then write 0 and accept, or enter an infinite loop. By lemma 19.25(a), fix b with $\varphi_b^B \simeq d^B$. Then $b \in B'$ iff $\varphi_b^B(b)$ is defined iff $h^B(b) = 0$ iff $b \notin B'$, a contradiction.

The displayed diagonal calculation is complete, but clause (a) rests on the relativized universal machine and clause (b) on the occurrence clause of lemma 19.25, both proof sketches; the status policy of the front matter forbids complete-proof status over a sketch dependency, which is why this theorem is a proof sketch.

Theorem 19.27 (Post)**.** *For every $n \in \mathbb{N}$ and every $A \subseteq \mathbb{N}$:*

1. $A \in \mathbf{\Sigma}^0_{n+1}$ *iff* A *is computably enumerable relative to* $\varnothing^{(n)}$*; and*
2. $A \in \mathbf{\Delta}^0_{n+1}$ *iff* $A \leq_T \varnothing^{(n)}$.
Status: cited theorem.

See [55] for a complete proof; the handbook treatment in [6] develops the relativized machinery in full. The cited proofs classify relations through prefix-form formulas; theorem 19.12 transfers those classifications to the classes fixed here. Post's theorem ties the syntactic hierarchy to iterated jumps, and the first three clauses of the strictness theorem above follow from it together with the jump theorem, theorem 19.26: $\varnothing^{(n+1)}$ is computably enumerable relative to $\varnothing^{(n)}$ but not Turing reducible to it, so $\mathbf{\Sigma}^0_{n+1} \neq \mathbf{\Delta}^0_{n+1}$. The final inclusion clause additionally uses the join of $\varnothing^{(n)}$ with its complement and the completeness of $\varnothing^{(n)}$ among sets computably enumerable relative to $\varnothing^{(n-1)}$; those steps are part of the cited proof.

Remark 19.28 (Convention bridge for the jump). Write $A \equiv_m B$, *many-one equivalence*, iff $A \leq_m B$ and $B \leq_m A$; the standing is a metalinguistic relation between subsets of $\mathbb{N}$, and it is used only in this remark and the results that cite it. Four computable table transformations align this jump with the conventions of the cited sources. (i) $K \equiv_m \varnothing'$: an ordinary transition table embeds as a query-free oracle table, and an oracle machine with oracle $\varnothing$ is simulated by answering every query negatively. (ii) The halting-based jump of the cited sources is $\equiv_m$ the present B': the map $e \mapsto e^*$, where e^* simulates e and, on any halt of the simulation, writes 0 and accepts, reduces halting to definedness; the map $e \mapsto e^{**}$, where e^{**} diverges unless the simulated halt is accepting with a valid numerical output, reduces definedness to halting. All four maps are computable table transformations, made uniform in their inputs by theorem 18.4. Jump monotonicity — $A \leq_T B$ implies $A' \leq_m B'$ — is used explicitly in this alignment: replace each A-query of an oracle machine with oracle A by a run of a fixed oracle-B decider for A, then align the self-application indices by the relativized parameter theorem, lemma 19.25(c), with padding. Hence Post's theorem is insensitive to the choice between the two conventions, and the citations stand. *Status: proof sketch — the four table transformations are elementary, but the monotonicity step runs through lemma 19.25(c), itself a proof sketch.*

Part VI

Formal Arithmetic and Incompleteness

Chapter 20

Arithmetic, Representation, and Coding

This part separates three levels throughout: arithmetic in the standard structure $\mathbb{N}$, formal derivability in a theory T, and metalogical reasoning about T. Gödel's original paper is [25]; modern treatments include [55, 9].

20.1 The language and standard structure of arithmetic

The language $\mathcal{L}_A$, the standard structure $\mathcal{N}$, and the numeral notation were fixed in definition 19.5. This part retains exactly that syntax and semantics.

Definition 20.1 (Robinson arithmetic)**.** Robinson arithmetic, written Q, is the finitely axiomatized $\mathcal{L}_A$-theory consisting of the universal closures of:

$$\begin{array}{ll} Sx \neq 0, & Sx = Sy \to x = y, \\ x \neq 0 \to \exists y\, x = Sy, & x + 0 = x, \\ x + Sy = S(x + y), & x \times 0 = 0, \\ x \times Sy = (x \times y) + x. & \end{array}$$

It has no induction scheme.

The incompleteness argument of Rosser evaluates bounded numerical comparisons inside Q. The two lemmas below supply exactly those evaluations for the object-language abbreviation $\leq$ registered in the hierarchy chapter, whose expansion $r \leq t := \exists z\,(z + r = t)$ places the witness on the left of $+$. Both proofs therefore use, besides the successor axioms, only the right-recursion axioms $x + 0 = x$ and $x + Sy = S(x + y)$.

Lemma 20.2 (Bounded-case enumeration in Q)**.** *For every $n \in \mathbb{N}$,*

$$\mathsf{Q} \vdash \forall y\,(y \leq \bar{n} \to (y = \bar{0} \vee y = \bar{1} \vee \cdots \vee y = \bar{n})).$$

Status: complete proof.

Proof. Metainduction on n; each instance is a Q-derivation using only the axioms $Sx \neq 0$, $Sx = Sy \to x = y$, $x \neq 0 \to \exists y\, x = Sy$, $x + 0 = x$, and $x + Sy = S(x + y)$.

Base $n = 0$. Reason in Q from the expansion of $y \leq \bar{0}$: fix z with $z + y = \bar{0}$. By the case axiom, $y = 0$ or $y = Sw$ for some w. In the second case $S(z + w) = z + Sw = \bar{0}$ contradicts $Sx \neq 0$. Hence $y = \bar{0}$.

Step from n to $n+1$. Reason in Q from $z + y = S\bar{n}$, noting that $\overline{n+1}$ is $S\bar{n}$. If $y = 0$, the disjunct $y = \bar{0}$ holds. If $y = Sw$, then $S(z + w) = z + Sw = S\bar{n}$, so S-injectivity gives $z + w = \bar{n}$, hence $w \leq \bar{n}$; the already-derived instance for n gives $w = \bar{0} \vee \cdots \vee w = \bar{n}$, and applying S to each disjunct gives $y = \bar{1} \vee \cdots \vee y = \overline{n+1}$. In both cases $y = \bar{0} \vee \cdots \vee y = \overline{n+1}$. □

Lemma 20.3 (Dichotomy in Q)**.** *For every $n \in \mathbb{N}$,*

$$\mathsf{Q} \vdash \forall y\,(y \leq \bar{n} \vee \bar{n} \leq y).$$

Status: complete proof.

Proof. Metainduction on n, from the same axiom list.

Base $n = 0$. For every y, the witness $z := y$ gives $y + \bar{0} = y$ by the axiom $x + 0 = x$, so $\bar{0} \leq y$.

Step from n to $n+1$. Fix y; by the case axiom, $y = 0$ or $y = Sw$ for some w. If $y = 0$, the witness $z := S\bar{n}$ gives $S\bar{n} + 0 = S\bar{n}$, so $y \leq S\bar{n}$. If $y = Sw$, the already-derived instance for n, applied to w, gives either some z with $z + w = \bar{n}$, whence $z + Sw = S(z + w) = S\bar{n}$ and $y \leq S\bar{n}$, or some u with $u + \bar{n} = w$, whence $u + S\bar{n} = S(u + \bar{n}) = Sw = y$ and $S\bar{n} \leq y$. □

Definition 20.4 (Peano arithmetic)**.** Peano arithmetic, written PA, is Q together with every instance of the induction scheme

$$(\varphi(0, \bar{z}) \wedge \forall x(\varphi(x, \bar{z}) \to \varphi(Sx, \bar{z}))) \to \forall x\, \varphi(x, \bar{z}),$$

where $\varphi(x, \bar{z})$ is any $\mathcal{L}_A$-formula and the parameters $\bar{z}$ remain universally quantified in the closure.

20.2 Standard and nonstandard models of arithmetic

Definition 20.5 (Standard and nonstandard models)**.** Let T be an $\mathcal{L}_A$-theory satisfied by $\mathcal{N}$. A model $\mathcal{M}$ of T is *standard* iff $\mathcal{M} \cong \mathcal{N}$ as an $\mathcal{L}_A$-structure, and it is *nonstandard* otherwise. These terms classify models of arithmetic by isomorphism type; they do not classify sentences or proof predicates. The distinction is stronger than agreement on all sentences, as the theorem and exercise below show, and it is needed to state what compactness produces.

Theorem 20.6 (Compactness produces nonstandard models)**.** *Assume the Axiom of Choice in the ambient set theory. Every $\mathcal{L}_A$-theory T satisfied by $\mathcal{N}$ has a countable nonstandard model. Status: complete proof.*

Proof. Add one new constant c and consider

$$T \cup \{c \neq \bar{n} : n \in \mathbb{N}\}.$$

Every finite subset has an expansion of $\mathcal{N}$ as a model: interpret c by a natural number different from the finitely many numerals mentioned. By first-order compactness, the whole set has a model $\mathcal{M}'$. In its $\mathcal{L}_A$-reduct $\mathcal{M}$, the element $c^{\mathcal{M}'}$ is unequal to $\bar{n}^{\mathcal{M}}$ for every standard n. If $\mathcal{M} \cong \mathcal{N}$, every element of M would be the value of some numeral, a contradiction. Thus the reduct is a nonstandard model of T. Applying theorem 13.4 in the expanded countable signature, with $c^{\mathcal{M}'}$ among the designated elements, makes the model countable while preserving all the displayed sentences. □

See [30, 44] for further compactness constructions of nonstandard models.

Example 20.7 (Nonstandard Peano arithmetic)**.** Since $\mathcal{N} \models \mathsf{PA}$, theorem 20.6 gives a countable model of PA containing an element unequal to every numeral. The theorem is an existence result: it does not identify that element with the value of any numeral, and it does not make the resulting model isomorphic to $\mathcal{N}$.

Exercise 20.8. Apply theorem 20.6 to $T = \mathrm{Th}(\mathcal{N})$. Prove that the resulting countable model agrees with $\mathcal{N}$ on every $\mathcal{L}_A$-sentence but is not isomorphic to $\mathcal{N}$. Explain why elementary equivalence therefore does not imply isomorphism.

Warning 20.9 (Standard truth versus formal proof). The notation $\mathcal{N} \models \sigma$ says that σ is true in the standard structure. The notation $T \vdash \sigma$ says that a finite derivation exists from the axioms of T in a fixed effective calculus. Passage from $T \vdash \sigma$ to $\mathcal{N} \models \sigma$ requires soundness of the fixed calculus together with the hypothesis $\mathcal{N} \models T$. Passage from $\mathcal{N} \models \sigma$ to $T \vdash \sigma$ fails in general even though the fixed calculus is complete—this failure is the content of the first incompleteness theorem below—and holds only for restricted sentence classes, such as Σ_1^0 sentences over theories extending Q.

20.3 Representability

Definition 20.10 (Numeralwise representation of a relation)**.** Let $T \supseteq \mathsf{Q}$, and let $R \subseteq \mathbb{N}^k$. A formula $\rho_R(x_1, \ldots, x_k)$ *numeralwise represents* R in T if, for every $\bar{n} \in \mathbb{N}^k$,

$$\begin{aligned} R(\bar{n}) \quad &\Longrightarrow \quad T \vdash \rho_R(\overline{\bar{n}}), \\ \neg R(\bar{n}) \quad &\Longrightarrow \quad T \vdash \neg\rho_R(\overline{\bar{n}}). \end{aligned}$$

The two clauses concern each standard tuple of numerals; they do not assert that T proves a universally quantified decision statement.

Definition 20.11 (Representation of a function)**.** Let $T \supseteq \mathsf{Q}$. A formula $\gamma_f(\bar{x}, y)$ *represents* a total function $f : \mathbb{N}^k \to \mathbb{N}$ in T if, for every $\bar{n} \in \mathbb{N}^k$,

$$T \vdash \forall y \, \big(\gamma_f(\overline{\bar{n}}, y) \leftrightarrow y = \overline{f(\bar{n})}\big).$$

Theorem 20.12 (Representability theorem)**.** *Every primitive recursive relation is numeralwise representable in* Q *by a* Σ_1^0 *formula, and every primitive recursive function is representable in* Q*. Every computably enumerable relation is definable in* $\mathcal{N}$ *by a* Σ_1^0 *formula. Status: cited theorem.*

See [55] for a complete proof.

The construction represents the initial functions directly and preserves representability through composition and primitive recursion by quantifying over coded finite computation sequences. The sequence-coding operations and the check that a code satisfies the recursion equations are primitive recursive. For a computably enumerable relation, a finite accepting computation supplies the existential witness. This paragraph records the construction used below; the cited source supplies the complete coding proof.

Example 20.13 (Representation of addition)**.** The formula

$$\mathsf{Add}(x, z, y) := x + z = y$$

represents the total function $(m, n) \mapsto m + n$ in Q. For each standard m, n, finitely many uses of the two addition axioms give $\mathsf{Q} \vdash \bar{m} + \bar{n} = \overline{m+n}$. Equality reasoning then gives

$$\mathsf{Q} \vdash \forall y \, \big(\mathsf{Add}(\bar{m}, \bar{n}, y) \leftrightarrow y = \overline{m+n}\big).$$

This numeral-by-numeral verification uses no induction scheme.

20.4 Gödel coding

Definition 20.14 (Gödel numbering)**.** A *Gödel numbering* for the chosen formal language and calculus is an injective effective map from symbols, finite strings, formulas, and finite derivations to natural numbers such that the basic syntactic operations and predicates required below are primitive recursive on codes.

The exact numerical values are conventional; the effectiveness properties are substantive. Fix one such numbering. Write $\ulcorner E \urcorner$ for the code of the expression or derivation E. In an arithmetic formula, $\overline{\ulcorner E \urcorner}$ is the numeral denoting that code.

Definition 20.15 (Computably enumerable theory)**.** Relative to the fixed Gödel numbering, a theory $T \subseteq \mathsf{Sent}(\mathcal{L}_A)$ is *computably enumerable* iff

$$\{\ulcorner \sigma \urcorner : \sigma \in T\}$$

is a computably enumerable set of natural numbers. This term classifies the given set of axioms T; it does not mean that T is deductively closed, and it does not define T as the set of its derivable consequences. The distinction is required because the proof predicate below verifies uses of members of T by finite enumeration-stage certificates.

Example 20.16 (A finite-sequence code)**.** One illustrative component code assigns to $(a_0, \ldots, a_{k-1})$ the natural number

$$\mathrm{pair}_{\mathbb{N}}\left(k, \prod_{i<k} p_i^{a_i+1}\right),$$

where p_i is the i-th prime. For $(2, 0, 1)$, the product is

$$2^3 \cdot 3^1 \cdot 5^2 = 600,$$

so the complete code is $\mathrm{pair}_{\mathbb{N}}(3, 600)$. The exponent $a_i + 1$ rather than a_i is what distinguishes trailing zeros; pairing with k additionally makes the length available by a projection rather than by factoring. A full arithmetization usually uses a coding with especially simple primitive recursive projection and concatenation operations.

Theorem 20.17 (Arithmetization of syntax)**.** *For the fixed numbering, each of the following is primitive recursive: being a symbol of a specified kind, being a term or formula, substitution of a numeral for a free variable, being an axiom instance of a fixed finitely axiomatized theory, and being a correct finite derivation from such axioms. For a computably enumerable theory, derivations may carry finite certificates identifying the stage at which each nonlogical axiom is enumerated, again giving a primitive recursive proof predicate. Status: proof sketch.*

Proof sketch. Proceed by bounded search through the finite coded expression or derivation. Formation trees, substitutions, and rule checks are finite data. Replace an unbounded appeal to computably enumerable axiom membership by a bounded verification of the enumeration-stage certificate stored in the derivation code.

Definition 20.18 (Proof and provability predicates)**.** Let T be computably enumerable and contain Q. Fix a primitive recursive relation $\mathsf{Proof}_T(p, x)$ meaning that p codes a correct T-proof whose concluding sentence has code x. By the Σ_1^0 clause of theorem 20.12, fix the same symbol for a specific Σ_1^0 formula that numeralwise represents this relation in Q. Define

$$\mathsf{Prov}_T(x) := \exists p\, \mathsf{Proof}_T(p, x).$$

The additional existential quantifier merges with the initial existential block of the chosen representation, so Prov_T is a Σ_1^0 formula by the syntactic hierarchy definition. The coded-computation-sequence apparatus behind the Σ_1^0 clause of theorem 20.12 — the construction paragraph following it — is the β coding of lemma 19.9, so a single in-document coding serves machine tables, computation histories, and proof sequences.

Warning 20.19 (Mentioning a sentence inside arithmetic). The expression $\mathsf{Prov}_T(\varphi)$ is ill-typed if φ is a formula rather than a number term. The correct expression is $\mathsf{Prov}_T(\overline{\ulcorner \varphi \urcorner})$, often abbreviated only after the coding convention has been declared.

Exercise 20.20. Define a primitive recursive relation saying that a finite sequence is a correct addition computation $a, a+1, \dots, a+b$. Explain how the relation supports representability of addition.

Exercise 20.21. Distinguish the following three assertions by giving their types: $T \vdash \sigma$, $\mathcal{N} \models \mathsf{Prov}_T(\overline{\ulcorner \sigma \urcorner})$, and $T \vdash \mathsf{Prov}_T(\overline{\ulcorner \sigma \urcorner})$. State one hypothesis that connects each adjacent pair.

Chapter 21

Diagonalization and the First Incompleteness Theorem

21.1 The diagonal lemma

Let $\mathsf{diag}(n)$ be the code of the result of substituting the numeral $\bar{n}$ for the distinguished free variable in the one-variable formula with code n. On nonformula codes, define an arbitrary default value. The function diag is primitive recursive.

Theorem 21.1 (Diagonal lemma)**.** *Let T be any theory extending Q. For every $\mathcal{L}_A$-formula $\theta(x)$ with at most the displayed free variable, there is a sentence G such that*

$$T \vdash G \leftrightarrow \theta(\overline{\ulcorner G \urcorner}).$$

Status: cited theorem.

See [9] for a complete proof. The following construction explains the fixed point used by subsequent complete proofs. Let $D(x, y)$ represent the function diag in Q, so for each standard n,

$$\mathsf{Q} \vdash \forall y \, (D(\bar{n}, y) \leftrightarrow y = \overline{\mathsf{diag}(n)}).$$

Define

$$\beta(x) := \exists y \, (D(x, y) \wedge \theta(y)).$$

Let $b = \ulcorner \beta \urcorner$, and let G be the sentence $\beta(\bar{b})$. By construction, $\mathsf{diag}(b) = \ulcorner G \urcorner$. Representability of diag gives, already in Q,

$$\begin{aligned} G &\leftrightarrow \exists y \, (D(\bar{b}, y) \wedge \theta(y)) \\ &\leftrightarrow \theta(\overline{\mathsf{diag}(b)}) \\ &\leftrightarrow \theta(\overline{\ulcorner G \urcorner}). \end{aligned}$$

Since $T \supseteq \mathsf{Q}$, T proves the same biconditional. The cited proof supplies the primitive-recursive verification and numeralwise representability of diag on which this explanation depends.

Remark 21.2. The lemma does not create a sentence that literally contains its own inscription. It creates a sentence provably equivalent to the result of applying θ to the numeral for that sentence's code.

21.2 Consistency and soundness conditions

Definition 21.3 (Consistency)**.** A theory T is *consistent* iff there is no sentence σ such that both $T \vdash \sigma$ and $T \vdash \neg\sigma$. In the fixed classical explosive calculus, this is equivalent to $T \nvdash \bot$. If $T \vdash 0 \neq S0$, then it is also equivalent to $T \nvdash 0 = S0$; in particular this specialization applies when $T \supseteq \mathsf{Q}$. Without the displayed derivability hypothesis, nonderivability of $0 = S0$ is not a general test for consistency.

Recall from definition 16.6 that T is deductively complete when, for every sentence σ in its language, either $T \vdash \sigma$ or $T \vdash \neg\sigma$. This property of a theory is distinct from semantic completeness of a calculus, while soundness and completeness of the fixed calculus identify it with the quantified semantic condition stated there.

Definition 21.4 (Σ^0_1-soundness)**.** A theory T is Σ^0_1-*sound* iff, whenever σ is a Σ^0_1 sentence and $T \vdash \sigma$, then $\mathcal{N} \models \sigma$.

Definition 21.5 (Omega-consistency)**.** A theory T is *omega-consistent* iff there is no formula φ with $\mathrm{FV}(\varphi) \subseteq \{x\}$ such that

$$T \vdash \exists x\, \varphi(x) \quad \text{and} \quad T \vdash \neg\varphi(\bar{n}) \text{ for every } n \in \mathbb{N}.$$

Both omega-consistency and Σ^0_1-soundness imply consistency, but consistency alone is weaker. Rosser's theorem will remove the stronger hypothesis from the incompleteness conclusion.

21.3 The Gödel sentence

Let T be a computably enumerable theory extending Q. Apply theorem 21.1, in the complexity-preserving form discussed below display (G), to $\theta(x) = \neg\mathsf{Prov}_T(x)$. Obtain a Π^0_1 sentence G_T satisfying

$$T \vdash G_T \leftrightarrow \neg\mathsf{Prov}_T(\overline{\ulcorner G_T \urcorner}). \tag{G}$$

The fixed-point biconditional is in fact derivable already in Q. The complexity-preservation clause is a standard refinement of the coding construction: choose a suitable representation of substitution on codes and put the resulting fixed point back into prenex form. The fixed point of a Π^0_1 formula may then be chosen Π^0_1. This clause is cited rather than proved here; see [55].

Theorem 21.6 (Gödel's first incompleteness theorem, original form)**.** *Let T be a computably enumerable theory extending Q.*
1. If T is consistent, then $T \nvdash G_T$.
2. If T is Σ^0_1-sound, then $T \nvdash \neg G_T$.
3. Omega-consistency also suffices for $T \nvdash \neg G_T$.
Consequently, if T is omega-consistent or Σ^0_1-sound, it is not deductively complete. Status: proof sketch.

Proof sketch. Suppose $T \vdash G_T$. Because a concrete T-proof exists and the proof predicate represents concrete proofs, $T \vdash \mathsf{Prov}_T(\overline{\ulcorner G_T \urcorner})$. But (G) also gives $T \vdash \neg\mathsf{Prov}_T(\overline{\ulcorner G_T \urcorner})$. Thus T is inconsistent. This proves (1).

If T is consistent, (1) says that no standard number codes a T-proof of G_T. Hence

$$\mathcal{N} \models \neg\mathsf{Prov}_T(\overline{\ulcorner G_T \urcorner}).$$

The biconditional (G) is already a theorem of Q, not merely an arbitrary theorem of T; since $\mathcal{N} \models \mathsf{Q}$, it follows that $\mathcal{N} \models G_T$. Therefore $\neg G_T$ is a false Σ_1^0 sentence. A Σ_1^0-sound T cannot prove it, establishing (2).

For (3), suppose $T \vdash \neg G_T$. By (G),

$$T \vdash \exists p\, \mathsf{Proof}_T(p, \overline{\ulcorner G_T \urcorner}).$$

For each standard n, consistency and (1) imply that n is not the code of a T-proof of G_T. Numeralwise representation of the primitive recursive proof relation gives

$$T \vdash \neg \mathsf{Proof}_T(\bar{n}, \overline{\ulcorner G_T \urcorner})$$

for every n. These statements violate omega-consistency. The argument is complete once the cited complexity-preserving fixed-point construction and the cited representability theorem are supplied in full; those dependencies are why the present status is a proof sketch.

Warning 21.7 (What the theorem does not say). The theorem does not say that every formal theory is deductively incomplete, that G_T is unprovable in every stronger theory, or that no deductively incomplete theory can prove any consistency statement. The hypotheses and the subscript T are essential.

21.4 Rosser's strengthening

Theorem 21.8 (Rosser)**.** *Let T be a consistent computably enumerable theory extending Q. Then there is a sentence R_T such that*

$$T \nvdash R_T \qquad \text{and} \qquad T \nvdash \neg R_T.$$

Thus consistency alone suffices for deductive incompleteness. Status: proof sketch.

The original source is [52].

Proof sketch. By theorem 21.1, fix a sentence R_T with

$$\mathsf{Q} \vdash R_T \leftrightarrow \forall y\, (\mathsf{Proof}_T(y, \overline{\ulcorner R_T \urcorner}) \to \exists z \le y\, \mathsf{Proof}_T(z, \overline{\ulcorner \neg R_T \urcorner})),$$

so that R_T says: every proof of R_T has a proof of $\neg R_T$ with no larger code. The bounded comparison uses the registered object-language abbreviation $\le$.

First branch: suppose $T \vdash R_T$, and let p be the least code of such a proof. Numeralwise representation of the proof relation gives $T \vdash \mathsf{Proof}_T(\bar{p}, \overline{\ulcorner R_T \urcorner})$, so the fixed point yields $T \vdash \exists z \le \bar{p}\, \mathsf{Proof}_T(z, \overline{\ulcorner \neg R_T \urcorner})$. Consistency gives $T \nvdash \neg R_T$, so numeralwise representation gives $T \vdash \neg \mathsf{Proof}_T(\bar{q}, \overline{\ulcorner \neg R_T \urcorner})$ for each $q \le p$. Lemma 20.2 turns the bounded existential into the finite disjunction over $z = \bar{0}, \dots, \bar{p}$, each disjunct refuted by the preceding line, so T is inconsistent, contradicting consistency. No metatheoretic proof of $\neg R_T$ is produced at any point.

Second branch: suppose $T \vdash \neg R_T$, and let q be the least code of such a proof; numeralwise representation gives $T \vdash \mathsf{Proof}_T(\bar{q}, \overline{\ulcorner \neg R_T \urcorner})$. Reason inside T from the assumption $\mathsf{Proof}_T(y, \overline{\ulcorner R_T \urcorner})$. Lemma 20.3 gives $y \le \bar{q} \vee \bar{q} \le y$. In the first case, lemma 20.2 gives $y = \bar{0} \vee \dots \vee y = \bar{q}$; consistency gives $T \nvdash R_T$, so numeralwise representation gives $T \vdash \neg \mathsf{Proof}_T(\bar{k}, \overline{\ulcorner R_T \urcorner})$ for each $k \le q$; every disjunct is contradicted, and the bounded existential follows from the contradiction. In the second case, $\bar{q} \le y$ makes $z := \bar{q}$ a witness for $\exists z \le y\, \mathsf{Proof}_T(z, \overline{\ulcorner \neg R_T \urcorner})$. Discharging the assumption and applying the fixed point yields $T \vdash R_T$, so T is inconsistent, again contradicting consistency.

A complete proof supplies the cited representability theorem and diagonal lemma in full; those dependencies are why the status remains a proof sketch. The bounded-comparison steps themselves are carried by lemmas 20.2 and 20.3, which have complete proofs.

Corollary 21.9 (Undecidability of theoremhood)**.** *If T is a consistent computably enumerable extension of Q, then the set of codes of theorems of T is computably enumerable but undecidable. Status: proof sketch.*

Proof sketch. The theorem set is computably enumerable by enumeration of finite T-proofs. If it were decidable, then for each sentence σ one could decide whether $T \vdash \sigma$. Run this decider on both R_T and $\neg R_T$; the two negative answers are compatible with decidability, so this alone is not a contradiction. Instead enumerate the sentences as $\sigma_0, \sigma_1, \ldots$. Starting with the empty finite set D_0, let $D_{n+1} = D_n \cup \{\sigma_n\}$ if $T \cup D_n \cup \{\sigma_n\}$ is consistent, and otherwise let $D_{n+1} = D_n \cup \{\neg\sigma_n\}$. The consistency test is decidable: by the deduction theorem, inconsistency of the finite extension is equivalent to T proving the negation of the conjunction of its finitely many added sentences. At least one choice preserves consistency, since otherwise $T \cup D_n$ would prove both σ_n and $\neg\sigma_n$.

The theory $T^* = T \cup \bigcup_n D_n$ is computably enumerable, consistent, extends Q, and is deductively complete because a side of every sentence is among its axioms. Rosser's theorem applied to T^* gives an independent sentence, a contradiction. This derivation uses the preceding proof sketch of Rosser's theorem; a complete proof supplies its bounded proof-comparison verification.

Remark 21.10. An alternative proof reduces the self-application domain set to theoremhood by uniform representability. The proof above makes explicit why independence of one fixed sentence does not by itself immediately contradict decidability of a set.

Exercise 21.11. In the proof sketch of theorem 21.6, identify the precise uses of effectiveness, representability, consistency, and Σ_1^0-soundness.

Exercise 21.12. Show that if T is consistent, then the Gödel sentence G_T constructed above is true in $\mathcal{N}$. Explain why this does not imply that $T \cup \{G_T\}$ proves every arithmetical truth.

Exercise 21.13. Why can the proof sketch of Rosser's theorem compare proof codes with a bounded quantifier without assuming omega-consistency? State the role of the least proof code.

Chapter 22

Provability, the Second Incompleteness Theorem, and Truth

22.1 Derivability conditions

Fix a computably enumerable theory $T \supseteq \mathsf{Q}$ and use the proof predicate and associated formula $\mathsf{Prov}_T(x)$ previously fixed in definition 20.18. To reduce clutter, write $\mathsf{Prov}_T(\ulcorner\varphi\urcorner)$ for $\mathsf{Prov}_T(\overline{\ulcorner\varphi\urcorner})$.

Definition 22.1 (Hilbert–Bernays–Löb derivability conditions)**.** The provability predicate satisfies the *derivability conditions* when, for all sentences φ and ψ:
1. if $T \vdash \varphi$, then $T \vdash \mathsf{Prov}_T(\ulcorner\varphi\urcorner)$;
2.
$$T \vdash \mathsf{Prov}_T(\ulcorner\varphi \to \psi\urcorner) \to \big(\mathsf{Prov}_T(\ulcorner\varphi\urcorner) \to \mathsf{Prov}_T(\ulcorner\psi\urcorner)\big);$$
3.
$$T \vdash \mathsf{Prov}_T(\ulcorner\varphi\urcorner) \to \mathsf{Prov}_T(\ulcorner\mathsf{Prov}_T(\ulcorner\varphi\urcorner)\urcorner).$$

The first clause is a metatheoretic implication; the latter two are theorem schemes internal to T. The three clauses are cited below as D1, D2, and D3.

For $T = \mathsf{PA}$ and its fixed proof predicate, all three derivability conditions hold: clause 1 is the displayed metatheoretic implication, while PA derives the schemes in clauses 2 and 3 by arithmetizing proof concatenation and proof verification [29, 41]. For an arbitrary computably enumerable $T \supseteq \mathsf{Q}$, the results below state the three conditions as hypotheses rather than inferring them from a vague strength requirement.

Definition 22.2 (Formal consistency sentence)**.** Fix $\bot_A$ as the false arithmetic sentence $0 = S0$. Define
$$\mathsf{Con}(T) := \neg\mathsf{Prov}_T(\overline{\ulcorner\bot_A\urcorner}).$$
This formula expresses consistency relative to the fixed numbering and proof predicate.

Warning 22.3 (External and internal consistency). The metalinguistic claim that T is consistent is not literally the formula $\mathsf{Con}(T)$. Correctness of the chosen proof predicate connects them in the standard structure $\mathcal{N}$; the second incompleteness theorem concerns whether T proves this particular internal sentence.

22.2 Löb's theorem

Theorem 22.4 (Löb)**.** *Let $T \supseteq \mathsf{Q}$ be computably enumerable, and assume its previously fixed provability predicate satisfies the three derivability conditions. For every sentence φ, if*

$$T \vdash \mathsf{Prov}_T(\ulcorner \varphi \urcorner) \to \varphi,$$

then $T \vdash \varphi$. Status: complete proof.

Proof. By the diagonal lemma, choose a sentence ψ such that

$$T \vdash \psi \leftrightarrow (\mathsf{Prov}_T(\ulcorner \psi \urcorner) \to \varphi). \tag{L1}$$

From the left-to-right direction of (L1), derivability conditions D1 and D2 give

$$T \vdash \mathsf{Prov}_T(\ulcorner \psi \urcorner) \to \mathsf{Prov}_T(\ulcorner \mathsf{Prov}_T(\ulcorner \psi \urcorner) \to \varphi \urcorner). \tag{L2}$$

Condition D3 gives

$$T \vdash \mathsf{Prov}_T(\ulcorner \psi \urcorner) \to \mathsf{Prov}_T(\ulcorner \mathsf{Prov}_T(\ulcorner \psi \urcorner) \urcorner). \tag{L3}$$

Apply condition D2, internalized modus ponens, to (L2) and (L3). Then

$$T \vdash \mathsf{Prov}_T(\ulcorner \psi \urcorner) \to \mathsf{Prov}_T(\ulcorner \varphi \urcorner).$$

By the theorem's hypothesis,

$$T \vdash \mathsf{Prov}_T(\ulcorner \psi \urcorner) \to \varphi.$$

The right-to-left direction of (L1) yields $T \vdash \psi$. Condition D1 now gives $T \vdash \mathsf{Prov}_T(\ulcorner \psi \urcorner)$, and the last display yields $T \vdash \varphi$. □

22.3 The second incompleteness theorem

Theorem 22.5 (Gödel's second incompleteness theorem)**.** *Let T be a consistent computably enumerable extension of Q whose provability predicate, previously fixed in definition 20.18, satisfies the derivability conditions. Then*

$$T \nvdash \mathsf{Con}(T).$$

Status: complete proof.

Proof. Suppose $T \vdash \mathsf{Con}(T)$. From $\neg\mathsf{Prov}_T(\overline{\ulcorner \bot_A \urcorner})$, assume $\mathsf{Prov}_T(\overline{\ulcorner \bot_A \urcorner})$; negation elimination yields $\bot$, and bottom elimination yields $\bot_A$. Implication introduction therefore gives

$$T \vdash \mathsf{Prov}_T(\overline{\ulcorner \bot_A \urcorner}) \to \bot_A.$$

The hypothesis of Löb's theorem now holds with $\varphi = \bot_A$. Therefore $T \vdash \bot_A$, contradicting the assumed consistency of T. □

Warning 22.6 (Scope of the second theorem)*.* The conclusion is not "T cannot prove any sentence expressing any notion of consistency." It concerns the arithmetized proof predicate fixed in definition 20.18 and the corresponding sentence $\mathsf{Con}(T)$. Other deliberately weak predicates can fail the derivability conditions.

Example 22.7 (Relative consistency is not excluded)**.** A stronger theory T' may prove $\mathsf{Con}(T)$; the second theorem places no prohibition on such a relative result. For example, sufficiently strong standard set theories verify the existence of the standard natural number structure and its satisfaction of the axioms of PA, from which they formalize $\mathsf{Con}(\mathsf{PA})$. This is a claim about what the stronger theory proves, not a claim that its consistency alone implies $\mathsf{Con}(T)$. The theorem says that a consistent T cannot prove its own fixed consistency sentence under the stated hypotheses.

22.4 Tarski's undefinability theorem

Definition 22.8 (Arithmetical truth definition)**.** An $\mathcal{L}_A$-formula $\mathsf{Tr}(x)$ is an *arithmetical truth definition* if, for every $\mathcal{L}_A$-sentence σ,

$$\mathcal{N} \models \mathsf{Tr}(\overline{\ulcorner \sigma \urcorner}) \leftrightarrow \sigma.$$

The biconditional is evaluated in the standard structure and ranges metalinguistically over all arithmetic sentences.

Theorem 22.9 (Tarski undefinability)**.** *There is no arithmetical truth definition for the full set of true $\mathcal{L}_A$-sentences. Status: complete proof.*

Proof. Suppose $\mathsf{Tr}(x)$ were such a formula. Apply the diagonal lemma to $\neg\mathsf{Tr}(x)$. There is a sentence L such that

$$\mathsf{Q} \vdash L \leftrightarrow \neg\mathsf{Tr}(\overline{\ulcorner L \urcorner}).$$

Because $\mathcal{N} \models \mathsf{Q}$, the biconditional is true in $\mathcal{N}$. But the assumed truth definition also gives

$$\mathcal{N} \models \mathsf{Tr}(\overline{\ulcorner L \urcorner}) \leftrightarrow L.$$

Combining the two yields $\mathcal{N} \models L \leftrightarrow \neg L$, impossible in the classical standard structure. □

Corollary 22.10. *The set of Gödel numbers*

$$\{\ulcorner \sigma \urcorner : \sigma \text{ is an arithmetic sentence and } \mathcal{N} \models \sigma\}$$

is not definable in $\mathcal{N}$ by any $\mathcal{L}_A$-formula. In particular, it is not computably enumerable.

Proof. A formula defining the displayed set in $\mathcal{N}$ would be an arithmetical truth definition. Every computably enumerable set is Σ_1^0-definable, so nondefinability implies that the set is not computably enumerable. □

Remark 22.11 (Partial truth definitions). For each fixed level of the arithmetical hierarchy, arithmetic can define a satisfaction predicate for formulas of that bounded complexity. Tarski's theorem excludes one arithmetic formula correct for all arithmetic sentences at once. This distinction is essential in reflection principles and proof theory [57].

22.5 A final audit table

Claim	Required hypotheses	Common invalid strengthening
First incompleteness, Gödel sentence	T computably enumerable, $T \supseteq \mathsf{Q}$; consistency for unprovability; omega-consistency or Σ_1^0-soundness for independence of G_T, that is, for underivability of both G_T and $\neg G_T$	Every consistent theory has the displayed Gödel sentence independent under no effectiveness or arithmetic-strength assumption.

Claim	Required hypotheses	Common invalid strengthening
First incompleteness, Rosser	T consistent, computably enumerable, and $T \supseteq \mathsf{Q}$	Every consistent theory whatever, with no computable-enumerability or Q-extension hypothesis, has an independent sentence.
Second incompleteness	$T \supseteq \mathsf{Q}$ consistent and computably enumerable; the previously fixed provability predicate satisfies all three derivability conditions	No theory can prove a consistency statement about itself or any weaker theory.
Löb's theorem	Diagonal lemma and all three derivability conditions	Any informal predicate called "provable" satisfies Löb's theorem.
Tarski undefinability	One arithmetic formula purporting to define truth for all arithmetic sentences in $\mathcal{N}$	No restricted satisfaction or partial truth predicate can be defined.

Exercise 22.12. Reprove theorem 22.5 directly from a Gödel sentence by showing in T that $\mathsf{Con}(T) \to G_T$. Identify the derivability conditions used in the internal argument.

Exercise 22.13. Let T satisfy the hypotheses of Löb's theorem. Prove that if $T \vdash \mathsf{Prov}_T(\ulcorner \varphi \urcorner) \leftrightarrow \varphi$, then $T \vdash \varphi$.

Exercise 22.14. Formulate a syntactic version of Tarski's theorem: show that no consistent theory extending Q can prove every biconditional

$$\mathsf{Tr}(\overline{\ulcorner \sigma \urcorner}) \leftrightarrow \sigma$$

for all arithmetic sentences σ. Use the diagonal lemma and state where consistency is required.

Exercise 22.15. Audit the sentence "Gödel proved that true statements cannot be proved." Replace it by three distinct correct statements concerning G_T, $\mathsf{Con}(T)$, and $\mathrm{Th}(\mathcal{N})$, each with exact hypotheses.

Part VII

Axiomatic Set Theory

Chapter 23

The Axioms of ZF and ZFC

Convention 23.1 (Scoped semantic notation)**.** Throughout these advanced chapters, $\mathcal{M}$ and $\mathcal{N}$ are the normative metavariables for semantic structures. The letter $\mathcal{H}$ is reserved for the explicitly defined Henkin structure. The word *model* is reserved for a structure together with an assertion that it satisfies a specified sentence or set of sentences. Letters assigned a different local role, such as forcing names or type metavariables, are declared at the start of that scope.

Set theory supplies a common metatheory for syntax, structures, proofs, and cardinality. It can also be studied as an object theory. These two roles must remain separate. In this part, ZF and ZFC are first-order theories in the one-sorted language $\mathcal{L}_{\in} = \{\in\}$, where equality is logical identity and $\in$ is a binary relation symbol. Unless a displayed formula is explicitly quoted, it is written in the metalanguage as a representative of an object-language axiom or axiom instance.

The treatment follows the standard organization of [33]. A relative-consistency assertion below is a metalogical implication between arithmetized consistency statements. It is not an assertion that the stronger theory is absolutely consistent.

23.1 The language and the axiom schemas

Convention 23.2 (Classes)**.** A formula $\varphi(x, \bar{p})$ may be used to abbreviate the collection

$$\{x : \varphi(x, \bar{p})\}.$$

This class notation is metalinguistic. It does not assert that the collection is a set, and it does not add class variables to $\mathcal{L}_{\in}$. A class is a set iff an instance of an axiom or a theorem supplies a set having the specified members.

Definition 23.3 (Extensionality)**.** The axiom of extensionality is

$$\forall a\, \forall b\, \big(\forall x\, (x \in a \leftrightarrow x \in b) \to a = b\big).$$

It states that membership determines set identity.

Definition 23.4 (Empty set)**.** The empty-set axiom is

$$\exists a\, \forall x\, (x \notin a).$$

Extensionality makes the witness unique; it is denoted by $\varnothing$.

Definition 23.5 (Pairing, union, and power set)**.** The pairing axiom is

$$\forall a\,\forall b\,\exists c\,\forall x\,(x \in c \leftrightarrow (x = a \vee x = b)).$$

The union axiom is

$$\forall a\,\exists u\,\forall x\,(x \in u \leftrightarrow \exists y\,(y \in a \wedge x \in y)).$$

The power-set axiom is

$$\forall a\,\exists p\,\forall x\,(x \in p \leftrightarrow x \subseteq a), \qquad x \subseteq a \;:\Longleftrightarrow\; \forall z\,(z \in x \to z \in a).$$

The unique witnesses are written $\{a, b\}$, $\bigcup a$, and $\mathcal{P}(a)$, respectively. The singleton $\{a\}$ abbreviates $\{a, a\}$.

Definition 23.6 (Infinity)**.** Let $S(x) := x \cup \{x\}$. The axiom of infinity is

$$\exists I\,(\varnothing \in I \wedge \forall x\,(x \in I \to S(x) \in I)).$$

It asserts the existence of an inductive set. The set ω is defined below as the least inductive set; that leastness is a theorem using separation, not part of the axiom of infinity.

Definition 23.7 (Separation schema)**.** For every formula $\varphi(x, \bar{p})$ of $\mathcal{L}_\in$ in which b is not free, the corresponding instance of separation is

$$\forall a\,\exists b\,\forall x\,(x \in b \leftrightarrow (x \in a \wedge \varphi(x, \bar{p}))).$$

The parameters $\bar{p}$ are arbitrary sets. Separation forms a subset of a previously given set; it does not assert an unrestricted comprehension principle.

Definition 23.8 (Replacement schema)**.** For every formula $\varphi(x, y, \bar{p})$ of $\mathcal{L}_\in$ in which b is not free, the corresponding instance of replacement is

$$\begin{aligned}\forall a\,[&\forall x \in a\,\exists! y\,\varphi(x, y, \bar{p}) \to \\ &\exists b\,\forall y\,(y \in b \leftrightarrow \exists x \in a\,\varphi(x, y, \bar{p}))].\end{aligned}$$

The antecedent is a functionality requirement on the set a; the quantifier abbreviation $\exists! y\,\psi(y)$ stands for $\exists y\,(\psi(y) \wedge \forall z\,(\psi(z) \to z = y))$ with z fresh. Replacement then asserts that the image is a set. The schema is not one first-order sentence: each eligible formula supplies one axiom instance.

Definition 23.9 (The set ω)**.** By infinity, fix an inductive set I_0. By separation, form

$$\omega := \{x \in I_0 : x \text{ belongs to every inductive set}\},$$

where the membership condition abbreviates the $\mathcal{L}_\in$-formula obtained from the displayed inductive condition of the axiom of infinity. Every member of ω belongs to every inductive set, so ω is included in every inductive set. The set ω is itself inductive, because $\varnothing$ belongs to every inductive set and inductive sets are closed under S. Extensionality makes ω unique. This discharges the promise recorded with the axiom of infinity.

Definition 23.10 (Foundation)**.** The axiom of foundation is

$$\forall a\,(a \neq \varnothing \to \exists x \in a\,\forall y \in x\,(y \notin a)).$$

Thus every nonempty set has a member disjoint from it.

Definition 23.11 (Axiom of Choice)**.** This definition introduces the object-language axiom of ZFC; it is distinct in standing from the metatheoretic Axiom of Choice of section 1.6, which governs the ambient set theory. Which of the two a later argument uses is determined by whether the assertion is a formula of $\mathcal{L}_{\in}$ or a metatheoretic principle. The *Axiom of Choice* states that for every set A of nonempty sets there is a function f with domain A such that $f(x) \in x$ for every $x \in A$: here the Kuratowski pair notation is the object-language abbreviation $\langle u, v\rangle := \{\{u\}, \{u, v\}\}$. In the following display, the other two abbreviations are object-language abbreviations for the indicated membership formulas,

$$\begin{aligned} \mathrm{Fun}(f) \quad &:\Longleftrightarrow \quad \big[\forall z \in f\, \exists u\, \exists v\, z = \langle u, v\rangle\big] \ \wedge \\ &\qquad\quad \forall u\, \forall v\, \forall w\, \big((\langle u, v\rangle \in f \wedge \langle u, w\rangle \in f) \to v = w\big), \\ \mathrm{dom}(f) = A \quad &:\Longleftrightarrow \quad \forall u\, \big(u \in A \leftrightarrow \exists v\, \langle u, v\rangle \in f\big). \end{aligned}$$

Thus an exact object-language instance is

$$\forall A\, \big[\forall x \in A\, (x \neq \varnothing) \to \exists f\ (\mathrm{Fun}(f) \wedge \mathrm{dom}(f) = A \wedge \forall x \in A\, \exists y\, (\langle x, y\rangle \in f \wedge y \in x))\big].$$

The first conjunct says that every member of f is an ordered pair and that two pairs in f with the same first coordinate have the same second coordinate. The domain clause and the final conjunct then give exactly one chosen member of each member of A. Pairwise disjointness is not required.

Definition 23.12 (The theories ZF and ZFC)**.** The theory ZF consists of extensionality, empty set, pairing, union, power set, infinity, foundation, every separation instance, and every replacement instance. The theory ZFC is ZF together with the Axiom of Choice.

Remark 23.13. Some axiom lists derive the empty-set axiom from another axiom and separation. That change is an alternative axiomatization, not a different set-theoretic claim. The active axiom list above is fixed for this document.

23.2 First constructions and exact uses of the axioms

Recall that the Kuratowski ordered pair is

$$\langle a, b\rangle := \{\{a\}, \{a, b\}\}.$$

A relation is a set of ordered pairs. A function is a relation having exactly one output for each member of its domain. These definitions are eliminable in the language of membership.

Lemma 23.14 (Ordered-pair identity)**.** *For all sets a, b, c, d,*

$$\langle a, b\rangle = \langle c, d\rangle \quad \Longleftrightarrow \quad (a = c \wedge b = d).$$

Proof. The right-to-left direction follows by substitution into the definition. For the converse, compare the members of the two Kuratowski pairs by extensionality. If $a = b$, the left pair is $\{\{a\}\}$, and equality forces $c = d = a$. If $a \neq b$, then $\{a\}$ and $\{a, b\}$ are distinct; equality identifies the singleton member and then the two-element member, yielding $a = c$ and $b = d$. □

Example 23.15 (Foundation excludes a membership loop)**.** Suppose $a \in a$. Pairing supplies $A = \{a\}$. Foundation applied to A requires an $x \in A$ such that no member of x belongs to A. The only possible x is a, but $a \in a$ and $a \in A$, a contradiction. Hence $a \notin a$ for every set a.

This argument proves one consequence of foundation. It does not prove that the nonexistence of membership loops by itself is equivalent to foundation.

Theorem 23.16 (Set recursion)**.** *Let G be a class operation specified by a formula such that, for every set x and every function g with domain x, there is exactly one set $G(x, g)$. Then there is a unique class function F satisfying*

$$F(x) = G(x, F \restriction x)$$

for every set x, where the recursion is taken over the well-founded membership relation. Every restriction of F to a set is a set function.

Proof sketch. Foundation supplies well-founded induction. The *transitive closure* of a set x is the least transitive set including x, obtained as $\bigcup_{n\in\omega} u_n$ with $u_0 = x$ and $u_{n+1} = \bigcup u_n$; replacement and union make it a set, and it is distinct from the reflexive transitive closure of a binary relation defined in chapter 3. Define approximations on the transitive closure of a given set and use replacement to collect the values. Extensionality proves uniqueness by choosing a membership-minimal input at which two proposed solutions differ. A complete proof requires the previously developed coding of functions and is therefore cited to [33]; this paragraph records the proof architecture, not a line-by-line proof.

Warning 23.17 (Unrestricted comprehension). The notation $\{x : \varphi(x)\}$ does not guarantee a set. In particular, the class $\{x : x \notin x\}$ cannot be a set. Separation permits $\{x \in a : x \notin x\}$ only after a bounding set a has been supplied.

Exercise 23.18. Identify the axioms used to construct $a \times b$ as a set. Your proof must give a bounding set for the ordered pairs before applying separation.

Exercise 23.19. Show that foundation excludes a two-cycle $a \in b \in a$. Do not appeal to a diagram; apply the axiom to a specifically constructed nonempty set.

Chapter 24

The Cumulative Hierarchy, Ordinals, and Cardinals

24.1 Transitive sets and ordinals

Transitive sets and ordinals were defined in definitions 1.22 and 1.23. This chapter uses those exact definitions inside set theory rather than introducing new terms for them.

For ordinals α and β, write $\alpha < \beta$ iff $\alpha \in \beta$. Define

$$\alpha + 1 := \alpha \cup \{\alpha\}.$$

An ordinal is a *successor ordinal* iff it has this form; a nonzero ordinal that is not a successor ordinal is a *limit ordinal*. These terms name ordinal forms, not arithmetic operations on arbitrary sets.

Theorem 24.1 (Trichotomy and transitivity of ordinals)**.** *For ordinals α and β, exactly one of*

$$\alpha \in \beta, \qquad \alpha = \beta, \qquad \beta \in \alpha$$

holds. Every member of an ordinal is an ordinal.

Proof sketch. The route runs through $\alpha \cap \beta$. First, $\alpha \cap \beta$ is an ordinal: it is transitive as an intersection of transitive sets, and $\in$ strictly well-orders it as a subset of α. Second, an ordinal that is a proper subset of an ordinal is a member of it: if $\gamma \subsetneq \delta$, the well-order of δ supplies the $\in$-least element ξ of $\delta \setminus \gamma$; minimality of ξ together with transitivity of δ puts every member of ξ in γ, and linearity of $\in$ on δ together with transitivity of γ puts every member of γ in ξ, so $\gamma = \xi \in \delta$. Now if $\alpha \cap \beta$ were a proper subset of both α and β, the second step would give $\alpha \cap \beta \in \alpha \cap \beta$, contradicting the irreflexivity of the strict well-order on the ordinal $\alpha \cap \beta$; hence $\alpha \cap \beta = \alpha$ or $\alpha \cap \beta = \beta$, and the second step converts the surviving inclusion, when proper, into $\alpha \in \beta$ or $\beta \in \alpha$. At most one alternative holds: any two of them combine, using transitivity of the sets involved, into an ordinal that is a member of itself. For the final claim, a member x of an ordinal α is a subset of α by transitivity of α, so $\in$ strictly well-orders x; and x is transitive because transitivity of α puts the members of members of x inside α, where the transitivity of the strict order $\in$ on α yields the required memberships. The omitted obligations are the two elementwise membership calculations in the second step; they are supplied by [33].

Theorem 24.2 (Transfinite induction)**.** *Let $\varphi(\alpha, \bar{p})$ be a formula. If*

$$\forall \alpha \, \left[(\forall \beta < \alpha \, \varphi(\beta, \bar{p})) \to \varphi(\alpha, \bar{p}) \right],$$

then $\varphi(\alpha, \bar{p})$ holds for every ordinal α.

Proof. If the conclusion fails at some ordinal δ, separation applied to $\delta + 1$ produces the nonempty set $\{\beta \in \delta + 1 : \neg\varphi(\beta, \bar{p})\}$. The ordinal well-order supplies its least member α. Every $\beta < \alpha$ satisfies φ, so the displayed hypothesis gives $\varphi(\alpha, \bar{p})$, a contradiction. □

Theorem 24.3 (Transfinite recursion)**.** *Let G be a definable class operation assigning one set $G(f)$ to every function f whose domain is an ordinal. There is a unique class function F on the ordinals such that*

$$F(\alpha) = G(F \restriction \alpha)$$

for every ordinal α. For each ordinal γ, the restriction $F \restriction \gamma$ is a set.

Proof sketch. For each ordinal γ, use transfinite induction to construct the unique function f_γ with domain γ satisfying $f_\gamma(\alpha) = G(f_\gamma \restriction \alpha)$. At a successor stage append the uniquely specified value; at a limit stage take the union of the coherent earlier functions. Replacement collects the earlier values and separation forms the required graphs. Transfinite induction proves $f_\beta = f_\gamma \restriction \beta$ whenever $\beta < \gamma$, so their union is the asserted class function; the same induction proves uniqueness. The omitted Separation and Replacement instances are supplied in [33, Chapter 2].

Replacement is essential in the general theorem: it collects the values already constructed at earlier stages.

24.2 The cumulative hierarchy and rank

Definition 24.4 (Cumulative hierarchy)**.** By transfinite recursion define

$$V_0 = \varnothing, \qquad V_{\alpha+1} = \mathcal{P}(V_\alpha), \qquad V_\lambda = \bigcup_{\beta<\lambda} V_\beta$$

when λ is a limit ordinal. The symbol

$$V = \bigcup_{\alpha\in\mathrm{Ord}} V_\alpha$$

is class notation, not a set-forming union; Ord denotes the class of all ordinals, which the Burali-Forti argument shows to be a proper class.

Theorem 24.5 (Hierarchy theorem)**.** *Assume* ZF*. Then:*
1. every V_α is transitive;
2. if $\alpha \le \beta$, then $V_\alpha \subseteq V_\beta$;
3. every set belongs to some V_α; and
4. for every set x, there is a least ordinal α such that $x \subseteq V_\alpha$.

Proof sketch. The first two clauses follow by transfinite induction. For the third, suppose some set x belongs to no stage. Foundation, applied to the set of members of the transitive closure of $\{x\}$ that belong to no stage, supplies a membership-minimal counterexample. All of its members occur at stages; replacement bounds their stage indices by a set of ordinals. Their union gives one stage containing every member, so the counterexample appears at the next stage. The fourth clause uses ordinal well-ordering. The dependency structure is recorded; routine coding details are omitted.

Definition 24.6 (Rank)**.** The rank of a set x is

$$\operatorname{rk}(x) := \min\{\alpha : x \subseteq V_\alpha\}.$$

Equivalently,

$$\operatorname{rk}(x) = \sup\{\operatorname{rk}(y) + 1 : y \in x\},$$

where, for a set S of ordinals, $\sup S := \bigcup S$ is the least ordinal that is at least every member of S. The equivalence is a theorem obtained from the hierarchy theorem and transfinite recursion.

Example 24.7 (The first stages)**.**

$$V_0 = \varnothing, \qquad V_1 = \{\varnothing\}, \qquad V_2 = \{\varnothing, \{\varnothing\}\},$$

and $V_3 = \mathcal{P}(V_2)$ has four members. Consequently $\operatorname{rk}(\varnothing) = 0$, $\operatorname{rk}(\{\varnothing\}) = 1$, and $\operatorname{rk}(V_2) = 2$. The index of the first stage containing x as a member is $\operatorname{rk}(x) + 1$; it is not $\operatorname{rk}(x)$.

24.3 Well-founded relations and the Mostowski collapse

Definition 24.8 (Well-founded relation)**.** Let $R \subseteq A \times A$. The relation R is well-founded on A iff every nonempty subset $B \subseteq A$ has an R-minimal member:

$$\forall B \subseteq A \, \big(B \neq \varnothing \to \exists b \in B \, \forall c \in B \, \neg(cRb)\big).$$

It is extensional iff distinct elements have distinct sets of R-predecessors.

Theorem 24.9 (Mostowski collapse)**.** *Let A be a set and let $R \subseteq A \times A$ be well-founded and extensional. There are a unique transitive set M and a unique isomorphism $\pi : (A, R) \to (M, \in)$. They satisfy*

$$\pi(a) = \{\pi(b) : bRa\}$$

for every $a \in A$.

Proof sketch. Well-founded recursion—recursion along the well-founded set relation R, obtained from the set-recursion theorem via the rank function of R—defines π by the displayed equation. Replacement makes each displayed collection a set. An induction on R proves that extensionality makes π injective. Its range is transitive by the defining equation, and the same equation proves preservation and reflection of the relations. A second well-founded induction proves uniqueness. The underlying recursion principle for arbitrary well-founded set relations and the separation and replacement instances are supplied in [33, Chapter 6].

24.4 Cardinals and the Axiom of Choice

Definition 24.10 (Equipotence)**.** Sets A and B are *equipotent*, written $A \approx B$, iff a bijection $A \to B$ exists. Thus a cardinal, in the sense recalled from definition 1.23, is an ordinal not equipotent with any smaller ordinal.

In ZF, a set A has an ordinal cardinal $|A|$ iff A is well-orderable. Do not write $|A|$ for an arbitrary set before well-orderability has been established. Under the Axiom of Choice, every set is well-orderable and therefore has a unique cardinal.

Theorem 24.11 (Hartogs)**.** *For every set A, there is a least ordinal $h(A)$ for which no injection $h(A) \to A$ exists. This theorem is provable in ZF, without the Axiom of Choice.*

Proof sketch. The well-orders carried by subsets of A form a set. Collapse each such well-order to its unique ordinal and use replacement to collect the resulting ordinals. Their supremum plus one cannot inject into A. Leastness follows from ordinal well-ordering. The set-coding of well-orders by subsets of A and the replacement instance are supplied in [33].

Definition 24.12 (Order vocabulary)**.** A *partial order* on a set A is a binary relation on A that is reflexive, transitive, and *antisymmetric*: whenever it relates x to y and y to x, then $x = y$. A *partially ordered set* is a set with a fixed partial order. A *chain* is a subset on which the order is total. An *upper bound* of a subset is an element that is greater than or equal to every member of that subset. A *maximal element* is an element strictly below no element; a *greatest member* is an element greater than or equal to every element. One well-order *end-extends* another iff it extends it and every added element is greater than every original element.

Theorem 24.13 (Equivalent forms of the Axiom of Choice)**.** *Over* ZF*, each of the following implies the others:*
1. *the Axiom of Choice;*
2. *every set can be well-ordered;*
3. *every partially ordered set in which every chain has an upper bound has a maximal element; and*
4. *every product of a set-indexed family of nonempty sets is nonempty.*

Proof sketch. The implication from the Axiom of Choice to the maximal-element principle is proved by choosing, whenever possible, a strict extension and applying transfinite recursion; Hartogs' theorem prevents the construction from continuing through all ordinals. The maximal-element principle applied to partial well-orders ordered by end-extension yields a well-order of an arbitrary set. A well-order of the union of a set-indexed family of nonempty sets selects the least member of each factor, giving a member of the product. Finally, a member of every such product is a choice function. This records the implication cycle and its use of replacement; the set-coding details are imported from [33, Chapter 5].

The four clauses of the preceding equivalence theorem are equivalent over the stated base theory; they are not definitions of one another. Any use of one clause in a proof must either cite that theorem or assume the clause directly.

Definition 24.14 (Aleph hierarchy, cardinal exponentiation, CH, and GCH)**.** By transfinite recursion, $\aleph_0 := \omega$; $\aleph_{\alpha+1}$ is the least cardinal greater than $\aleph_\alpha$, which exists by Hartogs' theorem; and $\aleph_\lambda := \sup\{\aleph_\beta : \beta < \lambda\}$ at a limit ordinal λ. Under the Axiom of Choice, define $2^\kappa := |\mathcal{P}(\kappa)|$ for every cardinal κ. The *continuum hypothesis* is the sentence

$$\mathsf{CH} := (2^{\aleph_0} = \aleph_1),$$

and the *generalized continuum hypothesis* is

$$\mathsf{GCH} := \forall\alpha\,(2^{\aleph_\alpha} = \aleph_{\alpha+1}),$$

each read as the evident $\mathcal{L}_\in$-formalization of the displayed cardinal equality.

Warning 24.15 (The Axiom of Choice hidden in cardinal notation). The assertion that any two cardinalities are comparable, the assertion that every set has an ordinal cardinal of the same size, and the unrestricted use of cardinal arithmetic all use consequences of the Axiom of Choice. A proof in ZF must not import those principles through notation.

Exercise 24.16. Prove that $\alpha \subseteq \beta$ whenever $\alpha \leq \beta$ are ordinals. Identify where transitivity is used, and explain why no transfinite induction is required.

Exercise 24.17. Compute $\mathrm{rk}(\{\varnothing, \{\varnothing\}\})$ directly from the recursive rank equation and verify the result from the hierarchy definition.

Exercise 24.18. Explain why the Mostowski collapse can fail to be injective when extensionality of R is removed. Supply a finite counterexample.

Chapter 25

Constructibility, Forcing, and Relative Consistency

25.1 The constructible hierarchy

Definition 25.1 (Definable subsets)**.** For a set A, let $\mathrm{Def}(A)$ be the collection of subsets of A definable over the structure $(A, \in)$ by a first-order formula with parameters from A:

$$\mathrm{Def}(A) := \{X \subseteq A : \begin{array}{c}\text{for some formula } \varphi(y, \bar{z}) \text{ and } \bar{a} \in A^{<\omega}, \\ X = \{x \in A : (A, \in), s_{x,\bar{a}} \models \varphi\}\end{array}\}.$$

Here $A^{<\omega}$ denotes the set of finite tuples from A, and $s_{x,\bar{a}}$ maps y to x and each displayed variable in $\bar{z}$ to the corresponding entry of $\bar{a}$; its values on other variables are irrelevant. The satisfaction relation in this definition is the set-sized satisfaction relation constructed in the metatheory. It is not a truth predicate internal to ZF for the entire universe.

Definition 25.2 (Constructible hierarchy)**.** Define by transfinite recursion

$$L_0 = \varnothing, \qquad L_{\alpha+1} = \mathrm{Def}(L_\alpha), \qquad L_\lambda = \bigcup_{\beta<\lambda} L_\beta$$

for limit λ, and write

$$L = \bigcup_{\alpha \in \mathrm{Ord}} L_\alpha.$$

A set is constructible iff it belongs to L.

The ordinary Tarskian satisfaction relation fixed earlier takes a set-sized structure on its left and therefore cannot take the proper class L as an argument. To state the next result without adding class structures or a class truth predicate, use the following formula-by-formula abbreviation. For a standard formula φ of $\mathcal{L}_\in$, let φ^L be its metalinguistic relativization to the displayed definable class: atomic formulas are unchanged, the operation commutes with the propositional connectives, and

$$(\exists x\, \psi)^L := \exists x\, (x \in L \wedge \psi^L), \qquad (\forall x\, \psi)^L := \forall x\, (x \in L \to \psi^L).$$

Here $x \in L$ is eliminated using the formula defining constructibility. Accordingly, $L \models \sigma$ abbreviates the ambient assertion σ^L, and $L \models T$ means that this holds for every standard sentence, including every displayed schema instance, represented by the theory expression T. This is a formula-by-formula metalinguistic convention. It neither treats the proper class L as a set-sized structure nor posits a single truth predicate for all formula codes.

Theorem 25.3 (The constructible class L)**.** *Assume* ZF *in the ambient universe. The class L is transitive and contains every ordinal.*

Proof sketch. The recursion defining L yields transitivity and containment of all ordinals: transfinite induction shows that every level L_α is transitive and that $\alpha \in L_{\alpha+1}$. The omitted verification is the level-by-level induction, supplied by [33].

Cited theorem (relativization scheme for the constructible class). For every axiom σ of $\mathsf{ZFC} + \mathsf{GCH} + (V = L)$, including every instance of the separation and replacement schemas,

$$\mathsf{ZF} \vdash \sigma^L,$$

where $(\cdot)^L$ is the relativization convention displayed above. Each instance of the scheme is a theorem of ZF about the definable class L; the proofs—reflection to set-sized levels, the canonical definable well-order of L, and condensation—are supplied by [33, Chapter 13, pp. 175–200]. No satisfaction relation and no consistency implication is asserted here; the exact relative-consistency statements are derived from this scheme in the final section of this chapter.

25.2 Forcing over a countable transitive set: a pedagogical device

Convention 25.4. For the remainder of this section, fix an externally countable transitive set M such that the membership structure $(M, \in)$ satisfies ZFC, and fix

$$(\mathbb{P}, \leq, 1_{\mathbb{P}}) \in M,$$

where $(\mathbb{P}, \leq)$ is a partially ordered set and $1_{\mathbb{P}}$ is its greatest member. The order is oriented so that $q \leq p$ means that q is stronger than p. In this section p, q, d range over members of $\mathbb{P}$, while μ, ν range over $\mathbb{P}$-names. This is an unnamed local convention on an ordinary poset, not the introduction of another kind of order. The countability and transitivity assumptions are explicit pedagogical hypotheses stronger than the arithmetical assertion $\mathsf{Con}(\mathsf{ZFC})$.

Remark 25.5 (Why the fixed hypothesis exceeds the consistency assertion)*.* The formal consistency sentence $\mathsf{Con}(\mathsf{ZFC})$ of definition 22.2 is a sentence of arithmetic, while ZFC is a theory in $\mathcal{L}_\in$; comparing the convention's hypothesis with that sentence requires a translation of arithmetic into the membership language, and none has been fixed so far. Fix the canonical interpretation: numbers are the finite ordinals, zero is $\varnothing$, successor is $S(x) = x \cup \{x\}$, and addition and multiplication are the recursive operations on ω. For an arithmetic sentence σ, write $\sigma^\in$ for its $\mathcal{L}_\in$-translation under this interpretation; the notation is a metalinguistic abbreviation whose scope is this chapter.

Suppose the hypothesized M exists. Then $(M, \in)$ is a set-sized structure satisfying ZFC, so soundness (theorem 11.6) gives metatheoretic consistency of ZFC. Correctness of the fixed proof predicate—the warning on external and internal consistency in the second-incompleteness chapter—then makes $\mathsf{Con}(\mathsf{ZFC})$ true in the standard structure $\mathcal{N}$. Absoluteness of arithmetic statements over transitive models containing ω [33, Chapter 12, pp. 155–174] gives $(M, \in) \models \mathsf{ZFC} + \mathsf{Con}(\mathsf{ZFC})^\in$, and soundness again gives consistency of $\mathsf{ZFC} + \mathsf{Con}(\mathsf{ZFC})^\in$. That consistency is not provable in $\mathsf{ZFC} + \mathsf{Con}(\mathsf{ZFC})^\in$ itself. Theorem 22.5 does not apply verbatim: it covers computably enumerable theories $T \supseteq \mathsf{Q}$ in the language of arithmetic, and $\mathsf{ZFC} + \mathsf{Con}(\mathsf{ZFC})^\in$ is a membership-language theory. It applies transported across the canonical interpretation, under which the fixed provability predicate still satisfies the three derivability conditions. So the fixed hypothesis strictly exceeds $\mathsf{Con}(\mathsf{ZFC})$. The omitted verifications are the translation convention itself, the absoluteness computation, and the interpreted derivability-condition bookkeeping. *Status: proof sketch.*

Definition 25.6 (Poset filter, dense set, and genericity)**.** *Necessity.* The qualifier *poset* is required because the previously used model-theoretic filter on $\mathcal{P}(I)$ and the filters used here have different kinds of members and closure clauses.

Definition. A poset filter on the fixed $(\mathbb{P}, \leq)$ is a nonempty set $G \subseteq \mathbb{P}$ satisfying

$$\begin{aligned} &p \in G \text{ and } p \leq q \quad \Longrightarrow \quad q \in G, \\ &p, q \in G \quad \Longrightarrow \quad \exists d \in G\, (d \leq p \wedge d \leq q). \end{aligned}$$

Thus the first clause closes G upward toward weaker members of $\mathbb{P}$, and the second makes it downward directed toward common strengthenings.

Standing. The phrase *poset filter* is a metalinguistic predicate of subsets G of the fixed poset $\mathbb{P}$; it is not an object-language formula or a semantic structure.

Nonredundancy. A model-theoretic filter on $\mathcal{P}(I)$ has subsets of I as its members, is upward closed under inclusion, and is closed under finite intersections. A poset filter is a subset of $\mathbb{P}$ and uses upward closure in $\leq$ together with directed common strengthenings. Neither closure specification supplies the other without additional structure.

Use. The qualified term is used below to define M-genericity, to interpret $\mathbb{P}$-names, and to prove monotonicity of the semantic forcing relation.

A set $D \subseteq \mathbb{P}$ is dense iff

$$\forall p \in \mathbb{P}\, \exists q \in D\, (q \leq p).$$

A poset filter $G \subseteq \mathbb{P}$ is M-generic iff

$$\forall D \in M\, (D \subseteq \mathbb{P} \text{ is dense} \to G \cap D \neq \varnothing).$$

Genericity is evaluated in the ambient universe. Because M is transitive, a statement built from membership whose quantifiers are all bounded by members of M—such as $p, q \in \mathbb{P}$, $q \leq p$, or $D \subseteq \mathbb{P}$—holds in $(M, \in)$ iff it holds in the ambient universe; this is verified by induction on such bounded statements, and it is the only transfer between M and the ambient universe that this section uses.

Lemma 25.7 (Existence of a generic filter under the fixed hypothesis)**.** *For every $p \in \mathbb{P}$, an M-generic poset filter containing p exists in the ambient universe.*

Proof. Externally enumerate the dense subsets of $\mathbb{P}$ belonging to M as $D_0, D_1, \ldots$. Choose $p_0 \leq p$, and recursively choose $p_{n+1} \leq p_n$ with $p_{n+1} \in D_n$. Let

$$G = \{q \in \mathbb{P} : \exists n\, (p_n \leq q)\}.$$

Then $p \in G$, G is upward closed, and any two members of G have a common strengthening $p_k \in G$ for a sufficiently large k. Moreover, $p_{n+1} \in G \cap D_n$ for every n. Hence G is an M-generic poset filter containing p. □

Definition 25.8 (Names, interpretation, and extension)**.** Define the hierarchy of $\mathbb{P}$-names by ordinal recursion:

$$V_\alpha^{\mathbb{P}} := \mathcal{P}\left(\left(\bigcup_{\beta<\alpha} V_\beta^{\mathbb{P}}\right) \times \mathbb{P}\right), \qquad V^{\mathbb{P}} := \bigcup_{\alpha \in \mathrm{Ord}} V_\alpha^{\mathbb{P}}.$$

A $\mathbb{P}$-name is a member of the class $V^{\mathbb{P}}$. Thus each member of a name ν is a pair $\langle \mu, p \rangle$, where μ has lower name rank and $p \in \mathbb{P}$. For a poset filter G, rank recursion defines

$$\nu^G := \{\mu^G : \exists p \in G\, (\langle \mu, p \rangle \in \nu)\}.$$

For each $x \in M$, the notation $\check{x}$ is needed to exhibit a $\mathbb{P}$-name in M whose interpretation is x. Define it by rank recursion:

$$\check{x} := \{\langle \check{y}, 1_{\mathbb{P}} \rangle : y \in x\}.$$

Its standing is a set of name–condition pairs, hence a $\mathbb{P}$-name, rather than the set x itself. This distinction is nonredundant because x need not have the pair structure required of a name. Because a nonempty poset filter contains $1_{\mathbb{P}}$, induction on the rank of x gives $\check{x} \in M \cap V^{\mathbb{P}}$ and $(\check{x})^G = x$. This calculation is the exact link by which members of M enter $M[G]$, and the notation is used below in the name for the Cohen real. It is not an appeal to the cited forcing theorem below. The associated extension is the set

$$M[G] := \{\nu^G : \nu \in M \cap V^{\mathbb{P}}\}.$$

Both recursions are well founded on ordinal rank; neither clause presupposes a forcing relation.

Definition 25.9 (Semantic forcing under the fixed hypothesis)**.** *Necessity.* The qualifier *semantic* is required to distinguish the exact external relation defined here from the separate recursively defined forcing relation whose construction is only cited below.

Definition. Let φ be a formula of the language of membership whose free variables are among the pairwise distinct variables $v_1, \dots, v_n$, let $\nu_1, \dots, \nu_n \in M \cap V^{\mathbb{P}}$, and let $p \in \mathbb{P}$. In this section the notation

$$p \Vdash^{\mathrm{sem}}_{M,\mathbb{P}} \varphi(\nu_1, \dots, \nu_n)$$

means exactly that, for every M-generic poset filter $G \subseteq \mathbb{P}$ with $p \in G$ and every assignment s_G into $M[G]$ satisfying $s_G(v_i) = \nu_i^G$ for $1 \le i \le n$,

$$(M[G], \in), s_G \models \varphi.$$

The preceding existence lemma prevents this universal condition from being vacuous.

Standing. For the fixed M and $\mathbb{P}$, semantic forcing is a metalinguistic relation among a member p of $\mathbb{P}$, a membership-language formula, and a tuple of $\mathbb{P}$-names.

Nonredundancy. This relation is not ordinary satisfaction: it quantifies over every M-generic poset filter containing p. It is also not the recursively defined relation used inside M to prove definability and preservation; no such recursive relation is defined in this manuscript.

Use. The notation is used immediately in the monotonicity lemma and later in the source-pinned comparison with recursive forcing.

Lemma 25.10 (Monotonicity of semantic forcing)**.** *Let the formula and names be as in the preceding definition. If $q \le p$ and*

$$p \Vdash^{\mathrm{sem}}_{M,\mathbb{P}} \varphi(\nu_1, \dots, \nu_n),$$

then

$$q \Vdash^{\mathrm{sem}}_{M,\mathbb{P}} \varphi(\nu_1, \dots, \nu_n).$$

Proof. Let G be any M-generic poset filter with $q \in G$, and let s_G be any assignment having the values required in the semantic-forcing definition. Since $q \le p$, upward closure of the poset filter gives $p \in G$. The hypothesis at p therefore yields $(M[G], \in), s_G \models \varphi$. Since G and s_G were arbitrary, the defining universal condition holds at q. □

Cited theorem (forcing theorem). Fix the countable transitive $(M, \in)$, the poset $\mathbb{P} \in M$, and the name interpretation already displayed. For every fixed standard formula $\varphi(v_1, \dots, v_n)$ of $\mathcal{L}_\in$, Jech's forcing development [33, pp. 201–224] constructs, by simultaneous recursion on name rank in the membership and equality cases and then by recursion on formula complexity, the source's forcing assertion for triples $(p, \varphi, \bar{\nu})$. For $p \in \mathbb{P}$, $\bar{\nu} \in (M \cap V^{\mathbb{P}})^n$, and every M-generic poset filter G, it proves the four claims below. In item 3, $\bar{\nu}^G$ abbreviates the interpreted tuple $(\nu_1^G, \dots, \nu_n^G)$, distinguishing it from the tuple of names and supplying exactly the parameter values used there.

1. for that fixed φ, an $\mathcal{L}_\in$-formula, with the fixed poset as a parameter, represents the recursive assertion over $(M, \in)$;
2. the recursive assertion at p holds iff $p \Vdash^{\mathrm{sem}}_{M,\mathbb{P}} \varphi(\bar{\nu})$;
3. $(M[G], \in) \models \varphi(\bar{\nu}^G)$ iff some $p \in G$ satisfies the recursive assertion for $\varphi(\bar{\nu})$; and
4. $(M[G], \in)$ satisfies ZFC and has the same ordinals as $(M, \in)$.

The first clause is formula-by-formula definability, not a single internal truth predicate ranging over all standard formula codes. The four conclusions are imported as a cited theorem; this manuscript has defined only the external semantic relation used in item 2.

Internal proof boundary. To turn that citation into a complete proof here would require printing the two simultaneous atomic clauses, the recursive clauses for the active primitive logical symbols $\bot, \neg, \wedge, \vee, \to, \forall, \exists$, and the precise recursion measure; proving existence, uniqueness, monotonicity, and formula-by-formula definability; proving the density and truth lemmas in every constructor case; and verifying every ZFC axiom in the extension, with the Replacement argument stated in full. Those source-dependent steps are not reproduced, so no local recursive relation or local completeness claim is introduced.

Example 25.11 (One Cohen real)**.** Let

$$\mathbb{P} = \mathrm{Fn}(\omega, 2, <\omega),$$

the finite partial functions from ω to 2, ordered by reverse inclusion. A name is necessary here because the generic function depends on the external filter G, while membership in $M[G]$ must be witnessed by an object already in M. Define

$$\dot{c} := \{\langle \langle \check{n}, i \rangle, p \rangle : p \in \mathbb{P},\ n \in \mathrm{dom}(p),\ p(n) = i\}.$$

The symbol $\dot{c}$ stands for a $\mathbb{P}$-name in M, not for the function c in the extension; the dot records precisely this type difference. This is nonredundant with $\check{x}$: check names interpret as fixed members of M, whereas $\dot{c}$ interprets differently with the generic filter. The ground-set-name recursion and separation in M give $\dot{c} \in M \cap V^{\mathbb{P}}$. Directly from name interpretation,

$$\begin{aligned}(\dot{c})^G &= \{\langle n, i \rangle : \exists p \in G\, (p(n) = i)\} \\ &= \bigcup G.\end{aligned}$$

Thus, if $c := \bigcup G$, then $c = (\dot{c})^G \in M[G]$; this immediate use is the reason for introducing $\dot{c}$. Directedness of G makes the union single-valued, and for each n the set

$$D_n = \{p \in \mathbb{P} : n \in \mathrm{dom}(p)\}$$

is dense and belongs to M, so genericity makes c a total function $\omega \to 2$. Write 2^ω for the set of total functions $\omega \to 2$. For $b \in M \cap 2^\omega$, the set

$$E_b = \{p \in \mathbb{P} : \exists n \in \mathrm{dom}(p)\, (p(n) \neq b(n))\}$$

belongs to M and is dense: extend any condition at one fresh coordinate with the value opposite to b. Since $G \cap E_b \neq \varnothing$, $c \neq b$. Hence $c \notin M$, while the displayed name calculation proves that the extension contains this new real. It does not by itself prove $\neg\mathsf{CH}$; that conclusion requires a poset adding enough reals together with cardinal-preservation arguments.

25.3 Exact relative-consistency status

Lemma 25.12 (Relativization of derivations)**.** *Let $\varphi_1, \dots, \varphi_k, \psi$ be formulas of $\mathcal{L}_\in$ whose free variables are among the pairwise distinct variables $v_1, \dots, v_m$. If $\{\varphi_1, \dots, \varphi_k\} \vdash \psi$ in the fixed calculus $\mathsf{ND}_{\mathsf{FOL}}$, then*

$$\mathsf{ZF} \vdash \forall v_1 \cdots \forall v_m \Big(\bigwedge_{j \le m} v_j \in L \wedge \bigwedge_{i \le k} \varphi_i^L \to \psi^L \Big).$$

For $m = 0$ or $k = 0$ the corresponding conjunction is absent from the displayed conclusion.

Proof. Strengthen the claim: for every derivation D with open assumptions Γ and conclusion χ, and for every finite list $u_1, \dots, u_r$ of pairwise distinct variables containing every variable free in $\Gamma \cup \{\chi\}$, the theory ZF derives the displayed guarded implication formed from Γ, χ, and $u_1, \dots, u_r$. The lemma is the case in which D derives ψ from $\varphi_1, \dots, \varphi_k$ and the list is $v_1, \dots, v_m$. Induct on D; throughout, argue inside ZF under the quantifiers $\forall u_1 \cdots \forall u_r$ and the guard hypotheses $u_j \in L$.

Two bookkeeping devices recur. First, $\mathsf{ZF} \vdash \exists u\,(u \in L)$: by the comprehension reading of Def, separation proves $\{x \in L_0 : \dots\} = \varnothing$ whatever the satisfaction clause returns over the empty domain, which the fixed structure convention excludes from semantic evaluation; hence $\varnothing \in \mathrm{Def}(L_0) = L_1$, so $\varnothing \in L$. This derivation does not route through the proof-sketch theorem that L contains every ordinal. Whenever the induction hypothesis supplies a claim whose variable list contains a variable u absent from the target node, instantiate the universally quantified guarded implication at this witness; the vanished variable and its guard disappear. Second, where the capture-avoiding substitution of the calculus renames bound variables, the relativizations of the formulas concerned are alpha-variants, because $(\cdot)^L$ inserts the same parameter-free guard under alpha-variant binders; the explicit α rule of $\mathsf{ND}_{\mathsf{FOL}}$ bridges them.

Assumptions, propositional rules, and classical rules. An assumption leaf χ maps to the relativized hypothesis χ^L of the guarded implication. The operation $(\cdot)^L$ commutes with the propositional connectives and $\bot^L = \bot$, so every propositional and classical rule instance maps to the same rule instance on the relativized premises, with discharged assumptions mapped to discharged relativized assumptions.

Quantifier rules. Every $\mathcal{L}_\in$-term is a variable, since the signature has no function or constant symbols. A $\forall E$ step from $\forall x\,\chi$ to $\chi[u/x]$ maps to a $\forall E$ step on $(\forall x\,\chi)^L = \forall x\,(x \in L \to \chi^L)$, followed by implication elimination against the guard $u \in L$ supplied by the antecedent. An $\exists I$ step from $\chi[u/x]$ to $\exists x\,\chi$ maps to conjunction introduction with the guard $u \in L$ followed by $\exists I$, concluding $\exists x\,(x \in L \wedge \chi^L)$. In both steps, if x is not free in χ, the instantiating variable is absent from the formulas; use the witness supplied by $\exists u\,(u \in L)$ in place of a guard from the antecedent. A $\forall I$ step with generalized variable x maps to implication introduction discharging the hypothesis $x \in L$, followed by $\forall I$; if x is not free in the premise, first conjoin the hypothesis $x \in L$ to the relativized premise and project the premise back out, so that the discharged assumption occurs in its derivation. An $\exists E$ step with eigenvariable y maps to $\exists E$ on the relativized existential premise; the minor derivation assumes $y \in L \wedge \chi^L[y/x]$, and conjunction eliminations recover the guard $y \in L$ and the relativized assumption. The eigenvariable conditions are unchanged, because the inserted guard $x \in L$ is parameter-free and adds no free variables.

Equality rules. Atomic formulas are unchanged by $(\cdot)^L$, so an $= I$ step concludes the unchanged atom. Substitution of variables for variables commutes with $(\cdot)^L$ up to the alpha-variant bookkeeping above, so an $= E$ step on φ maps to the $= E$ step on φ^L.

Alpha rule. An explicit α step maps to an α step by the second bookkeeping device.

At the root, the open relativized assumptions are $\varphi_1^L, \dots, \varphi_k^L$ and the guarded variables are $v_1, \dots, v_m$; implication introduction and universal introduction in the ambient ZF derivation yield the displayed conclusion. □

Theorem 25.13 (Relative consistency of CH)**.** *If* ZF *is consistent, then* $\mathsf{ZFC} + \mathsf{GCH} + (V = L)$ *is consistent. Hence if* ZFC *is consistent, then* $\mathsf{ZFC} + \mathsf{CH}$ *is consistent.*

Proof. Suppose a derivation concludes $\bot$ from axioms $\sigma_1, \dots, \sigma_k$ of $\mathsf{ZFC} + \mathsf{GCH} + (V = L)$. All the σ_i and $\bot$ are sentences, so lemma 25.12 with $m = 0$ and $\psi = \bot$ gives $\mathsf{ZF} \vdash \sigma_1^L \wedge \cdots \wedge \sigma_k^L \to \bot$, and the relativization scheme cited in section 25.1 gives $\mathsf{ZF} \vdash \sigma_i^L$ for each i. Conjunction introduction and implication elimination yield $\mathsf{ZF} \vdash \bot$, contradicting the consistency of ZF.

For the second claim, every ZF-derivation is a ZFC-derivation, so consistency of ZFC gives consistency of ZF and, by the first claim, of $\mathsf{ZFC} + \mathsf{GCH} + (V = L)$. That theory derives CH: universal elimination at the ordinal zero—formally, existential elimination from the ZFC-theorem that the ordinal zero exists, followed by universal elimination and equality substitutivity—produces the $\alpha = 0$ instance of the formalized GCH display, which is the formalized CH. Now suppose some derivation concludes $\bot$ with open assumptions in $\mathsf{ZFC} \cup \{\mathsf{CH}\}$. At least one open occurrence of CH remains in it, since otherwise its open assumptions lie in ZFC and $\mathsf{ZFC} \vdash \bot$, contrary to the assumed consistency of ZFC. Applying $\neg I$ at the root, discharging every such occurrence, gives $\mathsf{ZFC} \vdash \neg\mathsf{CH}$; monotonicity of the derivability relation gives $\mathsf{ZFC} + \mathsf{GCH} + (V = L) \vdash \neg\mathsf{CH}$; and negation elimination against the derivation of CH above gives $\mathsf{ZFC} + \mathsf{GCH} + (V = L) \vdash \bot$, contradicting the consistency established in the first claim. Hence $\mathsf{ZFC} + \mathsf{CH}$ is consistent. □

Cited theorem (relative consistency of the negation of CH). If ZFC is consistent, then $\mathsf{ZFC} + \neg\mathsf{CH}$ is consistent [33, Chapter 14, pp. 201–224]. The import is a family of ZFC-theorem schemes about the definable class V^B of B-valued names, where B is the Boolean completion of the poset $\mathrm{Fn}(\aleph_2 \times \omega, 2, <\omega)$ of finite partial functions from $\aleph_2 \times \omega$ to 2, and $\|\sigma\|_B$ denotes the defined ZFC-term assigning to a membership-language sentence σ with name parameters its Boolean truth value in B; both notations are scoped to this import. The cited schemes are:

1. $\|\sigma\|_B = 1$ for each axiom σ of ZFC, including every schema instance;
2. Boolean soundness: if $\{\sigma_1, \dots, \sigma_k\} \vdash \psi$, then $\mathsf{ZFC} \vdash \|\sigma_1\|_B \wedge \cdots \wedge \|\sigma_k\|_B \le \|\psi\|_B$, where $\|\bot\|_B = 0$ and ZFC proves B nontrivial, that is, $0 \neq 1$ in B;
3. B satisfies the countable chain condition, and chain-condition forcing preserves cardinals: $\|\check{\aleph}_1 = \aleph_1 \wedge \check{\aleph}_2 = \aleph_2\|_B = 1$; and
4. the distinctness density argument of the one-Cohen-real example, computed per pair of $\aleph_2$-indexed sections of the adjoined function, gives $\|2^{\aleph_0} \ge \check{\aleph}_2\|_B = 1$, hence $\|\neg\mathsf{CH}\|_B = 1$.

A derivation of $\bot$ from $\mathsf{ZFC} \cup \{\neg\mathsf{CH}\}$ would therefore give $\mathsf{ZFC} \vdash 1 = 0$ in B, contradicting nontriviality. This Boolean-valued route is the discharge demanded by the warning on countable transitive structures below; it hypothesizes no countable transitive model. The source states these schemes for its own calculus; consistency transfers to the fixed calculus $\mathsf{ND}_{\mathsf{FOL}}$ because soundness (theorem 11.6) and strong completeness (theorem 12.14) make derivability calculus-invariant.

Remark 25.14 (Arithmetized form). The two derivation transformations above are primitive recursive, so the implications $\mathsf{Con}(\mathsf{ZFC}) \to \mathsf{Con}(\mathsf{ZFC} + \mathsf{CH})$ and $\mathsf{Con}(\mathsf{ZFC}) \to \mathsf{Con}(\mathsf{ZFC} + \neg\mathsf{CH})$, read as arithmetized statements under the fixed proof predicate, are theorems of PA. The omitted verification

is the arithmetization, including a primitive recursive translation between the fixed calculus $\mathsf{ND}_{\mathsf{FOL}}$ and the source calculus of the cited theorem above: the semantic calculus-invariance argument there does not supply that translation; the deduction-theorem translation does. *Status: proof sketch.*

Theorem 25.15 (Nonderivability)**.** *If* ZFC *is consistent, then* $\mathsf{ZFC} \nvdash \mathsf{CH}$ *and* $\mathsf{ZFC} \nvdash \neg\mathsf{CH}$*.*

Proof. Suppose $\mathsf{ZFC} \vdash \mathsf{CH}$. A witnessing derivation has its open assumptions in $\mathsf{ZFC} \subseteq \mathsf{ZFC} \cup \{\neg\mathsf{CH}\}$, so monotonicity of the derivability relation gives $\mathsf{ZFC} + \neg\mathsf{CH} \vdash \mathsf{CH}$; the assumption rule gives $\mathsf{ZFC} + \neg\mathsf{CH} \vdash \neg\mathsf{CH}$; and negation elimination gives $\mathsf{ZFC} + \neg\mathsf{CH} \vdash \bot$. The contrapositive of the cited relative-consistency theorem for the negation of CH then refutes the consistency of ZFC, contrary to hypothesis. Symmetrically, if $\mathsf{ZFC} \vdash \neg\mathsf{CH}$, then monotonicity, the assumption rule, and negation elimination give $\mathsf{ZFC} + \mathsf{CH} \vdash \bot$, contradicting the consistency of $\mathsf{ZFC} + \mathsf{CH}$ supplied by theorem 25.13. □

Warning 25.16 (What the consistency theorem does not say). Neither construction proves $\mathsf{Con}(\mathsf{ZFC})$ without a consistency hypothesis. The exact second incompleteness result in theorem 22.5 applies to a consistent computably enumerable theory $T \supseteq \mathsf{Q}$ and the fixed provability predicate satisfying all three derivability conditions; under those hypotheses, T does not derive its displayed arithmetized consistency sentence.

Warning 25.17 (Countable transitive structures). The elementary presentation "take a countable transitive structure $(M, \in)$ satisfying ZFC and construct an M-generic filter" assumes more than the bare arithmetical statement $\mathsf{Con}(\mathsf{ZFC})$. Countability permits an external enumeration of the dense sets in M, and transitivity makes the name interpretation transparent. A proof of the displayed relative-consistency implication must discharge that pedagogical assumption by a syntactic, Boolean-valued, or suitable model-theoretic version of forcing. The cited relative-consistency theorem for the negation of CH above supplies exactly the Boolean-valued discharge.

Exercise 25.18. For Cohen forcing, prove that $D_n = \{p \in \mathbb{P} : n \in \operatorname{dom}(p)\}$ is dense for every $n \in \omega$. Then prove that $\bigcup G$ is a function.

Exercise 25.19. Distinguish the following three assertions: $L \models \mathsf{CH}$, $\mathsf{ZFC} \vdash V = L \to \mathsf{CH}$, and $\mathsf{Con}(\mathsf{ZFC}) \to \mathsf{Con}(\mathsf{ZFC} + \mathsf{CH})$. State the language level and required theorem for each implication between them.

Part VIII

Proof Theory

Chapter 26

Classical and Intuitionistic Sequent Calculi

Convention 26.1 (Proof-theory formula metavariables)**.** Throughout this part, A, B, C range over formulas unless a section explicitly declares a narrower local role. The symbols $\Gamma, \Delta, \Sigma, \Pi$ retain their normative standing as formula contexts. The simply typed lambda calculus section explicitly redeclares A, B as type metavariables and M, N as term metavariables only within that section.

The sequent calculi LK and LJ, and the cut-elimination theorem of chapter 27, originate with Gentzen [21, 22].

Structural proof theory studies derivations as formal objects. The active calculus, its context discipline, and its conversion relation must be fixed before a claim about cut elimination or normalization is meaningful. The presentation below follows the standard separation between sequent calculus and natural deduction in [60, 46].

26.1 Sequents and contexts

Convention 26.2 (Contexts)**.** In this chapter a context is a finite multiset of formulas. Commas denote multiset union. Exchange is therefore built into the representation of a context; weakening and contraction are not. If Γ is a context, then Γ, A adds one occurrence of A.

Definition 26.3 (Classical sequent)**.** A classical sequent is an expression

$$\Gamma \Rightarrow \Delta,$$

where Γ and Δ are finite multisets of formulas. Its semantic reading is

$$\bigwedge \Gamma \to \bigvee \Delta,$$

with $\bigwedge \varnothing = \top$ and $\bigvee \varnothing = \bot$. This reading states the intended semantics; a sequent is itself a syntactic object, not an object-language conditional.

Definition 26.4 (Intuitionistic sequent)**.** An intuitionistic sequent has the form

$$\Gamma \Rightarrow A,$$

with exactly one succedent formula. The restriction to one succedent is part of the definition of the calculus LJ, not a typographic abbreviation for a classical sequent.

26.2 The propositional rules of LK

The initial and structural rules are

$$\frac{}{A \Rightarrow A}\ \textsc{Id} \qquad \frac{}{\bot, \Gamma \Rightarrow \Delta}\ \bot L,$$

$$\frac{\Gamma \Rightarrow \Delta}{A, \Gamma \Rightarrow \Delta}\ \mathrm{WL} \qquad \frac{\Gamma \Rightarrow \Delta}{\Gamma \Rightarrow \Delta, A}\ \mathrm{WR},$$

$$\frac{A, A, \Gamma \Rightarrow \Delta}{A, \Gamma \Rightarrow \Delta}\ \mathrm{CL} \qquad \frac{\Gamma \Rightarrow \Delta, A, A}{\Gamma \Rightarrow \Delta, A}\ \mathrm{CR},$$

and

$$\frac{\Gamma \Rightarrow \Delta, A \qquad A, \Pi \Rightarrow \Sigma}{\Gamma, \Pi \Rightarrow \Delta, \Sigma}\ \textsc{Cut}.$$

The displayed occurrence of A in cut is the *cut formula*.

The logical rules for conjunction and disjunction are

$$\frac{A, B, \Gamma \Rightarrow \Delta}{A \wedge B, \Gamma \Rightarrow \Delta} \wedge L \qquad \frac{\Gamma \Rightarrow \Delta, A \qquad \Gamma \Rightarrow \Delta, B}{\Gamma \Rightarrow \Delta, A \wedge B} \wedge R,$$

$$\frac{A, \Gamma \Rightarrow \Delta \qquad B, \Gamma \Rightarrow \Delta}{A \vee B, \Gamma \Rightarrow \Delta} \vee L \qquad \frac{\Gamma \Rightarrow \Delta, A, B}{\Gamma \Rightarrow \Delta, A \vee B} \vee R.$$

The implication rules are

$$\frac{\Gamma \Rightarrow \Delta, A \qquad B, \Pi \Rightarrow \Sigma}{A \to B, \Gamma, \Pi \Rightarrow \Delta, \Sigma} \to L \qquad \frac{A, \Gamma \Rightarrow \Delta, B}{\Gamma \Rightarrow \Delta, A \to B} \to R.$$

Negation is the abbreviation $\neg A := A \to \bot$. If negation is instead made primitive, its rules must be added and the cut-elimination proof must include their cases.

26.3 Quantifier rules

For first-order formulas the quantifier rules are

$$\frac{A[t/x], \Gamma \Rightarrow \Delta}{\forall x\, A, \Gamma \Rightarrow \Delta} \forall L \qquad \frac{\Gamma \Rightarrow \Delta, A[y/x]}{\Gamma \Rightarrow \Delta, \forall x\, A} \forall R,$$

$$\frac{A[y/x], \Gamma \Rightarrow \Delta}{\exists x\, A, \Gamma \Rightarrow \Delta} \exists L \qquad \frac{\Gamma \Rightarrow \Delta, A[t/x]}{\Gamma \Rightarrow \Delta, \exists x\, A} \exists R.$$

Every displayed instance $A[t/x]$ uses the total capture-avoiding substitution fixed in the first-order syntax chapter; bound variables are renamed when required, so there is no additional definedness side condition. In $\forall R$ and $\exists L$, the eigenvariable y must not occur free in the conclusion sequent. This freshness requirement is a rule condition, not a convention that may be repaired after the inference.

Definition 26.5 (LK-derivation)**.** An LK-derivation is a finite tree of classical sequents in which every leaf is the conclusion of an instance of a displayed zero-premise rule, and every nonleaf is the conclusion of exactly one instance of one of the remaining displayed LK rules, whose premises are its immediate predecessors. A sequent is LK-derivable, written

$$\mathsf{LK} \vdash \Gamma \Rightarrow \Delta,$$

when it is the root of an LK-derivation. A derivation is cut-free when it contains no cut instance.

26.4 The calculus LJ

The calculus LJ uses the same finite-multiset antecedent contexts and the single-succedent sequent form $\Gamma \Rightarrow A$. Its complete primitive rule surface is as follows. The zero-premise and left-structural rules are

$$\frac{}{A \Rightarrow A}\ \textsc{Id} \qquad \frac{}{\bot, \Gamma \Rightarrow C}\ \bot L,$$

$$\frac{\Gamma \Rightarrow C}{A, \Gamma \Rightarrow C}\ \mathrm{WL} \qquad \frac{A, A, \Gamma \Rightarrow C}{A, \Gamma \Rightarrow C}\ \mathrm{CL}.$$

Exchange is built into finite-multiset contexts. The conjunction and disjunction rules are

$$\frac{A, B, \Gamma \Rightarrow C}{A \wedge B, \Gamma \Rightarrow C}\ \wedge L \qquad \frac{\Gamma \Rightarrow A \qquad \Gamma \Rightarrow B}{\Gamma \Rightarrow A \wedge B}\ \wedge R,$$

$$\frac{A, \Gamma \Rightarrow C \qquad B, \Gamma \Rightarrow C}{A \vee B, \Gamma \Rightarrow C}\ \vee L.$$

The two disjunction-right rules are

$$\frac{\Gamma \Rightarrow A}{\Gamma \Rightarrow A \vee B}\ \vee R_1 \qquad \frac{\Gamma \Rightarrow B}{\Gamma \Rightarrow A \vee B}\ \vee R_2.$$

The implication rules are

$$\frac{\Gamma \Rightarrow A \qquad B, \Pi \Rightarrow C}{A \to B, \Gamma, \Pi \Rightarrow C}\ \to L \qquad \frac{A, \Gamma \Rightarrow B}{\Gamma \Rightarrow A \to B}\ \to R.$$

The quantifier rules are

$$\frac{A[t/x], \Gamma \Rightarrow C}{\forall x\, A, \Gamma \Rightarrow C}\ \forall L$$

$$\frac{\Gamma \Rightarrow A[y/x]}{\Gamma \Rightarrow \forall x\, A}\ \forall R$$

$$\frac{A[y/x], \Gamma \Rightarrow C}{\exists x\, A, \Gamma \Rightarrow C}\ \exists L$$

$$\frac{\Gamma \Rightarrow A[t/x]}{\Gamma \Rightarrow \exists x\, A}\ \exists R.$$

As above, substitution is total and capture avoiding, and y must not occur free in the conclusion of $\forall R$ or $\exists L$. There is no right weakening or right contraction because there is exactly one succedent formula. Cut is

$$\frac{\Gamma \Rightarrow A \qquad A, \Pi \Rightarrow C}{\Gamma, \Pi \Rightarrow C}\ \textsc{Cut}.$$

Negation remains the defined connective $\neg A := A \to \bot$; there are no additional primitive negation rules.

Definition 26.6 (LJ-derivation)**.** An LJ-derivation is a finite rooted tree of intuitionistic sequents. A leaf is the conclusion of an instance of Id or $\bot L$, and every nonleaf is the conclusion of exactly one instance of one of the remaining displayed LJ rules, whose premises are its immediate predecessors. A sequent is LJ-derivable, written

$$\mathsf{LJ} \vdash \Gamma \Rightarrow A,$$

iff it is the root of such a tree. The derivation is cut-free iff none of its nodes is obtained by Cut.

Theorem 26.7 (Soundness of LK)**.** *Let Γ and Δ be finite contexts. If $\mathsf{LK} \vdash \Gamma \Rightarrow \Delta$, then every classical structure and assignment satisfying every formula in Γ satisfies at least one formula in Δ.*

Proof sketch. Induct on the derivation. Initial and structural cases follow directly from the semantic reading of a sequent. Each logical case unfolds the semantic clause for its principal connective. The quantifier cases use the substitution lemma; the eigenvariable cases use the fact that the eigenvariable is absent from the conclusion. The individual case calculations are omitted. Semantic soundness of LJ is stated only after its intuitionistic semantics is defined in chapter 31.

Example 26.8 (A cut-free derivation)**.** The commutativity sequent $A \wedge B \Rightarrow B \wedge A$ has the cut-free derivation

$$\dfrac{\dfrac{\dfrac{B \Rightarrow B}{A, B \Rightarrow B}\,\mathrm{WL} \qquad \dfrac{A \Rightarrow A}{A, B \Rightarrow A}\,\mathrm{WL}}{A, B \Rightarrow B \wedge A}\,\wedge R}{A \wedge B \Rightarrow B \wedge A}\,\wedge L.$$

The formulas above the last inference are subformulas of the endsequent. That observation becomes a theorem only after cut elimination.

26.5 Primitive, derived, and admissible rules

Definition 26.9 (Primitive rule)**.** A primitive rule is a rule occurring in the definition of the active calculus. Its status is syntactic: it is not inferred from soundness or semantic validity.

Definition 26.10 (Derived rule)**.** A rule is derived in a calculus when there is a uniform derivation of its conclusion from its premises using the primitive rules. Replacing an application of a derived rule by that derivation preserves the open leaves.

Definition 26.11 (Admissible rule)**.** A rule is admissible in a calculus when every substitution instance whose premises are derivable has a derivable conclusion. Adding an admissible rule does not enlarge the set of derivable endsequents.

Every derived rule is admissible. The converse need not hold. Admissibility is a closure property of the derivability relation; a derived rule supplies a uniform internal derivation of its conclusion from its premises as open leaves.

Example 26.12 (Cut after cut elimination)**.** Cut is primitive in the displayed formulations of LK and LJ. The cut-elimination theorem shows that cut is admissible in the cut-free fragments. This does not make cut a derived rule of the cut-free fragment: a single uniform cut-free derivation with the two arbitrary cut premises as open leaves would be a stronger assertion than admissibility.

Warning 26.13 (A sequent is not a consequence assertion). The sequent $\Gamma \Rightarrow \Delta$, the derivability assertion $\mathsf{LK} \vdash \Gamma \Rightarrow \Delta$, and its semantic validity are three different objects. Soundness and completeness connect them; typography does not identify them.

Definition 26.14 (Principal formula)**.** In an instance of a displayed logical rule, the *principal formula* is the compound formula occurrence displayed in the conclusion and introduced by that instance; the displayed occurrences in the premises from which it is formed are its *active formulas*, and all other formula occurrences are context. In an instance of weakening or contraction, the principal formula is the displayed weakened or contracted occurrence. A zero-premise rule instance has no principal formula.

Exercise 26.15. Give a cut-free LJ-derivation of $A \wedge (B \vee C) \Rightarrow (A \wedge B) \vee (A \wedge C)$. Mark the principal formula of every logical inference.

Exercise 26.16. Show that deleting the eigenvariable condition from $\forall R$ makes the calculus unsound by deriving a sequent and giving a first-order structure and assignment that satisfy every antecedent formula and falsify every succedent formula.

Chapter 27

Cut Elimination and Analytic Derivations

27.1 Measures on cuts

Definition 27.1 (Formula rank and derivation height)**.** The rank $r(A)$ of a formula is the number of logical symbols in it. The height $h(D)$ of a derivation is the greatest number of inference steps on a branch from a leaf to its root. These measures are different: formula rank measures syntax, whereas derivation height measures a proof tree.

The reduction below is run for the following generalization of cut, the *mix* (multicut) rule. Here A^k abbreviates k comma-separated occurrences of A. For LK, with $m, n \geq 1$, and with Δ and Σ permitted to contain further occurrences of A:

$$\frac{\Gamma \Rightarrow \Delta, A^m \qquad A^n, \Pi \Rightarrow \Sigma}{\Gamma, \Pi \Rightarrow \Delta, \Sigma}\ \textsc{Mix}.$$

For LJ the succedent is single, so the left premise's succedent multiplicity m is fixed at one: from $\Gamma \Rightarrow A$ and $A^n, \Pi \Rightarrow C$ with $n \geq 1$, Mix concludes $\Gamma, \Pi \Rightarrow C$. The displayed occurrences of A are the *mix formula* of the instance.

Lemma 27.2 (Equipollence of cut and mix)**.** *The calculus obtained from* LK *by replacing* Cut *by* Mix *derives exactly the* LK*-derivable sequents, and likewise for* LJ*. Consequently, a derivation in the mix formulation with no mix instance is a cut-free derivation, and eliminating mix eliminates cut. Status: complete proof.*

Proof. Cut is the mix instance $m = n = 1$, so reading each cut as a mix converts a derivation with cuts into a derivation in the mix formulation with the same endsequent. Conversely, an instance of Mix with multiplicities m and n is derived from the displayed primitive rules: $m - 1$ applications of CR to the left premise contract the succedent occurrences A^m to a single A, $n - 1$ applications of CL to the right premise contract A^n to a single A, and one Cut concludes $\Gamma, \Pi \Rightarrow \Delta, \Sigma$. Replacing every mix instance, uppermost first, by this derivation yields a derivation in the original formulation with the same endsequent. In LJ the left premise already has the single succedent A, so only CL is used. Hence the cut and mix formulations derive the same sequents, and a mix-elimination procedure eliminates cut: first read every cut as the mix instance $m = n = 1$, then eliminate the mixes. □

For an instance of mix with mix formula A and immediate subderivations D_1 and D_2, its rank is $r(A)$ and its height is $h(D_1) + h(D_2)$, the sum of the heights of those subderivations. The reduction is controlled by the lexicographic order on

$$\langle r(A), h(D_1) + h(D_2) \rangle.$$

For a cut, the mix instance $m = n = 1$, these measures are its *cut rank* and *cut height*.

Theorem 27.3 (Cut elimination for LK and LJ)**.** *Every* LK*-derivation can be transformed into a cut-free* LK*-derivation of the same endsequent. Every* LJ*-derivation can be transformed into a cut-free* LJ*-derivation of the same endsequent. Status: proof sketch.*

Proof sketch. By lemma 27.2, first replace every cut in the given derivation by the mix instance $m = n = 1$; the reduction below eliminates mixes, and steps 1, 2, 5, and 6 apply verbatim with "mix" for "cut" and the mix formula for the cut formula. Choose an uppermost cut of maximal cut rank.

1. If the cut formula is not principal in the final inference of one premise, and that inference is not a contraction of the cut formula, commute the cut above that inference. The cut rank is unchanged and the cut height decreases.
2. If the cut formula is introduced by weakening in one premise, delete the cut and restore the conclusion by weakenings and contractions.
3. If a premise of a mix ends in a contraction of the mix formula (CL above the right premise, CR above the left), replace that premise by the contraction's premise and raise the corresponding multiplicity by one: the rank is unchanged and the height decreases by one.
4. If the cut formula is principal on both sides, replace the cut by cuts on proper subformulas. For example, a cut between $\wedge R$ and $\wedge L$ is replaced by cuts on the two conjuncts. Every new cut has strictly smaller cut rank, so the number of cuts of maximal cut rank decreases. When the mix formula is principal on both sides with multiplicities (m, n), the principal reduction yields mixes on proper subformulas, of smaller rank, together with residual mixes on the remaining occurrences, of equal rank but smaller height.
5. If an initial sequent is a premise of the cut, delete the cut directly.
6. For quantifiers, rename eigenvariables before permutation and use the substitution lemma. This prevents capture and preserves the eigenvariable conditions.
7. Repeat according to the well-founded lexicographic measure. The resulting derivation has no cuts.

The calculus-specific table of commuting and principal reductions is the omitted verification; it occupies the complete proof in [60, 46].

Definition 27.4 (Subformula property)**.** A propositional derivation has the subformula property iff every formula occurrence in it is a subformula of a formula in its endsequent.

Lemma 27.5 (Subformula property of cut-free derivations)**.** *Every cut-free propositional* LK*- or* LJ*-derivation has the subformula property. Status: complete proof.*

Proof. In a cut-free derivation, inspect each rule upward from the endsequent. Every premise formula is either copied from the conclusion or is an immediate subformula of the principal formula. Induction on derivation height completes the argument. □

Corollary 27.6 (Propositional subformula property)**.** *Every derivable propositional* LK*- or* LJ*-sequent has a cut-free derivation with the subformula property. Status: proof sketch.*

Proof sketch. The cut-elimination theorem transforms a given derivation of the sequent into a cut-free derivation of the same endsequent, and lemma 27.5 shows that every such derivation has the subformula property. This derivation uses the preceding proof sketch of the cut-elimination theorem; a complete proof supplies its reduction case table.

Remark 27.7 (First-order qualification). For first-order cut-free derivations, quantifier rules introduce substitution instances such as $A[t/x]$. The literal propositional subformula property therefore fails. The correct statement is a subformula property modulo term substitution and eigenvariable renaming. State the selected formulation before using it in proof search.

Corollary 27.8 (Underivability of the empty sequent)**.** *There is no* LK*-derivation of the empty sequent* $\varnothing \Rightarrow \varnothing$*. Status: proof sketch.*

Proof sketch. If the empty sequent were derivable, cut elimination would give a cut-free derivation of it. The empty sequent is not an instance of a zero-premise rule, and the conclusion of every other cut-free rule contains at least one displayed formula occurrence. Hence no cut-free derivation has the empty sequent at its root, a contradiction. This derivation uses the preceding proof sketch of the cut-elimination theorem; a complete proof supplies its reduction case table.

27.2 What cut elimination establishes

Cut elimination establishes admissibility of cut and yields analytic derivations. Depending on the calculus, it supports consistency, interpolation, conservativity, and proof-search results. None of those consequences follows merely from the phrase "cut-free": each requires a separate argument using the exact rule set.

Cut elimination does not imply that removing cut makes proof search efficient. Elimination can cause a non-elementary increase in derivation size, and a cut-free search space may still be infinite because of quantifier terms.

Warning 27.9 (Calculus dependence). Adding equality rules, induction, fixed points, modal rules, or infinitary rules changes the cut reductions. A theorem for the displayed LK or LJ does not automatically apply to the extension.

Exercise 27.10. Write the principal cut reduction for a cut formula $A \wedge B$ when the left premise ends in $\wedge R$ and the right premise ends in $\wedge L$. Show that each new cut has smaller cut rank.

Exercise 27.11. Explain why the subformula property gives a decision procedure for propositional LJ, but not directly for first-order LJ.

27.3 Interpolation and definability

Interpolation is one of the consequences of cut elimination named in the preceding section; its statement, and the definability theorem derived from it, are recorded here at cited-theorem level.

Theorem 27.12 (Craig interpolation)**.** *Let* φ *and* ψ *be first-order sentences with* $\models \varphi \to \psi$*. Then there is a sentence* θ *such that* $\models \varphi \to \theta$*,* $\models \theta \to \psi$*, and every nonlogical symbol of* θ *occurs in both* φ *and* ψ*; equality and* $\bot$ *may occur in* θ *regardless. The degenerate cases are covered because* $\bot$ *and* $\neg\bot$ *are available as interpolants when* φ *is unsatisfiable or* ψ *is valid. Status: cited theorem.*

See [60] for the proof-theoretic route through Maehara's lemma over a cut-free sequent calculus, and [30] for the model-theoretic route through compactness and elementary diagrams.

Theorem 27.13 (Beth definability)**.** *Let T be a first-order theory in a signature containing a distinguished relation symbol R, and suppose T defines R implicitly: any two models of T with the same reduct to the signature without R interpret R identically. Then T defines R explicitly: some formula $\theta(\bar{x})$ not containing R satisfies $T \vdash \forall \bar{x}\,(R(\bar{x}) \leftrightarrow \theta(\bar{x}))$. Status: cited theorem.*

Beth definability follows from Craig interpolation; see [30]. The Robinson joint-consistency theorem is a further standard equivalent of interpolation and is stated and proved in the same source. The theorem is converse to the conservativity of definitional extensions (theorem 10.7): adding a defining axiom for R produces a theory that defines R explicitly, hence implicitly, while proving no new sentences of the smaller signature; Beth definability states that implicit definability already guarantees an explicit definition.

Chapter 28

Normalization, Proof Terms, and Ordinal Analysis

28.1 Natural-deduction normalization

The normalization theory of natural deduction developed in this chapter originates with Prawitz [48].

Definition 28.1 (Detour)**.** A detour is a segment of a natural-deduction derivation in which a formula is introduced by an introduction rule and is immediately used as the major premise of the matching elimination rule; the *major premise* of an elimination rule is the premise exhibiting the connective or quantifier that the rule eliminates. A derivation is normal when it has no detour and satisfies the required permutation conditions for disjunction and existential elimination.

For implication, the principal reduction has the schematic form

$$\dfrac{\dfrac{\begin{array}{c}[A]^{i}\\ \vdots D\\ B\end{array}}{A \to B}\,{\to}I_i \qquad \begin{array}{c}\vdots E\\ A\end{array}}{B}\,{\to}E \quad \rightsquigarrow \quad \begin{array}{c}\vdots E\\ A\\ \vdots D\\ B\end{array},$$

where $D[E/A^i]$ substitutes the derivation E for every discharged occurrence carrying label i. This is substitution of derivations, not term substitution inside formulas.

For the theorem below, the calculus is exactly the finite-tree natural-deduction presentation of chapters 6 and 11 with the rules $\wedge I, \wedge E_1, \wedge E_2, \vee I_1, \vee I_2, \vee E, \to I, \to E, \bot E, \forall I, \forall E, \exists I, \exists E$. It omits RAA, the primitive-negation rules, and the equality rules, and uses $\neg A := A \to \bot$. Its quantifier substitutions are the total capture-avoiding substitutions fixed earlier, and its discharge and eigenvariable conditions are literally those attached to the cited rules. This declaration fixes the derivation objects to which "reduces" and "normal" refer; it does not posit an unnamed standard calculus.

Theorem 28.2 (Normalization for intuitionistic natural deduction)**.** *Every derivation in the calculus just fixed reduces to a normal derivation with the same conclusion and no undischarged assumption*

that was not already undischarged in the original derivation. Equivalently, after identifying alpha-equivalent formulas, the resulting set of undischarged assumption formulas is a subset of the original set.

Proof sketch. Define maximal formulas and maximal segments. Principal reductions remove an introduction–elimination detour; permutation conversions move eliminations above disjunction or existential eliminations. A naive count of formula occurrences is insufficient because a reduction can duplicate a subderivation. The standard proof uses a lexicographic measure on the rank of maximal formulas and the lengths of maximal segments, or translates to a typed term calculus and proves strong normalization there. The full calculus-specific proof is given in [60].

Normalization and cut elimination express related proof transformations, but they are not one theorem. Normalization concerns a reduction relation on natural-deduction derivations; cut elimination concerns removal of a sequent rule.

28.2 The simply typed lambda calculus

Fix a set Base of base types. Types and terms are generated by the following scoped metavariables: in this section only, A, B range over types, M, N range over terms, and α ranges over members of Base. The symbol Θ_{ty} ranges over finite lists assigning distinct term variables to types; these lists are the typing contexts of this section. These are metalinguistic roles, not additional object-language variables.

$$A, B ::= \alpha \mid A \to B, \qquad \alpha \in \mathsf{Base},$$

and

$$M, N ::= x \mid \lambda x{:}A.M \mid MN.$$

The typing judgment $\Theta_{\mathrm{ty}} \vdash M : A$ is generated by

$$\frac{x{:}A \in \Theta_{\mathrm{ty}}}{\Theta_{\mathrm{ty}} \vdash x : A}\ \textsc{Var}, \qquad \frac{\Theta_{\mathrm{ty}}, x{:}A \vdash M : B}{\Theta_{\mathrm{ty}} \vdash \lambda x{:}A.M : A \to B} \to I,$$

In $\to I$, the variable x does not occur in Θ_{ty}; this can always be arranged by renaming the bound variable of the abstraction, and typing judgments are stable under such renaming of the subject term.

$$\frac{\Theta_{\mathrm{ty}} \vdash M : A \to B \qquad \Theta_{\mathrm{ty}} \vdash N : A}{\Theta_{\mathrm{ty}} \vdash MN : B} \to E.$$

The principal computation is beta reduction:

$$(\lambda x{:}A.M)N \longrightarrow_{\beta} M[N/x],$$

where term substitution is capture-avoiding.

Theorem 28.3 (Subject reduction)**.** *If $\Theta_{\mathrm{ty}} \vdash M : A$ and $M \longrightarrow_{\beta} N$, then $\Theta_{\mathrm{ty}} \vdash N : A$.*

Proof sketch. Prove the typing substitution lemma: if $\Theta_{\mathrm{ty}}, x{:}B \vdash M : A$ and $\Theta_{\mathrm{ty}} \vdash N : B$, then $\Theta_{\mathrm{ty}} \vdash M[N/x] : A$. Apply it to the contracted beta redex and use induction on the compatible closure of beta reduction for reductions inside a term.

Theorem 28.4 (Strong normalization)**.** *Every well-typed term in the simply typed lambda calculus has no infinite beta-reduction sequence.*

Proof sketch. Assign to each type a reducibility candidate. A term belongs to the candidate for a base type when it is strongly normalizing; it belongs to the candidate for $A \to B$ when application to every member of the candidate for A produces a member of the candidate for B. The fundamental lemma proves by induction on typing derivations that every well-typed term belongs to its type's candidate. Strong normalization follows. This proof sketch gives the logical-relations architecture; closure lemmas for the candidates are omitted.

28.3 Curry–Howard correspondence

Under the Curry–Howard correspondence, a proposition is represented by a type, a derivation by a typed term, implication introduction by lambda abstraction, implication elimination by application, and normalization by computation. The correspondence is structural: it does not identify every formal derivation with a program without first fixing a calculus and a term assignment.

Example 28.5 (Composition)**.** The term

$$\lambda f{:}(B \to C).\, \lambda g{:}(A \to B).\, \lambda x{:}A.\, f(gx)$$

has type

$$(B \to C) \to (A \to B) \to A \to C.$$

Reading types as propositions gives a proof of the corresponding intuitionistic formula. Each abstraction discharges one typed assumption, and each application is an implication-elimination step.

Products and sums correspond to conjunction and disjunction after their term constructors, eliminators, and reduction rules are defined. The empty type corresponds to $\bot$. Classical principles require additional control operators, translations, or axioms; they are not terms of the pure simply typed lambda calculus.

28.4 Scope of ordinal analysis

The landscape of ordinal-informative consistency analyses is surveyed in [10].

Cited theorem. Gentzen's consistency analysis of arithmetic, in the sequent-reduction presentation surveyed in [60, Chapter 10, especially Sections 10.1–10.4, pp. 318–337], assigns notations below ε_0 to arithmetic derivations so that each critical reduction strictly lowers the assigned notation. In a metatheory that accepts transfinite induction for the predicates and primitive-recursive ordering used in that particular reduction analysis, termination yields $\mathsf{Con}(\mathsf{PA})$. This is an analysis-relative imported result; it is not a definition of a canonical invariant and it does not assert that an unspecified ordinal "measures" a theory.

Formal objects needed to read the citation. The symbol ε_0 denotes the least nonzero ordinal α satisfying $\omega^\alpha = \alpha$. A standard external way to write ordinals below it uses finite Cantor-normal-form terms

$$a = \omega^{a_1} \cdot m_1 + \cdots + \omega^{a_k} \cdot m_k,$$

where $k \geq 1$, each m_i is a positive natural number, the exponent terms satisfy $a_1 > \cdots > a_k$, and 0 is also a term. Exponents are formed recursively by the same clause. Interpreting the symbols by

ordinal addition and exponentiation assigns each such finite term an ordinal below ε_0. This display supplies notation for the following worked calculation; it is not asserted to be the source's complete primitive-recursive coding.

For example, $\lambda = \omega^\omega$ is a limit notation. The sequence

$$\lambda[n] = \omega^n \qquad (n \geq 1)$$

satisfies $\lambda[n] < \lambda$ for every n and $\sup_{n\geq 1} \lambda[n] = \lambda$. Likewise, for $\mu = \omega^3$, define $\mu[n] = \omega^2 \cdot n$ for $n \geq 1$. Then

$$\mu[4] = \omega^2 \cdot 4 < \omega^3 = \mu.$$

This is the elementary shape of a fundamental-sequence descent: a limit notation is replaced by a selected smaller approximation. It does not show that any particular proof reduction receives μ, because that claim depends on the source's derivation coding and ordinal assignment.

The cited result therefore has three separately typed ingredients: finite derivation codes for the selected sequent presentation of first-order arithmetic; ordinal-notation codes with a primitive-recursive comparison and fundamental-sequence operation; and a primitive-recursive reduction on derivation codes together with an assignment from derivations to notation codes. Its decisive lemma says, case by case for that reduction, that a critical reduction strictly decreases the assigned notation. Transfinite induction for the predicates used in the reduction then excludes an infinite reduction sequence, and the source extracts the consistency statement from the resulting normal form.

Internal proof boundary. A complete proof in this manuscript would still have to reproduce the exact arithmetic rule surface and derivation coding, the complete notation grammar and algorithms, every reduction clause, the assignment, every case of the decrease lemma, the precise transfinite-induction schema, and the final consistency extraction. A later use of the technical phrase *proof-theoretic ordinal* for an arbitrary theory would additionally need a stated reduction notion, a leastness or supremum clause, and a comparison showing what is invariant under alternative presentations. No such general invariant is defined here.

The imported result is relative. It does not give a finitary proof of $\mathsf{Con}(\mathsf{PA})$ formalizable in PA itself. If PA is consistent, the second incompleteness theorem excludes such an internal proof for the standard arithmetized consistency statement. Stronger theories can require notation systems far beyond ε_0, and some strength comparisons are not captured by a single canonical ordinal.

Warning 28.6 (Unqualified strength claims). Do not infer that a larger displayed ordinal alone makes one theory stronger than another. Compare the theories, notation systems, base metatheories, and reflection principles used in the analyses.

Exercise 28.7. Type the term $\lambda x{:}A.\lambda y{:}B.x$, and give the corresponding natural-deduction derivation. Identify the structural weakening implicit in the unused variable y.

Exercise 28.8. Prove subject reduction for a single outermost beta redex. State the typing substitution lemma exactly and mark where capture avoidance is used.

Definition 28.9 (Abstract reduction relation and normalization)**.** An *abstract reduction relation* is a binary relation $\to$ on a set. An element is *strongly normalizing* iff no infinite $\to$-sequence starts from it; the relation is strongly normalizing iff every element is. An element is *weakly normalizing* iff some finite $\to$-sequence from it ends in an element with no $\to$-successor; the relation is weakly normalizing iff every element is. Beta reduction on well-typed terms is an abstract reduction relation, and the strong-normalization theorem above states that it is strongly normalizing.

Exercise 28.10. Explain why a strongly normalizing abstract reduction relation is weakly normalizing, and exhibit an abstract reduction relation that is weakly but not strongly normalizing.

Part IX

Second-Order and Higher-Order Logic

Chapter 29

Second-Order Syntax and Two Semantics

Second-order syntax permits quantification over relations on the first-order domain. That syntactic extension admits at least two distinct semantic regimes. Full semantics quantifies over every relation of the required arity. Henkin semantics quantifies over a specified collection of admissible relations. The distinction controls categoricity, compactness, Löwenheim–Skolem properties, and completeness [28, 7].

29.1 Second-order formulas

Definition 29.1 (Second-order signature)**.** A second-order signature $\mathcal{L}_2$ contains the constant, function, and relation symbols of a first-order signature. Its logical alphabet also contains, for every $n \geq 1$, a countably infinite set RelVar_n of n-ary relation variables. Relation variables are logical variables; nonlogical relation symbols remain members of $\mathcal{L}_2$.

First-order terms are generated as before. If $X \in \mathsf{RelVar}_n$ and $t_1, \ldots, t_n$ are terms, then $X(t_1, \ldots, t_n)$ is atomic. Formulas are closed under the propositional connectives, first-order quantifiers $\forall x, \exists x$, and second-order quantifiers $\forall X^n, \exists X^n$. The superscript records arity and may be omitted only when the arity is fixed by context.

Definition 29.2 (Second-order sentence)**.** A second-order sentence is a second-order formula having no free first-order variable occurrence and no free relation-variable occurrence.

Term substitution for first-order variables and relation-formula substitution for relation variables are different operations. For example, substituting a formula $\varphi(x_1, \ldots, x_n)$ for a free occurrence of X^n requires renaming every binder that could capture a variable free in φ.

29.2 Full semantics

Definition 29.3 (Full second-order assignment)**.** Let $\mathcal{M}$ be a $\mathcal{L}_2$-structure with domain M. A full second-order assignment s maps each first-order variable to an element of M and each n-ary relation variable to an arbitrary member of $\mathcal{P}(M^n)$.

The first-order atomic and connective clauses are unchanged. The new atomic and quantifier clauses are

$$\mathcal{M}, s \models X(t_1, \ldots, t_n) \quad \Longleftrightarrow \quad \langle [\![t_1]\!]_s^{\mathcal{M}}, \ldots, [\![t_n]\!]_s^{\mathcal{M}} \rangle \in s(X),$$

and

$$\mathcal{M}, s \models \forall X^n \varphi \iff \text{for every } R \subseteq M^n,\ \mathcal{M}, s[X \mapsto R] \models \varphi,$$
$$\mathcal{M}, s \models \exists X^n \varphi \iff \text{for some } R \subseteq M^n,\ \mathcal{M}, s[X \mapsto R] \models \varphi.$$

Definition 29.4 (Full semantic consequence)**.** For a set Γ of second-order formulas,

$$\Gamma \models_{\text{full}} \varphi$$

iff every structure under every full second-order assignment that satisfies every member of Γ also satisfies φ.

The word *full* modifies the range of second-order quantification. It does not mean that the first-order domain contains every set or that the structure is a set-theoretic universe.

Theorem 29.5 (Full comprehension validity)**.** *Let $\varphi(\bar{x})$ be a formula in which X^n is not free. Under full semantics the universal closure of*

$$\exists X^n \forall x_1 \cdots \forall x_n \left(X(\bar{x}) \leftrightarrow \varphi(\bar{x})\right)$$

is valid.

Proof. Fix $\mathcal{M}$ and a full second-order assignment s of all parameters. Let

$$R = \{\bar{a} \in M^n : \mathcal{M}, s[\bar{x} \mapsto \bar{a}] \models \varphi(\bar{x})\}.$$

Full semantics permits X to range over this particular subset of M^n. With $s(X) = R$, the biconditional follows from the definition of R. □

29.3 Full second-order categoricity

Let $\mathcal{L}_{\mathsf{N}} = \{0, S\}$, with a constant symbol 0 and a unary function symbol S. The full second-order Peano axioms are

$$\forall x\, (Sx \neq 0),$$
$$\forall x\, \forall y\, (Sx = Sy \to x = y),$$

and the second-order induction sentence

$$\forall X\ \left[(X(0) \wedge \forall x\, (X(x) \to X(Sx))) \to \forall x\, X(x)\right].$$

Theorem 29.6 (Categoricity of full second-order arithmetic)**.** *Every two structures satisfying the displayed second-order Peano axioms under full semantics are isomorphic. In particular, every such structure is isomorphic to $(\mathbb{N}, 0, n \mapsto n+1)$.*

Proof. Let $\mathcal{M} = (M, 0^{\mathcal{M}}, S^{\mathcal{M}})$ satisfy the axioms. Define $f : \mathbb{N} \to M$ recursively by

$$f(0) = 0^{\mathcal{M}}, \qquad f(n+1) = S^{\mathcal{M}}(f(n)).$$

The map f is injective. The proof is a metatheoretic induction on $n \in \mathbb{N}$ of the claim: for every $m \in \mathbb{N}$, if $f(n) = f(m)$, then $n = m$. Base case $n = 0$: suppose $f(0) = f(m)$ with $m \neq 0$. Then $m = k + 1$ for some $k \in \mathbb{N}$, so the supposition reads $0^{\mathcal{M}} = S^{\mathcal{M}}(f(k))$, contradicting the first axiom; hence $m = 0$. Induction step: assume the claim for n and suppose $f(n+1) = f(m)$. If $m = 0$, then $S^{\mathcal{M}}(f(n)) = 0^{\mathcal{M}}$, and the first axiom excludes this; so $m = k + 1$ for some $k \in \mathbb{N}$, and the

supposition reads $S^{\mathcal{M}}(f(n)) = S^{\mathcal{M}}(f(k))$. The second axiom gives $f(n) = f(k)$, the induction hypothesis gives $n = k$, and therefore $n + 1 = k + 1 = m$. The range $R \subseteq M$ of f contains $0^{\mathcal{M}}$ and is closed under $S^{\mathcal{M}}$. Under full semantics, the induction quantifier ranges over this actual subset R. Induction therefore gives $R = M$, so f is surjective. It preserves the constant and successor by construction and is therefore an isomorphism. □

If addition and multiplication are included with their recursive axioms, the same isomorphism preserves them by induction on $\mathbb{N}$. The categoricity claim depends on full semantics; it is not a theorem about arbitrary Henkin structures.

29.4 Failure of compactness and effective completeness

Theorem 29.7 (Failure of compactness under full semantics)**.** *Full second-order semantic consequence is not compact.*

Proof. Expand $\mathcal{L}_{\mathsf{N}}$ by a constant c. Let T contain the full second-order Peano axioms and, for every $n \in \mathbb{N}$, the sentence

$$c \neq S^n 0.$$

Every finite subset of T is satisfied under full semantics by a suitable structure: use the standard natural numbers and interpret c as a number larger than every index occurring in that finite subset. If one structure satisfied all of T under full semantics, categoricity would identify its domain with $\mathbb{N}$, while the displayed sentences would require c to differ from every natural number. Hence no such structure exists. □

Theorem 29.8 (No effective complete calculus for full validity)**.** *Here an effective calculus means one whose finite derivations are computably checkable, so its theorems form a computably enumerable set. There is no such calculus* D *for which, for every second-order sentence* σ*,*

$$\mathsf{D} \vdash \sigma \quad \Longleftrightarrow \quad \models_{\text{full}} \sigma.$$

Indeed, the set of full second-order validities is not computably enumerable.

Proof sketch. Use the finite full second-order axiomatization of arithmetic in the signature with $0, S, +, \times$. For each first-order arithmetic sentence σ, effectively form the second-order sentence $\mathsf{PA}_2 \to \sigma$. Full categoricity gives

$$\models_{\text{full}} (\mathsf{PA}_2 \to \sigma) \quad \Longleftrightarrow \quad (\mathbb{N}, 0, S, +, \times) \models \sigma.$$

If full second-order validities were computably enumerable, true first-order arithmetic would therefore be computably enumerable, contradicting the corollary of Tarski's undefinability theorem (theorem 22.9) that true arithmetic is not computably enumerable. The reduction and its effectiveness are the omitted steps controlled by [7, 54]; the computability proof itself is not reproduced.

This theorem excludes an effective, sound, and complete calculus for full validity. It does not exclude an ineffective presentation that takes all full validities as axioms. The effectiveness qualification is therefore part of the theorem, not an optional refinement; see [7, 54].

Theorem 29.9 (Categoricity of the full second-order reals)**.** *Let* $\mathcal{L}_{\mathsf{R}} = \{0, 1, +, \times, <\}$*. Take the finitely many first-order axioms stating that* $(0, 1, +, \times, <)$ *is an ordered field, together with the second-order completeness sentence*

$$\forall X \Big[\exists x\, X(x) \wedge \exists y\, \forall x\, (X(x) \to x \leq y) \;\to\; \exists z\, (\forall x\, (X(x) \to x \leq z) \wedge \forall y\, (\forall x\, (X(x) \to x \leq y) \to z \leq y))\Big],$$

where $u \le v$ abbreviates $u < v \lor u = v$ in this scope: every nonempty subset with an upper bound has a least upper bound. Any two $\mathcal{L}_{\mathsf{R}}$-structures satisfying these axioms under full semantics are isomorphic. In particular, every such structure is isomorphic to the ordered field $\mathbb{R}$, and its domain is uncountable. Status: cited theorem.

See [54] for the categoricity proof, which sends each element of one model to the element of the other determined by the same cut of the common rationals and verifies that this map is an ordered-field isomorphism; the ordered-field axioms abbreviated above are listed there. The uncountability of $\mathbb{R}$ is supplied by [33].

Full second-order logic also lacks the first-order Löwenheim–Skolem properties. The conjunction of the displayed second-order Peano axioms is satisfied by no uncountable structure under full semantics, so upward Löwenheim–Skolem fails. In the other direction, the axioms of theorem 29.9 are satisfied by the infinite structure $\mathbb{R}$, while theorem 29.9 makes every structure satisfying them under full semantics isomorphic to $\mathbb{R}$ and therefore uncountable. These axioms are thus satisfied by an infinite structure but by no countable structure; downward Löwenheim–Skolem fails.

Theorem 29.10 (Lindström)**.** *Let $\mathcal{L}$ be a regular abstract logic extending first-order logic: one closed under the Boolean connectives, quantification, and relativization, with satisfaction invariant under isomorphism. If $\mathcal{L}$ satisfies countable compactness and the downward Löwenheim–Skolem property to countable structures, then $\mathcal{L}$ has exactly the expressive power of first-order logic. Status: cited theorem.*

See [40] for the original proof and [18, Chapter XIII] for a complete textbook treatment, including the exact closure conditions abbreviated here as regularity. The theorem explains the pattern of this section: full second-order logic strictly exceeds first-order expressive power, so it must lose countable compactness or the downward Löwenheim–Skolem property, and this section shows that it loses both.

29.5 Henkin semantics

Definition 29.11 (Henkin structure)**.** A Henkin structure for the relational second-order language is a structure

$$\mathcal{H} = (M, (\mathcal{R}_n)_{n \ge 1}, \ldots),$$

where $M \neq \varnothing$, each $\mathcal{R}_n$ is a nonempty subset of $\mathcal{P}(M^n)$, and the remaining components interpret the nonlogical symbols. An n-ary relation variable ranges over $\mathcal{R}_n$, not over all of $\mathcal{P}(M^n)$.

A systematic treatment of Henkin semantics and of the translation between second-order logic and many-sorted first-order logic is [43].

When the calculus contains a comprehension schema, the semantic class must be restricted accordingly: every relation definable by an eligible formula with parameters from the Henkin domains must belong to the relevant $\mathcal{R}_n$. This closure is a hypothesis on the semantic class. It is not automatic from $\mathcal{R}_n \subseteq \mathcal{P}(M^n)$.

Definition 29.12 (Henkin semantic consequence)**.** Write

$$\Gamma \models_{\mathrm{H}} \varphi$$

when every Henkin structure under every sorted assignment satisfying Γ satisfies φ. A full second-order structure is the special case $\mathcal{R}_n = \mathcal{P}(M^n)$ for every n.

Cited theorem. For the typed formula system and general structures fixed in [28, pp. 81–91], a formula is a theorem of that source's system iff it is valid in that source's general structures. The hypotheses include the source's typed axiom and rule surface and its corresponding closure conditions; the conclusion is completeness for that paired syntax and semantics, not for full second-order validity. This is a cited theorem. It does not define derivability for this chapter, whose relational language has not imported that rule surface or selected a comprehension schema.

Worked many-sorted translation. The semantic reduction underlying Henkin completeness can nevertheless be stated exactly without choosing a proof calculus. Use a many-sorted first-order language with an object sort D, a relation sort S_n for each $n \geq 1$, and an $(n+1)$-ary predicate symbol E_n whose argument sorts are $(S_n, D, \ldots, D)$. All original constants, function symbols, and relation symbols keep their object-sort interpretations.

To distinguish a second-order formula from its many-sorted image, define the metalinguistic translation $\varphi \mapsto \varphi^\sharp$ recursively. It leaves object terms, equality, and nonlogical relation atoms unchanged, commutes with every propositional connective, and has the clauses

$$\begin{aligned} (X^n(t_1, \ldots, t_n))^\sharp &:= E_n(X, t_1, \ldots, t_n), \\ (\forall x\, \psi)^\sharp &:= \forall x{:}D\, \psi^\sharp, & (\exists x\, \psi)^\sharp &:= \exists x{:}D\, \psi^\sharp, \\ (\forall X^n\, \psi)^\sharp &:= \forall X{:}S_n\, \psi^\sharp, & (\exists X^n\, \psi)^\sharp &:= \exists X{:}S_n\, \psi^\sharp. \end{aligned}$$

This translation is not mere renaming: it replaces relation application by an incidence predicate and makes the two quantifier ranges into sorts. Its standing is a recursively eliminable metalinguistic operation, and it is used in the satisfaction equivalence below.

From a Henkin structure $\mathcal{H} = (M, (\mathcal{R}_n)_{n\geq 1}, \ldots)$, form the many-sorted structure $\mathcal{H}^\sharp$ by taking

$$D^{\mathcal{H}^\sharp} = M, \qquad S_n^{\mathcal{H}^\sharp} = \mathcal{R}_n, \qquad E_n^{\mathcal{H}^\sharp}(R, \bar{a}) \iff \bar{a} \in R.$$

This structure is needed because $\varphi^\sharp$ contains sorted variables and the incidence symbols E_n, which the original second-order structure does not interpret as first-order symbols. Its standing is a many-sorted first-order structure. It is nonredundant with $\mathcal{H}$: relations in $\mathcal{H}$ are values of second-order variables, whereas the same relations are first-order objects of the sorts S_n in $\mathcal{H}^\sharp$. The structure is used immediately in the following equivalence. For every second-order formula φ and every matching sorted assignment s,

$$\mathcal{H}, s \models \varphi \quad \iff \quad \mathcal{H}^\sharp, s \models \varphi^\sharp.$$

Indeed, induction on φ handles an atom $X^n(\bar{t})$ by the definition of E_n; first-order atoms because the object interpretations are unchanged; connectives by the induction hypothesis; object quantifiers because both sides range over M; and relation quantifiers because both sides range over $\mathcal{R}_n$. These cases exhaust the formula grammar.

If a selected calculus contains a comprehension instance for an eligible formula $\theta(x_1, \ldots, x_n, \bar{z}, \bar{Y})$, where X^n is not free in θ, its semantic counterpart is the many-sorted sentence

$$\forall \bar{z}\, \forall \bar{Y}\, \exists X{:}S_n\, \forall \bar{x}{:}D\, \big(E_n(X, \bar{x}) \leftrightarrow \theta^\sharp(\bar{x}, \bar{z}, \bar{Y})\big),$$

where $\bar{x} = (x_1, \ldots, x_n)$, and every displayed parameter is quantified in its declared sort. Thus the corresponding closure condition says exactly that the n-ary relation defined by each eligible instance belongs to $\mathcal{R}_n$; it is not automatic for an arbitrary family of relation domains.

Internal proof boundary. The translation proves a semantic equivalence, not a local completeness theorem. Before this chapter may assert Henkin completeness, compactness, or a downward Löwenheim–Skolem result for its own calculus, it must still list every axiom, rule, structural convention, equality and extensionality principle, and eligible comprehension instance; define finite derivations; match those instances with semantic closure; and carry the chosen many-sorted first-order completeness construction back to exactly that rule surface. No calculus name or derivability–validity biconditional is introduced here.

Henkin completeness and full categoricity concern different consequence relations. Combining them into the assertion "second-order logic is complete and categorical" suppresses the semantic parameter and is incorrect.

Example 29.13 (Reachability)**.** In a directed graph with edge relation E, the formula

$$\begin{aligned}\mathrm{Reach}(a,b) := \forall X\,[(X(a) \wedge \forall u\,\forall v\,((X(u) \wedge E(u,v)) \to X(v)))\\ \to X(b)]\end{aligned}$$

holds under full semantics iff a finite directed path leads from a to b. If a path exists, every edge-closed set containing a contains its successive vertices. Conversely, the actual set of vertices reachable from a is edge-closed; full quantification permits it as a value of X. The converse can fail in a Henkin structure if that reachable set is not among the admissible unary relations.

Warning 29.14 (Semantic parameter omitted)*.* Never write $\Gamma \models \varphi$ for a second-order argument before declaring full or Henkin semantics. The same sentence can have different status under the two consequence relations.

Exercise 29.15. Prove that the full second-order induction axiom is satisfied by the standard natural numbers. Your proof must quantify over an arbitrary subset of $\mathbb{N}$, not merely a definable subset.

Exercise 29.16. Explain precisely which step of the categoricity proof fails in a Henkin structure whose unary-relation domain does not contain the range of f.

Exercise 29.17. Use the compactness counterexample above to prove that no finitary sound and strongly complete calculus exists for full second-order semantic consequence. Distinguish this argument from the stronger non-enumerability theorem.

Chapter 30

Simple Type Theory and Higher-Order Logic

The term *higher-order logic* does not determine one calculus. A formal presentation must specify its types, term constructors, equality, logical constants, conversion rules, axioms, and semantic regime. This chapter fixes a representative classical simple-type syntax and two semantic classes. It does not select a proof calculus.

30.1 Types, terms, and formulas

Within this chapter, α and β are metalinguistic variables ranging over simple types, while F, A, B range over well-typed terms; a term of type o is a formula. These are scoped metalinguistic roles, not object-language variables. The type metavariables do not retain the earlier ordinal role and do not reuse the normative sentence metavariables σ, τ or substitution metavariable ρ. The symbols ι and o are object-language type constants.

Definition 30.1 (Simple types)**.** Let ι be the type of individuals and o the type of truth values. The simple types are generated by

$$\alpha, \beta ::= \iota \mid o \mid \alpha \to \beta.$$

This is a free grammar: $\iota \neq o$, neither base type is an arrow type, and

$$(\alpha \to \beta) = (\alpha' \to \beta') \quad \Longleftrightarrow \quad (\alpha = \alpha' \text{ and } \beta = \beta').$$

In particular, no finite simple type α satisfies $\alpha = \alpha \to o$.

For each type, fix variables and a possibly empty collection of nonlogical constants, each annotated with that type. The well-typed terms are the least family containing those variables and constants and closed under exactly these two formation clauses:

$$\frac{F : \alpha \to \beta \qquad A : \alpha}{FA : \beta}, \qquad \frac{x : \alpha \qquad B : \beta}{\lambda x{:}\alpha.B : \alpha \to \beta}.$$

Formulas are exactly the terms of type o. Equality at type α has type

$$=_\alpha : \alpha \to \alpha \to o,$$

and the universal quantifier at type α has type

$$\forall_\alpha : (\alpha \to o) \to o.$$

Thus $\forall x{:}\alpha.\, A$ abbreviates $\forall_\alpha(\lambda x{:}\alpha.A)$.

Alpha-equivalence, capture-avoiding substitution, beta reduction, and eta conversion are typed operations. A purported application with mismatched types is not a false formula; it is not a term.

30.2 Standard and Henkin semantics

Definition 30.2 (Standard type structure)**.** A standard type structure $\mathcal{M}$ assigns nonempty sets D_α to types such that

$$D_o = \{0, 1\}, \qquad D_{\alpha\to\beta} = D_\beta^{D_\alpha},$$

the full set of functions from D_α to D_β. Application is function application, lambda abstraction denotes the corresponding function, equality denotes identity at each type, and the logical constants have their classical meanings. Every admitted nonlogical constant $c{:}\alpha$ is assigned an element $c^{\mathcal{M}} \in D_\alpha$.

Definition 30.3 (Henkin type structure)**.** A Henkin type structure $\mathcal{M}$ assigns a nonempty set D_α to every type, with $D_o = \{0, 1\}$, and a selected set

$$D_{\alpha\to\beta} \subseteq D_\beta^{D_\alpha}$$

to every function type. The selected function sets must contain the denotation, under the recursion below, of every well-typed lambda term under every assignment. Application and the logical constants obey the same typed clauses as in a standard type structure, and every admitted nonlogical constant $c{:}\alpha$ is assigned an element $c^{\mathcal{M}} \in D_\alpha$.

In either kind of structure, a typed assignment s maps every variable $x{:}\alpha$ to a member of D_α. Denotation is the recursion

$$[\![x]\!]^{\mathcal{M}}_s = s(x), \qquad [\![c]\!]^{\mathcal{M}}_s = c^{\mathcal{M}},$$
$$[\![FA]\!]^{\mathcal{M}}_s = [\![F]\!]^{\mathcal{M}}_s([\![A]\!]^{\mathcal{M}}_s), \quad [\![\lambda x{:}\alpha.B]\!]^{\mathcal{M}}_s = (a \mapsto [\![B]\!]^{\mathcal{M}}_{s[x\mapsto a]}).$$

For a Henkin type structure, its closure requirement says precisely that the function in the last clause belongs to the selected $D_{\alpha\to\beta}$ for every well-typed abstraction and matching assignment. Equality at type α denotes the characteristic function of identity on D_α, and $\forall_\alpha$ returns 1 exactly on those predicates in $D_{\alpha\to o}$ that take value 1 at every member of D_α.

Every standard type structure is a Henkin type structure. The converse fails when a selected function space omits set-theoretic functions.

Cited theorem. For the formal theory of types and the general structures fixed in [28, pp. 81–91], soundness and completeness hold for that exact paired syntax and semantics. The hypotheses include the source's primitive typed constants, axioms, rules, conversion principles, and closure conditions. In its general structures a function domain need contain the functions denoted by the source's terms under assignments, rather than every set-theoretic function of the appropriate type. This is a cited theorem. It neither makes full-function semantics complete nor selects a derivability relation for the syntax-only presentation in this chapter.

Worked semantic calculation. The closure clause in a Henkin type structure is required already to interpret lambda abstraction. Let $x{:}\alpha$, let $B{:}\beta$, let $A{:}\alpha$, let $\mathcal{M}$ be a standard or Henkin type structure, and let s be a typed assignment. Capture-avoiding typed substitution satisfies the semantic substitution identity

$$[\![B[A/x]]\!]^{\mathcal{M}}_s = [\![B]\!]^{\mathcal{M}}_{s[x\mapsto [\![A]\!]^{\mathcal{M}}_s]}.$$

It is proved by induction on the typed construction of B: the variable case reads the updated assignment, application uses the two induction hypotheses, and abstraction first renames a bound variable away from the free variables of A and then applies the induction hypothesis under the updated assignment. Constants are unchanged. Consequently a beta redex has the same denotation as its contractum in both standard and Henkin structures:

$$\begin{aligned}[\![(\lambda x{:}\alpha.B)A]\!]^{\mathcal{M}}_s &= [\![\lambda x{:}\alpha.B]\!]^{\mathcal{M}}_s([\![A]\!]^{\mathcal{M}}_s)\\ &= [\![B]\!]^{\mathcal{M}}_{s[x\mapsto [\![A]\!]^{\mathcal{M}}_s]}\\ &= [\![B[A/x]]\!]^{\mathcal{M}}_s.\end{aligned}$$

The middle value is available because the denotation of the abstraction is a member of the selected domain $D_{\alpha\to\beta}$. Thus term closure is a definedness requirement, not an optional strengthening added only for a completeness proof.

Quantification makes the semantic difference visible. In $\forall f{:}(\alpha\to\beta).\,C$, a standard structure varies f over all of $D_\beta^{D_\alpha}$, whereas a Henkin structure varies it only over the declared $D_{\alpha\to\beta}$. The same typed formula can therefore have different validity status in the two semantic classes even though the beta calculation remains sound in both.

Internal proof boundary. An internal completeness theorem here would require a co-located list of every primitive typed constant, conversion rule, logical axiom, equality and extensionality principle, inference rule, and structural convention; an exact definition of finite derivations; semantic closure matching each syntactic principle; and a Henkin construction and truth lemma for precisely that package. These data are source-dependent and are not reproduced. The chapter therefore introduces no calculus symbol and asserts no local derivability–validity biconditional.

Example 30.4 (A polymorphism that simple types do not express)**.** At each fixed type α, the identity term

$$\lambda x{:}\alpha.x$$

has type $\alpha\to\alpha$. The displayed α is a metavariable in a schema of typed terms. It is not a type variable in the object language of simple type theory. Object-language quantification over types would require a different, explicitly polymorphic type system.

Example 30.5 (Typed representation of a binary relation)**.** A binary relation on individuals has type

$$\iota\to\iota\to o,$$

with application associated to the left. If $R:\iota\to\iota\to o$ and $a,b:\iota$, then $Rab:o$ is a formula. The terms Ra and R are not formulas: their types are $\iota\to o$ and $\iota\to\iota\to o$, respectively.

30.3 Type discipline and foundational scope

Simple types block unrestricted self-application. If $x:\alpha$, then xx would require simultaneously $x:\alpha\to\beta$ and $x:\alpha$, which the simple grammar cannot satisfy. This syntactic fact blocks the form

of self-membership used in Russell's paradox. It does not prove that every extension of type theory is consistent; consistency remains a metatheorem about a specified calculus.

Extensionality and the Axiom of Choice at higher types are separate principles. A system may take functional extensionality as an axiom,

$$\forall f, g{:}(\alpha \to \beta).\, (\forall x{:}\alpha.\, fx = gx) \to f = g,$$

while another builds it into equality semantics. A higher-order Axiom of Choice operator introduces additional axioms. Neither principle follows merely from the type grammar.

Warning 30.6 (Proof-assistant metalogics). A proof assistant based on a dependent type theory, a simple type theory, or a logical framework does not thereby prove a theorem about an encoded object logic. An encoding requires a theorem connecting encoded derivability or satisfaction to the intended external judgment.

Exercise 30.7. Assign types to every subterm of $\lambda R{:}(\iota \to \iota \to o).\lambda x{:}\iota.Rxx$.

Exercise 30.8. State the semantic clause for $\forall f{:}(\alpha \to \beta).\, A$ under standard semantics and under Henkin semantics. Identify the only difference.

Exercise 30.9. Explain why the assertion "higher-order logic is complete" is incomplete. Give one precise true statement and one precise false statement obtained by supplying a calculus and a semantic class.

Part X

Nonclassical Logic

Chapter 31

Intuitionistic Logic

Convention 31.1 (Nonclassical formula metavariables)**.** Throughout this part, A, B, C range over formulas and p, q range over propositional variables. The symbols $\Gamma, \Delta, \Sigma, \Pi$ retain their normative standing as formula contexts. Any narrower local role is declared before use.

Intuitionistic logic is specified positively by its calculus and semantics. It is not defined as classical logic with selected theorems deleted. This chapter uses LJ from chapter 26 as the calculus and Kripke structures for its semantics; see [37, 60].

31.1 Language, calculus, and Kripke structures

The propositional language contains $\wedge, \vee, \to, \bot$, and

$$\neg A := A \to \bot.$$

Because LJ-antecedents are finite multisets whereas premise collections may be arbitrary sets, derivability is defined by finite support:

$$\Gamma \vdash_{\mathsf{IPL}} A \quad :\Longleftrightarrow \quad \begin{array}{c}\text{there is a finite multiset context } \Gamma_0, \text{ every formula} \\ \text{occurring in which belongs to } \Gamma, \text{ such that } \quad \mathsf{LJ} \vdash \Gamma_0 \Rightarrow A.\end{array}$$

This defines the left side for every formula set Γ while using only contexts admitted by LJ. The unlifted sequent notation remains available for a particular finite multiset Γ_0.

Definition 31.2 (Intuitionistic Kripke frame and structure)**.** An intuitionistic Kripke frame is a partially ordered set $(W, \leq)$. A Kripke structure is $\mathcal{M} = (W, \leq, V)$, where $V : \mathsf{Prop} \to \mathcal{P}(W)$ is persistent:

$$w \in V(p) \text{ and } w \leq v \quad \Longrightarrow \quad v \in V(p).$$

The order represents growth of information. This interpretation motivates the semantics but is not part of its formal definition.

The forcing relation $\mathcal{M}, w \Vdash A$ is defined recursively:

$$\begin{aligned}
\mathcal{M}, w \Vdash p &\iff w \in V(p), \\
\mathcal{M}, w \nVdash \bot, & \\
\mathcal{M}, w \Vdash A \wedge B &\iff \mathcal{M}, w \Vdash A \text{ and } \mathcal{M}, w \Vdash B, \\
\mathcal{M}, w \Vdash A \vee B &\iff \mathcal{M}, w \Vdash A \text{ or } \mathcal{M}, w \Vdash B, \\
\mathcal{M}, w \Vdash A \to B &\iff \forall v \geq w\, (\mathcal{M}, v \Vdash A \to \mathcal{M}, v \Vdash B).
\end{aligned}$$

The arrow inside the final parenthesis is metalinguistic implication.

Lemma 31.3 (Persistence)**.** *For every formula A, if $w \leq v$ and $\mathcal{M}, w \Vdash A$, then $\mathcal{M}, v \Vdash A$.*

Proof. Proceed by structural induction on A. The atomic case is persistence of V; the conjunction and disjunction cases use the induction hypotheses. For implication, suppose $w \Vdash A \to B$ and $w \leq v$. If $v \leq u$ and $u \Vdash A$, then $w \leq u$, so $u \Vdash B$. Thus $v \Vdash A \to B$. The bottom case is vacuous. □

Definition 31.4 (Intuitionistic semantic consequence)**.**

$$\Gamma \models_{\mathsf{IPL}} A$$

iff, for every intuitionistic Kripke structure $\mathcal{M}$ and every world w, forcing all members of Γ at w implies forcing A at w.

Theorem 31.5 (Soundness and strong completeness of IPL)**.** *For every formula set Γ and formula A,*

$$\Gamma \vdash_{\mathsf{IPL}} A \quad \Longleftrightarrow \quad \Gamma \models_{\mathsf{IPL}} A.$$

Moreover, if $\Gamma \nvdash_{\mathsf{IPL}} A$ for finite Γ, there is a finite rooted Kripke structure with a world forcing every member of Γ and not forcing A.

Proof sketch. Soundness is induction on an LJ-derivation, using persistence in the implication rules. For completeness, extend a nonderiving premise set to prime theories—sets T that are consistent, are deductively closed under $\vdash_{\mathsf{IPL}}$, and contain, for every disjunction $A \vee B$ with $T \vdash_{\mathsf{IPL}} A \vee B$, the disjunct A or the disjunct B—and order them by inclusion. Deductive closure makes $T \vdash_{\mathsf{IPL}} A \vee B$ equivalent to $A \vee B \in T$; the defining condition is derivability from T, not derivability from the empty set. Force a propositional variable at a prime theory iff the variable belongs to it. The truth lemma $w \Vdash B \Longleftrightarrow B \in w$ is proved by structural induction; the implication case uses an extension lemma for failed implications. A finite filtration—a quotient identifying worlds that agree on the finitely many relevant subformulas—through the subformulas of $\Gamma \cup \{A\}$ yields the stated finite rooted Kripke structure and world. The prime-extension lemma, failed-implication extension lemma, and filtration verification are omitted and are supplied by the controlling sources.

Example 31.6 (Counterexample structure and world)**.** Let $W = \{w_0, w_1\}$ with $w_0 < w_1$, and let $V(p) = \{w_1\}$. At w_0, p is not forced. The formula $\neg p$ is also not forced, because the extension w_1 forces p and does not force $\bot$. Therefore

$$\mathcal{M}, w_0 \nVdash p \vee \neg p.$$

No world above w_0 forces $\neg p$, so $w_0 \Vdash \neg\neg p$. Since $w_0 \nVdash p$, the same structure refutes $\neg\neg p \to p$.

Example 31.7 (A constructive derivation)**.** The formula $A \wedge B \to B \wedge A$ is derivable in IPL. In LJ, from the initial sequents $A, B \Rightarrow B$ and $A, B \Rightarrow A$, apply $\wedge R$ to obtain $A, B \Rightarrow B \wedge A$, then $\wedge L$ to obtain $A \wedge B \Rightarrow B \wedge A$, and $\to R$ to conclude $\Rightarrow A \wedge B \to B \wedge A$. No classical principle is used.

Warning 31.8 (Disjunction at a world). The clause for $A \vee B$ requires one disjunct to be forced at the current world. It is not enough that every future world eventually forces some possibly varying disjunct.

Exercise 31.9. Give a Kripke structure and a world in it that falsify Peirce's formula $((A \to B) \to A) \to A$.

Exercise 31.10. Prove semantically that $\neg(A \vee B) \models_{\mathsf{IPL}} \neg A \wedge \neg B$. Then determine whether the converse holds.

31.2 Relations to classical derivability

Theorem 31.11 (Glivenko)**.** *For every propositional formula A and finite multiset Γ of propositional formulas, with both calculi restricted to their propositional rules: $\mathsf{LK} \vdash \Gamma \Rightarrow A$ iff $\mathsf{LJ} \vdash \Gamma \Rightarrow \neg\neg A$. Status: cited theorem.*

See [61, 60]. Glivenko's theorem fails for the quantifier rules; the negative translation repairs the failure symbol by symbol.

Theorem 31.12 (Gödel–Gentzen negative translation)**.** *Assign to every first-order formula A a formula A^N by recursion: $\bot^N := \bot$, and this clause takes priority in the sense that $\bot$ is formed by its own grammar clause, is not an atomic formula (section 8.2), and is not treated by the next clause; $A^N := \neg\neg A$ for atomic A; the translation commutes with $\wedge$, $\to$, and $\forall$; $(A \vee B)^N := \neg(\neg A^N \wedge \neg B^N)$; and $(\exists x\, A)^N := \neg\forall x\, \neg A^N$. Here $\neg$ is the abbreviation $\neg A := A \to \bot$ of section 26.2, so a negated formula is treated by the $\to$ clause; by unique readability (theorem 8.5), exactly one clause applies to each formula, and the recursion is well defined. Then $\mathsf{LK} \vdash \Gamma \Rightarrow A$ iff $\mathsf{LJ} \vdash \Gamma^N \Rightarrow A^N$, where Γ^N translates every member. Status: cited theorem.*

See [61].

Theorem 31.13 (Disjunction property)**.** *If $\mathsf{LJ} \vdash \Rightarrow A \vee B$, then $\mathsf{LJ} \vdash \Rightarrow A$ or $\mathsf{LJ} \vdash \Rightarrow B$. Status: complete proof.*

Proof. By cut elimination for LJ (chapter 27), fix a cut-free LJ-derivation of $\Rightarrow A \vee B$. Its final inference is not Id or $\bot L$, whose conclusions have nonempty antecedents, and not a left logical or left structural rule, for the same reason. The calculus has no right weakening or right contraction. Among the remaining right rules, only $\vee R_1$ and $\vee R_2$ conclude a sequent whose formula is a disjunction. Hence the final inference has premise $\Rightarrow A$ or $\Rightarrow B$. □

Classical derivability lacks this property: $P \vee \neg P$ is classically derivable while neither disjunct is.

Chapter 32

Modal, Temporal, and Epistemic Logic

32.1 Normal propositional modal logic

Modal formulas add $\Box A$ to the classical propositional grammar. Define

$$\Diamond A := \neg \Box \neg A.$$

The symbols have no fixed application until a class of frames and a satisfaction relation are specified.

Definition 32.1 (Kripke frame and Kripke structure)**.** A modal Kripke frame is a pair $\mathcal{F} = (W, R)$, where $W \neq \varnothing$ and $R \subseteq W \times W$. A modal Kripke structure is $\mathcal{M} = (W, R, V)$, where $V : \mathsf{Prop} \to \mathcal{P}(W)$. Satisfaction is classical for propositional connectives and

$$\mathcal{M}, w \models \Box A \quad \Longleftrightarrow \quad \forall v \, (wRv \to \mathcal{M}, v \models A).$$

Consequently, $\mathcal{M}, w \models \Diamond A$ iff some R-successor of w satisfies A.

Relational semantics for normal modal logic originates with [36].

Definition 32.2 (Structure validity and frame validity)**.** A formula is valid in a Kripke structure when it is satisfied at every world of that structure. It is valid on a frame when it is valid in every Kripke structure based on that frame. For a class $\mathcal{C}$ of frames, write $\models_{\mathcal{C}} A$ when A is valid on every frame in $\mathcal{C}$.

Definition 32.3 (Local and global modal consequence)**.** Local consequence is

$$\Gamma \models^{\ell}_{\mathcal{C}} A$$

iff, in every Kripke structure on a frame in $\mathcal{C}$ and at every world w, satisfaction of Γ at w implies satisfaction of A at w. Global consequence is

$$\Gamma \models^{g}_{\mathcal{C}} A$$

iff, in every such structure, satisfaction of every member of Γ at every world implies satisfaction of A at every world. These relations can differ.

The Hilbert calculus K consists of all uniform substitution instances of classically valid propositional formulas, modus ponens, the distribution schema

$$\Box(A \to B) \to (\Box A \to \Box B), \tag{K}$$

and necessitation

$$\frac{\vdash_{\mathsf{K}} A}{\vdash_{\mathsf{K}} \Box A}\ \textsc{Nec}.$$

Necessitation applies to theorems, not to a formula depending on an undischarged premise. Define

$$\begin{aligned} \mathsf{T} &= \mathsf{K} + (\Box A \to A), \\ \mathsf{S4} &= \mathsf{T} + (\Box A \to \Box\Box A), \\ \mathsf{S5} &= \mathsf{T} + (\Diamond A \to \Box\Diamond A). \end{aligned}$$

For $\mathsf{L} \in \{\mathsf{K}, \mathsf{T}, \mathsf{S4}, \mathsf{S5}\}$ and a premise set Γ, an L-derivation from Γ is a finite sequence whose lines are members of Γ, instances of the axiom schemas of L, consequences of earlier lines by modus ponens, or formulas $\Box A$ for which A has a separate no-premise L-derivation. Write $\Gamma \vdash_{\mathsf{L}} A$ when such a sequence ends in A. Thus necessitation remains theorem-only even in a derivation from premises. This definition supplies the premise-sensitive standing needed by strong completeness.

Theorem 32.4 (Frame correspondence and completeness)**.** *In each row, let* L *be the displayed calculus and let* $\mathcal{C}_{\mathsf{L}}$ *be the displayed class of frames:*

calculus	*frame class*
K	*all frames*
T	*reflexive frames*
$\mathsf{S4}$	*reflexive transitive frames*
$\mathsf{S5}$	*frames whose accessibility relation is an equivalence relation.*

Then, for every premise set Γ *and formula* A*,*

$$\Gamma \vdash_{\mathsf{L}} A \quad\Longleftrightarrow\quad \Gamma \models^{\ell}_{\mathcal{C}_{\mathsf{L}}} A.$$

Thus each calculus is sound and strongly complete for local consequence on its paired frame class.

Proof sketch. Soundness checks each axiom against its frame condition and verifies modus ponens and necessitation. Completeness constructs the canonical structure of maximally consistent sets. Define wRv when every $\Box A \in w$ has $A \in v$, and prove the truth lemma by induction. The axioms T, 4, and 5 force the canonical relation to be reflexive, transitive, and Euclidean; reflexivity plus Euclideanness yields an equivalence relation. See [8] for the detailed canonical-structure arguments.

Example 32.5 (Three counterexample structures and worlds)**.** 1. Let $W = \{w\}$, let $R = \varnothing$, and make p false at w in the resulting structure $\mathcal{M}$. Then $\mathcal{M}, w \models \Box p$ and $\mathcal{M}, w \not\models p$, so the structure and world falsify $\Box p \to p$. This refutes T on a nonreflexive frame.

2. Let wRv and vRu, but not wRu. Make p true at v and false at u. If v is the only successor of w, then the resulting structure $\mathcal{M}$ has $\mathcal{M}, w \models \Box p$ but $\mathcal{M}, w \not\models \Box\Box p$. Thus the structure and world falsify $\Box p \to \Box\Box p$, refuting 4 on a nontransitive frame.

3. Let wRv and wRu, let v have no successors, and make p true only at u in the resulting structure $\mathcal{M}$. Then $\mathcal{M}, w \models \Diamond p$, while $\mathcal{M}, v \not\models \Diamond p$; hence $\mathcal{M}, w \not\models \Box\Diamond p$. Thus the structure and world w falsify $\Diamond p \to \Box\Diamond p$, refuting 5 on a non-Euclidean frame.

32.2 Linear temporal logic

Linear temporal logic over discrete infinite time adds the operators X (next) and U (until):

$$A ::= p \mid \bot \mid \neg A \mid (A \wedge A) \mid \mathsf{X}A \mid (A\ \mathsf{U}\ A).$$

Because this grammar retains only $\neg$ and $\wedge$ as primitive Boolean connectives, the following local eliminable abbreviations are necessary for the calculus below:

$$A \vee B := \neg(\neg A \wedge \neg B), \qquad A \to B := \neg(A \wedge \neg B),$$
$$A \leftrightarrow B := (A \to B) \wedge (B \to A), \qquad \top := \neg\bot.$$

Their standing is metalinguistic abbreviation, not additional temporal constructors. Each expands to a formula of the displayed grammar, differs from a primitive constructor precisely by being eliminable, and is used in the axiom schemas, rules, and definitions of F and G below. A temporal structure is an infinite word $\mathcal{M} = (\mathbb{N}, V)$, with $V : \mathbb{N} \to \mathcal{P}(\mathsf{Prop})$. Atomic and Boolean satisfaction is defined pointwise by

$$\begin{aligned} \mathcal{M}, n \models p &\iff p \in V(n), \\ \mathcal{M}, n \not\models \bot, & \\ \mathcal{M}, n \models \neg A &\iff \mathcal{M}, n \not\models A, \\ \mathcal{M}, n \models A \wedge B &\iff (\mathcal{M}, n \models A \text{ and } \mathcal{M}, n \models B). \end{aligned}$$

The temporal clauses are

$$\mathcal{M}, n \models \mathsf{X}A \iff \mathcal{M}, n+1 \models A,$$

and

$$\mathcal{M}, n \models A \,\mathsf{U}\, B \iff \exists m \geq n\, [\mathcal{M}, m \models B \ \wedge \ \forall k\, (n \leq k < m \to \mathcal{M}, k \models A)].$$

Define

$$\mathsf{F}A := \top \,\mathsf{U}\, A, \qquad \mathsf{G}A := \neg\mathsf{F}\neg A.$$

The temporal validity condition used below is

$$\mathcal{M}, n \models A \quad \text{for every temporal structure } \mathcal{M} \text{ and every } n \in \mathbb{N}.$$

It is a property of one formula, not an undefined premise-sensitive consequence relation.

Fix the following Hilbert calculus for theoremhood in this language. A derivation is a finite sequence of formulas in which each line is a uniform substitution instance of a classically valid propositional formula, an instance of one of the three schemas

$$\begin{aligned} \mathsf{X}(A \to B) &\to (\mathsf{X}A \to \mathsf{X}B), \\ \mathsf{X}\neg A &\leftrightarrow \neg\mathsf{X}A, \\ A \,\mathsf{U}\, B &\leftrightarrow B \vee (A \wedge \mathsf{X}(A \,\mathsf{U}\, B)), \end{aligned}$$

or follows from earlier lines by one of the rules

$$\frac{A \qquad A \to B}{B}\ \text{MP}, \qquad \frac{A}{\mathsf{X}A}\ \text{XNec}, \qquad \frac{B \vee (A \wedge \mathsf{X}C) \to C}{A \,\mathsf{U}\, B \to C}\ \text{UInd}.$$

All metavariables in these schemas and rules range over temporal formulas. Write $\vdash_{\mathsf{LTL}} A$ when such a no-premise derivation ends in A. This is a syntactic theoremhood relation, distinct from temporal validity, and it is introduced because the next theorem compares those two standings. Since derivations have no open premises, both necessitation and the induction rule apply only to theorems.

Theorem 32.6 (Soundness and completeness of linear temporal logic)**.** *For every temporal formula A,*

$$\vdash_{\mathsf{LTL}} A \quad \Longleftrightarrow \quad \mathcal{M}, n \models A \text{ for every temporal structure } \mathcal{M} \text{ and every } n \in \mathbb{N}.$$

This is weak completeness for validity, not a strong-completeness assertion for arbitrary premise sets. It concerns discrete infinite linear time; finite traces, dense time, and branching time require different semantics and calculi.

Proof sketch. Soundness of the first two temporal schemas and of XNec follows directly from the unique successor. The unfolding schema follows by separating the cases where the witness for until is the current position or a later position. For UInd, assume its premise is valid and reason backward from the least witness for B to show C at each preceding position at which A holds. Completeness closes a formula under subformulas, negation, and the temporal obligations generated by until; it constructs maximally consistent closure sets linked by the X requirements, proves that every pending until obligation is eventually realized, and unravels the resulting path into a temporal structure. The truth lemma then yields a falsifying temporal structure for every underivable formula. The consistency, eventuality-realization, and truth-lemma case calculations are omitted—this is why the theorem is presented with a proof sketch rather than a complete proof—and are controlled by [20].

Example 32.7 (Eventuality is not recurrence)**.** Let p hold only at time 1. At time 0, $\mathsf{F}p$ is true, but $\mathsf{GF}p$ is false because no occurrence of p remains after time 1. Thus $\mathsf{F}p \to \mathsf{GF}p$ is not valid: the displayed temporal structure and position 0 falsify it.

32.3 Epistemic logic

Fix a nonempty set I of agents. The language adds $K_i A$ for each $i \in I$. An epistemic frame is

$$(W, (\sim_i)_{i \in I}),$$

where $W \neq \varnothing$ and each $\sim_i \subseteq W \times W$. An epistemic structure is $\mathcal{M} = (W, (\sim_i)_{i \in I}, V)$, where $V : \mathsf{Prop} \to \mathcal{P}(W)$, with the classical connective clauses and

$$\mathcal{M}, w \models K_i A \quad \Longleftrightarrow \quad \forall v \, (w \sim_i v \to \mathcal{M}, v \models A).$$

For knowledge, each $\sim_i$ is normally required to be an equivalence relation. The multi-agent calculus used below has all uniform substitution instances of classically valid propositional formulas and, for each $i \in I$, the schemas

$$K_i(A \to B) \to (K_i A \to K_i B), \qquad K_i A \to A,$$
$$K_i A \to K_i K_i A, \qquad \neg K_i A \to K_i \neg K_i A.$$

For a premise set Γ, a derivation is a finite sequence in which each line is a member of Γ, an instance of one of the displayed schemas, follows by modus ponens, or has the form $K_i A$ where A has a separate no-premise derivation. Write $\Gamma \vdash_{\mathsf{S5}_I} A$ when such a derivation ends in A. This theorem-only restriction on necessitation is part of the calculus. The subscript I records that the construction is the componentwise multi-agent extension of the previously defined $\mathsf{S5}$ calculus; it is needed to distinguish the agent-indexed language and adds no interaction axioms between distinct agents.

Theorem 32.8 (Multi-agent $\mathsf{S5}$)**.** *For every premise set Γ and formula A, $\Gamma \vdash_{\mathsf{S5}_I} A$ iff A is a local consequence of Γ over epistemic frames in which every $\sim_i$ is an equivalence relation.*

Proof sketch. Soundness checks each agent-indexed axiom against reflexivity, transitivity, and Euclideanness of $\sim_i$; necessitation is theorem-only. For completeness, take maximally consistent sets as worlds and define $w \sim_i v$ iff every $K_i A \in w$ has $A \in v$. The truth lemma is a structural induction. The three non-distribution schemas make each canonical relation reflexive, transitive, and Euclidean, hence an equivalence relation; no step relates $\sim_i$ to $\sim_j$ for $i \neq j$. Lindenbaum extension, the modal existence lemma, and the full truth-lemma calculation are omitted; the one-agent instances are controlled by [8, Chapter 4].

For a finite group $G \subseteq I$, the language further adds the operators E_G and C_G, with $E_G A$ and $C_G A$ formulas whenever A is a formula. Everyone-knows is definable by

$$E_G A := \bigwedge_{i \in G} K_i A.$$

Common knowledge $C_G A$ receives a satisfaction clause of its own: $C_G A$ is true at w iff A is true at every world reachable from w by a path of length $n \geq 0$ whose edges belong to $\bigcup_{i \in G} \sim_i$; the length-zero path makes w itself reachable, so the clause is reflexive. Common knowledge is therefore a reflexive-transitive closure modality; it is not a finite iteration of E_G bounded in advance.

Example 32.9 (Truth without knowledge)**.** Let $w \sim_i v$, let p be true at w, and let p be false at v. Then p is true at w but $K_i p$ is false there. The structure does not refute factivity $K_i p \to p$: equivalence makes $w \sim_i w$, so knowledge still implies truth at the actual world.

Warning 32.10 (Changing the accessibility condition)*.* If an epistemic relation is not reflexive, $K_i A \to A$ can fail. Such a modality may represent belief or information rather than factive knowledge, but the change must be made in both terminology and frame class.

Exercise 32.11. Give a two-world Kripke structure and a world that falsify $\Diamond p \to \Box \Diamond p$ on a reflexive but non-Euclidean frame.

Exercise 32.12. Evaluate $p \mathbin{\mathsf{U}} q$ at every position of a five-step prefix in which p holds at positions $0, 1, 2$ and q first holds at position 3. State what must be specified beyond the prefix.

Exercise 32.13. Give an epistemic structure in which every agent in G knows p at w, but $C_G p$ fails at w.

Chapter 33

Many-Valued and Paraconsistent Logic

33.1 Logical matrices

Definition 33.1 (Logical matrix)**.** A finite logical matrix for a propositional signature is the structure

$$\mathcal{M} = (\mathsf{V}, D, (f_c)_{c \in \mathcal{L}}),$$

where V is a finite nonempty set of values, $D \subseteq \mathsf{V}$ is the set of designated values, and each n-ary connective c is interpreted by a function $f_c : \mathsf{V}^n \to \mathsf{V}$. A valuation maps propositional variables to V and extends homomorphically to every formula.

Definition 33.2 (Matrix consequence)**.** For a formula set Γ,

$$\Gamma \models_{\mathcal{M}} A$$

iff every valuation assigning a designated value to every member of Γ also assigns a designated value to A.

The number of values does not determine a consequence relation. The operations and the designated set are equally essential.

33.2 A value-labelled tableau calculus

For a fixed finite matrix $\mathcal{M}$, a labelled formula $A{:}a$ asserts that A has exact value $a \in \mathsf{V}$. Expand

$$c(A_1, \ldots, A_n){:}a$$

into one branch for every tuple $\langle a_1, \ldots, a_n \rangle$ satisfying $f_c(a_1, \ldots, a_n) = a$, placing $A_1{:}a_1, \ldots, A_n{:}a_n$ on that branch. A branch closes when it contains $p{:}a$ and $p{:}b$ for the same variable p and distinct values $a \neq b$, or when it contains a labelled formula $c(A_1, \ldots, A_n){:}a$ for which no tuple satisfies $f_c(a_1, \ldots, a_n) = a$, so that its expansion produces no branches.

To test $\Gamma \models_{\mathcal{M}} B$, start one tableau for each assignment of a designated label to every member of Γ and a nondesignated label to B. The tableau succeeds when every branch of every such root closes.

Definition 33.3 (Saturated branch)**.** A branch is *saturated* iff, for every labelled formula $c(A_1, \ldots, A_n){:}a$ on it, there is at least one tuple $\langle a_1, \ldots, a_n \rangle$ with $f_c(a_1, \ldots, a_n) = a$ such that $A_1{:}a_1, \ldots, A_n{:}a_n$ all lie on the branch.

Theorem 33.4 (Finite-matrix tableau soundness and completeness)**.** *For every finite matrix $\mathcal{M}$, finite formula set Γ, and formula B, the value-labelled tableau for Γ and B closes iff $\Gamma \models_{\mathcal{M}} B$. Status: complete proof.*

Proof. Call a valuation v *compatible* with a branch iff $v(A) = a$ for every labelled formula $A{:}a$ on the branch. Full expansion terminates: expanding a labelled occurrence removes it from the unexpanded stock of its branch and adds only labelled occurrences of its immediate subformulas, so the sum of the complexities of the unexpanded labelled occurrences on a branch strictly decreases at each step; each expansion produces at most $|\mathsf{V}|^n$ successor branches, so the fully expanded tree is finite.

Suppose first that $\Gamma \not\models_{\mathcal{M}} B$. Fix a valuation v assigning a designated value to every member of Γ and a nondesignated value to B. Among the started roots is the one labelling each $C \in \Gamma$ with $v(C)$ and B with $v(B)$, and v is compatible with it. If v is compatible with a branch and the labelled formula $c(A_1, \dots, A_n){:}a$ on it is expanded, then

$$f_c(v(A_1), \dots, v(A_n)) = v(c(A_1, \dots, A_n)) = a,$$

so $\langle v(A_1), \dots, v(A_n)\rangle$ is among the expansion tuples—in particular the expansion is not empty—and the successor branch it produces is again compatible with v. Induction on the number of expansion steps therefore keeps one branch compatible with v at every stage of this root. A branch compatible with v satisfies neither closure condition: labels $p{:}a$ and $p{:}b$ with $a \neq b$ would give $a = v(p) = b$, and the displayed calculation supplies a tuple for every labelled compound formula. Hence this root never closes, and the tableau does not close.

Conversely, suppose the tableau does not close: some root, fully expanded, retains a branch that does not close. That branch is saturated: every labelled compound formula on it has been expanded, the second closure condition excludes an empty expansion, and the branch passes through one of the produced successor branches, which placed the components of a witnessing tuple on it. Define a valuation v as follows: a variable p labelled on the branch carries exactly one label $p{:}a$, because the first closure condition fails, and $v(p) := a$; every variable not labelled on the branch receives one fixed value of the nonempty set V. By induction on formula complexity, $v(A) = a$ for every labelled formula $A{:}a$ on the branch. If A is a variable, this is the definition of v. If $A = c(A_1, \dots, A_n)$, saturation supplies a tuple $\langle a_1, \dots, a_n\rangle$ with $f_c(a_1, \dots, a_n) = a$ and every $A_i{:}a_i$ on the branch; the induction hypothesis for the immediate subformulas A_i gives $v(A_i) = a_i$, and the homomorphic extension of v gives

$$v(A) = f_c(v(A_1), \dots, v(A_n)) = f_c(a_1, \dots, a_n) = a.$$

Applied to the root labels, v assigns every member of Γ its designated root label and B its nondesignated root label, so $\Gamma \not\models_{\mathcal{M}} B$. □

33.3 Kleene's three-valued matrix

Let

$$\mathsf{V} = \{0, \tfrac{1}{2}, 1\}, \qquad D = \{1\},$$

and define

$$\neg a = 1 - a, \qquad a \wedge b = \min(a, b), \qquad a \vee b = \max(a, b).$$

This matrix determines the strong Kleene consequence relation for the displayed language; the matrix is denoted $\mathbf{K}_3$, and $\models_{\mathbf{K}_3}$ is its matrix consequence relation.

Example 33.5 (Excluded middle is not designated)**.** If $v(p) = \frac{1}{2}$, then

$$v(p \vee \neg p) = \max(\tfrac{1}{2}, \tfrac{1}{2}) = \tfrac{1}{2} \notin D.$$

Therefore $\not\models_{\mathbf{K}_3} p \vee \neg p$. The calculation is a countervaluation, not a derivation.

33.4 The Logic of Paradox

Use the same value set and operations, but designate

$$D_{\mathsf{LP}} = \{\tfrac{1}{2}, 1\}.$$

Interpret $\frac{1}{2}$ as carrying both positive and negative support. The resulting matrix defines the Logic of Paradox LP. Its value-labelled tableau is sound and complete by the finite-matrix theorem.

Definition 33.6 (Paraconsistent consequence relation)**.** A consequence relation is paraconsistent iff there are formulas A, B such that

$$A, \neg A \not\models B.$$

This definition denies unrestricted explosion. It does not assert that any particular contradiction is derivable or true.

Example 33.7 (Counterexample valuation to explosion in LP)**.** Set

$$v(p) = \tfrac{1}{2}, \qquad v(q) = 0.$$

Both p and $\neg p$ receive the designated value $\frac{1}{2}$, while q is nondesignated. Hence

$$p, \neg p \not\models_{\mathsf{LP}} q.$$

At the same valuation, $p \vee \neg p$ is designated. Paraconsistency and the failure of excluded middle are therefore independent properties; compare the two choices of designated values on the same algebra.

Warning 33.8 (Truth values are formal data). Labels such as “both” or “indeterminate” are interpretations of matrix values. The matrix consequence relation is determined by the truth-value set, operations, and designated set. An informal label cannot replace those data.

Exercise 33.9. Construct the complete value-labelled tableau for $p \wedge q \models_{\mathbf{K}_3} p$. Explain why the same tableau shape is sound for LP.

Exercise 33.10. Find a formula that is valid in LP but not valid in $\mathbf{K}_3$, and verify both claims by the three possible values of its propositional variable.

Chapter 34

Relevance, Substructural Systems, and Linear Logic

34.1 Structural rules as logical data

In a sequent calculus, structural rules determine how formula occurrences may be used:

$$\frac{\Gamma \Rightarrow B}{\Gamma, A \Rightarrow B}\ \textsc{Weakening}, \qquad \frac{\Gamma, A, A \Rightarrow B}{\Gamma, A \Rightarrow B}\ \textsc{Contraction},$$

and exchange permutes adjacent occurrences. A substructural logic omits or controls at least one structural rule. The context representation must therefore remain visible: sets build exchange and contraction into syntax, multisets build in exchange only, and sequences build in neither.

34.2 Routley–Meyer semantics for relevance

The formula language used in this section is exactly

$$A ::= p \mid \neg A \mid (A \wedge A) \mid (A \vee A) \mid (A \to A), \qquad p \in \mathsf{Prop}.$$

No consequence relation or proof calculus is determined by this formation grammar alone.

Definition 34.1 (Routley–Meyer frame and structure)**.** A Routley–Meyer frame is a tuple

$$\mathcal{F} = (W, N, R, *),$$

where $W \neq \varnothing$, $N \subseteq W$ is a nonempty set of normal worlds, $R \subseteq W^3$, and $* : W \to W$ is an involution. The base normality condition used here is

$$n \in N \quad \Longrightarrow \quad (R(n, u, v) \Longleftrightarrow u = v).$$

A Routley–Meyer structure is $\mathcal{M} = (W, N, R, *, V)$, where the frame components satisfy the displayed conditions and $V : \mathsf{Prop} \to \mathcal{P}(W)$ is a valuation.

Atomic satisfaction is

$$\mathcal{M}, w \models p \quad \Longleftrightarrow \quad w \in V(p).$$

Satisfaction for conjunction and disjunction is local and classical. The relevance-sensitive clauses are

$$\mathcal{M}, w \models \neg A \quad \Longleftrightarrow \quad \mathcal{M}, w^* \not\models A,$$

and

$$\mathcal{M}, w \models A \to B \quad \iff \quad \forall u, v\,((R(w,u,v) \wedge \mathcal{M}, u \models A) \to \mathcal{M}, v \models B).$$

Validity requires satisfaction at every normal world. Different relevance logics impose additional first-order conditions on R, $*$, and N, and use matching Hilbert, display, or labelled calculi. No such calculus is selected by the semantic clauses alone.

Worked semantic calculation. At a normal world $n \in N$, the displayed base condition reduces implication to global inclusion of truth sets:

$$\begin{aligned}\mathcal{M}, n \models A \to B &\iff \forall u, v\,((u = v \wedge \mathcal{M}, u \models A) \Longrightarrow \mathcal{M}, v \models B)\\ &\iff \forall u \in W\,(\mathcal{M}, u \models A \Longrightarrow \mathcal{M}, u \models B).\end{aligned}$$

Thus implication at n is not evaluated merely from the truth values of A and B at n. At a nonnormal world, the ternary relation can instead test an A-world and a possibly different B-world. The four-world countermodel below exploits exactly that difference.

For negation, the star operation supplies the comparison point. Let $W = \{w, w^*\}$, let $N = \{w\}$, let star swap the two worlds, put

$$R = \{\langle w, x, x\rangle : x \in W\}, \qquad V(p) = \{w\},$$

and let $\mathcal{M} = (W, N, R, *, V)$, with the values of other propositional variables arbitrary. Then $\mathcal{M}, w \models \neg p$ holds while $\mathcal{M}, w^* \not\models \neg p$. This is a direct calculation from the semantic clause, not an assertion that every relevance logic validates the same additional negation principles.

Cited theorem. Routley and Meyer [53, pp. 53–73] pair the entailment systems specified in that source with ternary relational semantics and prove the corresponding source-scoped semantic results. The controlling hypotheses are the selected source calculus and its matching postulates on the ternary relation, normal points, and star operation; changing either side changes the result. This citation supports the displayed semantic architecture and the need for a matched pair. It is not a completeness theorem for the deliberately broad base frames defined in this chapter.

Internal proof boundary. A local completeness theorem would require selecting one existing system and reproducing its exact formula language; every axiom and inference rule; the context, label, and structural policies; all side conditions; finite derivation objects; and the precise corresponding frame postulates. A complete proof would then need rule-by-rule soundness and a canonical-model or saturated-branch construction with an existence lemma and truth lemma for every constructor. Those source-dependent data are not supplied. The chapter therefore uses only the explicitly displayed satisfaction clauses and normal-world validity, and introduces no calculus name or derivability–validity biconditional.

Example 34.2 (Failure of irrelevant weakening)**.** Let $W = \{n, a, u, v\}$ consist of four distinct worlds, let $N = \{n\}$, let $*$ be the identity, and put

$$R = \{\langle n, x, x\rangle : x \in W\} \cup \{\langle a, u, v\rangle\}.$$

Let $V(p) = \{a\}$, $V(q) = \{u\}$, and let $\mathcal{M} = (W, N, R, *, V)$. Then $q \to p$ is false at a, since $R(a, u, v)$, q is true at u, and p is false at v. Since $R(n, a, a)$ and p is true at a,

$$\mathcal{M}, n \not\models p \to (q \to p).$$

Thus the classical weakening principle need not be relevantly valid in this structure.

The example establishes failure in the displayed base structure. It does not by itself classify the formula in every system called a relevance logic.

34.3 Intuitionistic multiplicative linear logic

The formulas of the multiplicative fragment are

$$A ::= p \mid \mathbf{1} \mid A \otimes A \mid A \multimap A.$$

Sequents have one succedent and multiset antecedents. The calculus contains identity and cut together with

$$\frac{\Gamma, A, B \Rightarrow C}{\Gamma, A \otimes B \Rightarrow C}\otimes L, \qquad \frac{\Gamma \Rightarrow A \qquad \Delta \Rightarrow B}{\Gamma, \Delta \Rightarrow A \otimes B}\otimes R,$$

$$\frac{\Gamma, A \Rightarrow B}{\Gamma \Rightarrow A \multimap B}\multimap R, \qquad \frac{\Gamma \Rightarrow A \qquad \Delta, B \Rightarrow C}{\Gamma, \Delta, A \multimap B \Rightarrow C}\multimap L,$$

and

$$\frac{}{\Rightarrow \mathbf{1}}\,\mathbf{1}R, \qquad \frac{\Gamma \Rightarrow C}{\Gamma, \mathbf{1} \Rightarrow C}\,\mathbf{1}L.$$

Weakening and contraction are absent. Exchange is present because contexts are multisets.

Definition 34.3 (Commutative residuated partially ordered monoid)**.** A commutative residuated partially ordered monoid is

$$\mathcal{A} = (A, \leq, \otimes, 1, \multimap)$$

such that $\leq$ is a partial order on A, $(A, \otimes, 1)$ is a commutative monoid, $\otimes$ is monotone in each argument with respect to $\leq$, and

$$a \otimes b \leq c \quad \Longleftrightarrow \quad b \leq a \multimap c.$$

A valuation extends by

$$[\![A \otimes B]\!] = [\![A]\!] \otimes [\![B]\!], \qquad [\![A \multimap B]\!] = [\![A]\!] \multimap [\![B]\!], \qquad [\![\mathbf{1}]\!] = 1.$$

The sequent $A_1, \ldots, A_n \Rightarrow B$ is valid when

$$[\![A_1]\!] \otimes \cdots \otimes [\![A_n]\!] \leq [\![B]\!]$$

under every valuation in every such monoid.

Theorem 34.4 (Soundness, completeness, and cut elimination)**.** *The displayed multiplicative intuitionistic linear calculus is sound and complete for commutative residuated partially ordered monoids. Cut can be eliminated: every derivable sequent has a cut-free derivation; equivalently, cut is admissible in the cut-free fragment of the calculus.*

Proof sketch. Soundness is a rule-by-rule use of monoid laws and residuation. Algebraic completeness orders formulas by derivability and forms the Lindenbaum quotient; tensor supplies the monoid operation and linear implication supplies its residual. Cut elimination uses principal and commuting reductions adapted to context splitting. The broader classical linear calculus and its proof-theoretic semantics originate with [23].

Example 34.5 (Counterexample algebra and valuations)**.** Let $A = \mathbb{Z}$, let the order be equality, define

$$a \otimes b = a + b, \qquad 1^{\mathcal{A}} = 0, \qquad a \multimap b = b - a.$$

This is a commutative residuated partially ordered monoid because

$$a + b = c \quad \Longleftrightarrow \quad b = c - a.$$

If $v(p) = v(q) = 1$, then $p \Rightarrow p \otimes p$ would require $1 = 2$, so it is invalid; unrestricted contraction would derive it from $p, p \Rightarrow p \otimes p$. Similarly, $p, q \Rightarrow p$ would require $2 = 1$, so unrestricted left weakening is invalid.

The exponential $!A$ marks formulas for which controlled structural rules are available. A standard extension includes dereliction, weakening, contraction, and promotion in the forms

$$\frac{\Gamma, A \Rightarrow B}{\Gamma, !A \Rightarrow B}, \qquad \frac{\Gamma \Rightarrow B}{\Gamma, !A \Rightarrow B}, \qquad \frac{\Gamma, !A, !A \Rightarrow B}{\Gamma, !A \Rightarrow B},$$

$$\frac{!\Gamma \Rightarrow A}{!\Gamma \Rightarrow !A}.$$

The side condition in promotion requires every antecedent formula to be exponential. Adding ! changes both the syntax and the cut-elimination proof.

34.4 Relations among the systems

Relevance logic and linear logic are both substructural, but they are not one logic. Relevance systems constrain implication so that premises bear an appropriate relation to conclusions; many retain contraction while rejecting unrestricted weakening. Linear logic tracks multiplicity and normally rejects both weakening and contraction outside exponentials. A noncommutative substructural calculus also rejects exchange and uses sequence contexts, as in Lambek calculi.

Warning 34.6 (Resource metaphors)*.* Calling a formula a "resource" can motivate a rule, but the formal content is the context representation and the presence or absence of structural rules. The metaphor cannot establish soundness, completeness, or cut elimination.

Exercise 34.7. Verify soundness of $\multimap R$ and $\multimap L$ directly from the residuation law.

Exercise 34.8. Give a cut-free derivation of $A \multimap B, A \Rightarrow B$. Explain why no weakening is required.

Exercise 34.9. For each of exchange, weakening, and contraction, state how representing a context as a sequence, multiset, or set affects whether the rule is built into syntax.

Exercise 34.10. The formula $p \to (q \to p)$ is classically valid and fails in the displayed Routley–Meyer structure. Identify the exact semantic clause that makes the counterexample possible and explain why an ordinary binary Kripke relation would not encode the same conditional clause.

Part XI

Formal Practice, Automation, and Verification

Chapter 35

Constructing Derivations, Models, and Countermodels

This chapter collects methods that apply across the logical systems already defined. A method is not an inference rule. It guides the search for a derivation or semantic object; the completed result must still be checked by the grammar, calculus, and semantics of the active logical system.

35.1 A derivation-search method

Method 35.1 (Backward and forward search)**.** Given premises Γ, conclusion φ, and calculus $\mathcal{C}$:
1. Copy the exact judgment $\Gamma \vdash_{\mathcal{C}} \varphi$.
2. Identify the principal symbol of φ, its main connective in the propositional case.
3. Work backward through introduction rules whose conclusion has that principal symbol.
4. Work forward through elimination rules whose major premise is already available.
5. Record every assumption to be discharged and the rule that will discharge it.
6. For a quantifier rule, verify the displayed total capture-avoiding substitution and, when present, write the eigenvariable condition before applying the rule.
7. When the branches meet, rewrite the result as a derivation tree and check every leaf, rule instance, and discharge mark.

Example 35.2 (Mixed search)**.** To derive $(P \to R) \wedge (Q \to R)$ from $P \vee Q$ and $(P \vee Q) \to R$, backward search on the conjunction creates the two targets $P \to R$ and $Q \to R$. Each target creates an assumption, to be discharged by implication introduction. Forward search applies the second premise to the first premise and yields R, independently of either assumption. Because a discharging application must discharge at least one open occurrence of its assumption, each implication is then concluded through a conjunction detour: assume P, conjoin it with the derived R by $\wedge I$, recover R by $\wedge E_2$, and discharge the occurrence of P used by $\wedge I$; the case for Q is symmetric. The derivation is longer than necessary, and that fact is itself useful: the stronger intermediate result R should be derived first, after which both implications follow by the detours just described.

35.2 A propositional countervaluation method

To refute $\Gamma \models \varphi$, the target conditions are

$$\widehat{v}(\gamma) = \mathbf{T} \quad (\gamma \in \Gamma), \qquad \widehat{v}(\varphi) = \mathbf{F}.$$

Work inward from these target values. A false implication requires a true antecedent and false consequent; a true conjunction requires two true conjuncts; a false disjunction requires two false disjuncts. When a branch assigns both truth values to the same propositional variable, close that branch. A surviving total valuation is a countervaluation.

Example 35.3. For $(P \to Q) \lor (Q \to P)$, an attempted false valuation requires both implications false. The first requires P true and Q false; the second requires Q true and P false. The requirements conflict, so no countervaluation exists. This search establishes validity only because every way for the outer disjunction to be false was exhausted.

35.3 A first-order semantic-refutation method

Method 35.4 (Small-structure search)**.** To refute a first-order semantic consequence:
1. Negate the conclusion at the metalinguistic level by requiring it not to be satisfied.
2. Expand the semantic clauses for the premises and failed conclusion.
3. Start with a one-element domain unless a premise or failed conclusion requires distinct elements.
4. Introduce witnesses for existential requirements and counterexamples for failed universal requirements.
5. Interpret each relation and function symbol only as far as the expanded conditions require.
6. Complete every partially specified function interpretation to a total function on the domain; the remaining values are unconstrained, and any choice may be recorded.
7. Check every premise from the original formulas, not merely the expanded notes.
8. Check that the conclusion fails under the required variable assignment, or in the structure when the conclusion is a sentence.

Example 35.5. The sentence $\forall x \exists y\, R(x, y)$ does not semantically entail $\exists y \forall x\, R(x, y)$. Let the domain be $\{0, 1\}$, and interpret R as $\{\langle 0, 0\rangle, \langle 1, 1\rangle\}$. Every element is related to itself, so the premise is true. No single element is related to both elements, so the conclusion is false.

35.4 Writing a metatheoretic proof

A metatheoretic proof should expose its induction object and invariant.

Method 35.6 (Metatheorem template)**.** 1. State the theorem with the language, calculus, semantic class, and all side hypotheses.
2. Expand the target technical term into its definition.
3. Identify the construction or induction object: formula, term, derivation, finite set, enumeration stage, or computation.
4. State the invariant in a quantified sentence.
5. Prove every base case and constructor case.
6. Isolate uses of compactness, the Axiom of Choice, classical reasoning, or an imported theorem.
7. Restate exactly the conclusion obtained; do not strengthen it in the final sentence.

Warning 35.7. An induction on formula complexity is not complete when it proves only the connective cases and omits the quantifier cases. An induction on a derivation is not complete when it checks only logical rules and omits structural rules or initial sequents present in the calculus.

35.5 Failure analysis

When a proposed proof fails, classify the failure before changing the theorem.

Failure	Meaning
No derivation found	The search may be incomplete; this alone gives no semantic conclusion.
Semantic refutation constructed	Semantic consequence fails; by soundness, derivability in the sound calculus also fails.
Rule side condition fails	That rule instance is unavailable; a different derivation may exist.
Construction invariant fails	The proposed construction does not prove the theorem; a corrected invariant or construction is required.
Hypothesis cannot be verified	The cited theorem cannot yet be applied.
Proof assistant rejects a term	The term does not type-check in the current environment; the informal theorem may still be correct.
Solver returns unknown	No satisfiability or validity conclusion follows from that response.

Exercise 35.8. Develop a countermodel to $\forall x(P(x) \lor Q(x)) \models (\forall x P(x)) \lor (\forall x Q(x))$. Prove that a one-element domain cannot serve and that a two-element domain can.

Exercise 35.9. Write a proof plan for the substitution lemma. Identify the induction object, the strengthened induction statement, and the quantifier case in which bound variable renaming is required.

Chapter 36

Automated Reasoning

Automated reasoning separates a calculus from a search procedure. A calculus determines which inferences are licensed. A search procedure decides which licensed inference to try next. Soundness is a property of the inferences; termination and efficiency are properties of the search procedure. The handbook source for the calculi and procedures of this chapter is [51].

36.1 Propositional satisfiability

The terms *propositional literal*, *clause*, and *conjunctive normal form* retain their definitions from the classical propositional chapter. In particular, a clause C is a finite set of propositional literals, not a formula, and $\mathrm{form}_{\mathsf{CPL}}(C)$ is its exact formula interpretation. Let S be a finite set of clauses. A valuation satisfies S iff it satisfies $\mathrm{form}_{\mathsf{CPL}}(C)$ for every $C \in S$. The empty set of clauses is satisfied by every valuation, whereas a set containing the empty clause is satisfied by none.

Definition 36.1 (Unit propagation)**.** Let S be a finite set of clauses. If S contains a unit clause $\{L\}$, *unit propagation* assigns L true, deletes every clause containing L, and deletes the complementary literal from every remaining clause.

Definition 36.2 (Pure-literal elimination)**.** If a propositional variable occurs in a finite clause set with only one polarity, *pure-literal elimination* assigns the truth value that satisfies all of those occurrences and deletes the clauses containing them.

The Davis–Putnam–Logemann–Loveland (DPLL) procedure recursively applies unit propagation, optionally applies pure-literal elimination, and otherwise branches on an unassigned variable [16, 15]. The following pseudocode specifies the logical core; it does not specify a branching heuristic. The set V of variables occurring in the original input is retained unchanged by every recursive call.

Listing 36.1: Logical core of a DPLL satisfiability procedure

```
DPLL(S, valuation, V):
  repeatedly unit-propagate S, extending valuation by each forced value
  if S has no clauses:
    extend valuation to every unassigned variable in V with value false
    return SAT with the resulting total valuation
  if S contains the empty clause: return UNSAT
  choose an unassigned variable p
  r := DPLL(S with p=true, valuation extended by p=true, V)
  if r is SAT: return r
  return DPLL(S with p=false, valuation extended by p=false, V)
```

Theorem 36.3 (Correctness of the displayed DPLL procedure)**.** *For every finite set S of clauses, the displayed procedure terminates. It returns* SAT *with a satisfying valuation iff S is satisfiable, and it returns* UNSAT *iff S is unsatisfiable.*

Proof. Maintain the invariant that every deleted original clause already contains a literal made true by the current partial valuation, and every undeleted clause is its residual after removing literals made false. Unit propagation preserves this invariant: every satisfying extension of a unit clause makes its literal true. If no residual clauses remain, every original clause is already true; assigning arbitrary values to the remaining input variables cannot falsify a clause that already contains a true literal. Thus the displayed completion by **F** returns a total satisfying valuation on V.

At a branch on p, every total valuation makes either p true or false, so the current residual set is satisfiable iff at least one branch is satisfiable. Each branch assigns a previously unassigned variable; there are finitely many variables, so every path terminates. Induction on the number of unassigned variables now establishes both return specifications. □

36.2 Equivalence and equisatisfiability

Definition 36.4 (Equisatisfiability)**.** Formulas φ and ψ, possibly in different signatures, are *equisatisfiable* iff the two formulas are either both satisfiable or both unsatisfiable.

Logical equivalence is stronger: it compares truth under corresponding valuations, whereas equisatisfiability compares only existence of a satisfying valuation. Tseitin clausification [62] introduces a fresh variable for each selected subformula and constrains that variable to have the same truth value as the subformula. If p_φ names the complete input formula φ, the output also contains the unit clause $\{p_\varphi\}$, which requires the root variable to be true. Call the resulting finite clause set S_φ. The conjunction of the $\mathrm{form}_{\mathsf{CPL}}$-images in any enumeration of S_φ is equisatisfiable with φ in the expanded signature, and S_φ can have size linear in the size of φ. The output is not literally the same formula and need not be logically equivalent in the original signature.

Example 36.5 (Tseitin constraint)**.** For a subformula $\theta = (P \wedge Q)$, introduce fresh p_θ and add the finite set of clauses

$$S_\theta = \{\{\neg p_\theta, P\}, \{\neg p_\theta, Q\}, \{p_\theta, \neg P, \neg Q\}\}.$$

Conjoining the three $\mathrm{form}_{\mathsf{CPL}}$-images in the displayed clause order gives a formula logically equivalent to

$$(\neg p_\theta \vee P) \wedge (\neg p_\theta \vee Q) \wedge (p_\theta \vee \neg P \vee \neg Q),$$

which expresses $p_\theta \leftrightarrow (P \wedge Q)$. Every valuation of P, Q extends uniquely to p_θ while satisfying S_θ. If θ is the complete input, the Tseitin output is $S_\theta \cup \{\{p_\theta\}\}$; the added unit clause requires the extension to make θ true.

36.3 First-order unification

The definitions of finite-support simultaneous term substitution, composition, the instance preorder, unifier, and most general unifier are exactly those in the first-order resolution chapter. In this chapter only, substitutions are written $\theta_{\mathrm{u}}, \rho_{\mathrm{u}}, \eta_{\mathrm{u}}$. The subscript is necessary because θ already denotes a formula in the preceding Tseitin example; it changes no definition. Composition is oriented by

$$(\theta_{\mathrm{u}}\eta_{\mathrm{u}})(x) = (\theta_{\mathrm{u}}(x))\eta_{\mathrm{u}}.$$

Thus θ_{u} is a most general unifier of atoms A and B iff $A\theta_{\mathrm{u}} = B\theta_{\mathrm{u}}$ literally and every unifier ρ_{u} has the form $\theta_{\mathrm{u}}\eta_{\mathrm{u}}$ for some term substitution η_{u}.

The unification procedure performs the occurs check when it considers an equation $x \doteq t$. If x occurs properly in t, the procedure fails. Thus the atoms $P(x)$ and $P(f(x))$ have no unifier: their argument equation is $x \doteq f(x)$. Assigning $x \mapsto f(x)$ does not unify the atoms because their simultaneously substituted arguments are $f(x)$ and $f(f(x))$.

Example 36.6. The atoms $P(f(x, g(a)))$ and $P(f(g(y), g(y)))$ have the most general unifier $\theta_{\mathrm{u}} = \{x \mapsto g(a), y \mapsto a\}$. Applying it to either atom yields $P(f(g(a), g(a)))$.

36.4 First-order resolution

The term *clause* retains its standing as a finite set of first-order literals. Its formula interpretation is $\mathrm{ucl}_{\mathcal{L}}(C)$, the universal closure of $\mathrm{form}_{\mathcal{L}}(C)$. Before each inference, rename the two parent clauses apart. If $A \in E$, $\neg B \in F$, and θ_{u} is a most general unifier of the atoms A and B, the active resolution rule is

$$\frac{E \qquad F}{((E \setminus \{A\}) \cup (F \setminus \{\neg B\}))\theta_{\mathrm{u}}}\ \textsc{Resolution}.$$

The conclusion is the resolvent. Factoring is also part of the active calculus: if two same-polarity literals in E have unifiable underlying atoms and θ_{u} is a most general unifier, infer $E\theta_{\mathrm{u}}$.

Theorem 36.7 (Soundness of resolution)**.** *Every structure satisfying the parent clauses of a resolution or factoring inference satisfies its conclusion.*

Proof. Renaming apart preserves the universally closed formula interpretations. Substitution preserves satisfaction of each parent clause. If an assignment falsified every literal in the displayed resolvent, satisfaction of the first substituted parent would force $A\theta_{\mathrm{u}}$, while satisfaction of the second would force $\neg(B\theta_{\mathrm{u}})$. The two atoms are literally identical, a contradiction. Factoring is sound because simultaneous substitution preserves clause satisfaction. □

For a finite equality-free clause set, resolution with factoring is refutationally complete: after adding a constant if the signature has no ground term, the set is unsatisfiable iff the empty clause has a resolution derivation (theorem 13.29; [49]). Bare resolution is not complete for logical equality. Modern saturation calculi add ordering, redundancy, and equality reasoning; those additions change the calculus and must be stated before their completeness results are used [5].

36.5 Equality and superposition

Replacing equals by equals is semantically valid, but unrestricted replacement creates an impractical search space. A superposition calculus integrates oriented equality reasoning with resolution by selecting eligible terms and using a reduction ordering. The ordering restrictions and redundancy criterion are part of the calculus, not optional implementation details.

Warning 36.8. Treating equality as an uninterpreted binary relation without the identity axioms changes the semantics. Conversely, passing a built-in equality symbol to a prover whose completeness theorem assumes a superposition calculus requires the prover's actual equality rules, not ordinary resolution alone.

36.6 SAT, SMT, and first-order proving

Definition 36.9 (Satisfiability modulo theories)**.** Given a specified background theory T, the *satisfiability modulo theories problem* asks whether some model of T satisfies the input formula under some variable assignment; equivalently, whether T together with the existential closure of the input formula is satisfiable.

An SMT solver commonly combines Boolean search with decision procedures for fragments such as equality with uninterpreted functions, linear arithmetic, arrays, or bit-vectors. The exact input grammar, sort system, theory symbols, commands, and responses are standardized by SMT-LIB. Version 2.7 specifies a many-sorted language, operational commands, and logical semantics [56]. A response of `sat` should be accompanied by a model when one is needed; a response of `unsat` should be accompanied by a checkable proof or trusted-solver assumption when the conclusion is used in a high-assurance development.

The TPTP language supplies standardized formula roles, names, syntax, and annotations for automated theorem proving [58]. A parser-accepted file is not thereby a correctly formalized theory. The declarations, formula roles, conjecture polarity, and imported axioms must still be audited.

36.7 Complexity and practical status

A *decision problem* is, under the fixed coding, a set of natural numbers. It is *polynomial-time decidable* iff some deterministic Turing machine decides it within $p(n)$ steps on every input of length n, for some polynomial p. The class NP consists of the decision problems A for which there are a polynomial-time decidable binary relation R and a polynomial q such that $x \in A$ iff some witness w whose length is at most $q(n)$, where n is the length of x, satisfies $R(x, w)$. A problem is NP-*complete* iff it belongs to NP and every member of NP reduces to it by a polynomial-time computable many-one reduction. Propositional satisfiability is NP-complete [14]. Whether propositional satisfiability is decidable in worst-case polynomial time is the open P versus NP question; no worst-case polynomial-time decision procedure is known. This complexity result does not imply that every large instance is difficult; modern solvers exploit structure. It also does not justify replacing a proof with a solver response whose trust path is unstated.

First-order validity is computably enumerable and undecidable. A complete first-order prover may therefore search forever on some nonvalid inputs. A timeout means only that the configured run did not finish.

Method 36.10 (Automated-result audit)**.** Record:
1. the exact input language and version;
2. the intended logical semantics;
3. every declaration and nonlogical axiom;
4. transformations, including whether they preserve equivalence or only satisfiability;
5. the calculus and search program;
6. the result status and resource limit;
7. the proof certificate or model, when available; and
8. the independent checker and its trusted base.

Exercise 36.11. Run the logical core of DPLL by hand on

$$\{\{P, Q\}, \{\neg P, R\}, \{\neg Q, R\}, \{\neg R\}\}.$$

Give the sequence of unit propagations and the final result.

Exercise 36.12. Show that the atoms $P(f(x, x))$ and $P(f(g(y), y))$ have no unifier. Identify the equation that triggers the occurs check.

Exercise 36.13. Transform the negation of $\forall x(P(x) \to \exists y\, R(x, y))$ into a finite set of first-order clauses whose universal-closure interpretations are equisatisfiable with the negated sentence in the expanded signature. At every step state whether the transformation preserves logical equivalence or only equisatisfiability.

Chapter 37

Mechanized Proof Checking

37.1 Kernel, elaboration, and automation

Definition 37.1 (Proof term)**.** Relative to a type-theoretic proof assistant, a *proof term* is a term whose type is the formal statement proved, according to the assistant's kernel type theory.

Definition 37.2 (Kernel)**.** A *kernel* is the component that checks declarations and proof terms against the primitive typing and conversion rules of the implemented foundation.

Definition 37.3 (Elaboration)**.** *Elaboration* is the transformation of surface-language input containing notation, omitted arguments, coercions, and tactic output into a kernel-level term with all information required for checking.

Definition 37.4 (Tactic)**.** A *tactic* is a program that transforms a proof state or constructs a candidate proof term. A tactic is not an additional inference rule when its output is checked by the kernel.

Lean 4, Rocq, and Isabelle expose different foundations and user languages. Their official manuals distinguish surface syntax from kernel or framework logic [38, 39, 17, 50, 31]. A theorem checked in one assistant is relative to the definitions, axioms, imports, and kernel of that development. For a survey of the mechanization of mathematics, see [3].

37.2 Propositions as types

In a propositions-as-types foundation, a proposition is represented by a type and a proof by a term of that type. Implication corresponds to a function type: from a term of type $A \to B$ and a term of type A, function application produces a term of type B. Universal quantification corresponds to a dependent function type when the quantified object may occur in the result.

This correspondence does not identify every object-language formula in this reference with a proposition of the assistant without an encoding. A deep embedding represents object-language syntax by a datatype and defines an object-level derivability or satisfaction relation. A shallow embedding maps object-language operations directly to operations in the assistant's foundation. The latter can be concise, but it may make adequacy and the exact object logic harder to isolate.

Method 37.5 (Encoding audit)**.** For an encoded logical system, require:
1. a representation of signatures, terms, formulas, and variable binding;
2. a definition of substitution with a capture argument;

3. a representation of structures and satisfaction or of derivations;
4. a theorem connecting the representation to the intended paper definition;
5. explicit treatment of quotienting, alpha-equivalence, or de Bruijn indices;
6. soundness of any reflected decision procedure; and
7. a list of axioms and unsafe or opaque extensions used.

37.3 Trusted base

Definition 37.6 (Trusted base)**.** For a particular checked result, the *trusted base* is the set of components and assumptions whose correctness is required but is not itself established by the checked proof term.

The trusted base normally includes the kernel executable and hardware. It may also include axioms, foreign code, an external solver, generated proof certificates, or an unverified parser, depending on the workflow. The trusted base is result-specific; naming the proof assistant is not a complete trust report.

Method 37.7 (Mechanized-result report)**.** Report separately:
1. the natural-language theorem intended;
2. the exact formal theorem statement;
3. the correspondence argument between them;
4. the assistant and exact version;
5. the imported modules;
6. the axioms printed or otherwise inspected;
7. the kernel-check result;
8. external tools and certificate checking; and
9. remaining human review of definitions and fidelity.

37.4 A small propositions-as-types example

The term

$$\lambda f.\, \lambda g.\, \lambda x.\, f(g(x))$$

has type

$$(B \to C) \to (A \to B) \to A \to C.$$

Under propositions as types it is a proof of the corresponding implicational formula. Each lambda abstraction introduces an assumption, and each application eliminates an implication. The type checker verifies the local rule structure; it does not decide whether A, B, C faithfully encode the informal concepts intended by an application.

Warning 37.8. The phrase "formally verified" is incomplete without the formal statement and trusted base. A proof of a weakened translation, a proof containing an unintended axiom, or a proof about an incorrect definition may be kernel-correct and still fail to establish the intended theorem.

Exercise 37.9. Give proof terms for $A \to A$, $(A \wedge B) \to A$, and $(A \to B) \to (B \to C) \to (A \to C)$ in a simply typed lambda calculus with product types.

Exercise 37.10. Compare a deep and shallow embedding of propositional conjunction. State one adequacy theorem required by each representation.

Chapter 38

Writing and Typesetting Formal Logic

38.1 Academic prose carries the argument

Every display belongs to a grammatical sentence. Use prose to state why an equation or derivation appears, which hypotheses are active, and what the display establishes. Symbols should compress repeated structure after it has been defined; they should not replace the definition.

Method 38.1 (Paragraph structure)**.** A proof paragraph should normally have four components:
1. the local target;
2. the definition or prior result used;
3. the derivation, construction, or calculation; and
4. the exact consequence for the theorem.

When the calculation has several lines, align relation symbols and justify each nontrivial transition.

The following rules govern academic style in this reference.

1. State the language, calculus, semantic class, and ambient metatheory before asserting derivability, validity, completeness, or consistency.
2. Quantify every object on which a claim depends. Replace "for suitable objects" by the exact domain and hypotheses.
3. Use *definition*, *calculation*, *derivation*, and *countermodel* only for the corresponding evidence. A diagram or suggestive example is not a derivation.
4. Use only the proof-status descriptions *complete proof*, *proof sketch*, and *cited theorem*. A proof sketch must state its omitted obligations; a cited theorem must reproduce the hypotheses used and identify its source.
5. Replace evaluative adverbs such as "clearly" and "obviously" by the definition, rule instance, or finite calculation that licenses the sentence.
6. Keep object-language formulas distinct from metalinguistic assertions. In particular, $\varphi \leftrightarrow \psi$ is a formula, whereas $\varphi \equiv \psi$ is a declared metalinguistic relation.
7. State the induction object, induction hypothesis, and every constructor case. If a simultaneous or strengthened induction is required, state it before the base case.
8. End a proof paragraph with the exact clause established. Do not leave the reader to infer whether the argument proved existence, uniqueness, equivalence, or only one implication.

38.2 One symbol, one role within a declared scope

Maintain a notation register during drafting. For every symbol record its standing, type, first definition, scope, and conflicts. A glyph is not globally reserved merely because one chapter assigns it a role. For example, Σ is a formula context in a displayed calculus and is also part of Σ^0_n in the arithmetical hierarchy. These roles are legitimate because each scope is declared before use. International conventions for mathematical symbols are catalogued in [32]; where the registered notation of this document departs from them, the register governs. Within one declared scope, however, a glyph must not change standing silently.

Overloading $\models$ is controlled by argument shape:

$$\mathcal{M}, s \models \varphi, \qquad \mathcal{M} \models \sigma, \qquad \Gamma \models \varphi.$$

These respectively denote satisfaction, truth of a sentence in a structure, and semantic consequence. The propositional, modal, and matrix chapters register the analogous shapes $v \models \varphi$, $\mathcal{M} \models T$, $\mathcal{M}, w \models A$, and $\models_{\mathbf{K}_3}$; no meaning outside these declared shapes is assigned to the glyph.

The unification section of the automated-reasoning chapter (section 36.3) uses $\theta_{\mathrm{u}}, \rho_{\mathrm{u}}, \eta_{\mathrm{u}}$ for substitutions. The visible subscript distinguishes those maps from the formula θ in the Tseitin example while preserving the substitution definitions and composition convention of the first-order resolution chapter.

38.3 A complete before-and-after formalization

Consider the sentence "Clearly, if every P is a Q and something is a P, then something is a Q." As written, it does not identify a signature, a calculus, the status of the letters, or the inference that uses the existential premise. It is therefore not yet a formal-logical claim.

Fix a first-order signature with unary relation symbols P, Q, use the natural-deduction calculus $\mathsf{ND}_{\mathsf{FOL}}$, and let x, y be object-language variables. The corrected claim is

$$\forall x(P(x) \to Q(x)),\ \exists x P(x) \vdash_{\mathsf{ND}_{\mathsf{FOL}}} \exists x Q(x).$$

Here is a complete derivation. The superscript i marks the assumption discharged by $\exists E$:

1.	$\forall x(P(x) \to Q(x))$	premise,
2.	$\exists x P(x)$	premise,
3.	$[P(y)]^i$	assumption,
4.	$P(y) \to Q(y)$	1, $\forall E$,
5.	$Q(y)$	3, 4, $\to E$,
6.	$\exists x Q(x)$	5, $\exists I$,
7.	$\exists x Q(x)$	2, 3–6, $\exists E$ (i).

Choose y absent from the conclusion, from every other open assumption, and from the existential premise. Those are exactly the active $\exists E$ side conditions. Lines 4–6 show that an arbitrary witness for the premise supplies a witness for the conclusion; line 7 discharges the only open assumption. The revised passage therefore fixes the language, calculus, metavariable standing, rule applications, discharge, and freshness condition that the original sentence omitted.

38.4 LaTeX structure

Use semantic markup in the source. A macro name should record the formal-logical role, not the desired visual spacing. For example:

Listing 38.1: Semantic macros for logic

```
\newcommand{\Fm}{\mathsf{Fm}}
\newcommand{\Sent}{\mathsf{Sent}}
\newcommand{\FV}{\operatorname{FV}}
\newcommand{\derives}{\vdash}
\newcommand{\entails}{\models}
```

Use `amsmath` alignment environments; do not use raw double-dollar displays. Use `amsthm` environments to distinguish definitions, theorems, examples, and remarks. Label every referenced theorem or equation and use cross-reference commands rather than fixed numbers. The AMS guidance and `amsmath` documentation provide the publication conventions used here [1, 2].

38.5 Auditing source and rendered output

An audit of rendered output includes verifying that every rendered glyph is the intended character; the normative catalogue of mathematical symbol characters is [64].

The following small example exhibits the whole path from semantic source to a rendered claim. The source deliberately contains a typed signature, quantified semantic parameters, semantic macros, and a stable label:

Listing 38.2: Source for a semantic implication

```
\begin{theorem}\label{thm:sample-semantic-implication}
Let \(\Lang\) contain unary relation symbols \(P,Q\).
For every \(\Lang\)-structure \(\mathcal M\) and assignment \(s\),
if \(\mathcal M,s\entails P(x)\) and
\(\mathcal M,s\entails\forall y(P(y)\to Q(y))\), then
\(\mathcal M,s\entails Q(x)\).
\end{theorem}
\begin{proof}
By the universal clause, if \(P^{\mathcal M}(s(x))\) then
\(Q^{\mathcal M}(s(x))\).  The first hypothesis gives
\(P^{\mathcal M}(s(x))\), so \(Q^{\mathcal M}(s(x))\), which is exactly
\(\mathcal M,s\entails Q(x)\).
\end{proof}
```

The rendered statement must visibly read as a claim about every $\mathcal{L}$-structure and assignment; both hypotheses must remain inside the conditional, and $\models$ must render as semantic satisfaction rather than derivability. The rendered proof must expose the instance $s(x)$, because that calculation is the bridge between the universal semantic clause and the conclusion.

Audit this example in the following order.

1. *Source audit.* Confirm that P, Q are relation symbols, $\mathcal{M}$ is a structure, s is an assignment, and x, y are object-language variables. Confirm that the label is unique and that a later reference uses `\cref` rather than a typed theorem number.
2. *Formal-logical audit.* Expand the satisfaction clauses. The hypotheses become $P^{\mathcal{M}}(s(x))$ and $\forall a \in |\mathcal{M}|(P^{\mathcal{M}}(a) \to Q^{\mathcal{M}}(a))$; instantiation at $s(x)$ yields the conclusion. This independently checks the prose proof.

3. *Compilation audit.* Compile enough times to resolve the label and bibliography. Reject undefined references, missing citations, duplicate labels, malformed environments, and any overfull box that obscures a symbol.
4. *Rendered-page audit.* Inspect the theorem number, line breaks, subscripts, turnstile/model glyphs, proof-ending mark, and surrounding page break. Verify that no line makes the quantifier appear to govern less text than the source says it governs.
5. *Extraction audit.* Extract text from the final PDF and check that the theorem heading, label target, hypotheses, and conclusion remain searchable. Text extraction supplements visual inspection; it does not replace it.

Passing only the source or only the rendered page is insufficient: the audit requires agreement among the formal claim, the LaTeX source, compilation records, cross-reference target, and visible output.

38.6 Rule displays

Each rule display must show:

- all premises;
- the conclusion;
- the rule name;
- discharged assumptions; and
- freshness, admissibility, or occurrence side conditions.

If contexts are sequences, multisets, or sets, state that choice before the rule sheet. Structural rules depend on it.

38.7 Worked correction of an invalid rule application

The following tempting use of universal introduction is invalid:

$$\frac{P(x) \vdash P(x)}{P(x) \vdash \forall x P(x)}\ \textsc{invalid}\ \forall I.$$

The variable x occurs free in the open assumption $P(x)$, contrary to the side condition $x \notin \mathrm{FV}(\Gamma)$. The failure is semantic as well: on a two-element domain, let P hold only of the element assigned to x. Then $P(x)$ is satisfied under that assignment while $\forall x P(x)$ is not, so $\{P(x)\} \not\models \forall x P(x)$: the semantic counterpart of the conclusion judgment fails.

One corrected derivation begins with a genuinely universal premise:

$$\frac{\dfrac{\forall y P(y) \vdash \forall y P(y)}{\forall y P(y) \vdash P(x)}\ \forall E}{\forall y P(y) \vdash \forall x P(x)}\ \forall I \quad x \notin \mathrm{FV}(\{\forall y P(y)\}).$$

Universal elimination supplies the arbitrary instance $P(x)$; now x is absent from the only open assumption, so universal introduction is licensed. The correction changes the premise rather than hiding the violated side condition.

38.8 Citation practice

Separate formal-logical proof from historical attribution. A citation can identify a source or authorize an imported theorem, but it does not turn an unstated theorem into a proved theorem. When

importing a result, reproduce the hypotheses and conclusion needed. When citing a continuously updated manual, record the version or retrieval date.

Primary sources control historical priority and original theorem statements. Canonical monographs control modern notation and standard proofs. Official manuals control current software and language behavior. Secondary summaries may orient research but should not control a disputed technical claim when the owning source is available.

38.9 Revision audit

Method 38.2 (Release audit for a formal document)**.** 1. Search for every technical term at first use and verify a definition.
2. Search every symbol and verify that it retains one role within each declared scope.
3. Check every theorem for quantified objects and exact hypotheses.
4. Check every use of soundness, completeness, consistency, compactness, decidability, and categoricity for the intended meaning.
5. Check every substitution for capture avoidance.
6. Check every quantifier rule for its side condition.
7. Check every imported theorem against the cited source.
8. Check that proof sketches are labelled and that no later result treats a sketch as a formal derivation.
9. Compile until there are no undefined references, missing citations, duplicate labels, or overfull boxes that damage readability.
10. Inspect the rendered pages, including rule displays, tables, index, and bibliography.
11. Report formal-logical review, machine checking, compilation, visual inspection, and human pedagogical review separately.

Exercise 38.3. Rewrite a proof paragraph that uses “clearly” twice and omits the type of a function. Replace each occurrence with the exact definition, calculation, or typed declaration required.

Exercise 38.4. Create a notation register for one chapter of a logic text. Identify every overloaded glyph and either remove the overloading or specify how argument shape disambiguates it.

Chapter 39

Research Workflow and Reproducibility

39.1 Table for theorem status

Maintain a table with one row per result:

Field	Content	Purpose	Failure detected
Identifier	Stable label	Cross-reference	Duplicate or drifting names
Statement	Exact quantified formula	Scope	Strength mismatch
Dependencies	Definitions and prior theorems	Order	Definition after use
Status	complete proof, proof sketch, or cited theorem	Evidence	Unsupported result
System	Language, calculus, semantics	Meaning	Logic shift
Verification	Checks actually performed	Reporting	Compilation reported as proof

39.2 Counterexample-first testing

Before investing in a proof, test the statement against boundary cases:

- empty premise set;
- one-element and two-element structures;
- empty relation and universal relation;
- formulas with no free variable occurrence;
- vacuous quantification;
- inconsistent theories;
- finite versus infinite models;
- countable versus uncountable signatures; and
- constructive versus classical metalogic.

These tests do not prove the statement. They expose missing hypotheses and incorrect generality early.

39.3 Reproducible computational supplements

A computational supplement should include exact input files, tool versions, commands, expected results, and certificate-checking steps. Generated examples should have fixed seeds. If a proof assistant is used, the source should build from a clean environment and print or otherwise expose the axiom surface of the principal results.

39.4 Independent review

An author should ask a reviewer to begin from the theorem statements and seek counterexamples before reading the proofs. A second pass should trace every definition through the proof. A third pass should compare the final statement with its formalization or computational encoding. These passes test different failure modes and should be recorded separately.

Warning 39.1. No finite audit establishes that a long reference contains no formal-logical error. A release report should state the checks completed and the review not performed. It should not replace those facts with a global assurance.

Appendix A

Registered Notation

This is a scoped register, not a list of global reservations. Each row governs only the scope named in its second column. The same glyph may have another standing in a disjoint scope only after that standing has been declared before substantive use. Within one row's scope, the registered standing and meaning remain fixed.

Notation	Declared scope	Standing	Meaning
$\mathcal{L}$	after a language declaration	signature	Nonlogical symbols with declared arities or sorts.
x, y, z	first-order syntax, semantics, and rule sheets	object-language variables	Variables occurring in terms and formulas.
r, t	first-order term formation, substitution, and identity rules	term metavariables	Members of $\mathsf{Term}(\mathcal{L})$.
φ, ψ, χ	propositional and first-order chapters, unless a local declaration says otherwise	formula metavariables	Members of the active formula set.
σ, τ	theory, model-theory, and arithmetic passages after the sentence set is fixed	sentence metavariables	Members of $\mathsf{Sent}(\mathcal{L})$.
θ, ρ, η	first-order resolution chapter	simultaneous term substitutions	Finite-support variable-to-term maps with the action and composition declared there.
$\theta_{\mathrm{u}}, \rho_{\mathrm{u}}, \eta_{\mathrm{u}}$	automated-reasoning chapter in this part	simultaneous term substitutions	Local visible aliases for the preceding substitution role, used to avoid collision with a formula metavariable in the Tseitin example.
$\Gamma, \Delta, \Sigma, \Pi$	a natural-deduction or sequent-calculus display that declares its context representation	formula contexts	Finite sets, multisets, or sequences exactly as declared by that active calculus.

Notation	Declared scope	Standing	Meaning
$\mathcal{A}_{\text{in}}, \mathcal{A}_{\text{tape}}$	Turing-machine definitions	alphabets	Input and tape alphabets, respectively, with no formula-context standing in that scope.
u, v, w	formal-language and computability passages that declare an alphabet	strings	Finite words over the declared alphabet.
Σ^0_n, Π^0_n	arithmetical-hierarchy section	syntactic classes of formulas	Classes fixed by the declared normalization and unbounded quantifier alternation.
$\mathbf{\Sigma}^0_n, \mathbf{\Pi}^0_n, \mathbf{\Delta}^0_n$	arithmetical-hierarchy section	definability classes of relations	Classes fixed by definability in $\mathcal{N}$ using formulas from the corresponding syntactic class.
α, β	cumulative-hierarchy passages	ordinal metavariables	Ordinals in the ambient set theory.
α, β	simple-type-theory passages after the local type grammar	type metavariables	Simple types; this scope is disjoint from the preceding ordinal scope.
T, U	first-order theories and metatheory after a signature is fixed	theories	Sets of sentences in that signature.
$\mathcal{M}, \mathcal{N}$	semantic and model-theory passages	structures	Structures for the active signature, with nonempty domains unless a local convention says otherwise.
$\lvert\mathcal{M}\rvert$	whenever $\mathcal{M}$ is a structure	set	Domain of $\mathcal{M}$.
v	propositional and finite-matrix semantics	valuation	Map from propositional variables to the active truth-value set; its unique homomorphic extension $\widehat{v}$ to formulas is fixed by the active semantics.
s	first-order semantics	variable assignment	Map from first-order variables to $\lvert\mathcal{M}\rvert$.
$\mathcal{M}, s \models \varphi$	first-order semantics	satisfaction	φ is satisfied by $\mathcal{M}$ under s.
$\mathcal{M} \models \sigma$	sentence semantics	truth	Sentence σ is true in $\mathcal{M}$.
$\Gamma \models \varphi$	after a semantic class is declared	semantic consequence	Every interpretation and assignment in that class satisfying Γ satisfies φ.
$\Gamma \vdash_{\mathcal{C}} \varphi$	after calculus $\mathcal{C}$ is declared	derivability	A derivation exists in $\mathcal{C}$.
$\varphi \equiv \psi$	after a semantic class is declared	metalinguistic relation	The formulas have the stated logical equivalence in that class.
$\varphi \leftrightarrow \psi$	a language containing biconditional	formula constructor	Object-language biconditional.

Notation	Declared scope	Standing	Meaning
$a := b$	definitions in the metalanguage	metalinguistic abbreviation	The expression on the left is defined by the expression on the right.
$\varphi[t/x]$	first-order syntax	formula operation	Capture-avoiding substitution of t for free occurrences of x in φ.
$\mathrm{FV}(\varphi)$, $\mathrm{FV}(\Gamma)$	first-order syntax	set-valued operation	Variables with a free occurrence in φ, and their union over all members of the formula context Γ, respectively.
$\mathrm{Mod}(T)$	model theory	class	Structures satisfying every sentence of T.
$\mathrm{Th}(\mathcal{M})$	model theory	theory	Sentences true in $\mathcal{M}$.
R^*	relation-closure and common-knowledge passages	relation	Reflexive transitive closure of the locally declared relation R.
$\bot, \top$	a formula language that declares these constants	formulas	The bottom formula and the defined top-formula abbreviation.

A.1 Structure convention

Unless a chapter explicitly states otherwise, every first-order structure and every sort domain is nonempty. This convention affects quantifier laws, Skolemization, and translations between many-sorted and one-sorted logic.

A.2 Natural numbers convention

$\mathbb{N}$ contains 0. A finite ordinal n is $\{0, \ldots, n-1\}$. The word “positive” therefore means greater than 0, not merely a member of $\mathbb{N}$.

Appendix B

Binding and Precedence

The official grammar is primary. The following precedence convention permits parentheses to be omitted in expository formulas after unique readability has been established.

Rank	Symbols	Association
1	function application, predicate application	determined by arity
2	$=$	nonassociative
3	$\neg, \Box, \Diamond, \forall x, \exists x$	prefix; governs the immediately following formula
4	$\wedge$	left associative
5	$\vee$	left associative
6	$\rightarrow$	right associative
7	$\leftrightarrow$	nonassociative

Thus $\neg P \wedge Q \rightarrow R$ abbreviates $((\neg P) \wedge Q) \rightarrow R$, while $P \rightarrow Q \rightarrow R$ abbreviates $P \rightarrow (Q \rightarrow R)$. Parentheses must be restored whenever omission could make the intended parse uncertain. In particular, $\forall x\, P(x) \rightarrow Q(x)$ abbreviates $(\forall x\, P(x)) \rightarrow Q(x)$; write $\forall x\, (P(x) \rightarrow Q(x))$ when the implication belongs to the quantifier's scope.

Appendix C

Rule Sheets

C.1 Classical natural deduction

Contexts in this sheet are finite sets of formulas. A displayed rule is a rule schema: uniformly substituting formulas for its formula metavariables produces a rule instance. In the propositional portion, $\vdash$ abbreviates $\vdash_{\mathsf{ND}_{\mathsf{CPL}}}$; in the quantifier and identity portion it abbreviates $\vdash_{\mathsf{ND}_{\mathsf{FOL}}}$. The latter calculus contains every rule of the former. Assumptions enter derivations through

$$\frac{}{\Gamma \cup \{\varphi\} \vdash \varphi}\ \text{ASSUMPTION.}$$

Conjunction

$$\frac{\Gamma \vdash \varphi \qquad \Delta \vdash \psi}{\Gamma \cup \Delta \vdash \varphi \wedge \psi}\ \wedge I \qquad \frac{\Gamma \vdash \varphi \wedge \psi}{\Gamma \vdash \varphi}\ \wedge E_1 \qquad \frac{\Gamma \vdash \varphi \wedge \psi}{\Gamma \vdash \psi}\ \wedge E_2.$$

Disjunction

$$\frac{\Gamma \vdash \varphi}{\Gamma \vdash \varphi \vee \psi}\ \vee I_1 \qquad \frac{\Gamma \vdash \psi}{\Gamma \vdash \varphi \vee \psi}\ \vee I_2$$

$$\frac{\Gamma \vdash \varphi \vee \psi \qquad \Delta \cup \{\varphi\} \vdash \chi \qquad \Sigma \cup \{\psi\} \vdash \chi}{\Gamma \cup \Delta \cup \Sigma \vdash \chi}\ \vee E.$$

The displayed occurrences of φ and ψ in the minor derivations are discharged.

Implication

$$\frac{\Gamma \cup \{\varphi\} \vdash \psi}{\Gamma \vdash \varphi \to \psi}\ \to I \qquad \frac{\Gamma \vdash \varphi \to \psi \qquad \Delta \vdash \varphi}{\Gamma \cup \Delta \vdash \psi}\ \to E.$$

The displayed occurrence of φ in implication introduction is discharged.

Primitive negation, the bottom formula, and RAA

$$\frac{\Gamma \cup \{\varphi\} \vdash \bot}{\Gamma \vdash \neg\varphi}\ \neg I \qquad \frac{\Gamma \vdash \varphi \qquad \Delta \vdash \neg\varphi}{\Gamma \cup \Delta \vdash \bot}\ \neg E$$

$$\frac{\Gamma \vdash \bot}{\Gamma \vdash \varphi}\ \bot E \qquad \frac{\Gamma \cup \{\neg\varphi\} \vdash \bot}{\Gamma \vdash \varphi}\ \text{RAA.}$$

The rules $\to I$, $\neg I$, and RAA discharge the displayed assumption. Negation is primitive in the active classical language. Removing RAA gives the primitive-negation presentation of its intuitionistic fragment.

Quantifiers

$$\frac{\Gamma \vdash \varphi}{\Gamma \vdash \forall x\, \varphi}\ \forall I \quad x \notin \mathrm{FV}(\Gamma)$$

$$\frac{\Gamma \vdash \forall x\, \varphi}{\Gamma \vdash \varphi[t/x]}\ \forall E$$

$$\frac{\Gamma \vdash \varphi[t/x]}{\Gamma \vdash \exists x\, \varphi}\ \exists I$$

$$\frac{\Gamma \vdash \exists x\, \varphi \qquad \Delta \cup \{\varphi[y/x]\} \vdash \psi}{\Gamma \cup \Delta \vdash \psi}\ \exists E.$$

Side condition. y is absent from ψ, from every other open assumption, and from $\exists x\, \varphi$. The displayed assumption $\varphi[y/x]$ is discharged. In the contextual display, "every other open assumption" means every formula in $\Gamma \cup \Delta$; the side condition is therefore literally the one used by the active natural-deduction calculus.

Alpha-equivalence

$$\frac{\Gamma \vdash \varphi}{\Gamma \vdash \psi}\ \alpha \quad \varphi \equiv_\alpha \psi.$$

This is the explicit bound-variable conversion rule of the active calculus; it does not permit replacement by an arbitrary logically equivalent formula.

Logical identity

$$\frac{}{\vdash t = t} = I \qquad \frac{\Gamma \vdash r = t \qquad \Delta \vdash \varphi[r/x]}{\Gamma \cup \Delta \vdash \varphi[t/x]} = E.$$

Identity elimination is symmetric because symmetry is derivable from these rules. Every substitution displayed in this sheet is the total capture-avoiding substitution fixed in the syntax chapter, so no separate substitution side condition is imposed.

C.2 Classical sequent calculus

A sequent is $\Gamma \Rightarrow \Delta$, with finite multisets on both sides. It is interpreted as $\bigwedge \Gamma \to \bigvee \Delta$, with the empty conjunction interpreted as $\top$ and empty disjunction as $\bot$.

The propositional rules below are exactly the active calculus $\mathsf{LK}_{\mathsf{CPL}}$. The subsequent quantifier and identity rules extend it to $\mathsf{LK}_{\mathsf{FOL}}$.

Initial sequents and cut

$$\frac{}{\Gamma, \varphi \Rightarrow \varphi, \Delta}\ \text{ID} \qquad \frac{}{\Gamma, \bot \Rightarrow \Delta}\ \bot L$$

$$\frac{\Gamma \Rightarrow \Delta, \varphi \qquad \Sigma, \varphi \Rightarrow \Pi}{\Gamma, \Sigma \Rightarrow \Delta, \Pi}\ \text{CUT}.$$

Structural rules

$$\frac{\Gamma \Rightarrow \Delta}{\varphi, \Gamma \Rightarrow \Delta}\ \mathrm{WL} \qquad \frac{\Gamma \Rightarrow \Delta}{\Gamma \Rightarrow \Delta, \varphi}\ \mathrm{WR}$$

$$\frac{\varphi, \varphi, \Gamma \Rightarrow \Delta}{\varphi, \Gamma \Rightarrow \Delta}\ \mathrm{CL} \qquad \frac{\Gamma \Rightarrow \Delta, \varphi, \varphi}{\Gamma \Rightarrow \Delta, \varphi}\ \mathrm{CR}.$$

Exchange is built into the multiset convention.

Logical rules

$$\frac{\Gamma \Rightarrow \Delta, \varphi}{\Gamma, \neg\varphi \Rightarrow \Delta}\ \neg L \qquad \frac{\Gamma, \varphi \Rightarrow \Delta}{\Gamma \Rightarrow \Delta, \neg\varphi}\ \neg R$$

$$\frac{\varphi, \psi, \Gamma \Rightarrow \Delta}{\varphi \wedge \psi, \Gamma \Rightarrow \Delta}\ \wedge L \qquad \frac{\Gamma \Rightarrow \Delta, \varphi \qquad \Gamma \Rightarrow \Delta, \psi}{\Gamma \Rightarrow \Delta, \varphi \wedge \psi}\ \wedge R$$

$$\frac{\varphi, \Gamma \Rightarrow \Delta \qquad \psi, \Gamma \Rightarrow \Delta}{\varphi \vee \psi, \Gamma \Rightarrow \Delta}\ \vee L \qquad \frac{\Gamma \Rightarrow \Delta, \varphi, \psi}{\Gamma \Rightarrow \Delta, \varphi \vee \psi}\ \vee R$$

$$\frac{\Gamma \Rightarrow \Delta, \varphi \qquad \Gamma, \psi \Rightarrow \Delta}{\Gamma, \varphi \to \psi \Rightarrow \Delta}\ \to L \qquad \frac{\varphi, \Gamma \Rightarrow \Delta, \psi}{\Gamma \Rightarrow \Delta, \varphi \to \psi}\ \to R.$$

Quantifiers

$$\frac{\Gamma, \varphi[t/x] \Rightarrow \Delta}{\Gamma, \forall x\, \varphi \Rightarrow \Delta}\ \forall L \qquad \frac{\Gamma \Rightarrow \Delta, \varphi[t/x]}{\Gamma \Rightarrow \Delta, \exists x\, \varphi}\ \exists R$$

$$\frac{\Gamma \Rightarrow \Delta, \varphi[y/x]}{\Gamma \Rightarrow \Delta, \forall x\, \varphi}\ \forall R \qquad \frac{\Gamma, \varphi[y/x] \Rightarrow \Delta}{\Gamma, \exists x\, \varphi \Rightarrow \Delta}\ \exists L.$$

In $\forall R$, the eigenvariable y is free in no formula of $\Gamma, \Delta, \forall x\, \varphi$. In $\exists L$, it is free in no formula of $\Gamma, \Delta, \exists x\, \varphi$. Every displayed substitution is total and capture-avoiding.

Logical identity

$$\frac{}{\Rightarrow t = t}\ = I \qquad \frac{\Gamma, \varphi[r/x] \Rightarrow \Delta}{\Gamma, r = t, \varphi[t/x] \Rightarrow \Delta}\ = L.$$

The symmetric replacement instance is derivable. These displays, together with the propositional rules, are the complete active first-order sequent calculus used in this reference.

C.3 Modal calculus for K

The Hilbert calculus K contains all substitution instances of classical classically valid propositional formulas, the axiom schema

$$\Box(\varphi \to \psi) \to (\Box\varphi \to \Box\psi),$$

modus ponens, and necessitation:

$$\frac{\vdash_{\mathsf{K}} \varphi}{\vdash_{\mathsf{K}} \Box\varphi}\ \mathrm{NEC}.$$

Necessitation applies to theorems, not to an arbitrary formula depending on an undischarged assumption. Adding $\Box\varphi \to \varphi$, $\Box\varphi \to \Box\Box\varphi$, and $\Diamond\varphi \to \Box\Diamond\varphi$ yields the standard systems T, $\mathsf{S4}$, and $\mathsf{S5}$ in the combinations defined in the modal chapter.

Appendix D

Qualifying-Examination Problems and Selected Solutions

D.1 Problem 1: substitution and satisfaction

Let φ be a first-order formula, let x be a variable, let t be a term, let $\mathcal{M}$ be a structure, and let s be a variable assignment. Prove

$$\mathcal{M}, s \models \varphi[t/x] \quad \Longleftrightarrow \quad \mathcal{M}, s[x \mapsto [\![t]\!]_s^{\mathcal{M}}] \models \varphi.$$

Solution. First prove the term-substitution lemma by structural induction on terms. Then proceed by structural induction on φ. Atomic formulas follow from the term lemma. For each connective, unfold its semantic clause and apply the induction hypotheses to its immediate subformulas. Let $Q \in \{\forall, \exists\}$ and consider $Qy\,\psi$. If $y = x$, substitution stops at the binder. On the right side, the quantifier update of x overrides the preceding update of x, so the truth value of $Qx\,\psi$ is independent of that preceding value.

Now suppose $y \neq x$. If y occurs in t, the deterministic capture-avoiding definition first renames the binder to a variable fresh for t and ψ. Thus it remains to consider $y \notin \mathrm{Var}(t)$. For each $a \in |\mathcal{M}|$, the assignments

$$s[y \mapsto a][x \mapsto [\![t]\!]_s^{\mathcal{M}}] \quad \text{and} \quad s[x \mapsto [\![t]\!]_s^{\mathcal{M}}][y \mapsto a]$$

agree because $x \neq y$, and term evaluation of t is unchanged by the update at y. Apply the induction hypothesis to ψ under each updated assignment and then apply the semantic clause for Q. Alpha-invariance justifies the preliminary binder renaming. Substitution is total and capture-avoiding, so the displayed definition requires no additional definedness hypothesis. □

D.2 Problem 2: compactness

Let $\mathcal{L}$ contain a binary relation symbol $<$ and constants c_n for $n \in \mathbb{N}$. Let T say that $<$ is a strict linear order and include $c_{n+1} < c_n$ for every n. Prove that T has a model, and explain why this does not prove that every strict linear order has an infinite descending chain.

Solution. Every finite subset $T_0 \subseteq T$ mentions only finitely many of the descending-chain axioms. Interpret the relevant constants as a finite descending segment of $\mathbb{Z}$ with its usual order, and interpret unused constants arbitrarily. Then T_0 has a model. Compactness gives a model of all of T, in which the interpretations of the c_n form an infinite descending chain. The conclusion is existential: at least one strict linear order with such a chain exists. It makes no universal assertion about all strict linear orders; $\mathbb{N}$ with its usual order is a counterexample to that universal assertion. □

D.3 Problem 3: incompleteness hypotheses

State a version of the first incompleteness theorem for a computably enumerable theory T, and explain why each hypothesis is needed.

Solution. One standard version is: if T is a consistent, computably enumerable theory in the language of arithmetic extending a fixed weak theory sufficient to represent the required computable syntactic relations, then there is a sentence G_T such that $T \nvdash G_T$; with the stronger ω-consistency hypothesis in Gödel's original form, $T \nvdash \neg G_T$ as well. Rosser's refinement replaces ω-consistency by consistency for a differently constructed sentence. The computably enumerable axiom set makes provability arithmetically expressible; arithmetic strength supports coding and diagonalization; consistency prevents the theory from deriving every sentence; and the strengthened hypothesis in the original proof excludes derivation of the negation. No version licenses a claim about an arbitrary collection of axioms lacking these properties. □

D.4 Problem 4: modal frame condition

Prove that $\Box P \to P$ is valid on a frame iff the accessibility relation is reflexive.

Solution. If the relation is reflexive and $\mathcal{M}, w \models \Box P$, then every accessible world satisfies P; since w is accessible from itself, $\mathcal{M}, w \models P$. Conversely, suppose the frame is not reflexive at w. Let $\mathcal{M}$ be the structure on that frame whose valuation makes P true at every world accessible from w and false at w. Then $\mathcal{M}, w \models \Box P$ but $\mathcal{M}, w \not\models P$, so the formula is not valid on the frame. □

D.5 Problem 5: cut and subformula property

Explain why cut elimination implies a subformula property for the cut-free classical propositional sequent calculus, and state one consequence.

Solution. Inspect every propositional rule except cut. Each formula in a premise is either copied from the conclusion or is an immediate subformula of the principal formula in the conclusion. Induction upward through a cut-free derivation therefore shows that every formula occurrence is a subformula of a formula in the end sequent, apart from the declared treatment of structural copies. Cut is the rule that can introduce an arbitrary intermediate formula and then remove it. A consequence is consistency: a cut-free derivation of the empty sequent would have a last rule, but no rule of the cut-free calculus—initial, structural, or logical—has the empty sequent as its conclusion, since every weakening or contraction conclusion contains a displayed formula occurrence. For the first-order calculus, quantifier premises contain substitution instances. Its analytic property must therefore be stated modulo term substitution and eigenvariable renaming rather than as the literal subformula property just proved. □

D.6 Additional problems

1. Prove Łoś's theorem for existential formulas from its induction hypothesis, explicitly identifying the use of the Axiom of Choice in selecting coordinatewise witnesses.
2. Give a complete Henkin construction for a countable consistent theory and prove the truth lemma for its term structure.
3. Prove that the full second-order Peano axioms are categorical, and state why the proof uses full semantics.

4. Construct a three-valued matrix in which explosion fails. State the designated values and calculate one countervaluation.
5. Formalize one theorem in a proof assistant and record the exact theorem declaration, printed axiom surface, build instructions, and a paragraph comparing the declaration with the informal theorem.

Appendix E

Exercise Feedback

This appendix follows the exercise order in the reference. Every heading gives the exact source label and links back to the exercise. "Solution" marks a complete answer at the level requested. "Solution sketch" gives the decisive construction or invariant and an explicit criterion by which a completed proof can be checked.

E.1 Language levels, formal objects, and proof writing

ex:levels-01. *Solution.* Relative to classical propositional logic, P and $P \wedge Q$ are object-language formulas, while φ, the phrase "is valid," and $\models$ belong to the metalanguage. A left-parenthesis token used in the fully parenthesized formula $(P \wedge Q)$ is object-language punctuation; the phrase "the left parenthesis" that mentions that token is metalinguistic.

ex:levels-02. *Solution.* Put $v(P) = \mathbf{F}$ and $v(Q) = \mathbf{T}$. Then $v(P \to Q) = \mathbf{T}$ and $v(Q) = \mathbf{T}$, while $v(P) = \mathbf{F}$. Thus this single row satisfies both premises and falsifies the conclusion.

ex:strings-01. *Solution.* Let B be the least set of strings such that $\epsilon \in B$, $w \in B$ implies $(w) \in B$, and $u, v \in B$ implies $uv \in B$. Write $L(w), R(w)$ for the two parenthesis counts. Structural induction gives $L(\epsilon) = R(\epsilon) = 0$; if $L(w) = R(w)$, then both counts increase by one in (w); and if the equality holds for u, v, then $L(uv) = L(u) + L(v) = R(u) + R(v) = R(uv)$.

ex:strings-02. *Solution.* Give every initial derivation height 0, and define the height of a derivation ending with immediate subderivations $D_1, \ldots, D_k$ to be $1 + \max_i h(D_i)$. Each immediate subderivation has smaller height. Repeating that strict inequality down the finite constructor path shows that every proper subderivation has strictly smaller height.

ex:writing-foundations-01. *Solution.* A correct completeness statement is: for every set Γ of first-order sentences and sentence σ, if $\Gamma \models \sigma$ over all nonempty first-order structures, then $\Gamma \vdash_{\mathsf{ND}_{\mathsf{FOL}}} \sigma$. A false correction is: for every arithmetic sentence σ, if $\mathcal{N} \models \sigma$, then $\mathsf{PA} \vdash \sigma$. The first fixes a calculus and semantic class; the second fails because true arithmetic is not the theorem set of PA.

ex:writing-foundations-02. *Solution.* The original leaves "theory," "true," the intended structures, and the proof calculus unspecified. Deductive completeness is: for every sentence σ, either $T \vdash \sigma$ or $T \vdash \neg\sigma$. The separate semantic property is that all models of T agree on every sentence.

For a satisfiable T, soundness and completeness of the fixed first-order calculus make these two properties equivalent.

ex:set-prerequisites-01. *Solution.* Let $i : A \to \mathbb{N}$ be an injection. For each $b \in f[A]$, define

$$j(b) := \min\{i(a) : a \in A \text{ and } f(a) = b\}.$$

The set minimized over is a nonempty subset of $\mathbb{N}$. If $j(b) = j(b')$, injectivity of i gives the same selected preimage, so $b = b'$. Thus $j : f[A] \to \mathbb{N}$ is an injection. This also covers $A = \varnothing$, when j is the empty function.

ex:set-prerequisites-02. *Solution.* Define $[a]_E + [b]_E = [a + b]_E$. If aEa' and bEb', then $5 \mid (a - a')$ and $5 \mid (b - b')$, so $5 \mid ((a + b) - (a' + b'))$. Therefore $[a + b]_E = [a' + b']_E$, which proves independence from both representatives.

ex:set-prerequisites-03. *Solution.* For a family $(A_n)_{n\in\mathbb{N}}$, the usual proof first selects, for every n, one enumeration or surjection $e_n : \mathbb{N} \to A_n$ (with a separate convention for empty members), and only then maps (n, k) to $e_n(k)$. The simultaneous selection of the sequence $(e_n)_n$ is the simultaneous selection step; pairing $\mathbb{N}^2$ with $\mathbb{N}$ is not. If the enumerations are part of the given data, that step is unnecessary.

E.2 Classical propositional logic

ex:prop-metatheory-01. *Solution.* The full parse is $((\neg P \wedge Q) \to (R \vee S))$. Its subformulas are $P, Q, R, S, \neg P, (\neg P \wedge Q), (R \vee S)$, and the whole implication. Every binary node shown is fixed by the declared precedence, and implication associativity is not needed because there is only one implication token.

ex:prop-metatheory-02. *Solution.* Let $l(\varphi)$ count syntax-tree leaves—occurrences of propositional variables or $\bot$—and let $b(\varphi)$ count binary-connective occurrences. For a variable or $\bot$, $l = 1 = 0 + 1 = b + 1$. A negation has the same leaves and binary nodes as its sole subformula, so the induction hypothesis is preserved. If $\varphi \circ \psi$ is formed with a binary connective, then

$$l(\varphi \circ \psi) = l(\varphi) + l(\psi) = (b(\varphi) + 1) + (b(\psi) + 1) = b(\varphi \circ \psi) + 1.$$

These are all constructors, so structural induction proves the invariant.

ex:prop-metatheory-03. *Solution.* Set $P = \mathbf{F}, Q = \mathbf{T}$. Both $P \to Q$ and Q are true, but P is false, so semantic consequence fails. This valuation inspects truth values only; nonexistence of a derivation follows separately from soundness of the specified calculus.

ex:prop-metatheory-04. *Solution.* Let P mean that the process terminates, Q that the input is finite, and R that the search is bounded. Taking "or" as inclusive before applying the stated restriction, the phrase following "only if" requires exactly one of Q and R whenever P holds. A faithful translation is

$$P \to \big((Q \vee R) \wedge \neg(Q \wedge R)\big).$$

The stronger formula $(P \to (Q \vee R)) \wedge \neg(Q \wedge R)$ would forbid $Q \wedge R$ even when the process does not terminate; that global restriction is not stated by the sentence.

ex:prop-metatheory-05. *Solution.* A conjunctive normal form is $(\neg P \vee Q) \wedge (\neg Q \vee R)$. A disjunctive normal form is $(\neg P \wedge \neg Q) \vee (Q \wedge R)$. In the four rows with $Q = \mathbf{F}$, the formula is true iff $P = \mathbf{F}$; in the four rows with $Q = \mathbf{T}$, it is true iff $R = \mathbf{T}$. Those two cases are exactly the two displayed disjuncts.

ex:prop-metatheory-06. *Solution.* Assume $P \to R$, then $Q \to R$, then $P \vee Q$. For the left case assume P and infer R; for the right case assume Q and infer R. Disjunction elimination discharges the two case assumptions. Three successive applications of $\to I$ discharge $P \vee Q$, $Q \to R$, and $P \to R$, in that order, yielding the required nested implication.

ex:prop-metatheory-07. *Solution.* Assume $P \wedge \neg P$. Conjunction elimination gives P and $\neg P$; negation elimination gives $\bot$; negation introduction discharges the assumption and yields $\neg(P \wedge \neg P)$. The decisive final rule is $\neg I$, which is available in both displayed classical and intuitionistic natural deduction; no reductio rule is used.

ex:prop-metatheory-08. *Solution sketch.* Apply $\to R$ to reduce the target to $P \to Q, Q \to R, P \Rightarrow R$. Apply $\to L$ to $Q \to R$; its right premise is the identity sequent $R \Rightarrow R$. Its left premise derives Q by applying $\to L$ to $P \to Q$, with identities $P \Rightarrow P$ and $Q \Rightarrow Q$. Context weakening supplies the displayed companions. A checked tree contains no cut, and every branch ends in identity.

ex:prop-metatheory-09. *Solution sketch.* Reflexivity, symmetry, and transitivity follow from equality of truth values under every valuation. For congruence, assume corresponding inputs have equal truth values; each primitive connective is a fixed truth function, so applying it to equal input tuples gives equal outputs. Check the unary negation case and each binary truth table separately; these are all primitive constructors.

ex:prop-metatheory-10. *Solution sketch.* Choose the underivable formula P. Then $\{\neg P\}$ is consistent; extend it to a maximal consistent set Δ. Define $v(Q) = \mathbf{T}$ iff $Q \in \Delta$. The truth lemma, proved by induction on formulas, gives $v(\varphi) = \mathbf{T}$ iff $\varphi \in \Delta$. Since $\neg P \in \Delta$, maximal consistency gives $P \notin \Delta$, hence $v(P) = \mathbf{F}$. A complete answer must include the truth-lemma cases for every primitive connective.

ex:prop-metatheory-11. *Solution.* The clause set is $\{\{P, Q\}, \{\neg P, R\}, \{\neg Q, R\}, \{\neg R\}\}$. Resolve the second and fourth clauses to obtain $\{\neg P\}$, and the third and fourth to obtain $\{\neg Q\}$. Resolve $\{P, Q\}$ with $\{\neg P\}$ to get $\{Q\}$, then with $\{\neg Q\}$ to obtain the empty clause.

ex:prop-metatheory-12. *Solution.* A tautological clause is true under every valuation, so removing it changes neither the satisfying valuations nor satisfiability. For the second claim take $S = \{\{P\}, \{Q\}\}$. The valuation $P = Q = \mathbf{T}$ satisfies the clause $\{P\}$ that is deleted, but the reduced set $\{\{Q\}\}$ is also satisfied by $P = \mathbf{F}, Q = \mathbf{T}$, whereas S is not. Thus satisfaction by the current valuation does not establish equivalence under all valuations.

E.3 Classical first-order logic

ex:fol-deduction-01. *Solution.* The free-variable set is $\{x, y, z\}$. The occurrence of x in $P(x, z)$ is free; the occurrences of x in $Q(x, y)$ and $R(x, y, z)$ are bound by $\forall x$. The occurrence of y in

$Q(x, y)$ is free because it lies outside the scope of $\exists y$; the occurrence of y in $R(x, y, z)$ is bound by that existential quantifier. Both occurrences of z are free.

ex:fol-deduction-02. *Solution.* In the first formula, rename the bound y to fresh z before substitution, obtaining $(\exists y R(x, y))[y/x] = \exists z R(y, z)$. In the second, rename bound x to fresh z, because the replacing term contains x; the result is $(\forall x(P(x) \to Q(y)))[f(x)/y] = \forall z(P(z) \to Q(f(x)))$. In each result the formerly free variables in the replacing term remain free.

ex:fol-deduction-03. *Solution sketch.* Prove simultaneously: term values agree whenever two assignments agree on the term's variables, and formula truth agrees whenever they agree on the formula's free variables. The variable case is the assignment hypothesis; the binary-function case applies both term induction hypotheses; the binary-relation case applies term coincidence to both arguments. Boolean cases use formula induction. In a quantifier case, update both assignments at the bound variable by the same arbitrary element, then use the induction hypothesis. The proof for an arbitrary signature differs only by replacing two arguments with the declared finite arity of the active symbol.

ex:fol-deduction-04. *Solution.* Choose y absent from both premises and the conclusion. Under the assumption $P(y)$, to be discharged by $\exists E$, universal elimination gives $P(y) \to Q(y)$; implication elimination gives $Q(y)$; existential introduction gives $\exists x Q(x)$. Apply $\exists E$ to the premise $\exists x P(x)$ and discharge $P(y)$. The eigenvariable y is absent from the conclusion, every other open assumption, and the existential premise.

ex:fol-deduction-05. *Solution.* From $x = y$ and $y = z$, apply equality elimination to the latter equality with the formula $\varphi(w) := x = w$. Its required minor premise is $\varphi(y)$, namely $x = y$, and its conclusion is $\varphi(z)$, namely $x = z$. Thus $x = y, y = z \vdash x = z$.

ex:fol-deduction-06. *Solution.* Use one domain containing people and papers, unary predicates $\mathsf{Rev}, \mathsf{Paper}$, and binary relations $\mathsf{Read}, \mathsf{Cites}$. The translations are

$$\forall x\big(\mathsf{Rev}(x) \to \exists y(\mathsf{Paper}(y) \land \mathsf{Read}(x, y) \land \forall z((\mathsf{Paper}(z) \land \mathsf{Read}(x, z)) \to z = y))\big),$$
$$\exists y(\mathsf{Paper}(y) \land \forall x(\mathsf{Rev}(x) \to \mathsf{Read}(x, y))),$$
$$\forall x\big(\mathsf{Rev}(x) \to \exists y(\mathsf{Paper}(y) \land \mathsf{Read}(x, y) \land \neg\mathsf{Cites}(y, y))\big).$$

The last formula makes the nonvacuous reading explicit: every reviewer read at least one paper that did not cite itself.

ex:fol-deduction-07. *Solution.* If a $\mathcal{L}_{\text{grp}}$-structure satisfies the three displayed sentences, the interpretation of $\cdot$ is associative, e is a two-sided identity, and $i(a)$ is a two-sided inverse of each a; hence the reduct is a group. Conversely, interpret $\cdot, e, i$ in any group by its multiplication, identity, and inverse operation. The group laws verify each universal sentence under every assignment. Equality is logical identity in both directions, so the model class is exactly the class of groups in that signature.

ex:fol-deduction-08. *Solution.* In the signature with one binary relation E, take

$$T_{\text{graph}} = \{\forall x\, \neg E(x, x), \quad \forall x \forall y(E(x, y) \leftrightarrow E(y, x))\}.$$

Its models are precisely nonempty simple undirected graphs. The first axiom forbids loops, and logical identity needs no additional equivalence axioms.

ex:fol-deduction-09. *Solution.* Begin with the identity sequent $P(y) \Rightarrow P(y)$. Apply $\forall L$ to obtain $\forall x P(x) \Rightarrow P(y)$, then $\exists R$ to obtain $\forall x P(x) \Rightarrow \exists x P(x)$. No cut occurs. The semantic soundness of choosing the instance denoted by y uses that assignments into the domain exist, hence that the domain is nonempty.

ex:fol-cardinality-normalforms-01. *Solution sketch.* For $\mathcal{M} \subseteq \mathcal{N}$, prove by structural induction that formulas with parameters from M have the same truth value in both structures. Atomic and Boolean cases use the substructure hypothesis and induction. For $\exists x \varphi$, the forward direction keeps a witness in M; the reverse direction uses the Tarski–Vaught witness condition to replace an N-witness by one in M, then applies induction. For $\forall x \varphi$, one direction restricts the quantified domain, while the other argues that a counterexample in N would make $\exists x \neg \varphi$ true and hence have an M-witness. A complete proof must state both quantifier directions without invoking duality as a substitute for these arguments.

ex:fol-cardinality-normalforms-02. *Solution sketch.* Let A_0 be A together with m_0 and the interpretations of the constant symbols. Given countable A_n, let A_{n+1} add the witness value $F_\varphi(\bar{a})$ for every formula $\exists y\, \varphi(y, \bar{x})$ and tuple $\bar{a}$ from A_n; the witness functions for the formulas $\exists y\, (y = f(\bar{x}))$ put the value of every signature function on every tuple from A_n into A_{n+1}. These prescriptions explicitly give A_0, A_1, A_2, and countability follows from the countable signature. If a finite tuple lies in $\bigcup_n A_n$, it lies in some common stage; the value of every signature function on that tuple lies in the next stage. This is the required closure check.

ex:fol-cardinality-normalforms-03. *Solution.* Add constants $c_0, c_1, \ldots$ and all inequalities $c_i \neq c_j$ for $i \neq j$ to T. Every finite subset has a model: choose a sufficiently large finite model of the mentioned part of T and interpret the finitely many constants distinctly. Compactness gives a model of the whole expanded theory, whose reduct is an infinite model of T. No map from any particular finite model was constructed, so no embedding claim follows.

ex:fol-cardinality-normalforms-04. *Solution.* Suppose an empty-signature sentence σ held exactly in finite nonempty structures. Add, for each $n \geq 1$, a sentence saying that at least n distinct elements exist. Every finite subset together with σ has a sufficiently large finite model. Compactness therefore gives an infinite model of σ, contradicting the assumed characterization.

ex:fol-cardinality-normalforms-05. *Solution.* Successively applying quantifier negation, implication elimination, De Morgan's law, and vacuous-variable movement gives

$$\begin{aligned} &\neg \forall x (P(x) \to \exists y (Q(y) \land R(x, y))) \\ &\equiv \exists x (P(x) \land \neg \exists y (Q(y) \land R(x, y))) \\ &\equiv \exists x \forall y \big(P(x) \land (\neg Q(y) \lor \neg R(x, y))\big). \end{aligned}$$

The variables are distinct, and moving $\forall y$ across the conjunction is licensed because $y \notin \mathrm{FV}(P(x))$. The final formula is rectified and in prenex normal form.

ex:fol-cardinality-normalforms-06. *Solution.* Replace u by a new constant c; replace y by a new unary function $f(x)$; and replace w by a new binary function $g(x, z)$. The universal closure is

$$\forall x \forall z\, R(c, x, f(x), z, g(x, z)).$$

The arities $0, 1, 2$ record exactly the preceding universal variables on which the corresponding existential witness may depend.

ex:fol-cardinality-normalforms-07. *Solution.* Let the original sentence be $\forall x \exists y R(x, y)$, and take the two-element structure with $R = \{(0, 0), (1, 1)\}$. Its reduct satisfies the sentence. Expanding by the identity function makes $\forall x R(x, f(x))$ true, while expanding by the constant-zero function makes it false at 1. Thus Skolemization is a claim about existence of a suitable expansion and its reduct, not logical equivalence of formulas from different signatures.

ex:fol-cardinality-normalforms-08. *Solution.* The first pair produces $y = f(x)$ and $x = g(y)$, hence $x = g(f(x))$; the occurs check proves there is no unifier. The second pair has most general unifier $\{x \mapsto y\}$ (equivalently, with variables reversed, $\{y \mapsto x\}$). The third produces $x = y$ and $x = f(y)$, hence $y = f(y)$; the occurs check again proves there is no finite-term unifier.

ex:fol-cardinality-normalforms-09. *Solution.* After Skolemizing the existential premise with new constant c, the clauses are $\{\neg P(x), Q(x)\}$, $\{P(c)\}$, and $\{\neg Q(z)\}$, with parent variables renamed apart. Resolve the first two using $\{x \mapsto c\}$ to obtain $\{Q(c)\}$. Resolve that clause with the third using $\{z \mapsto c\}$ to obtain the empty clause.

ex:fol-cardinality-normalforms-10. *Solution.* A time bound truncates the search, so the result reports only that no refutation was found among the inferences explored within that resource limit. Refutational completeness guarantees eventual discovery for an unsatisfiable set only under an unbounded fair search. It does not convert finite unsuccessful search into a model or a satisfiability certificate.

E.4 Model theory

ex:maps-elementarity-01; alias ex:diagram-test. *Solution.* Interpret each new constant c_a in $\mathcal{N}$ by the same element $a \in M \subseteq N$. Substructure preservation of constants and functions, and agreement of relations on tuples from M, make every true or negated atomic $\mathcal{L}(M)$-sentence from $\mathcal{M}$ true in the expansion of $\mathcal{N}$. The expansion satisfies the elementary diagram iff, for every $\mathcal{L}$-formula $\varphi(\bar{x})$ and $\bar{a} \in M$, $\mathcal{M} \models \varphi(\bar{a})$ iff $\mathcal{N} \models \varphi(\bar{a})$, which is precisely $\mathcal{M} \preccurlyeq \mathcal{N}$.

ex:maps-elementarity-02. *Solution.* Every formula in the language expanded by constants for A becomes a formula of the original language with the corresponding members of A as parameters. Since $\mathcal{M} \preccurlyeq \mathcal{N}$, the two structures agree on that parameter instance. Translating back gives elementarity of the two constant expansions.

ex:maps-elementarity-03.
Solution. Work in the equality-only signature. Let $M_n = \{0, 1, \ldots, n\}$, with inclusions as substructures. Their union is $\mathbb{N}$, but M_0 is not elementary in the union: with parameter 0, the formula $\exists x\,(x \neq 0)$ is false in M_0 and true in $\mathbb{N}$.

ex:types-ultraproducts-01; alias ex:type-space-basis. *Solution.* A complete type contains both φ and ψ iff it contains $\varphi \wedge \psi$, giving the intersection identity; it does not contain φ iff completeness puts $\neg\varphi$ in it, giving the complement identity. For the equality-only theory of a two-element structure, $S_1(T)$ has one point, isolated by $[x = x]$. The space $S_2(T)$ has the equality type and inequality type, whose singleton basic opens are $[x_1 = x_2]$ and $[x_1 \neq x_2]$, respectively.

ex:types-ultraproducts-02; alias ex:homogeneity-back-step. *Solution.* The inverse $h^{-1} : h[A] \to M$ is elementary. Apply κ-homogeneity to extend it to an elementary map $g : h[A] \cup \{d\} \to M$. Elementary maps are injective, so g^{-1} is elementary on its range; it extends h, because $g(h(a)) = a$, and its range contains d, because $g^{-1}(g(d)) = d$. The relevant domains have size below κ.

ex:types-ultraproducts-03; alias ex:omit-constant-type. *Solution.* Suppose that $\theta(x)$ were principal for $p(x) = \{x \neq c_n : n \in \mathbb{N}\}$. Choose a model $\mathcal{M} \models T$ and $a \in M$ with $\mathcal{M} \models \theta(a)$. If $a = c_k^{\mathcal{M}}$ for some k, then this same witness shows that $T \not\models \forall x(\theta(x) \to x \neq c_k)$, contrary to principality. Otherwise $a \neq c_n^{\mathcal{M}}$ for every n. The formula θ mentions only finitely many constants, so choose c_m that it does not mention and form $\mathcal{M}'$ by changing only the interpretation of c_m to a. The interpretations of all constants remain pairwise distinct, because a differed from every old constant interpretation; hence $\mathcal{M}' \models T$. The truth of $\theta(a)$ is unchanged, but $\mathcal{M}' \not\models a \neq c_m$, again contradicting principality. Thus p is nonprincipal. In the displayed model every element equals some c_n, so no element realizes p.

ex:types-ultraproducts-04. *Solution.* Define $F([a]_{\mathcal{U}}) = a(i_0)$. Equality modulo an ultrafilter principal at i_0 means exactly equality at i_0, so F is well-defined and injective. Every $b \in M_{i_0}$ is the image of a choice function with value b at i_0, so F is surjective. Functions and relations are interpreted coordinatewise, hence F is an isomorphism.

ex:types-ultraproducts-05. *Solution.* In the ultrapower over $\mathbb{N}$ by a nonprincipal ultrafilter, let $a = [i \mapsto i]_{\mathcal{U}}$. For each standard n, the set $\{i : n < i\}$ is cofinite and therefore belongs to $\mathcal{U}$. By Łoś's theorem, $d(n) < a$. Thus a is greater than every diagonal image while remaining one element of the ultrapower.

ex:qe-categoricity-01. *Solution sketch.* Enumerate the domains of a countable dense order without endpoints M and of $\mathbb{Q}$. Starting from the empty partial isomorphism, alternately map the next unused element of M into the unique open interval determined by its already mapped neighbors, and map the next unused rational back into the corresponding interval of M. Density and the absence of endpoints supply a fresh point in every step. The union is total, surjective, and order preserving and reflecting. Check all four interval cases: two bounds, lower bound only, upper bound only, and no bounds.

ex:qe-categoricity-02. *Solution.* Given $\varphi(\bar{x})$, choose quantifier-free $\psi(\bar{x})$ with $T \models \forall \bar{x}(\varphi \leftrightarrow \psi)$. Since both structures model T, each agrees with ψ on φ. A substructure and its superstructure agree on quantifier-free formulas with parameters from the substructure, so they agree on φ. If either structure fails to model T, the relative equivalence to ψ is not available there, and the argument stops.

ex:qe-categoricity-03; alias ex:back-and-forth-equivalence. *Solution sketch.* Enumerate two countable models and maintain a finite partial bijection preserving the equivalence relation. To add an element equivalent to an old domain element, choose a fresh member of the corresponding infinite target class. To add an element in a new class, choose a fresh target class, possible because infinitely many classes remain. Apply the symmetric construction on alternate steps. The union is a total isomorphism; checking the two extension cases proves preservation and reflection of equivalence.

E.5 Computability and arithmetical hierarchy

ex:turing-machines-01. *Solution.* Complement a decider by exchanging its two halting answers. For finite union or intersection, run the finitely many deciders and combine their answers by disjunction or conjunction. For computably enumerable sets, dovetail the recognizers: accept a union when one accepts, and accept an intersection after every recognizer has accepted its common input. No complement construction is used, because a recognizer's failure to accept need not halt.

ex:universal-reductions-01. *Solution.* Each property depends only on the computed partial function, not on its program text, and is nontrivial. The everywhere-undefined function is not total while the zero function is; the former's range omits 0 while the latter's contains 0. Under the exercise's stated nonempty-domain convention for a constant partial function, the zero function has the third property and the everywhere-undefined function does not. Rice's theorem therefore makes all three index sets undecidable.

ex:decision-hierarchy-01. *Solution sketch.* With the fixed bounded arithmetic computation formula $\mathsf{Comp}(e, x, s)$, nonempty domain is defined by $\exists x \exists s\, \mathsf{Comp}(e, x, s)$, a Σ_1^0 formula. Given e, effectively produce a program that is undefined except that on input 0 it simulates the normalized wrapper for program e on input e, accepts with a valid numerical output iff $\varphi_e(e)$ is defined, and diverges otherwise. Its domain is nonempty iff $e \in K$. The uniform program-index map is the required many-one reduction; writing it via the parameter theorem completes the formal check. Since K is $\mathbf{\Sigma}_1^0$-complete under many-one reductions, this proves the same completeness classification for the nonempty-domain index set.

ex:decision-hierarchy-02. *Solution.* Using the same bounded arithmetic formula Comp, finite domain has the immediate form $\exists b \forall x > b \forall s\, \neg\mathsf{Comp}(e, x, s)$, hence belongs to $\mathbf{\Sigma}_2^0$. Infinite domain has form $\forall b \exists x > b \exists s\, \mathsf{Comp}(e, x, s)$, hence belongs to $\mathbf{\Pi}_2^0$. Cofinite domain has form $\exists b \forall x > b \exists s\, \mathsf{Comp}(e, x, s)$, hence belongs to $\mathbf{\Sigma}_3^0$. These are upper bounds only; no completeness claim follows without reductions.

ex:decision-hierarchy-03. *Solution.* Enumerate all finite derivations in a fixed effective, sound, and complete first-order calculus and output their conclusions. Every output is valid by soundness. Conversely, completeness is used exactly to infer that every semantically valid sentence has some finite derivation and therefore eventually appears. The enumeration never ranges directly over structures.

E.6 Arithmetic and incompleteness

ex:arithmetic-coding-01; alias ex:nonstandard-true-arithmetic. *Solution.* Taking $T = \mathrm{Th}(\mathcal{N})$, the compactness construction produces a countable $\mathcal{M} \models T$ with an element unequal to

every numeral. Since T contains exactly the sentences true in $\mathcal{N}$, $\mathcal{M}$ and $\mathcal{N}$ agree on every arithmetic sentence. They are not isomorphic, because every element of $\mathcal{N}$ is denoted by a numeral and the displayed element of $\mathcal{M}$ is not. Thus elementary equivalence is strictly weaker than isomorphism.

ex:arithmetic-coding-02. *Solution sketch.* Fix primitive recursive coding of finite sequences with length and component functions. Let $A(c, a, b)$ say: c has length $b+1$, its zeroth component is a, and for every $i < b$ its $(i+1)$-st component is the successor of its i-th component. Bounded quantification over components keeps A primitive recursive. The last component is $a + b$; representability of A, together with existence and uniqueness of the coded recursion, yields a representing formula for addition. A completed account must verify the sequence-code operations and the unique last value.

ex:arithmetic-coding-03. *Solution.* The first assertion is a metalinguistic claim that a T-derivation of σ exists. The second is a semantic assertion in the standard arithmetic structure about the numeral coding σ. The third is again a metalinguistic derivability claim, now with an arithmetized provability sentence as conclusion. Correct representation of the proof predicate connects an actual proof with standard truth of the second assertion; if $T \supseteq \mathsf{Q}$, numeralwise representability gives the first derivability condition and connects an actual proof to the third. The reverse move from T-provability to standard truth requires the corresponding soundness hypothesis.

ex:first-incompleteness-01. *Solution sketch.* Effectiveness makes finite T-proof checking primitive recursive and Prov_T arithmetically expressible. Representability turns a concrete proof into a T-proof of its proof-code instance and verifies each standard nonproof code. Consistency rules out $T \vdash G_T$, since the fixed point would then yield both provability and its negation. Σ^0_1-soundness rules out $T \vdash \neg G_T$, because $\neg G_T$ is a false Σ^0_1 sentence once consistency shows G_T true in $\mathcal{N}$. These four uses should appear at those exact transitions.

ex:first-incompleteness-02. *Solution.* Consistency gives $T \nvdash G_T$. Hence no standard number codes a T-proof of G_T, so $\mathcal{N} \models \neg\mathsf{Prov}_T(\ulcorner G_T \urcorner)$. The fixed point is already provable in Q, which is true in $\mathcal{N}$, so $\mathcal{N} \models G_T$. This semantic conclusion alone does not show that $T \cup \{G_T\}$ is consistent: that additionally requires $T \nvdash \neg G_T$. Under, for example, the stated Σ^0_1-soundness hypothesis, that condition holds; then the one-sentence extension is effective and consistent, so an incompleteness theorem supplies another sentence it does not decide. Thus adjoining G_T does not produce all arithmetical truths under the strengthened hypothesis, while mere consistency licenses no such claim about the extension.

ex:first-incompleteness-03. *Solution sketch.* Once a proof with least standard code m is assumed, every comparison with a code below m is a bounded finite check whose true and false instances are numeralwise representable in Q; lemma 20.2 turns the bounded existential over codes at most m into a finite disjunction, and lemma 20.3 splits an arbitrary proof code against $\bar{m}$ inside Q. The Rosser sentence uses that bounded comparison to force an opposite proof with no larger code, contradicting consistency. Omega-consistency is not needed because the argument never infers a universal standard claim from all of its numeral instances; the least code converts the relevant search into a single bounded verification.

ex:second-incompleteness-01. *Solution sketch.* From the fixed point $G_T \leftrightarrow \neg\mathsf{Prov}_T(\ulcorner G_T \urcorner)$, internalize the implication $G_T \to \neg\mathsf{Prov}_T(\ulcorner G_T \urcorner)$ using derivability conditions 1 and 2. Condition 3 gives $\mathsf{Prov}_T(\ulcorner G_T \urcorner) \to \mathsf{Prov}_T(\ulcorner \mathsf{Prov}_T(\ulcorner G_T \urcorner) \urcorner)$; internalized modus ponens then yields

$\mathsf{Prov}_T(\ulcorner G_T \urcorner) \to \mathsf{Prov}_T(\ulcorner \bot_A \urcorner)$. Therefore $\mathsf{Con}(T)$ implies $\neg\mathsf{Prov}_T(\ulcorner G_T \urcorner)$, and the fixed point gives $\mathsf{Con}(T) \to G_T$. If T proved its consistency, it would prove G_T, contradicting consistency. A full derivation must display the instances of all three conditions just identified.

ex:second-incompleteness-02.
Solution. The assumed biconditional includes $T \vdash \mathsf{Prov}_T(\ulcorner \varphi \urcorner) \to \varphi$. This is exactly the hypothesis of Löb's theorem, so $T \vdash \varphi$. The reverse half of the biconditional is not needed.

ex:second-incompleteness-03. *Solution.* Suppose one formula $\mathsf{Tr}(x)$ had all the stated biconditionals provable in consistent $T \supseteq \mathsf{Q}$. By the diagonal lemma choose L with $T \vdash L \leftrightarrow \neg\mathsf{Tr}(\overline{\ulcorner L \urcorner})$. The assumed instance for L gives $T \vdash \mathsf{Tr}(\overline{\ulcorner L \urcorner}) \leftrightarrow L$. Propositional reasoning combines them to derive $L \leftrightarrow \neg L$, and hence a contradiction. Consistency is used only at the final step to rule out that derivation.

ex:second-incompleteness-04. *Solution.* Three correct replacements are: (i) if T is a consistent computably enumerable extension of Q, then its fixed Gödel sentence G_T is not provable in T, and it is true in $\mathcal{N}$; (ii) if in addition the fixed provability predicate satisfies the derivability conditions, then $T \nvdash \mathsf{Con}(T)$; and (iii) $\mathrm{Th}(\mathcal{N})$ contains every true arithmetic sentence by definition, is deductively complete, and is not computably enumerable. These claims concern different theories, predicates, and evidence.

E.7 Set theory and forcing

ex:zf-zfc-01. *Solution sketch.* Form $a \cup b$ using Pairing and Union. For $x \in a, y \in b$, the Kuratowski pair $\langle x, y\rangle = \{\{x\}, \{x, y\}\}$ belongs to $\mathcal{P}(\mathcal{P}(a \cup b))$. This double power set is the required bounding set. Apply Separation to it with the formula $\exists x \in a \exists y \in b\, z = \langle x, y\rangle$. Extensionality identifies the separated set with $a \times b$. A complete derivation should cite Pairing, Union, Power Set twice, Separation, and Extensionality at these exact points.

ex:zf-zfc-02. *Solution.* By Pairing form the nonempty set $c = \{a, b\}$. Foundation supplies $x \in c$ with $x \cap c = \varnothing$. If $x = a$, then $b \in a \cap c$; if $x = b$, then $a \in b \cap c$. Both cases contradict the supplied disjointness, so no two-cycle $a \in b \in a$ exists.

ex:cumulative-hierarchy-01. *Solution.* Fix β and argue directly from transitivity; no transfinite induction is required. If $\alpha = \beta$, inclusion is immediate. If $\alpha < \beta$, then $\alpha \in \beta$ by the ordinal order definition; since every ordinal β is transitive, every member of α is also a member of β, so $\alpha \subseteq \beta$. The use of transitivity is exactly the step from $u \in \alpha \in \beta$ to $u \in \beta$. The argument invokes no induction hypothesis, which is why no transfinite induction is required.

ex:cumulative-hierarchy-02. *Solution.* Put $X = \{\varnothing, \{\varnothing\}\}$. Since $\mathrm{rk}(\varnothing) = 0$ and $\mathrm{rk}(\{\varnothing\}) = 1$,

$$\mathrm{rk}(X) = \sup\{0 + 1, 1 + 1\} = 2.$$

Also $X \subseteq V_2$, so $X \in V_3$, while $\{\varnothing\} \notin V_1$, so $X \notin V_2$. This confirms the rank calculation from the hierarchy stages.

ex:cumulative-hierarchy-03. *Solution.* Let $X = \{a, b\}$ with $a \neq b$ and let $R = \varnothing$. The relation is well-founded but not extensional, because a and b have the same predecessor set. The collapse recursion sends both to $\{\} = \varnothing$, so the collapse map is not injective.

ex:constructibility-forcing-01. *Solution.* For a finite partial function p, if $n \in \mathrm{dom}(p)$ then $p \in D_n$; otherwise $p \cup \{(n, 0)\}$ extends p and belongs to D_n. Thus D_n is dense. If $(n, i) \in p \in G$ and $(n, j) \in q \in G$, directedness of the filter gives $r \in G$ extending both p, q. Since r is a function, $i = j$. Hence $\bigcup G$ is a function; meeting every D_n further makes its domain all of ω.

ex:constructibility-forcing-02. *Solution sketch.* $L \models \mathsf{CH}$ is an external semantic claim about the constructible universe, proved from the fine structure and definable well-order of L. The formula $\mathsf{ZFC} \vdash V = L \to \mathsf{CH}$ is a syntactic derivability claim obtained by formalizing that result in ZFC. The implication $\mathsf{Con}(\mathsf{ZFC}) \to \mathsf{Con}(\mathsf{ZFC} + \mathsf{CH})$ is a metatheoretic relative-consistency claim: from a model or syntactic consistency construction for ZFC, pass to its constructible inner model, which satisfies $V = L$ and hence CH. A completed proof must state the soundness/completeness or syntactic formalization used when moving between model existence and consistency.

E.8 Sequent calculus, normalization, and typed proof terms

ex:lk-lj-01. *Solution sketch.* Apply $\wedge L$ to the antecedent, making A and $B \vee C$ available; then apply $\vee L$ to $B \vee C$. In the B-branch, identities for A, B and $\wedge R$ yield $A \wedge B$, followed by the left $\vee R$ injection. In the C-branch, the analogous steps yield $A \wedge C$, followed by the right injection. The principal formulas, from bottom upward, are $A \wedge (B \vee C)$, $B \vee C$, the concluding disjunction, and the relevant conjunction. The finished tree contains no cut and has one succedent at every node.

ex:lk-lj-02. *Solution.* Without the eigenvariable condition, identity $P(y) \Rightarrow P(y)$ would permit $\forall R$ to infer $P(y) \Rightarrow \forall x P(x)$. Take domain $\{0, 1\}$, interpret P as $\{0\}$, and assign y to 0. The antecedent is true under that assignment and the succedent is false. Thus the relaxed rule derives an invalid sequent.

ex:cut-elimination-01. *Solution sketch.* Suppose the left cut premise obtains $A \wedge B$ by $\wedge R$ from derivations of A and B, while the right cut premise obtains its sequent by $\wedge L$ from a derivation with both A, B in the antecedent. First cut the derivation of B against the right premise to remove B, retaining A; then cut the derivation of A against that result. Context contractions or exchanges are applied as licensed by the active calculus. Both new cut formulas, A and B, are proper subformulas of $A \wedge B$, so each has strictly smaller cut rank.

ex:cut-elimination-02. *Solution.* In propositional LJ, a cut-free backwards search uses only finitely many subformulas of the endsequent; with the declared context representation, only finitely many relevant sequents can arise, so exhaustive search with repetition control decides derivability. In first-order LJ, quantifier rules introduce arbitrarily many term instances, and the set of possible terms is generally infinite. Literal subformula finiteness therefore does not by itself bound proof search.

ex:normalization-curry-howard-01. *Solution.* The term has type $A \to (B \to A)$. In the typing calculus, the hypothesis $y : B$ enters the context and is never consumed by the VAR rule; that unused context entry is the structural weakening implicit in $\lambda x{:}A.\lambda y{:}B.x$. In the tree natural-deduction

calculus, a discharging application must discharge at least one open occurrence, so the corresponding derivation of $A \to (B \to A)$ routes the assumption B through a conjunction detour: assume A and B, form $B \wedge A$ by $\wedge I$, recover A by $\wedge E_2$, discharge B to obtain $B \to A$, and discharge A. Both discharged occurrences are genuinely used by $\wedge I$.

ex:normalization-curry-howard-02. *Solution sketch.* The required typing substitution lemma is: if $\Theta_{\text{ty}}, x{:}A \vdash M : B$ and $\Theta_{\text{ty}} \vdash N : A$, then $\Theta_{\text{ty}} \vdash M[N/x] : B$, after capture-avoiding renaming of bound variables. Typing inversion on $\Theta_{\text{ty}} \vdash (\lambda x{:}A.M)N : B$ gives its two premises; the lemma then types the reduct $M[N/x]$ by B. Capture avoidance is used in the lambda-abstraction case of the lemma when the binder occurs free in N. Checking that case and the variable case completes the outer-redex proof.

ex:normalization-curry-howard-03. *Solution.* If every reduction sequence from a term terminates, then in particular at least one terminating sequence exists, so strong normalization implies weak normalization. For the converse failure, take a relation with $a \to a$ and $a \to b$, where b has no successor. The object a has the terminating path $a \to b$, but also the infinite path $a \to a \to a \to \cdots$.

E.9 Second-order, typed, intuitionistic, and modal systems

ex:second-order-logic-01. *Solution.* Let $X \subseteq \mathbb{N}$ be arbitrary, not assumed definable, and suppose $0 \in X$ and $n \in X \Rightarrow n + 1 \in X$. External induction on n gives $n \in X$ for every $n \in \mathbb{N}$, hence $X = \mathbb{N}$. Since full second-order semantics ranges its unary-relation variables over every subset of $\mathbb{N}$, this verifies the full induction axiom in the standard structure.

ex:second-order-logic-02. *Solution.* The categoricity proof forms the range of the recursively defined map $f : \mathbb{N} \to M$ and instantiates second-order induction with that range to show it is all of M. In a Henkin structure, the unary-relation domain may omit $\text{ran}(f)$. The induction axiom then cannot be instantiated with that subset, so the surjectivity step fails.

ex:second-order-logic-03. *Solution sketch.* Any finitary strongly complete calculus would make full second-order consequence compact: from $\Gamma \models \varphi$, strong completeness gives a finite derivation, which uses only a finite $\Gamma_0 \subseteq \Gamma$, so $\Gamma_0 \models \varphi$ by soundness. The displayed full-semantics counterexample to compactness contradicts this. This argument excludes finitary sound strong completeness; the stronger claim that full second-order validities are not computably enumerable requires a separate effectivity theorem.

ex:simple-type-theory-01. *Solution.* Here $R : \iota \to \iota \to o$, $x : \iota$, $Rx : \iota \to o$, and $Rxx : o$. Therefore $\lambda x{:}\iota.Rxx : \iota \to o$, and the whole term has type $(\iota \to \iota \to o) \to (\iota \to o)$.

ex:simple-type-theory-02. *Solution.* Under standard semantics, $\forall f{:}(\alpha \to \beta).A$ is true under assignment s iff A is true under $s[f \mapsto g]$ for every function $g : D_\alpha \to D_\beta$. Under Henkin semantics the same clause quantifies only over the designated domain $D_{\alpha\to\beta}$, whose members need not be all such functions. The quantifier range is the only difference in the two clauses.

ex:simple-type-theory-03. *Solution.* A precise true statement is: the standard typed deductive calculus is sound and complete for its declared class of Henkin structures. A precise false statement is: some finitary effective sound calculus is strongly complete for standard full higher-order semantics. Thus the bare word "complete" omits both the calculus and the semantic class and can conceal a change from Henkin to full semantics.

ex:intuitionistic-logic-01. *Solution.* Let $w < v$, and let $\mathcal{M}$ make A true only at v and B true nowhere. At both w and v, $A \to B$ fails because $\mathcal{M}, v \models A$ and $\mathcal{M}, v \not\models B$. Hence $\mathcal{M}, w \models (A \to B) \to A$ vacuously, while $\mathcal{M}, w \not\models A$. Peirce's formula therefore fails at w.

ex:intuitionistic-logic-02. *Solution.* If $\mathcal{M}, w \models \neg(A \vee B)$, then no extension of w satisfies $A \vee B$; consequently no extension satisfies A, and none satisfies B. Thus $\mathcal{M}, w \models \neg A \wedge \neg B$. Conversely, if $\mathcal{M}, w \models \neg A \wedge \neg B$, an extension satisfying $A \vee B$ would satisfy one disjunct and contradict the corresponding negation. Hence the converse also holds intuitionistically.

ex:modal-temporal-epistemic-01. *Solution.* Let $W = \{w, u\}$ and $R = \{(w, w), (w, u), (u, u)\}$. This frame is reflexive and is not Euclidean, since wRw and wRu but not uRw. Let $\mathcal{M}$ be the structure on this frame that makes p true only at w. Then $\mathcal{M}, w \models \Diamond p$, but $\mathcal{M}, u \not\models \Diamond p$; because wRu, $\mathcal{M}, w \not\models \Box\Diamond p$.

ex:modal-temporal-epistemic-02. *Solution.* At positions $0, 1, 2$, choose position 3 as the witness for q; p holds at every earlier position, so $p \,\mathsf{U}\, q$ is true. It is also true at position 3, with the current position as witness and an empty intervening interval. Its value at position 4 is undetermined from the data: one must specify the truth of p, q there and on the infinite continuation, because a past occurrence of q cannot witness a future-until formula.

ex:modal-temporal-epistemic-03. *Solution.* Take agents a, b and worlds w, u, v. Let the a-equivalence classes be $\{w, u\}$, $\{v\}$, and the b-classes be $\{w\}$, $\{u, v\}$. Make p true at w, u and false at v. At w, both agents know p, but the two-step path $w \sim_a u \sim_b v$ reaches a world where p is false. Therefore $C_{\{a,b\}}p$ fails at w.

E.10 Many-valued, relevance, and linear logic

ex:many-valued-paraconsistent-01. *Solution sketch.* For $\mathbf{K}_3$, the two roots are $(p \wedge q){:}1, p{:}0$ and $(p \wedge q){:}1, p{:}\frac{1}{2}$. Expanding the conjunction labelled 1 forces $p{:}1, q{:}1$, so each root closes on conflicting labels for p. For LP, test designated premise labels $1, \frac{1}{2}$ against the sole nondesignated conclusion label 0. Every branch of the min-expansion of a designated conjunction gives p a designated value, so every branch again conflicts with $p{:}0$. Listing all $\min(a, b) = \frac{1}{2}$ input pairs completes the LP tree.

ex:many-valued-paraconsistent-02. *Solution.* Use $p \vee \neg p$. At values $0, \frac{1}{2}, 1$, it has values $1, \frac{1}{2}, 1$, respectively. All three are designated in LP, whereas $\frac{1}{2}$ is not designated in $\mathbf{K}_3$. Hence the formula is LP-valid but not K3-valid.

ex:relevance-linear-01. *Solution.* Write g for the tensor of Γ. The premise of $\multimap R$ is $g \otimes a \leq b$, which is equivalent by residuation to $g \leq a \multimap b$, its conclusion. For $\multimap L$, suppose $g \leq a$ and $d \otimes b \leq c$. Since $a \otimes (a \multimap b) \leq b$, monotonicity gives $d \otimes g \otimes (a \multimap b) \leq d \otimes b \leq c$, exactly the conclusion sequent's inequality.

ex:relevance-linear-02. *Solution.* Use identities $A \Rightarrow A$ and $B \Rightarrow B$ as the two premises of $\multimap L$. It yields $A, A \multimap B \Rightarrow B$, and exchange built into multiset contexts gives $A \multimap B, A \Rightarrow B$. Every antecedent occurrence is consumed by the rule, so no weakening is required.

ex:relevance-linear-03. *Solution.* Sequence contexts build in none of exchange, weakening, or contraction. Multiset contexts identify permutations and thus build in exchange, but retain multiplicity and do not build in weakening or contraction. Set contexts ignore order and multiplicity, so exchange and contraction are built in; weakening is still a substantive monotonicity principle because a set does not identify Γ with $\Gamma \cup \{A\}$.

ex:relevance-linear-04. *Solution.* The decisive clause says that $A \to B$ at w tests every pair u, v with $R(w, u, v)$, requiring truth of A at u to yield truth of B at the potentially different world v. In the displayed structure, $R(a, u, v)$, q holds at u, and p fails at v, so $q \to p$ fails at a, which then defeats the outer conditional at the normal world. An ordinary binary Kripke clause evaluates antecedent and consequent at one accessible world and therefore does not encode this ternary split between the two tested worlds.

E.11 Formal practice, automation, mechanization, and typesetting

ex:formal-practice-01. *Solution.* On a one-element domain $\{a\}$, the premise makes $P(a) \vee Q(a)$ true, so one disjunct is true; that disjunct then holds of every domain element, making one universal true. For a two-element countermodel, take domain $\{a, b\}$, let $P = \{a\}$ and $Q = \{b\}$. Every element satisfies $P \vee Q$, but neither predicate holds of every element, so the conclusion is false.

ex:formal-practice-02. *Solution sketch.* Induct structurally on the formula, after proving the term-substitution lemma by induction on terms. Strengthen the claim to all structures, assignments, variables, and replacing terms at once: $\mathcal{M}, s \models \varphi[t/x]$ iff $\mathcal{M}, s[x \mapsto [\![t]\!]^{\mathcal{M}}_s] \models \varphi$. Atomic and Boolean cases use the corresponding induction hypotheses. In $\forall y\psi$ and $\exists y\psi$, if $y = x$ substitution stops below the binder; if $y \neq x$ but $y \in \mathrm{Var}(t)$, first rename y fresh. A completed proof must show the two assignment updates commute after that renaming.

ex:automated-reasoning-01. *Solution.* The unit clause $\{\neg R\}$ sets $R = \mathbf{F}$. Then $\{\neg P, R\}$ and $\{\neg Q, R\}$ reduce to units $\{\neg P\}$ and $\{\neg Q\}$, setting $P = Q = \mathbf{F}$. The first clause $\{P, Q\}$ then becomes empty. DPLL returns UNSAT.

ex:automated-reasoning-02. *Solution.* Decomposing the two atoms gives $x \doteq g(y)$ and $x \doteq y$. Substituting the second equation into the first gives $y \doteq g(y)$. Since y occurs properly in $g(y)$, the occurs check fails, and no finite-term unifier exists.

ex:automated-reasoning-03. *Solution.* Before Skolemization, equivalence-preserving steps give

$$\neg\forall x(P(x) \to \exists y R(x, y)) \equiv \exists x(P(x) \wedge \forall y \neg R(x, y)).$$

Replace the existential witness by a new constant c. This step preserves only equisatisfiability between the original signature and suitable expansions. The remaining universal formula is equivalent in the expanded signature to the universal closures of the clauses $\{P(c)\}$ and $\{\neg R(c, y)\}$. Thus only the Skolem step weakens equivalence to equisatisfiability.

ex:mechanization-01. *Solution.* Suitable terms are

$$\lambda x{:}A.x, \qquad \lambda p{:}(A \times B).\pi_1 p, \qquad \lambda f{:}(A \to B).\lambda g{:}(B \to C).\lambda x{:}A.g(fx).$$

They have types $A \to A$, $(A \times B) \to A$, and $(A \to B) \to (B \to C) \to (A \to C)$, respectively, by variable, product-elimination, and function-elimination typing rules.

ex:mechanization-02. *Solution sketch.* In a deep embedding, let Form_d be an inductive datatype with constructor $\mathsf{And} : \mathsf{Form}_d \to \mathsf{Form}_d \to \mathsf{Form}_d$, and define $\mathsf{eval}_d(A, v)$ by recursion. The required adequacy theorem is

$$\mathsf{eval}_d(A, v) = \mathsf{true} \quad \Longleftrightarrow \quad v \models_{\mathrm{CPL}} A$$

for every object-language formula A and valuation v; its conjunction case uses $\mathsf{eval}_d(\mathsf{And}(A, B), v) = \mathsf{eval}_d(A, v) \wedge_{\mathrm{Bool}} \mathsf{eval}_d(B, v)$.

For a shallow embedding, fix for each valuation v a translation $[\![-]\!]^s_v$ from object-language formulas to host propositions and stipulate

$$[\![A \wedge B]\!]^s_v := [\![A]\!]^s_v \wedge_{\mathrm{host}} [\![B]\!]^s_v.$$

Its required adequacy theorem is

$$[\![A]\!]^s_v \quad \Longleftrightarrow \quad v \models_{\mathrm{CPL}} A$$

for every A and v. The deep representation retains syntax for induction, substitution, and quotation; the shallow representation supplies those operations only if additional encoded syntax is introduced.

ex:latex-style-01. *Solution.* Replace "Let f be the inclusion. Clearly atomic formulas are preserved. Clearly the result follows by induction" with: "Let $f : M \hookrightarrow N$ be the inclusion map of the $\mathcal{L}$-substructure $\mathcal{M} \subseteq \mathcal{N}$. For every function symbol g, substructure closure gives $f(g^{\mathcal{M}}(\bar{a})) = g^{\mathcal{N}}(f\bar{a})$; for every relation symbol R, the substructure definition gives $R^{\mathcal{M}}(\bar{a})$ iff $R^{\mathcal{N}}(f\bar{a})$. These are the atomic induction cases. The Boolean constructor cases follow from their truth clauses and the induction hypotheses." The typed map and the two omitted justifications are now explicit.

ex:latex-style-02. *Solution.* For a first-order resolution chapter, register x, y, z as object variables, r, t as terms, A, B as atoms, C, D as finite literal clauses, and $\theta_{\mathrm{u}}, \rho_{\mathrm{u}}, \eta_{\mathrm{u}}$ as finite-support term substitutions. Register $\mathrm{ucl}_{\mathcal{L}}(C)$ as the formula interpretation of a clause. Do not also use C for a calculus or θ_{u} for a formula in that chapter. If $\models$ occurs, its argument shapes must separately declare structure satisfaction and clause set consequence, as in the registered-notation appendix.

E.12 Keys for the qualifying problems

The five labelled qualifying problems retain their existing solutions. This list makes their keys part of the same deterministic lookup system.

prob:substitution. *Solution.* The solution immediately following the problem identifies the term-substitution lemma, sets up the structural induction, and displays the quantified assignment-commutation calculation; the term-lemma induction and the atomic and connective cases are left to the reader.

prob:compactness. *Solution.* The complete solution immediately following the problem builds models for finite subsets, applies compactness, and separates the existential model claim from the rejected assertion about all orders.

prob:incompleteness. *Solution.* The solution immediately following the problem states the computably enumerable axiom-set, arithmetic-strength, and consistency hypotheses and distinguishes the original Gödel conclusion from Rosser's refinement.

prob:modal-frame. *Solution.* The complete solution immediately following the problem proves the reflexive direction by the satisfaction clause and constructs a valuation at a nonreflexive world for the converse.

prob:cut-subformula. *Solution.* The solution immediately following the problem traces the subformula invariant through cut-free propositional rules, derives consistency as a consequence, and states the required first-order qualification modulo term substitution and eigenvariable renaming.

Appendix F

Editorial and Formal-Logical Release Checklist

F.1 Definitions and notation

- Every technical term is defined before use.
- Every new technical name has necessity, an exact eliminable definition or explicit axiom, standing, nonredundancy, and later use at its first introduction.
- Every symbol has one registered role and type within each declared scope.
- The signature, grammar, binding, and precedence rules are explicit.
- Every recursive definition has a well-founded basis.
- Every quotient definition is proved independent of representatives.

F.2 Theorems and proofs

- Every theorem states all quantified objects and hypotheses.
- Every derivability statement identifies a calculus.
- Every semantic statement identifies a structure class.
- Every completeness, consistency, compactness, categoricity, and decidability claim identifies the object having that property.
- Every induction identifies the induction object and all cases.
- Every quantifier-rule side condition is checked.
- Every substitution is capture-avoiding.
- Every proof sketch is labelled and is not cited later as a complete proof.
- Every relative-consistency theorem remains explicitly relative.

F.3 Sources and computation

- Historical claims use primary sources when available.
- Current tool behavior uses exact-version official manuals.
- Automated transformations state equivalence or equisatisfiability.
- Solver results include input, status, resource limit, and certificate or trust assumption.
- Mechanized proofs report formal statement, correspondence, imports, axioms, tool version, and kernel result.

F.4 Production

- The source compiles without undefined references or missing citations.
- Duplicate labels and multiply defined macros are absent.
- The index and bibliography are generated.
- No display is clipped or overlaps surrounding text.
- Headers, footers, page numbers, tables, and rule sheets are inspected.
- Formal-logical proof review, static checks, compilation, visual inspection, review by an automated software agent (reported with the agent identity and configuration), and human review are reported separately.

Bibliography

[1] American Mathematical Society. *AMS Author Handbook: Journal Classes*. American Mathematical Society, current online edition. https://www.ams.org/arc/journals/packages/proc/proc_amslatex/Author_Handbook_Journals.pdf.

[2] American Mathematical Society and the LaTeX Project. *User's Guide for the amsmath Package*. https://ctan.org/pkg/latex-amsmath.

[3] Jeremy Avigad. The mechanization of mathematics. *Notices of the American Mathematical Society*, 65(6):681–690, 2018. https://www.ams.org/journals/notices/201806/rnoti-p681.pdf.

[4] Matthias Baaz, Uwe Egly, and Alexander Leitsch. Normal form transformations. In J. Alan Robinson and Andrei Voronkov, editors, *Handbook of Automated Reasoning*, volume I, pages 273–333. Elsevier, 2001. doi:10.1016/B978-044450813-3/50007-2.

[5] Leo Bachmair and Harald Ganzinger. Resolution theorem proving. In J. Alan Robinson and Andrei Voronkov, editors, *Handbook of Automated Reasoning*, volume I, pages 19–99. Elsevier, 2001. doi:10.1016/B978-044450813-3/50004-7.

[6] Jon Barwise, editor. *Handbook of Mathematical Logic*. North-Holland, 1977.

[7] Jon Barwise and Solomon Feferman, editors. *Model-Theoretic Logics*. North-Holland, 1985; Cambridge University Press digital edition, 2017. doi:10.1017/9781316717158.

[8] Patrick Blackburn, Maarten de Rijke, and Yde Venema. *Modal Logic*. Cambridge University Press, 2001. doi:10.1017/CBO9781107050884.

[9] George S. Boolos, John P. Burgess, and Richard C. Jeffrey. *Computability and Logic*. 5th edition. Cambridge University Press, 2007. doi:10.1017/CBO9780511804076.

[10] Samuel R. Buss, editor. *Handbook of Proof Theory*. Elsevier, 1998.

[11] C. C. Chang and H. J. Keisler. *Model Theory*. 3rd edition. North-Holland, 1990.

[12] Ian Chiswell and Wilfrid Hodges. *Mathematical Logic*. Oxford University Press, 2007. doi:10.1093/oso/9780198571001.001.0001.

[13] Alonzo Church. An unsolvable problem of elementary number theory. *American Journal of Mathematics*, 58(2):345–363, 1936. doi:10.2307/2371045.

[14] Stephen A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, pages 151–158, 1971. doi:10.1145/800157.805047.

[15] Martin Davis, George Logemann, and Donald W. Loveland. A machine program for theorem-proving. *Communications of the ACM*, 5(7):394–397, 1962. doi:10.1145/368273.368557.

[16] Martin Davis and Hilary Putnam. A computing procedure for quantification theory. *Journal of the ACM*, 7(3):201–215, 1960. doi:10.1145/321033.321034.

[17] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, pages 625–635. Springer, 2021. doi:10.1007/978-3-030-79876-5_37.

[18] H.-D. Ebbinghaus, J. Flum, and W. Thomas. *Mathematical Logic*. 2nd edition. Springer, 1994. doi:10.1007/978-1-4757-2355-7.

[19] Herbert B. Enderton. *A Mathematical Introduction to Logic*. 2nd edition. Academic Press, 2001.

[20] Dov Gabbay, Amir Pnueli, Saharon Shelah, and Jonathan Stavi. On the temporal analysis of fairness. In *Proceedings of the Seventh ACM Symposium on Principles of Programming Languages*, pages 163–173, 1980. doi:10.1145/567446.567462.

[21] Gerhard Gentzen. Untersuchungen über das logische Schließen I. *Mathematische Zeitschrift*, 39:176–210, 1934. doi:10.1007/BF01201353.

[22] Gerhard Gentzen. Untersuchungen über das logische Schließen II. *Mathematische Zeitschrift*, 39:405–431, 1935. doi:10.1007/BF01201363.

[23] Jean-Yves Girard. Linear logic. *Theoretical Computer Science*, 50:1–101, 1987. doi:10.1016/0304-3975(87)90045-4.

[24] Kurt Gödel. Die Vollständigkeit der Axiome des logischen Funktionenkalküls. *Monatshefte für Mathematik und Physik*, 37:349–360, 1930. doi:10.1007/BF01696781.

[25] Kurt Gödel. Über formal unentscheidbare Sätze der *Principia Mathematica* und verwandter Systeme I. *Monatshefte für Mathematik und Physik*, 38:173–198, 1931. doi:10.1007/BF01700692.

[26] Paul R. Halmos. How to write mathematics. *L'Enseignement Mathématique*, 16(1):123–152, 1970. doi:10.5169/seals-43857.

[27] Leon Henkin. The completeness of the first-order functional calculus. *The Journal of Symbolic Logic*, 14(3):159–166, 1949. doi:10.2307/2267044.

[28] Leon Henkin. Completeness in the theory of types. *The Journal of Symbolic Logic*, 15(2):81–91, 1950. doi:10.2307/2266967.

[29] David Hilbert and Paul Bernays. *Grundlagen der Mathematik*, volume 2. Julius Springer, Berlin, first edition, 1939. WorldCat catalog record.

[30] Wilfrid Hodges. *Model Theory*. Cambridge University Press, 1993. doi:10.1017/CBO9780511551574.

[31] The Isabelle Development Team. *Isabelle2025-2 Documentation*. `https://isabelle.in.tum.de/documentation.html`.

[32] International Organization for Standardization. *ISO 80000-2:2019, Quantities and Units – Part 2: Mathematics*. 2nd edition, corrected 2021. `https://www.iso.org/standard/64973.html`.

[33] Thomas Jech. *Set Theory: The Third Millennium Edition, Revised and Expanded*. 3rd edition. Springer, 2003. doi:10.1007/3-540-44761-X.

[34] Stephen Cole Kleene. *Introduction to Metamathematics*. North-Holland, 1952.

[35] Donald E. Knuth, Tracy L. Larrabee, and Paul M. Roberts. *Mathematical Writing*. Mathematical Association of America, 1989. https://cs-faculty.stanford.edu/~knuth/klr.html.

[36] Saul A. Kripke. Semantical analysis of modal logic I: Normal modal propositional calculi. *Zeitschrift für mathematische Logik und Grundlagen der Mathematik*, 9(5–6):67–96, 1963. doi:10.1002/malq.19630090502.

[37] Saul A. Kripke. Semantical analysis of intuitionistic logic I. In J. N. Crossley and M. A. E. Dummett, editors, *Formal Systems and Recursive Functions*, pages 92–130. North-Holland, 1965. doi:10.1016/S0049-237X(08)71685-9.

[38] Lean Project. *The Lean Language Reference*, version 4.32.1. Retrieved 9 August 2026. https://lean-lang.org/doc/reference/4.32.1/.

[39] Jeremy Avigad, Leonardo de Moura, Soonho Kong, and Sebastian Ullrich. *Theorem Proving in Lean 4*. Official online tutorial, retrieved 9 August 2026. https://docs.lean-lang.org/theorem_proving_in_lean4/.

[40] Per Lindström. On extensions of elementary logic. *Theoria*, 35(1):1–11, 1969. doi:10.1111/j.1755-2567.1969.tb00356.x.

[41] M. H. Löb. Solution of a problem of Leon Henkin. *The Journal of Symbolic Logic*, 20(2):115–118, 1955. doi:10.2307/2266895.

[42] Jerzy Łoś. On the extending of models I. *Fundamenta Mathematicae*, 42:38–54, 1955. doi:10.4064/fm-42-1-38-54.

[43] María Manzano. *Extensions of First Order Logic*. Cambridge University Press, 1996.

[44] David Marker. *Model Theory: An Introduction*. Springer, 2002. doi:10.1007/b98860.

[45] David Marker. *An Invitation to Mathematical Logic*. Springer, 2024. doi:10.1007/978-3-031-55368-4.

[46] Sara Negri and Jan von Plato. *Structural Proof Theory*. Cambridge University Press, 2001. doi:10.1017/CBO9780511527340.

[47] The Open Logic Project. *The Open Logic Text*. Release 9f12419, 25 May 2026. https://builds.openlogicproject.org/open-logic-complete.pdf.

[48] Dag Prawitz. *Natural Deduction: A Proof-Theoretical Study*. Almqvist & Wiksell, 1965; Dover reprint, 2006.

[49] J. A. Robinson. A machine-oriented logic based on the resolution principle. *Journal of the ACM*, 12(1):23–41, 1965. doi:10.1145/321250.321253.

[50] The Rocq Development Team. *The Rocq Prover Reference Manual*, version 9.1.1. https://rocq-prover.org/doc/V9.1.1/refman/index.html.

[51] J. Alan Robinson and Andrei Voronkov, editors. *Handbook of Automated Reasoning*. Elsevier and MIT Press, 2001.

[52] J. Barkley Rosser. Extensions of some theorems of Gödel and Church. *The Journal of Symbolic Logic*, 1(3):87–91, 1936. doi:10.2307/2269028.

[53] Richard Routley and Robert K. Meyer. The semantics of entailment—II. *Journal of Philosophical Logic*, 1(1):53–73, 1972. doi:10.1007/BF00649991.

[54] Stewart Shapiro. *Foundations without Foundationalism: A Case for Second-Order Logic*. Oxford University Press, 1991. doi:10.1093/0198250290.001.0001.

[55] Joseph R. Shoenfield. *Mathematical Logic*. Addison-Wesley, 1967.

[56] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. *The SMT-LIB Standard*, version 2.7, release 7 July 2025. https://smt-lib.org/papers/smt-lib-reference-v2.7-r2025-07-07.pdf.

[57] Alfred Tarski. *Logic, Semantics, Metamathematics: Papers from 1923 to 1938*. 2nd edition, edited by John Corcoran, translated by J. H. Woodger. Hackett, 1983.

[58] Geoff Sutcliffe and contributors. *The TPTP Language: Hyperlinked BNF*, BNF revision 9.3.0.3. Retrieved 9 August 2026. SHA-256 of the retrieved UTF-8 page content: 6bcbfc86f2e606e3b0db8c5b1de294093b2753c868296b5078ad028b9f863243. https://tptp.org/UserDocs/TPTPLanguage/SyntaxBNF.html.

[59] Boris A. Trakhtenbrot. The impossibility of an algorithm for the decision problem on finite classes. *Doklady Akademii Nauk SSSR*, 70:569–572, 1950.

[60] A. S. Troelstra and H. Schwichtenberg. *Basic Proof Theory*. 2nd edition. Cambridge University Press, 2000. doi:10.1017/CBO9781139168717.

[61] A. S. Troelstra and D. van Dalen. *Constructivism in Mathematics: An Introduction*. North-Holland, 1988.

[62] G. S. Tseitin. On the complexity of derivation in propositional calculus. In Jörg H. Siekmann and Graham Wrightson, editors, *Automation of Reasoning 2: Classical Papers on Computational Logic 1967–1970*, pages 466–483. Springer, 1983. Originally published in 1968. doi:10.1007/978-3-642-81955-1_28.

[63] A. M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, series 2, 42:230–265, 1937. doi:10.1112/plms/s2-42.1.230.

[64] The Unicode Consortium. *The Unicode Standard*, version 17.0, Chapter 22: Symbols. https://www.unicode.org/versions/Unicode17.0.0/core-spec/chapter-22/.

Index

Σ^0_1-soundness, 122
LJ-derivation, 150
LK-derivation, 149
Łoś's theorem, 88

A functionally complete basis, 29
Abstract reduction relation and normalization, 159
Adequacy of the numbering: enumeration theorem, 101
Admissible rule, 151
Aleph hierarchy, cardinal exponentiation, CH, and GCH, 137
Algebraically closed fields, 91
Alpha-equivalence, 44
Argument, 11
Arithmetical truth definition, 127
Arithmetization of syntax, 119
Atomic and elementary diagrams, 81
Axiom of Choice, 5, 132
Axiomatization, 50

Back-and-forth test, 92
Beth definability, 155
Binary relation, 3
Bounded verification of computations, 110

Calculus, 8
Cantor, 4
Cantor–Schröder–Bernstein, 4
Capture-avoiding substitution in a formula, 44
Cardinal comparison, 4
Cartesian product, 3
Categoricity, 91
Categoricity of full second-order arithmetic, 162
Categoricity of the full second-order reals, 163
Class, 2
Classical sequent, 148
Commutative residuated partially ordered monoid, 185
Compactness of type spaces, 83
Compactness produces nonstandard models, 117
Completeness, 10
Completeness of the first-order sequent calculus, 63
Completeness of the sequent calculus, 36
Complexity and subformula, 24
Composition and instance preorder, 72
Computably enumerable set, 98
Computably enumerable theory, 119
Configuration and computation, 95
Conjunctive and disjunctive normal forms, 27
Conservativity of definitional extensions, 51
Consistency, 34, 122
Constants for parameters, 81
Constructible hierarchy, 139
Correctness of the displayed DPLL procedure, 192
Countable categoricity of dense linear order, 93
Countable set, 4
Countermodel, 11
Craig interpolation, 154
Cumulative hierarchy, 135
Cut elimination for LK and LJ, 153

Decidable and computably enumerable relations, 98
Decidable set, 98
Deductive completeness, semantic completeness, and model completeness, 90
Definability classes of relations, 107
Definable subsets, 139
Defined connectives, 23
Dense linear orders, 90
Derivability, 10
Derived rule, 151
Detour, 156
Diagonal lemma, 121
Diagram tests for embeddings, 81
Disjunction property, 174
Downward Löwenheim–Skolem, 66

Elaboration, 196
Elementary chain, 82
Elementary chain theorem, 82
Elementary map, 80
Elementary substructure, 65, 80
Embedding, 78
Empty set, 130
Equipotence, 136
Equisatisfiability, 192
Equisatisfiability of Skolemization, 70
Equivalence class and quotient, 4
Equivalence closure, 15
Equivalence relation, 4
Equivalent forms of computable enumerability, 99
Equivalent forms of the Axiom of Choice, 137
Existence of saturated elementary extensions, 85
Extensional index property, 104
Extensionality, 130

Failure of compactness under full semantics, 163
Filter, 86
Finite coding over an infinite cardinal, 6
Finite partial isomorphism, 92
Finite sequence, 12
Finite set, 4
Finite-matrix tableau soundness and completeness, 181
Finite-support simultaneous term substitution, 71
First-order clause and formula interpretation, 71
First-order compactness, 64
First-order consistency, 58
First-order derivability, 53
First-order formula, 42
First-order formula complexity, 43
First-order literal and negation normal form, 66
First-order signature, 41
First-order structure, 46
First-order unification, 73
First-order unification procedure, 73
Formal consistency sentence, 125
Formal language, 8
Formula rank and derivation height, 152
Foundation, 131
Frame correspondence and completeness, 176
Free variables, 42
Full comprehension validity, 162
Full second-order assignment, 161
Full semantic consequence, 162
Function, 3
Functional completeness, 29

Gödel numbering, 119
Gödel's first incompleteness theorem, original form, 122
Gödel's second incompleteness theorem, 126
Gödel–Gentzen negative translation, 174
Glivenko, 174
Ground term and ground instance, 74

Hartogs, 136
Henkin semantic consequence, 164
Henkin structure, 164
Henkin type structure, 168
Herbrand theorem for clauses, 74
Hierarchy classifications of program domains and totality, 111
Hierarchy theorem, 135
Hilbert–Bernays–Löb derivability conditions, 125
History coding, 110
Homogeneity, 85
Homomorphism, 78

Image and inverse image, 3
Inductively generated set, 13
Infinity, 131
Injection, surjection, and bijection, 3
Intuitionistic Kripke frame and structure, 172
Intuitionistic semantic consequence, 173
Intuitionistic sequent, 148
Isomorphism and automorphism, 78

Jump theorem, 113

Kernel, 196
Kleene recursion theorem, 103
Kripke frame and Kripke structure, 175

Löb, 126
Language and standard structure of arithmetic, 106
Lifting lemma, 74
Lindström, 164
Local and global modal consequence, 175
Logical matrix, 180
Logical system, 8

Machine decision and recognition, 98
Many-one hardness and completeness for a class, 111
Many-one reduction, 102
Matrix consequence, 180
Maximal consistent set, 34
Metalanguage, 9
Model, 10
Morley's categoricity theorem, 93
Mostowski collapse, 136
Multi-agent S5, 178

Names, interpretation, and extension, 141
Natural-deduction derivation, 30
Negation normal form, 27
Negation, conjunctive, and disjunctive normal form, 27
Negative solution to the classical Entscheidungsproblem, 105
No effective complete calculus for full validity, 163
Nonderivability, 146
Normalization for intuitionistic natural deduction, 156
Numeralwise representation of a relation, 118

Object language, 8
Omega-consistency, 122
Omitting types theorem, 86
Oracle Turing machine and Turing reducibility, 112
Order vocabulary, 137
Ordered pair, 2
Ordinal and cardinal, 5

Pairing, union, and power set, 131
Paraconsistent consequence relation, 182
Parameter theorem, 102
Partial computable function, 96
Partial function, 3
Peano arithmetic, 117
Poset filter, dense set, and genericity, 141
Post, 113
Power set, 2
Prefix normal form, 109
Prenex formula, 67
Prenex normal form, 69
Primitive recursive functions, 97
Primitive recursive relation, 97
Primitive rule, 151
Principal and nonprincipal types, 86
Principal formula, 151
Proof and provability predicates, 119
Proof term, 196
Proof theory, 9
Properties of a binary relation, 3
Propositional alphabet, 23
Propositional compactness, 35
Propositional derivability, 31
Propositional formula, 23
Propositional literal, clause, and formula interpretation, 26
Pure-literal elimination, 191

Quantifier elimination, 90

Rank, 136
Realization, 84
Reflexive transitive closure, 15
Refutational completeness of first-order resolution, 74
Refutational completeness of propositional resolution, 37
Relative consistency of CH, 145
Renaming apart, 72
Replacement schema, 131
Representability theorem, 118
Representation of a function, 118
Resolvent, 36
Rice, 104
Robinson arithmetic, 116
Rosser, 123
Routley–Meyer frame and structure, 183
Ryll–Nardzewski, 93

Satisfaction, 10, 47
Satisfiability modulo theories, 194
Saturated branch, 180
Saturation, 84
Saturation supplies homogeneity, 85
Second-order sentence, 161
Second-order signature, 161
Semantic consequence, 9
Semantic forcing under the fixed hypothesis, 142
Semantic notions, 25
Semantics, 8
Separation schema, 131
Sequent, 32

Set recursion, 133
Signature extension, expansion, and reduct, 46
Simple types, 167
Skolemization, 69
Soundness, 10
Soundness and completeness of linear temporal logic, 178
Soundness and strong completeness of IPL, 173
Soundness of LK, 150
Soundness of classical propositional natural deduction, 33
Soundness of first-order natural deduction, 55
Soundness of first-order resolution and factoring, 74
Soundness of propositional resolution, 37
Soundness of resolution, 193
Soundness of the classical propositional sequent calculus, 33
Soundness of the first-order sequent calculus, 55
Soundness, completeness, and cut elimination, 185
Standard and nonstandard models, 117
Standard type structure, 168
Strictness of the arithmetical hierarchy, 112
String, 12
Strong completeness of classical first-order logic, 62
Strong completeness of classical propositional natural deduction, 35
Strong normalization, 157
Structural induction, 13
Structural recursion, 14
Structure validity and frame validity, 175
Subformula property, 153
Subject reduction, 157
Subset, 2
Substitution in a term, 43
Substitution lemma, 47
Substructure, 65, 79
Syntactic arithmetical hierarchy, 106
Syntax, 9

Tactic, 196
Tarski undefinability, 127
Tarski–Vaught test, 65, 80
Term, 41
Term structure, 61
Term value, 46
The constructible class L, 140
The set ω, 131
The theories ZF and ZFC, 132
Theorem of a calculus, 10
Theory, model, satisfiability, and structure theory, 50
Trakhtenbrot, 105
Transfinite induction, 134
Transfinite recursion, 5, 135
Transitive set, 5
Trichotomy and transitivity of ordinals, 134
Trusted base, 197
Truth in a structure, 10
Truth values and valuation, 25
Truth, validity, and semantic consequence, 48
Turing jump, 112
Turing machine, 95
Type, 83
Types over a theory and type spaces, 83

Ultrafilter Lemma, 6
Ultraproduct, 88
Undecidability of the self-application domain set, 103
Unifier and most general unifier, 73
Unique readability, 24
Unique readability for first-order expressions, 42
Unit propagation, 191
Universal computation, 102
Upward Löwenheim–Skolem, 66

Validity, 10
Variable assignment and update, 46
Variables occurring in a term, 42

Well-founded relation, 136
Well-ordering theorem, 5
Witness property, 59